

FINAL YEAR ENGINEERING PROJECT

RECONSTRUCTING AUDIO FROM SILENT VIDEO

Complete Technical, Learning and Viva Handbook

Lip Reading and Speech Generation · Action Recognition and Sound Generation · Acoustic Eye
Three independent subsystems, running locally on one laptop

Author	Bilal Ashfaq
Machine	Apple M4, 17.18 GB unified memory, macOS 26.2
Compute	Metal Performance Shaders (MPS) and CPU. No CUDA anywhere
Subsystem 1	Auto-AVSR (Conformer + Transformer), 250.4 M parameters
Subsystem 2	Qwen2.5-VL-3B-Instruct + MOSS-SoundEffect v2.0
Subsystem 3	Phase-based Visual Microphone (Davis et al., 2014)

Every number, filename, model name and claim in this handbook was read out of the actual repository on this machine. Nothing is assumed and nothing is invented.

Contents

PART 0 — HOW TO USE THIS HANDBOOK	1
Chapter 0 — How to use this handbook	2
0.1 The three registers	2
0.2 The boxes in the margin	2
0.3 Reading routes	2
0.4 What this handbook will not do	3
0.5 One warning about vocabulary	3
PART 1 — THE PROJECT AT A GLANCE	4
Chapter 1 — What this project actually is	5
1.1 Title	5
1.2 The problem statement	5
1.3 Motivation: why this matters	5
1.4 Existing systems, and what is wrong with them	6
1.5 The proposed system	7
1.6 Objectives	7
1.7 Key features	8
1.8 The complete workflow, at the level of a sentence each	8
Chapter 2 — The three subsystems, side by side	10
2.1 One table you should be able to reproduce from memory	10
2.2 Why three separate systems and not one	10
2.3 Where the engineering depth is	11
Chapter 3 — Explaining the project out loud	13
3.1 The 30-second explanation	13
3.2 The 2-minute explanation	13
3.3 The 5-minute explanation	13
Chapter 4 — Map of the repository	16
4.1 Top level	16
4.2 Subsystem 2 in detail: Module3_Fresh/	16
4.3 Subsystem 1 in detail: 02-Auto-AVSR-Test/	17
4.4 Subsystem 3 in detail: Acoustic eye/acoustic-eye/	17
4.5 What is deliberately not in version control	17
Chapter 5 — The technology stack	20
5.1 Complete stack table	20
5.2 Why FastAPI for two subsystems and Flask for the other	21
5.3 Why React for one frontend and plain JavaScript for the others	22

PART 2 — FOUNDATIONS	23
Chapter 6 — Artificial intelligence, machine learning, deep learning	24
6.1 The three words, in order of size	24
6.2 Parameters, and what "3 billion parameters" means	24
6.3 Precision: float32, bfloat16, and why it matters here	25
6.4 Training, inference, and which one this project does	25
6.5 Supervised learning, and the vocabulary around datasets	26
Chapter 7 — Neural networks and the layer types this project uses	28
7.1 A neuron, a layer, a network	28
7.2 Convolutional neural networks (CNNs)	28
7.3 Recurrent networks: RNN, LSTM, GRU	29
7.4 The Transformer and attention	30
7.5 The Conformer	31
Chapter 8 — Sequence modelling, CTC and decoding	33
8.1 The alignment problem	33
8.2 CTC: Connectionist Temporal Classification	33
8.3 The CTC lag correction - a real bug, found by measurement	34
8.4 Logits, softmax and log-probabilities	34
8.5 Beam search decoding	34
8.6 Tokens and SentencePiece	35
Chapter 9 — Generative models: VAE, diffusion, flow matching	37
9.1 Latent space and the autoencoder	37
9.2 Diffusion, in the way that actually makes sense	37
9.3 Flow matching	38
9.4 The Diffusion Transformer (DiT)	39
9.5 Classifier-free guidance (CFG)	39
9.6 Negative prompts	40
9.7 Seeds and determinism	40
Chapter 10 — Vision-language and multimodal models	42
10.1 What "multimodal" means	42
10.2 Why a VLM instead of an action classifier	42
10.3 How Qwen sees video here	42
Chapter 11 — Audio and video fundamentals	45
11.1 Digital audio	45
11.2 Decibels, dBFS, RMS, peak and crest factor	45
11.3 Dynamic range	46
11.4 Spectrum, spectrogram, STFT	46
11.5 Spectral flatness and harmonic ratio	47

11.6 Effective bits	47
11.7 Envelope, onset, attack and the distinction that mattered	48
11.8 Digital video	48
11.9 AAC and why the audio duration does not exactly match	49
11.10 The mixing chain vocabulary	49
PART 3 — SUBSYSTEM 1: LIP READING AND SPEECH GENERATION	50
Chapter 12 — What lip reading is, and the proof that it is really happening	51
12.1 The purpose of this subsystem	51
12.2 What "lip reading" and "visual speech recognition" mean	51
12.3 Why audio is unnecessary - and in fact forbidden - in this module	51
12.4 The complete pipeline	52
12.5 Video preprocessing: validation and standardisation	53
12.6 Face detection and the mouth region of interest	53
12.7 The audio-independence experiment	54
Chapter 13 — The Auto-AVSR model	57
13.1 What it is	57
13.2 The architecture, layer by layer	57
13.3 Why each component exists	58
13.4 Why this model was selected	58
13.5 Alternatives considered	59
13.6 Why CPU and not MPS	59
13.7 Loading the model once	59
Chapter 14 — From encoder output to a sentence	61
14.1 The decode, executed inline and why	61
14.2 Reading a decoder score	61
Chapter 15 — Word timing from the CTC head	64
15.1 The problem	64
15.2 The algorithm	64
15.3 The lag correction	65
15.4 Pause detection, and why it comes from the video	65
Chapter 16 — Speech synthesis and placement	67
16.1 The TTS stack	67
16.2 Choosing a voice: gender detection	67
16.3 Four lessons that shaped the placement code	67
16.4 The placement algorithm	68
Chapter 17 — Datasets behind Subsystem 1	70
17.1 Dataset characteristics that matter to your results	70
17.2 Dataset limitations and biases - state these honestly	70

Chapter 18 — Subsystem 1: the web application	72
18.1 The API	72
18.2 Error handling	72
18.3 Security details worth pointing out	73
18.4 The frontend	73
Chapter 19 — Subsystem 1: results, limitations and challenges	75
19.1 What was measured	75
19.2 What was verified qualitatively	75
19.3 The honest gap: there is no word error rate	75
19.4 Limitations	76
19.5 Challenges faced, and how each was solved	76
Chapter 20 — Subsystem 1: important code, explained	79
20.1 run_server.py - the entry point	79
20.2 app/video_processing.py - standardize()	79
20.3 app/inference.py - the shape contract	80
20.4 app/tts.py - the Kokoro worker protocol	80
Chapter 21 — The rejected first attempt: AV-HuBERT	82
21.1 What was tried	82
21.2 What AV-HuBERT is	82
21.3 Why it was set aside	82
PART 4 — SUBSYSTEM 2: ACTION RECOGNITION AND SOUND GENERATION	84
Chapter 22 — The nine-stage pipeline	85
22.1 Purpose and contract	85
22.2 The nine stages	85
22.3 Stage-by-stage detail	86
22.4 Why 60 seconds and not longer	86
22.5 Expected processing time	86
Chapter 23 — Module 2: action recognition with Qwen2.5-VL	89
23.1 What this stage does	89
23.2 The design decision that makes this work	89
23.3 The prompt	89
23.4 The windowing	90
23.5 Frame extraction - video stream only	90
23.6 Running the model	91
23.7 The memory guard	91
23.8 Parsing the response	92
23.9 Merging windows into spans	92
23.10 The reference clip, end to end	92

23.11 Recorded performance	93
Chapter 24 — Timeline resolution	95
24.1 The problem: overlapping spans are ambiguous	95
24.2 Midpoint boundary resolution	95
24.3 Suspect flagging	95
24.4 The resolved timeline for the reference clip	96
24.5 Validation	96
24.6 The bug that this stage caused, and how it was fixed	96
24.7 Which array the pipeline uses, and why	97
Chapter 25 — From an action phrase to a Foley prompt	99
25.1 What this layer does	99
25.2 The resolution tiers	99
25.3 The 17 curated classes	100
25.4 An example prompt, in full	100
25.5 Keyword matching, and the bug that shaped it	101
25.6 Deliberate silences	102
25.7 The verb fallback, and the hole it closed	102
25.8 Open-vocabulary prompt synthesis	103
Chapter 26 — MOSS-SoundEffect v2.0	106
26.1 The model	106
26.2 The three components	106
26.3 Duration handling, and the parameter that was added	106
26.4 Phased inference: the memory problem and its solution	107
26.5 The two MPS compatibility fixes	107
26.6 The validated settings	109
26.7 Recorded generation performance	109
26.8 The content-addressed cache	110
26.9 Multi-candidate generation	110
Chapter 27 — The Foley quality gate	113
27.1 Why generated audio cannot be trusted	113
27.2 The six gates	113
27.3 Why the gates are multi-criteria	113
27.4 Why gate 4 exists separately from gate 3	114
27.5 The quality score	114
27.6 What happens to a rejected asset	114
27.7 The second, independent limit in the mixer	115
27.8 What the gate actually did, across the whole corpus	115
Chapter 28 — Case study: the cup-pickup failure	118
28.1 Why cup pickup was attempted at all	118

28.2 Attempt 1 - MOSS v1	118
28.3 Attempt 2 - MOSS v2, with a longer negative prompt	119
28.4 Attempt 3 - extraction from an existing asset	120
28.5 The decision in the standalone build: silence	120
28.6 The rescue in the web pipeline	120
28.7 How to tell this story in a viva	121
Chapter 29 — Visual event localisation	123
29.1 The problem restated	123
29.2 Frame motion in region bands	123
29.3 The four detection strategies	124
29.4 Confidence is not uniform, and it is reported	126
29.5 The wide search for footsteps	126
29.6 The results on the reference clip	126
Chapter 30 — Temporal alignment	129
30.1 Segment selection: cutting a 10-second asset down	129
30.2 The alignment rule	130
30.3 The alignment plan	130
30.4 Why residual error accumulates for rhythmic actions	130
30.5 Merging consecutive intervals - and why only some of them	131
Chapter 31 — Mixing and rendering	133
31.1 The per-clip chain	133
31.2 Why levels differ by class	133
31.3 The peak cap, and the bug it caused	133
31.4 End-of-video handling	134
31.5 The bus	134
31.6 Mix validation	135
31.7 The final render	135
Chapter 32 — Model selection: everything that was rejected	137
32.1 Action recognition	137
32.2 Sound generation	137
32.3 The direct A/B: MOSS against Stable Audio	138
32.4 The runner for the alternative backend	139
Chapter 33 — Subsystem 2: results	141
33.1 Functional verification	141
33.2 Synchronisation accuracy - the headline result	141
33.3 The aggregate picture across all recorded jobs	142
33.4 The current end-to-end result on the reference clip	142
33.5 The 19-check quality gate on the validated build	143
33.6 The two ablations, and what they prove	144

33.7 Output sparsity - a property, not a defect	145
PART 5 — SUBSYSTEM 3: ACOUSTIC EYE	146
Chapter 34 — The visual microphone principle	147
34.1 What this subsystem does	147
34.2 The physics	147
34.3 The sampling theorem, which bounds everything	147
34.4 The algorithm	148
34.5 Why phase, and not simply pixel brightness	148
34.6 The core code	149
Chapter 35 — Implementation, and the eleven fixes	151
35.1 What was adapted, and from where	151
35.2 The eleven documented fixes	151
35.3 The three parameters that matter for high-speed footage	152
35.4 The eight pipeline stages	153
35.5 The API	153
35.6 The "audio in text" report	154
35.7 Cost	154
Chapter 36 — Subsystem 3: results	156
36.1 The controlled characterisation	156
36.2 Ordinary footage: what actually happens	157
36.3 The high-speed demonstration	157
36.4 Testing	158
36.5 Limitations, stated as the project states them	158
PART 6 — THE WEB APPLICATIONS	159
Chapter 37 — Subsystem 2: the backend	160
37.1 Three processes, isolated by design	160
37.2 The layers	160
37.3 The job model	160
37.4 The background worker	161
37.5 Real progress, not a fake bar	162
37.6 The endpoints	162
37.7 Streaming upload with the limit enforced during the stream	162
37.8 Status codes	163
37.9 Failure messages	163
Chapter 38 — Subsystem 2: the React frontend	165
38.1 The stack	165
38.2 React concepts you need to be able to explain	165

38.3 The application state machine	165
38.4 The useJob polling hook	166
38.5 The API client	167
38.6 TypeScript interfaces as a contract	168
38.7 The action timeline visualisation	168
38.8 Honest reporting in the interface	168
Chapter 39 — The complete request flow	171
39.1 The diagram	171
39.2 What the background thread does	171
39.3 What the browser does while that happens	173
39.4 Every arrow, explained	173
PART 7 — CODE ARCHITECTURE AND INTEGRATION	174
Chapter 40 — Who calls whom	175
40.1 Subsystem 2 call graph	175
40.2 Subsystem 1 call graph	175
40.3 Subsystem 3 call graph	176
40.4 Import discipline	176
Chapter 41 — File-by-file reference	178
41.1 Subsystem 2 backend	178
41.2 Subsystem 2 frontend	179
41.3 The MOSS wrapper and the standalone scripts	179
41.4 Subsystem 1 and 3	180
Chapter 42 — Environments and process isolation	182
42.1 The five environments	182
42.2 How a subprocess is invoked	182
42.3 Structured error propagation	183
42.4 Protecting validated work	183
Chapter 43 — Data flow: what happens to the video file	185
43.1 Subsystem 2, stage by stage	185
43.2 Important data formats	186
43.3 Where things are written	186
PART 8 — IMPORTANT CODE, LINE BY LINE	188
Chapter 44 — Imports: what they are and why they are there	189
44.1 The backend imports	189
Chapter 45 — Snippet 1: the application entry point	192
Chapter 46 — Snippet 2: the upload endpoint	194
Chapter 47 — Snippet 3: starting the job	196

Chapter 48 — Snippet 4: the pipeline orchestrator	198
Chapter 49 — Snippet 5: resolving an action to a Foley class	201
Chapter 50 — Snippet 6: the cache key	204
Chapter 51 — Snippet 7: the quality gate	206
Chapter 52 — Snippet 8: true attack detection	208
Chapter 53 — Snippet 9: footstep detection	211
Chapter 54 — Snippet 10: matching the generated cadence to the filmed gait	213
Chapter 55 — Snippet 11: the mixer	216
Chapter 56 — Snippet 12: the final render	219
Chapter 57 — Snippet 13: the MOSS denoising loop	221
Chapter 58 — Snippet 14: the frontend polling hook	223
Chapter 59 — Snippet 15: CTC forced alignment	225
 PART 9 — MODELS, DATASETS, ACCURACY AND TESTING	226
Chapter 60 — Every model, in one place	227
60.1 Models actually used in the delivered system	227
60.2 Models evaluated and rejected	229
Chapter 61 — Every dataset, in one place	231
61.1 The honest summary first	231
61.2 Datasets behind the lip-reading checkpoint	231
61.3 Datasets behind the other models	232
61.4 The project's own evaluation material	232
61.5 Dataset vocabulary, defined	232
Chapter 62 — Accuracy and performance, carefully separated	235
62.1 Model accuracy: what exists and what does not	235
62.2 Synchronisation accuracy	235
62.3 Audio quality	236
62.4 Application performance	236
62.5 Reliability	237
Chapter 63 — Testing	239
63.1 Subsystem 2 test suites	239
63.2 The four most valuable tests, and why	239
63.3 A test that documents a limitation	240
63.4 Running them	240
63.5 Subsystem 3 tests	240
63.6 What is not tested	240
 PART 10 — CHALLENGES, LIMITATIONS AND FUTURE WORK	241

Chapter 64 — Challenges: cause, solution, result	242
64.1 Model and environment challenges	242
64.2 Recognition challenges	242
64.3 Generation challenges	243
64.4 Synchronisation and mixing challenges	244
64.5 Application challenges	245
Chapter 65 — Limitations	247
65.1 The limitation that matters most	247
65.2 Subsystem 1 limitations	247
65.3 Subsystem 2 limitations	247
65.4 Subsystem 3 limitations	248
65.5 Cross-cutting limitations	248
65.6 Ethical and privacy considerations	249
Chapter 66 — Future scope	251
66.1 Currently implemented	251
66.2 Future work, in priority order	251
 PART 11 — VIVA QUESTION BANK	 253
Chapter 67 — Level 1: basic project questions	254
Chapter 68 — Level 2: module and pipeline questions	256
Chapter 69 — Level 3: machine-learning questions	258
Chapter 70 — Level 4: deep-learning questions	260
Chapter 71 — Level 5: model questions	262
Chapter 72 — Level 6: dataset questions	264
Chapter 73 — Level 7: backend questions	265
Chapter 74 — Level 8: frontend questions	267
Chapter 75 — Level 9: API and integration questions	269
Chapter 76 — Level 10: audio and video processing questions	270
Chapter 77 — Level 11: synchronisation questions	272
Chapter 78 — Level 12: architecture questions	274
Chapter 79 — Level 13: difficult technical questions	275
Chapter 80 — Level 14: tricky and hostile questions	278
Chapter 81 — Level 15: "why did you use this?"	280
 PART 12 — PRESENTATION SCRIPT	 282
Chapter 82 — The slide-by-slide script	283
Slide 1 - Title	283
Slide 2 - The problem	283

Slide 3 - Existing approaches and why they failed here	283
Slide 4 - Objectives	284
Slide 5 - System overview	284
Slide 6 - Subsystem 1: lip reading	284
Slide 7 - Proving there is no audio path	284
Slide 8 - Word timing from the CTC head	284
Slide 9 - Subsystem 2: the nine stages	285
Slide 10 - The central problem: where to put the sound	285
Slide 11 - Visual event localisation	285
Slide 12 - Alignment on true attack times	285
Slide 13 - Not trusting the generated audio	286
Slide 14 - Why one number is not enough	286
Slide 15 - When it fails, it says so	286
Slide 16 - Results	286
Slide 17 - Subsystem 3: the visual microphone	287
Slide 18 - Limitations	287
Slide 19 - Future work	287
Slide 20 - Conclusion	288
82.1 The live demonstration	288
 PART 13 — LAST NIGHT BEFORE THE VIVA	 289
Chapter 83 — The project in ten lines	290
Chapter 84 — The numbers you must know	291
Chapter 85 — Technologies, models and datasets, in three tables	292
Chapter 86 — Important files	293
Chapter 87 — Important algorithms	294
Chapter 88 — Things you must remember	295
Chapter 89 — The fifty questions most likely to be asked	296
 PART 14 — GLOSSARY AND APPENDICES	 298
Chapter 90 — Glossary A to Z	299
A	299
B	299
C	299
D	300
E	300
F	300
G	301
H	301

I	301
L	301
M	302
N	302
O	302
P	302
Q	302
R	302
S	303
T	303
V	303
W	304
Z	304
Chapter 91 — Appendix A: how this handbook was verified	306
91.1 Source files read in full	306
91.2 Documentation read in full	306
91.3 Data read and recomputed	306
91.4 Commands executed while writing	307
91.5 Discrepancies found between the documentation and the code	307
Chapter 92 — Appendix B: how to run everything	310
92.1 Subsystem 2 - action recognition and sound generation	310
92.2 Subsystem 1 - lip reading and speech generation	310
92.3 Subsystem 3 - Acoustic Eye	310
92.4 Integrity checks before a demonstration	310
92.5 Rebuilding this handbook	311

PART 0

HOW TO USE THIS HANDBOOK

Read this page first. It tells you which parts to read, in which order, and what each symbol in the margin means.

Chapter 0 — How to use this handbook

This document has one job: to make you able to explain, defend and extend your own project, from a standing start, without needing anyone else in the room.

It is not a project report. A project report is written **for** an examiner. This is written **for you**, so that you can then talk to the examiner. It therefore repeats itself deliberately: every important idea appears three times, in three different registers.

0.1 The three registers

Almost every technical idea in this handbook is explained in the same order.

Register	What it does	How to spot it
Simple explanation	Explains the idea in everyday language, with no jargon, as if to a friend who does not study computing	Green box marked SIMPLE EXPLANATION
Technical explanation	Gives the proper definition, the correct terminology, and the mechanism	Ordinary body text under a numbered section
In our project	Says exactly where that idea appears in your code, with the real filename and the real numbers	Purple box marked EVIDENCE IN THE PROJECT

Read the simple explanation first. Then read the technical explanation. Then read the project evidence. If you can only remember one of the three under pressure, remember the project evidence, because that is the one an examiner cannot get from a textbook.

0.2 The boxes in the margin

Box	Meaning
SIMPLE EXPLANATION	Plain-language version. Safe to say to anybody.
EVIDENCE IN THE PROJECT	A measured fact, a filename, or a line of your code. This is your proof.
PROFESSOR MAY ASK	A question an examiner is likely to ask at exactly this point, with the answer.
REMEMBER THIS	A number or a sentence worth memorising verbatim.
CAUTION	A trap: something that sounds right but is wrong, or a claim you must not overstate.
QUESTION	A full viva question with a short answer, a detailed answer and a project-specific answer.

0.3 Reading routes

You do not have to read this cover to cover.

If you have one evening. Read Part 1 (overview), then Part 13 (last night before viva), then the question bank in Part 11 at Levels 1, 2 and 15.

If you have one week. Read Parts 1 and 2 (foundations) first, so the vocabulary stops being frightening. Then read the part for whichever subsystem you are being asked about most. Then Part 11.

If you want to actually understand the whole system. Read in order. Parts 1 to 5 are the science and engineering, Parts 6 to 8 are the software, Parts 9 and 10 are the honest evaluation, Parts 11 to 14 are exam preparation.

0.4 What this handbook will not do

It will not claim results the project did not produce. Where a number does not exist, it says so. Where a component is incomplete, it says so. Where the project's own written documentation has fallen out of date relative to the code, this handbook follows the code and points out the discrepancy.

CAUTION: Two claims that are commonly overstated

Do not say "my system understands what people are saying". Subsystem 1 produces a best guess from lip motion, and lip reading is genuinely ambiguous.

Do not say "my system recovers sound from any video". Subsystem 3 is bounded by the sampling theorem: at 30 frames per second you can only recover frequencies below 15 Hz, which is below human hearing. This is physics, not a bug, and saying so out loud is a strength, not a weakness.

0.5 One warning about vocabulary

Throughout the project's own files, the three subsystems are sometimes called "Module 1", "Module 2" and "Module 3", and sometimes "S1", "S2" and "S3". These names do **not** line up one-to-one, and this confuses people, including examiners.

Name used in files	What it actually refers to
Module 1	Lip reading (the folder 02-Auto-AVSR-Test)
Module 2	Action recognition only (Qwen2.5-VL), a stage inside the sound-generation web app
Module 3	Sound generation and synchronisation, the rest of that same web app
S1	Subsystem 1 in the research paper: lip reading and speech generation
S2	Subsystem 2 in the research paper: action recognition and Foley generation, which contains Module 2 and Module 3
S3	Subsystem 3 in the research paper: Acoustic Eye, the visual microphone

This handbook uses **Subsystem 1, 2 and 3** throughout, and says "Module 2" or "Module 3" only when naming a stage inside Subsystem 2, because that is what the code calls them.

REMEMBER THIS

Three subsystems. Subsystem 2 contains the things the code calls Module 2 and Module 3. If an examiner says "module", ask which one they mean; that alone shows you know the architecture.

PART 1

THE PROJECT AT A GLANCE

Title, problem, motivation, objectives, the three subsystems, the technology stack, and the three rehearsed explanations you can give a professor in 30 seconds, 2 minutes and 5 minutes.

Chapter 1 — What this project actually is

1.1 Title

Reconstructing Audio from Silent Video: A Three-Path System for Speech, Foley and Surface-Vibration Recovery on Consumer Hardware.

That is the title of the research paper written for this project, which lives in `paper/main.tex`. The three web applications carry their own working titles:

- **"Give a silent video its voice back"** — the lip-reading application.
- **"Transform silent videos into synchronized sound"** — the action-recognition and

Foley application.

- **"Recovering Acoustic Information from Visual Vibrations"** — Acoustic Eye.

1.2 The problem statement

An enormous amount of recorded video carries no usable audio.

- Surveillance and dashboard cameras very often record picture only.
- Archive and historical footage may have lost its sound track, or never had one.
- Ordinary phone recordings are routinely ruined by wind noise, handling noise, a muted microphone, or a codec failure.
- Video shot for social platforms is frequently stripped of audio for rights reasons.

In every one of those cases the **picture still contains information about the sound**. A person's lips move in a way that constrains what they could have said. A mug meeting a table implies a particular contact sound at a particular instant. Surfaces in the frame physically vibrate in response to the sound field around them.

The problem this project attacks is therefore:

KEY POINT: The problem in one sentence

Given a video with no audio track, reconstruct a plausible, temporally correct audio track using only the pixels.

1.3 Motivation: why this matters

Accessibility. Silent footage is unusable for anyone who relies on sound, and hard to follow for everyone else. Reconstructed audio makes silent archives watchable.

Forensics and investigation. Surveillance footage without audio loses half its evidential value. Even approximate reconstruction of what was said or what was handled adds information.

Film and content production. Foley - the craft of making everyday sound effects to match action on screen - is skilled, slow, manual work. Automating a first pass of it is economically meaningful.

Scientific interest. The visual microphone result (Davis et al., SIGGRAPH 2014) showed that sound is literally recoverable from the vibration of objects in high-speed video. Reimplementing and characterising that is worthwhile in itself.

Engineering interest, which is the honest core of this project. Making generative audio land at the right *instant* turns out to be far harder than generating the audio at all, and no amount of model quality fixes bad placement. That problem, and the way it was solved and measured, is the real contribution.

1.4 Existing systems, and what is wrong with them

Three families of existing approach were evaluated on this machine, and all three were found wanting for this task. The evidence is in the project's own result files.

(a) Closed-vocabulary action recognisers. These classify a clip into one of a fixed list of labels. Two were tested:

Model	What it returned for the test clip	Why that is wrong
VideoMAE (Kinetics fine-tune)	"shredding paper" at confidence 0.550, with no relevant label in the top ten	Kinetics labels describe whole-clip human activities, not object-contact events. There is no label for "a mug is set on a table".
X-CLIP	"pouring liquid" at 0.21 to 0.38 in every window, collapsing to one segment	Same structural problem, plus no temporal discrimination at all.

Table 1.1 - Measured failures of closed-vocabulary recognisers. Sources: 03-FoleyCrafter-Test/action-recognition/results/videomae_test_result.json and ../temporal_results.json.

The failure is not that the models are bad. It is that the label set is organised around the wrong unit. Foley needs *events*, and Kinetics-style labels describe *activities*.

(b) Video-conditioned audio generators. These take the video itself as the conditioning signal and produce audio end-to-end. Two were tried:

Model	Output	Measured failure
FoleyCrafter	16 kHz mono, 3.0 s	A continuous noise bed, not events: 25 "events" detected at 8.6 Hz, 0 percent of the file below -20 dB
MMAudio v1 / v2	44.1 kHz mono	86 percent and 96 percent digital silence, and the sound was placed on the wrong action

These are the natural competitors to this project's approach, and it is important to be fair about them: they are strong published systems. They failed **here**, on **this material**, within **17 GB of unified memory and no CUDA**. That is a statement about the deployment constraint, not about the models in general, and the handbook says so consistently.

(c) Text-to-audio generators used naively. Three were tried and all three failed the same way:

Model	Output	Failure
Stable Audio Open 1.0	44.1 kHz stereo, 4.5 s	96.2 percent silence, 2 clicks of 0.04 s
Stable Audio Open Small	44.1 kHz stereo, 3.0 s	95.6 percent silence
AudioLDM 2	16 kHz mono, 10 s	91.9 percent silence, 30 ms impulses

Table 1.2 - The "silence plus clicks" signature. Source: Module3_Fresh/results/text_to_audio_model_evaluation.md.

Three different architectures producing an almost identical failure signature is a clue, and the project chased it: **every one of those runs asked for 2.0 to 4.5 seconds of audio from models trained to generate 10 to 47 seconds**. Asking a generative model for output far shorter than its training regime is a plausible cause of out-of-regime behaviour. The conclusion drawn was procedural rather than about model identity:

KEY POINT: The rule that came out of the failure analysis

Generate at the model's native duration and crop afterwards. Never ask a generative audio model for a duration far below what it was trained on.

This is exactly what the final system does: it generates 10 seconds and uses only the segment it needs.

(d) The deeper problem nobody else solves for you. Even a perfect generator does not tell you *when* to play the sound. An action recogniser says "walking, 1.5 to 2.5 seconds". But a footstep is not audible across a span; it is audible at an *instant* - the moment the foot plants. Stretching a two-second walking clip across a two-second label produces audio that is present and completely, visibly wrong.

1.5 The proposed system

The proposed system reconstructs audio along **three independent paths**, each attacking a different kind of acoustic information, all running locally on one laptop.

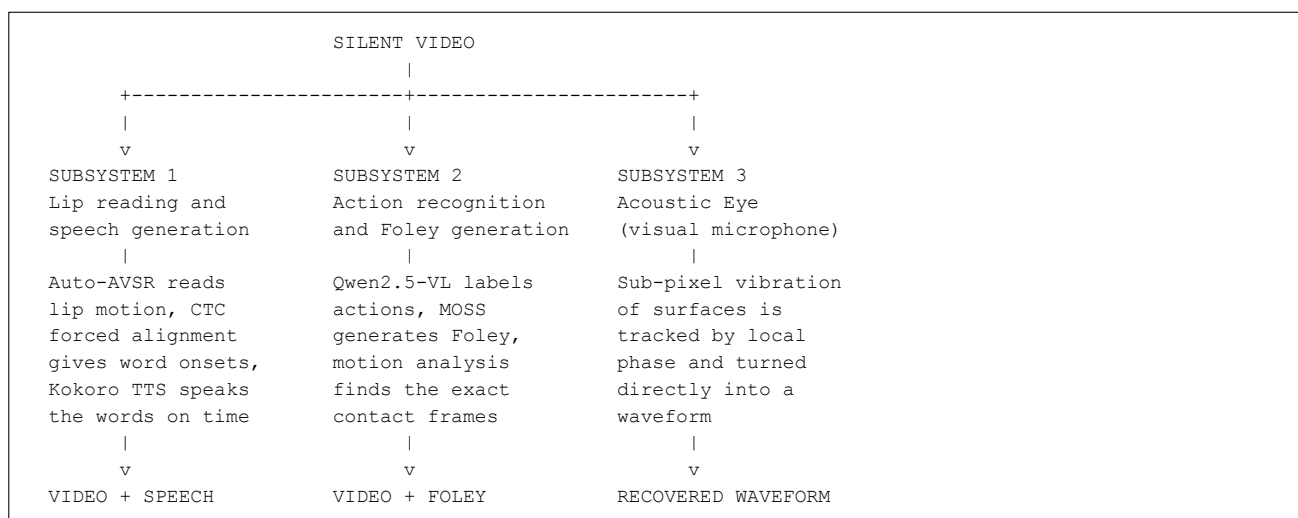


Figure 1.1 - The three paths. They are complementary, not competing: speech, physical interaction, and the acoustic field itself.

The three paths share two contracts:

1. **Input contract.** Only the video stream is ever decoded. If the uploaded file

already contains an audio track, that track is never read, never decoded, and never used. This is enforced in code with `ffmpeg -map 0:v:0` and is proved experimentally in Subsystem 1 (Section 12.7).

1. **Output contract.** The picture is stream-copied, never re-encoded. The output video

is bit-identical to the input in its picture stream; only an audio track is added.

1.6 Objectives

Primary objective. Reconstruct plausible and *temporally correct* audio from silent video, on consumer hardware, with no cloud inference.

Secondary objectives, in the order the project actually pursued them:

1. Prove that recognition is happening from pixels alone, not from a hidden audio path.
2. Make generated sound land on the correct frame, not merely inside the correct label.
3. Refuse to output audio that is measurably unusable, rather than covering the gap with something plausible-sounding.

1. Keep every model inside its own environment, so that three mutually incompatible dependency stacks can coexist on one machine.
1. Report every limitation honestly, including the ones that make the system look worse.

1.7 Key features

Feature	Where it lives	Why it matters
Visual-event synchronisation	<code>backend/services/synchronization.py</code>	Sound is anchored to measured contact frames, not label boundaries. This is the project's core contribution.
True envelope attack alignment	<code>attack_times()</code> in the same file	Onset-strength peaks lead or lag the real transient by -96 to +250 ms. Using them caused a real, audible 96 ms error.
Multi-criteria quality gate	<code>backend/services/foley_validation.py</code>	Six independent gates. 29.6 percent of the 54 generated assets were rejected by it.
Multi-candidate generation	<code>generate_best()</code> in <code>sound_generation.py</code>	Up to three seeds per sound class, stopping early. This rescued the cup-pickup class, which failed on seeds 42 and 43 and passed on seed 44 with a score of 85.8.
Honest silence	<code>pipeline.py</code> , stage 6	An action with no usable sound is left silent and reported, never filled with a substitute.
Open-vocabulary Foley	<code>backend/services/prompt_synthesis.py</code>	Any action phrase, even one nobody wrote a class for, gets a prompt and a synchronisation strategy.
Process isolation	<code>backend/core/config.py</code>	Each model runs as a subprocess in its own virtual environment. Memory is returned to the operating system unconditionally at process exit.
Content-addressed cache	<code>cache_key()</code> in <code>sound_generation.py</code>	Generation costs about 4.7 minutes per asset. A repeated request returns in seconds.
Real progress reporting	<code>core/jobs.py</code> and <code>pipeline.py</code>	The percentage shown in the browser is a function of work actually completed. No progress value is fabricated on the client.

1.8 The complete workflow, at the level of a sentence each

Input. A silent video file. MP4, MOV, AVI, M4V or MKV, at most 200 MB and at most 60 seconds, for the Foley application; MP4, MOV or M4V for the lip-reading application.

Processing. For the Foley application, nine stages: upload, validation, action recognition, timeline resolution, Foley generation, Foley quality validation, visual synchronisation, audio mixing, and rendering.

Output. An MP4 whose picture stream is bit-identical to the input and which now carries an AAC audio track at 192 kbit/s, 48 kHz mono, generated from a 48 kHz mono PCM 16-bit master.

Chapter 2 — The three subsystems, side by side

2.1 One table you should be able to reproduce from memory

	Subsystem 1	Subsystem 2	Subsystem 3
Name	Lip Reading and Speech Generation	Action Recognition and Sound Generation	Acoustic Eye
Kind of information	Speech	Physical interaction	The acoustic field itself
Recognition model	Auto-AVSR vsr_trlrs2lrs3vox2avsp_base, 250,383,410 parameters	Qwen2.5-VL-3B-Instruct	none - this is signal processing, not learning
Generation model	Kokoro TTS (ONNX), Piper fallback	MOSS-SoundEffect v2.0	none
Backend framework	Flask	FastAPI	FastAPI
Frontend	Vanilla HTML, CSS, JavaScript	React 18 + Vite 6 + TypeScript + Tailwind	Vanilla HTML, CSS, JavaScript
Compute device	CPU only	Apple MPS	CPU
Folder	02-Auto-AVSR-Test/	Module3_Fresh/	Acoustic eye/acoustic-eye/
Status	Working, verified on 7 recordings	Working end to end, 64 automated tests pass	Working; characterised on synthetic stimuli and demonstrated on one high-speed clip

Table 2.1 - The three subsystems. Learn this table; it answers about a third of all viva questions on its own.

2.2 Why three separate systems and not one

Three reasons, all of them real constraints rather than design preference.

Reason 1: the information is of different kinds. Speech is carried by articulator motion. Foley is carried by contact events. Surface vibration is carried by sub-pixel displacement. No single model reads all three.

Reason 2: the dependency stacks are mutually incompatible. This is not a convenience argument; the environments genuinely cannot coexist.

Subsystem	Python	Key pins	Why the pin is forced
1 (lip reading)	3.11	torch 2.5.1, torchvision 0.20.1, numpy 1.26.4, av 13.1.0, mediapipe 0.10.21	<code>torchvision.io.read_video</code> is the repository's only video loader and was removed in torchvision 0.26. torchvision 0.20.1 references <code>av.AVError</code> , which PyAV 14 renamed. MediaPipe 0.10.21 declares <code>numpy<2</code> .
1 (its TTS voice)	3.11, separate venv	kokoro-onnx	Kokoro needs <code>numpy>=2</code> , which directly contradicts the MediaPipe pin above.
2 (action recognition)	3.10	torch 2.13	The validated Qwen environment.
2 (sound generation)	3.12	torch 2.9.1	MOSS-SoundEffect requires Python 3.12 and torch 2.9.

You cannot install `numpy==1.26.4` and `numpy>=2` in the same interpreter. That single conflict is why the lip-reading application drives its own text-to-speech engine as a separate process over a pipe.

Reason 3: memory. Qwen2.5-VL peaks near 12 GB and MOSS near 12.1 GB, on a machine with 17.18 GB total. They can never be resident at the same time.

PROFESSOR MAY ASK: Why did you not just use one big model that does everything?

Because no such model runs in 17 GB on Apple Silicon, and because the three problems have different structure. Video-to-audio models that do try to do it end to end were evaluated - MMAudio and FoleyCrafter - and did not produce usable output on this hardware and this material. The measured results are in `Module3_Fresh/results/mmaudio_model_evaluation.md` and `foleycrafter_model_evaluation.md`.

2.3 Where the engineering depth is

Be honest and specific about this in a viva, because it is a strength.

- **Subsystem 1** is largely careful *integration*: the model, its preprocessing

geometry, its tokenizer and its beam-search configuration are used unmodified from the official Auto-AVSR repository. The original work is the input standardisation, the CTC-based word timing, the phrase-level speech placement, and the proof of audio independence.

- **Subsystem 2** is where the original engineering is. The generation model is

off-the-shelf, but the *placement*, the *validation*, the *level policy*, the *open vocabulary*, the *cache*, the *memory-phased inference wrapper* and the *whole nine- stage pipeline* are this project's own work.

- **Subsystem 3** is a faithful adaptation of a published algorithm with eleven

documented robustness fixes, wrapped in a web application. The original contributions are the fixes, the parameter controls that matter for high-speed footage, and the characterisation.

REMEMBER THIS

If asked "what is your contribution?", the sharpest answer is: *placement and validation*. Anyone can call a generative model. Making the sound land on the right frame, and refusing to ship sound that measurement says is unusable, is the work.

Chapter 3 — Explaining the project out loud

These are scripts. Read them out loud several times until they are yours. They are deliberately written the way people actually speak, not the way reports are written.

3.1 The 30-second explanation

> "My project takes a video that has no sound and generates the sound for it, using only > the picture. It does this in three ways. First, it reads the speaker's lips and > produces speech. Second, it recognises physical actions - walking, drinking, putting a > cup down - and generates matching sound effects that are aligned to the exact frame > where the action happens. Third, it implements the visual microphone, which recovers > sound from the tiny vibrations of objects in the frame. Everything runs locally on a > laptop; nothing goes to the cloud."

3.2 The 2-minute explanation

> "The starting point is that a lot of video has no usable audio - surveillance footage, > archive material, phone videos ruined by wind. But the picture still carries > information about the sound, so the question is whether you can get the audio back. >> I built three independent paths. >> The first reads lips. It uses Auto-AVSR, a visual speech recognition model with 250 > million parameters, which takes an 88 by 88 pixel crop of the mouth for every frame and > produces a sentence. I then use the model's own CTC head to work out when each word > starts, and a text-to-speech engine speaks the sentence timed to those moments. >> The second is the main one. A vision-language model, Qwen2.5-VL, watches the video in > two-second windows and describes the physical action in each one. Those descriptions > are turned into text prompts for a sound-generation model called MOSS-SoundEffect, > which produces the Foley audio. But here is the part that actually mattered: an action > label like 'walking, 1.5 to 2.5 seconds' does not tell you when to play a footstep, > because a footstep is an instant, not a span. So I run a separate frame-level motion > analysis that finds the exact frames where a foot plants, or a mug touches a table, and > I anchor each generated sound to those instants. On my reference clip the worst > alignment error is 20 milliseconds, which is less than half a video frame. >> The third path is the visual microphone: sound makes objects vibrate by a fraction of a > pixel, and if the camera is fast enough you can measure that vibration and turn it back > into a waveform. >> The other thing I would highlight is that the system does not trust its own generated > audio. Every generated file is measured before it is allowed into the mix, and about > 30 percent of everything I generated was rejected. When a sound fails, the system > leaves that interval silent and tells the user why, rather than substituting something."

3.3 The 5-minute explanation

Use this when you are given the floor properly. It is structured in five beats of about a minute each.

Beat 1 - the problem.

> "A large fraction of recorded video has no usable audio track. Surveillance cameras > often record picture only. Archive footage may have lost its sound. Phone recordings > are routinely spoiled by wind or a muted microphone. In all of those cases the visual > record still constrains what the audio was: lips move in a way that limits what could > have been said, a mug meeting a table implies a specific contact sound at a specific > instant, and surfaces in the frame physically vibrate in response to the sound around > them. My project asks whether that information can be turned back into audio, on > ordinary hardware."

Beat 2 - what I built.

> "Three independent subsystems, all local, all on one Apple M4 laptop with 17 gigabytes > of memory and no CUDA. >> Subsystem 1 reads lips and generates timed speech. Subsystem 2 recognises actions and > generates synchronised

Foley. Subsystem 3 recovers sound from surface vibration. > > They share two contracts. Only the video stream is ever decoded, so any audio in the > uploaded file is never read - I proved that experimentally, not just by assertion. And > the picture is stream-copied to the output, never re-encoded, so the output video is > bit-identical in its picture stream."

Beat 3 - the technical core.

> "The interesting engineering is in the second subsystem, and there are two problems > there that were harder than I expected. > > The first is *where* to put a generated sound. Action recognition gives you intervals, > but Foley events are instants. I decode the frames to 320 by 180 greyscale and compute > the mean absolute inter-frame difference inside a band of the frame - the lower third > for footsteps, the upper half for drinking, the middle for the table. Then the > detection rule depends on the physics of the action: a footstep is a motion peak > resolved to the *following* minimum, because the swing is the peak and the plant is > what you hear; a sip is a sustained motion *minimum*, because a sip is the mug held > still at the lips; a contact is the last motion peak before rest. > > The second problem is that generative audio models fail in ways that ordinary > processing makes worse. Sometimes MOSS returns a near-silent, near-constant file with > no usable signal. A level-setting algorithm will faithfully try to raise that to > target, apply forty-something decibels of gain, and turn quantisation noise into > audible hiss. So every generated file is measured raw, before any gain, against six > gates."

Beat 4 - results.

> "On the reference clip, alignment error measured on the *rendered audio* - not asserted > from the plan - is at worst 20 milliseconds across seven events. One frame at 24 frames > per second is 41.7 milliseconds, so every event is inside half a frame. > > Across the 54 audio assets I generated during development, the quality gate rejects 16, > which is 29.6 percent. Eleven of those would have needed more than 25 decibels of > automatic make-up gain, with a median of 29.1 and a maximum of 42.1. > > And I have two results that argue against trusting a single quality score. Two > different generation backends had almost identical median scores, 54.5 and 53.2, but > their median harmonic ratios differ by a factor of 22 - one of them produced literal > sine waves for object contacts. And a setting change that *raised* measured dynamic > range from 43.7 to 61.1 decibels simultaneously halved the number of audible transients > in a walking sound. The scalar improved while the audio got worse."

Beat 5 - limitations and what I would do next.

> "The limiting component is action recognition. On one test video it missed a cup > placement entirely and emitted a single stirring action under three different labels. > Sound generation can only be as good as the timeline it is given, and that is where the > remaining quality is. > > The synchronisation result is seven events on one clip, on the build that was tuned > against it. The listening judgements were made by me alone. And the visual microphone > is bounded by the sampling theorem - at 60 frames per second you can only recover > frequencies below 30 hertz, which excludes speech entirely. > > If I continued, the first thing I would fix is the action timeline, because it bounds > everything downstream."

REMEMBER THIS

The single most impressive sentence in the 5-minute version is: "*alignment error measured on the rendered audio, not asserted from the plan*". It tells an examiner you know the difference between what your program intended and what it produced.

Chapter 4 — Map of the repository

4.1 Top level

```
Silent-Video-Project/
|
+-- 01-Lip-Reading/          SUBSYSTEM 1, FIRST ATTEMPT (AV-HuBERT) - superseded
+-- 02-Auto-AVSR-Test/       SUBSYSTEM 1, DELIVERED (Auto-AVSR + Kokoro TTS)
+-- 03-FoleyCrafter-Test/    MODEL EVALUATION GROUND + Module 2 source code
|   +-- action-recognition/  the validated Qwen action-recognition implementation
|       +-- qwen/            venv-qwen (Python 3.10, torch 2.13)
|       +-- stable-audio/    venv-stable-audio (alternative Foley backend)
|       +-- audioldm2/       evaluated and rejected
|       +-- mmaudio/         evaluated and rejected
|       +-- videomae/        evaluated and rejected
+-- Module3_Fresh/           SUBSYSTEM 2, DELIVERED (the main web application)
+-- Acoustic_ey/             SUBSYSTEM 3, DELIVERED (visual microphone)
+-- paper/                   IEEE-format research paper + experiment scripts
+-- handbook/                this handbook
+-- README.md                repository README for Subsystem 2
```

Figure 4.1 - Top-level layout. Sizes on disk: *Module3_Fresh* 16 GB, *03-FoleyCrafter-Test* 7.4 GB, *02-Auto-AVSR-Test* 3.2 GB, *01-Lip-Reading* 2.6 GB, *Acoustic_ey* 553 MB.

4.2 Subsystem 2 in detail: *Module3_Fresh/*

```
Module3_Fresh/
+-- backend/
|   +-- main.py              FastAPI application object
|   +-- api/routes.py        every HTTP endpoint
|   +-- core/
|       | +-- config.py      paths, interpreters, defaults, limits
|       | +-- jobs.py        job store, 9-stage state machine, worker thread
|       +-- services/
|           | +-- video_service.py  ffprobe validation
|           | +-- action_recognition.py  subprocess -> venv-qwen
|           | +-- prompt_map.py      17 curated Foley classes
|           | +-- prompt_synthesis.py open-vocabulary prompt writer
|           | +-- sound_generation.py subprocess -> MOSS; cache; candidate selection
|           | +-- foley_validation.py the six quality gates and the 0-100 score
|           | +-- synchronization.py frame motion -> visual events -> alignment plan
|           | +-- audio_processing.py per-clip polish, bus mix, limiter
|           | +-- video_render.py    ffmpeg mux
|           | +-- pipeline.py        stage orchestration
|       +-- runners/
|           | +-- run_module2.py     executed by venv-qwen
|           | +-- run_stable_audio.py executed by venv-stable-audio
|       +-- tests/                  test_suite.py (42), test_foley_validation.py (22),
|                                   e2e_gate.py, e2e_demo.py, e2e_upload.py
+-- frontend/src/                  React 18 + TypeScript + Tailwind
+-- moss/
|   +-- checkpoints/MOSS-SoundEffect-v2.0/ 10 GB of weights
|   +-- MOSS-TTS/                        upstream repo, pinned, never modified
|   +-- scripts/moss_generate.py          the phased generation driver
|   +-- scripts/moss_phased.py            component loaders
|   +-- scripts/mps_compat.py             the MPS float64 shim
|   +-- venv-moss/                        Python 3.12, torch 2.9.1
+-- scripts/                            the validated stand-alone Module 3 implementation
+-- data/{uploads,jobs,generated,outputs} runtime data and the Foley cache
+-- results/                             the engineering record: evaluations, QA, analyses
+-- docs/{architecture,api,pipeline,deployment}.md
+-- audio/, input/, output/              the validated demo assets and rendered results
+-- HANDOFF.md, README.md
```

Figure 4.2 - Subsystem 2. This is the folder to open if an examiner asks to see code.

4.3 Subsystem 1 in detail: 02-Auto-AVSR-Test/

```

02-Auto-AVSR-Test/
+-- app/
|   +-- api.py           Flask routes: /, /health, /predict, /generate
|   +-- inference.py     AutoAVSRLipReader - loads the checkpoint once
|   +-- video_processing.py validation + FFmpeg standardisation to 25 fps SDR
|   +-- timing.py        word onsets by CTC forced alignment
|   +-- tts.py           voice selection; Kokoro primary, Piper fallback
|   +-- sync.py          phrase placement, pacing, mux
|   +-- gender.py        speaker gender from the face (overridable)
|   +-- templates/index.html the whole UI
|   +-- static/{css,js}  335 lines of JavaScript, 459 lines of CSS
+-- auto_avsr/          the official mpc001/auto_avsr checkout, pinned at 182b628
+-- checkpoints/        vsr_trlrs2lrs3vox2avsp_base.pth, 955 MB
+-- preprocess_video.py the official Auto-AVSR preprocessing chain
+-- tts_worker.py       Kokoro worker, runs in venv-tts, driven over stdin
+-- run_server.py       entry point
+-- outputs/            7 recorded inference transcripts
+-- requirements.txt     the pinned stack, with the reason for every pin

```

Figure 4.3 - Subsystem 1.

4.4 Subsystem 3 in detail: Acoustic eye/acoustic-eye/

```

Acoustic eye/
+-- acoustic-eye/
|   +-- backend/
|   |   +-- main.py       FastAPI app; also serves the frontend
|   |   +-- config.py     every tunable parameter, with a rationale comment
|   |   +-- api/routes.py /health /upload /process /process-local /status /result
|   |   +-- processing/
|   |   |   +-- video_reader.py robust decode and real frame counting
|   |   |   +-- visual_microphone.py the phase-based core (steerable pyramid)
|   |   |   +-- signal_processing.py high-pass, mains notch, low-pass, spectral subtraction
|   |   |   +-- audio_writer.py  WAV writing + waveform and spectrogram PNGs
|   |   |   +-- text_report.py   plain-English description of the recovered signal
|   |   |   +-- pipeline.py     8-stage orchestration
|   +-- frontend/        index.html + css + js
|   +-- tests/           37 pytest tests across 4 files
+-- visual-mic-master/   the MIT-licensed reference implementation adapted from
+-- recovered/           the recovered audio from a high-speed clip

```

Figure 4.4 - Subsystem 3.

4.5 What is deliberately not in version control

The working tree is roughly 28 GB. Almost all of that is virtual environments and model weights, which are reproducible from the setup documentation and in some cases cannot be redistributed.

Excluded	Size	Why
Virtual environments	about 3 GB	Reproducible from <code>pip install</code>
MOSS-SoundEffect v2.0 weights	10 GB	Too large; Apache-2.0 so redistribution would be legal, but pointless
Stable Audio Open 1.0 weights	5 GB	Stability AI Community licence, non-commercial , redistribution not permitted
Auto-AVSR checkpoint	955 MB	Subject to the licence terms of LRS2, LRS3, VoxCeleb2 and AVSpeech
moos/MOSS-TTS/	26 MB	Must stay pristine; cloned from upstream at commit 58b20a0
node_modules/	80 MB	<code>npm install</code>

PROFESSOR MAY ASK: Why is the model repository not vendored into your project?

Because every compatibility fix lives in a wrapper *outside* it, and the build asserts that `git status --porcelain` inside `mooss/MOSS-TTS` returns empty. If I had edited the upstream source, I could not prove that my results come from the published model rather than from something I changed.

Chapter 5 — The technology stack

5.1 Complete stack table

Technology	Version	Purpose	Where used	Why selected	Alternative considered
Python	3.12	Subsystem 2 backend and MOSS	Module3_Fresh	Required by MOSS-SoundEffect	none - forced
Python	3.10	Action recognition environment	venv-qwen	The validated Qwen environment	none - forced
Python	3.11	Subsystem 1 and its TTS	02-Auto-AVSR-T est	torchvision 0.20.1 wheels	none - forced
PyTorch	2.9.1 (S2), 2.5.1 (S1), 2.13 (Qwen)	Deep-learning runtime	all model code	MPS support on Apple Silicon	TensorFlow - no MPS parity for these checkpoints
FastAPI	current	REST API, Subsystems 2 and 3	backend/main.py	Typed, async, generates interactive docs at /docs for free	Flask - used for Subsystem 1
Uvicorn	current	ASGI server	both FastAPI apps	The standard FastAPI server	Gunicorn
Flask	3.0.3	REST API, Subsystem 1	app/api.py	Simple, synchronous, matches a single-decode-at-a-time model	FastAPI
React	18.3.1	Subsystem 2 frontend	frontend/src	Component state maps cleanly onto a polled job	plain JS - used for the other two
Vite	6	Frontend build tool and dev server	vite.config.ts	Fast, and its dev proxy forwards /api to the backend	webpack
TypeScript	5.6.3	Type safety in the frontend	frontend/src	Strict mode catches shape mismatches with the API at compile time	plain JavaScript
Tailwind CSS	3.4.17	Styling	frontend	Utility classes, no separate stylesheet to drift	hand-written CSS
Qwen2.5-VL-3B-Instruct	3 B params	Action recognition	run_module2.py	Free-text output, so the vocabulary is open; fits the memory budget in bfloat16	VideoMAE, X-CLIP - both rejected on measurement
MOSS-SoundEffect v2.0	1.3 B DiT	Foley generation	moss_generate.py	48 kHz, Apache-2.0, explicit human-action Foley category, duration control to 30 s	Stable Audio Open, AudioLDM 2, FoleyCrafter, MMAudio, AudioGen
Stable Audio Open 1.0	-	Alternative Foley backend	run_stable_audio.py	Retained as a switchable backend so the two can be compared on one pipeline	-
Auto-AVSR	250.4 M params	Visual speech recognition	app/inference.py	Visual-only checkpoint, published 20.3 percent WER on LRS3	AV-HuBERT - tried first, see Chapter 21

Technology	Version	Purpose	Where used	Why selected	Alternative considered
Kokoro (ONNX)	v1.0	Text to speech	<code>tts_worker.py</code>	Markedly more natural than Piper on this material; 24 kHz	Piper - kept as automatic fallback
Piper	-	Fallback TTS	<code>app/tts.py</code>	Offline, native arm64, in-process	macOS <code>say</code> - second fallback
MediaPipe	0.10.21	Face landmark detection	Auto-AVSR preprocessing	The detector the official repository calls	RetinaFace - also supported upstream
OpenCV	via mediapipe	Gender classifier, video reading	<code>gender.py</code> , <code>video_reader.py</code>	Already present; <code>cv2.dnn</code> needs no extra dependency	-
pyrtools	current	Complex steerable pyramid	<code>visual_microphone.py</code>	The pyramid implementation the reference uses; pure Python	writing one by hand
NumPy	1.26.4 (S1) / 2.x (S2)	Array maths	everywhere	-	-
SciPy	current	Filters, peak finding, STFT	sync, signal processing	<code>find_peaks</code> , <code>butter</code> , <code>iirnotch</code> , <code>stft</code>	-
librosa	current	Audio feature analysis	<code>foley_validation.py</code>	RMS frames, spectral flatness, harmonic separation	hand-written DSP
soundfile	current	WAV read and write	mixing, validation	Reliable PCM_16 output	<code>scipy.io.wavfile</code> - used by the reference, replaced
FFmpeg / FFprobe	system	Decode, standardise, mux	all three subsystems	The only tool that does all of it correctly	-
psutil	current	Memory guards	both model runners	Aborts cleanly below 1.5 GB available	-
pytest	current	Subsystem 3 tests	<code>acoustic-eye/tests</code>	-	-

5.2 Why FastAPI for two subsystems and Flask for the other

This is a genuinely good viva question and the answer is not "I felt like it".

Subsystem 2 needs a job model. Foley generation takes about 4.7 minutes per asset. An HTTP request cannot stay open that long, so the work has to run in the background and the client has to poll. FastAPI gives typed request and response models, automatic interactive documentation, and clean background handling.

Subsystem 1 does not. A lip-reading inference takes about 1.6 seconds and the whole request completes in under 6 seconds. There is no job, no polling and no progress. A synchronous Flask endpoint is the correct shape, and `threaded=False` deliberately serialises decoding so CPU inference stays predictable.

KEY POINT: The general principle

Choose the framework that matches the *shape of the work*, not the framework that is fashionable. Long work needs jobs and polling. Short work needs a synchronous request.

5.3 Why React for one frontend and plain JavaScript for the others

Subsystem 2's interface has real client state: a phase machine (`idle`, `ready`, `processing`, `done`, `error`), a polled job status, an action timeline that appears part-way through processing, and a results view. React's `useState` and `useEffect` model that directly, and TypeScript's strict mode catches any mismatch between the shapes the API returns and the shapes the components consume.

Subsystems 1 and 3 have a single form, a progress list and a result. That is roughly 335 lines of JavaScript for Subsystem 1, and adding a framework and a build step for that would be worse, not better.

PART 2

FOUNDATIONS

Every machine-learning, deep-learning, audio and video concept that this project actually uses, explained from zero. Nothing here is included unless it appears in the code.

Chapter 6 — Artificial intelligence, machine learning, deep learning

6.1 The three words, in order of size

SIMPLE EXPLANATION: The nesting-doll version

Artificial intelligence is the big box: any program that does something we would call intelligent. Machine learning is a smaller box inside it: programs that get better by looking at examples instead of being told the rules. Deep learning is a smaller box inside that: machine learning using neural networks with many layers.

Technically.

- **Artificial intelligence (AI)** is the field concerned with building systems that

perform tasks associated with human intelligence. It includes approaches with no learning at all, such as search and rule-based expert systems.

- **Machine learning (ML)** is the subset in which the system's behaviour is determined

by *parameters fitted to data* rather than by rules written by a programmer.

- **Deep learning (DL)** is the subset of ML in which the model is a neural network with

many stacked layers, so that early layers learn simple features and later layers learn compositions of them.

EVIDENCE IN THE PROJECT: Where each appears in this project

Every model in this project is deep learning: Auto-AVSR, Qwen2.5-VL and MOSS-SoundEffect are all deep neural networks.

Subsystem 3, the Acoustic Eye, contains **no machine learning at all**. It is classical signal processing: a complex steerable pyramid, phase differencing, and Butterworth filters. Saying this clearly is a strength - it shows you know the difference between "AI" and "an algorithm".

6.2 Parameters, and what "3 billion parameters" means

A neural network is a very large function with adjustable numbers in it. Those numbers are the **parameters** (also called *weights*). Training is the process of choosing values for them; inference is the process of using them.

Model in this project	Parameters	What that costs in memory
Auto-AVSR VSR base	250,383,410	About 1 GB at float32, and this project runs it on CPU
Qwen2.5-VL-3B-Instruct	about 3 billion	Peaks near 12 GB in bfloat16 with activations
MOSS-SoundEffect v2.0 total	3,508.21 M	10.59 GB if all three components load in their as-shipped precision
- its Diffusion Transformer	1,416.05 M	2.85 GB resident when cast to bfloat16
- its Qwen3 text encoder	1,720.57 M	3.44 GB resident
- its DAC variational autoencoder	371.59 M	0.74 GB resident
VideoMAE (evaluated, rejected)	86,534,800	-

Table 6.1 - Parameter counts. The MOSS figures come from `results/documentation/01_MODULE3_TECHNICAL_DOCUMENTATION.md`; the Auto-AVSR figure is printed by the model loader itself and appears in all seven transcripts in `02-Auto-AVSR-Test/outputs/`.

PROFESSOR MAY ASK: How many parameters does your system have in total?

Do not answer with one number, because the models never run at the same time. Say: "Auto-AVSR is 250 million, Qwen2.5-VL is about 3 billion, and MOSS is 3.5 billion across three components which are loaded one at a time. They are never co-resident - that is the whole point of the phased design."

6.3 Precision: float32, bfloat16, and why it matters here

A parameter is a number, and a number takes space. `float32` uses 4 bytes; `bfloat16` uses 2. Halving the storage halves the memory.

bfloat16 ("brain float 16") keeps the same *exponent range* as `float32` but throws away precision in the mantissa. For neural network inference that trade is usually free, because networks care about dynamic range far more than about the last few decimal places.

EVIDENCE IN THE PROJECT: How this project uses bfloat16

Qwen2.5-VL is loaded with `torch_dtype=torch.bfloat16` in `backend/runners/run_module2.py`.

For MOSS the situation is more interesting. The upstream pipeline only forwards `torch_dtype` to the text encoder, so the Diffusion Transformer and the decoder load in `float32` - 10.59 GB of resident weights on a 17.18 GB machine, which produced about +9.8 GB of swap growth during loading alone. The wrapper in `moss/scripts/moss_phased.py` casts parameters to `bfloat16` itself, and this costs no inference precision because `MossSoundEffectPipeline.__call__` already wraps the whole engine call in `torch.autocast(device_type, dtype=torch.bfloat16)`. The forward pass computes in `bfloat16` either way; `float32` storage merely doubled the memory.

Measured effect on the walking generation: swap growth fell from +9.80 GB to +0.01 GB.

CAUTION: The one thing you must not cast

The wrapper casts **parameters only**, never buffers. The Diffusion Transformer carries three `complex128` rotary position tables. A blanket `.to(dtype=bfloat16)` silently discards their imaginary part - PyTorch even warns "Casting complex values to real discards the imaginary part" - which destroys rotary position encoding entirely. See `cast_params_only()` in `moss_phased.py`.

6.4 Training, inference, and which one this project does

Training adjusts parameters using labelled data and an optimisation algorithm. It is expensive, needs a dataset, and needs gradients.

Inference runs a trained model forward to get an output. It is comparatively cheap and needs no labels.

REMEMBER THIS

This project performs no training and no fine-tuning of any kind. Every model is used with published pre-trained weights, unmodified. If an examiner asks "what did you train?", the correct and honest answer is "nothing - I used pre-trained checkpoints and built the system around them, and the engineering is in the pipeline, the placement and the validation."

That is not a weakness, and you should not present it as one. Training a 3-billion- parameter vision-language model requires hardware this project does not have and a dataset it does not have. Every serious deployed system in industry uses pre-trained weights.

EVIDENCE IN THE PROJECT: Evidence that no training happens

There is no optimiser, no loss function, no backward pass and no dataset loader anywhere in the delivered code. In `app/inference.py` the very first thing the loader does is `torch.set_grad_enabled(False)`. In `run_module2.py` inference runs inside with `torch.inference_mode()::` In `moss_generate.py` every forward pass is inside with `torch.no_grad()::`

6.5 Supervised learning, and the vocabulary around datasets

Term	Simple meaning	Where it matters here
Training set	The examples the model learns from	LRS2, LRS3, VoxCeleb2 and AVSpeech for Auto-AVSR
Validation set	Held-out examples used to tune choices during training	Used by the model authors, not by this project
Test set	Held-out examples used once, to report a final number	The 20.3 percent word error rate on LRS3 is a test-set number reported by the Auto-AVSR authors
Ground truth	The correct answer for an example	This project does not have ground-truth transcripts for its own recordings. See Section 19.3.
Label	The category or text attached to an example	"walking", or the sentence a person said
Annotation	The human act of producing labels	none done in this project
Augmentation	Making training data artificially more varied (crops, noise, speed changes)	used by the model authors; not by this project
Overfitting	The model memorises the training data and fails on new data	a training-time concern; not applicable here
Underfitting	The model is too simple to capture the pattern	same
Generalisation	Performing well on data never seen in training	highly relevant: everything in this project is a generalisation test, because none of the models ever saw this footage

PROFESSOR MAY ASK: Is your system overfitted?

"Overfitting is a property of a training run, and I did not train anything, so the term does not apply to my models. What *is* relevant is generalisation, and there my honest position is that the synchronisation result is measured on one clip using the build that was tuned against it, so I quote it as a demonstration that sub-frame placement is achievable, not as a general accuracy figure."

Chapter 7 — Neural networks and the layer types this project uses

7.1 A neuron, a layer, a network

SIMPLE EXPLANATION: What a neural network really is

Imagine a very long chain of simple steps. Each step takes some numbers in, multiplies them by its own set of dials, adds them up, applies a simple "bend" so the result is not just a straight line, and passes the answer to the next step. Training is turning all the dials until the last step gives the right answers. There are hundreds of millions of dials, which is why it needs a lot of examples.

Technically. A single artificial neuron computes a weighted sum of its inputs plus a bias, then applies a non-linear **activation function**:

```
x1 --w1--\
x2 --w2---> sum(wi*xi) + b --> f(.) --> y
x3 --w3--/
                        ^
                        |
                    activation function
                    (ReLU, GELU, SiLU, sigmoid, ...)
```

Figure 7.1 - One neuron. A layer is many neurons in parallel; a network is many layers in series.

Without the activation function, stacking layers would be pointless: a composition of linear maps is still a linear map. The non-linearity is what lets depth buy expressive power.

7.2 Convolutional neural networks (CNNs)

SIMPLE EXPLANATION: Why convolution exists

If you want to find an edge in a photograph, the same little edge-detecting pattern works everywhere in the image. A fully connected layer would have to learn that pattern separately for every pixel position, which is wasteful. A convolution learns it once and slides it over the whole image.

The pieces.

- A **filter** (or kernel) is a small grid of weights, typically 3x3 or 5x5.
- **Convolution** slides that filter over the input, computing a weighted sum at every position.
- The result is a **feature map**: one image-sized array per filter, showing where that filter's pattern is present.
- **Stride** is how far the filter jumps between positions. A stride of 2 halves the output size.
- **Padding** adds a border so the output does not shrink.
- **Pooling** downsamples a feature map, usually by taking the maximum in each small window. It buys translation tolerance and reduces size.
- **Channels**: a colour image has 3 input channels; a layer with 64 filters has 64

output channels.

Stacking convolutions builds a hierarchy: early layers respond to edges and blobs, middle layers to parts, late layers to whole objects.

3-D convolution. A `Conv3d` slides a filter over *space and time together*, so it can respond to motion rather than to a still pattern. This is exactly what lip reading needs: a still image of a mouth is far less informative than the way the mouth moves.

EVIDENCE IN THE PROJECT: Where CNNs appear in this project

1. **Auto-AVSR's frontend is a `Conv3dResNet`.** It accepts a tensor of shape

`(B, T, 1, 88, 88)`: batch, time, one greyscale channel, 88 by 88 pixels. The 3-D convolution stem sees the mouth move over time. The transcripts in `02-Auto-AVSR-Test/outputs/` print exactly this: `Frontend : Conv3dResNet (Conv3d stem -> cannot accept audio).`

1. **The gender classifier in `app/gender.py`** is the Levi and Hassner (2015)

convolutional network, loaded through `cv2.dnn.readNet` with a 227 by 227 input.

1. **Qwen2.5-VL's vision encoder** is a Vision Transformer, which uses convolution only

for the initial patch embedding.

1. **The DAC decoder inside MOSS** is a convolutional decoder that turns a latent

sequence back into a waveform.

PROFESSOR MAY ASK: Why does a `Conv3d` frontend prove your lip reading is not cheating?

Because a `Conv3d` layer accepts a five-dimensional pixel tensor and nothing else. There is no code path by which a one-dimensional audio waveform could enter it. The claim is therefore structural, not merely empirical - though it was also verified empirically by stripping the audio track and confirming a bit-identical output. See Section 12.7.

7.3 Recurrent networks: RNN, LSTM, GRU

SIMPLE EXPLANATION: What "recurrent" means

A recurrent network reads a sequence one step at a time and keeps a running memory. It is the difference between reading a sentence word by word while remembering what came before, and looking at the whole sentence as an unordered bag of words.

RNN. At each time step, the network combines the current input with its own previous hidden state to produce a new hidden state. In principle this can remember arbitrarily far back.

The vanishing gradient problem. In practice a plain RNN cannot. During training, the gradient signal is multiplied by a similar factor at every step going backwards through time; if that factor is below one the signal shrinks exponentially and the network never learns long-range dependencies.

LSTM (Long Short-Term Memory). Adds an explicit *cell state* that flows along the sequence with only additive updates, plus three learned *gates* that control it:

Gate	What it decides
Forget gate	how much of the existing cell state to keep
Input gate	how much of the new candidate value to write

Gate	What it decides
Output gate	how much of the cell state to expose as the hidden state

Because the cell state is updated additively rather than multiplicatively, gradients survive far longer.

GRU (Gated Recurrent Unit). A simplification with two gates (reset and update) and no separate cell state. Fewer parameters, often similar quality.

CAUTION: An honest note about RNNs in this project

No LSTM or GRU appears anywhere in the delivered code. Every sequence model here is a Transformer or a Conformer. You should still understand LSTMs, because examiners ask about them and because they are the historical route to the Transformer - but do not claim your project uses one.

7.4 The Transformer and attention

SIMPLE EXPLANATION: Attention in one paragraph

When you read the sentence "the mug that Sarah put on the table was hot", to understand "was hot" you need to look back at "the mug", not at "the table". Attention is a mechanism that lets every position in a sequence look at every other position and decide, for itself, which ones matter. It replaces "remember as you go" with "look at everything and weigh it".

Technically. Self-attention transforms each position of a sequence into three vectors:

- a **query** (what am I looking for?),
- a **key** (what do I offer?),
- a **value** (what do I actually contribute?).

For every position, the model computes the dot product of its query with every key, scales the result, applies a **softmax** to turn the scores into weights that sum to one, and returns the weighted sum of the values.

```

position i                all positions j
  |                        |
  query_i . key_j --> score_ij --> softmax over j --> weight_ij
                                     |
                                     output_i = sum_j ( weight_ij * value_j ) <+

```

Figure 7.2 - Scaled dot-product attention, for one query position.

Multi-head attention runs several attention operations in parallel with different learned projections, so different heads can specialise - one on nearby positions, one on long-range structure - and their outputs are concatenated.

Why Transformers replaced RNNs. Every position is computed independently of every other, so the whole sequence can be processed in parallel on a GPU. An RNN is inherently sequential.

Positional encoding. Because attention has no notion of order on its own, position information must be added. The two schemes that matter here:

- **Sinusoidal / absolute** encodings add a fixed pattern of sines and cosines of

different frequencies to each position.

- **RoPE (rotary position embedding)** *rotates* the query and key vectors by an angle

proportional to position, so that the dot product between two positions depends on their *relative* distance. This is what MOSS's Diffusion Transformer uses, and it is why those `complex128` buffers exist.

EVIDENCE IN THE PROJECT: Transformers and attention in this project

- **Auto-AVSR's encoder is a Conformer** (convolution-augmented Transformer) and its

decoder is a **Transformer decoder**. The source is at `02-Auto-AVSR-Test/auto_avsr/espnet/nets/pytorch_backend/encoder/conformer_encoder.py` and `.../decoder/transformer_decoder.py`.

- **Qwen2.5-VL** is a Transformer-based vision-language model.

- **MOSS-SoundEffect's denoiser is a Diffusion Transformer** with 30 layers, model

dimension 1536, 12 attention heads and a feed-forward dimension of 8960 - those exact numbers are in `moss/checkpoints/MOSS-SoundEffect-v2.0/transformer/config.json`.

- **MOSS's text encoder is Qwen3**, itself a Transformer, producing a context tensor of

shape `(1, 512, 2048)` - visible in every generation record under `phase1`.

- **The RoPE buffers** `freqs_cis_0`, `freqs_cis_1` and `freqs_cis_2` are the rotary

tables handled specially in `cast_params_only()`.

7.5 The Conformer

A **Conformer** is a Transformer encoder block with a convolution module inserted inside it. The intuition is that attention is excellent at global, long-range relationships and convolution is excellent at local, fine-grained patterns, and speech needs both: the local shape of a phoneme and the global structure of a sentence.

A Conformer block is, in order: a half-step feed-forward module, a multi-head self-attention module, a convolution module, a second half-step feed-forward module, and a layer normalisation.

EVIDENCE IN THE PROJECT: Why the Conformer matters to you

Auto-AVSR's encoder is a Conformer. In viva you can say: "the encoder is a Conformer, so it combines self-attention for long-range context with convolution for local articulator detail, which is exactly the right inductive bias for reading lips."

Chapter 8 — Sequence modelling, CTC and decoding

8.1 The alignment problem

A lip-reading model sees, say, 89 video frames and must output a sentence of 5 words. It is not told which frames correspond to which word. That is the **alignment problem**, and it is the reason CTC exists.

8.2 CTC: Connectionist Temporal Classification

SIMPLE EXPLANATION: CTC without the maths

The model produces one guess per frame - possibly "nothing here", possibly a letter or word piece. Because a sound lasts several frames, the same symbol repeats. CTC's rule is: collapse repeats, then delete the special "nothing here" symbol, and what is left is your answer. Because many different frame-by-frame paths collapse to the same answer, CTC trains by adding up the probability of *all* of them.

Technically. CTC (Graves et al., ICML 2006) introduces an extra **blank** symbol. The network emits a probability distribution over the vocabulary plus blank at every time step. A frame-level path is mapped to an output string by:

1. collapsing consecutive repeated symbols, then
2. removing all blanks.

The probability of an output string is the sum of the probabilities of every path that maps to it, computed efficiently by dynamic programming (the forward-backward algorithm). The CTC loss is the negative log of that sum.

Peakiness. A trained CTC model is characteristically *peaky*: it emits blank for most frames and spikes on one frame per symbol. That has a practical consequence used in this project: CTC onsets are reliable, but CTC does not tell you how long a word lasted.

Forced alignment. If you already know the correct output string, you can ask which frame-level path is the most probable one that produces it. That is **forced alignment**, and it gives every symbol a frame index.

frames:	1	2	3	4	5	6	7	8	9	10	11	12
CTC out:	_	_	HELLO	_	_	_	EVERYONE	_	_	HOW	_	_
			^				^			^		
			onset of word 1				onset of word 2			onset of word 3		
			(_ = blank)									

Figure 8.1 - CTC peakiness and what forced alignment extracts from it.

EVIDENCE IN THE PROJECT: CTC in this project

Auto-AVSR is a **hybrid CTC-attention** model. It has both an attention decoder and a CTC head, and the beam search combines them with weights `decoder 0.9, ctc 0.1` - printed in every transcript in `02-Auto-AVSR-Test/outputs/`.

The CTC head is then reused for something the model was not built for: **timing**. `app/timing.py` calls `torchaudio.functional.forced_align(logp, targets, blank=model.blank)` on the per-frame CTC log-posteriors, using the already-decoded token sequence as the target. Collapsing the resulting frame path gives one onset per token, and tokens beginning with the SentencePiece word-start marker start a new word.

The result is a word onset every 40 ms - one video frame at 25 fps - **derived from the video itself**, not estimated.

PROFESSOR MAY ASK: Why is 40 milliseconds the resolution of your word timings?

Because the model runs at 25 frames per second, so the CTC head emits one distribution per frame, and one frame is 1/25 of a second, which is 40 ms. The timing resolution is the frame rate; it cannot be finer without a faster camera or a different model.

8.3 The CTC lag correction - a real bug, found by measurement

A CTC peak marks the frame at which the model became *confident* about a symbol. That is later than the frame at which the mouth *began forming* the sound. If you place synthesised speech on the raw CTC onsets, the speech lags the lips.

`app/timing.py` measures the lag directly rather than guessing it:

1. `motion_onset(sample)` takes the model input tensor itself, crops the inner-mouth

region `[:, 28:68, 20:76]`, computes the mean absolute inter-frame difference, and finds the first frame from which movement stays above 35 percent of its range for at least three consecutive frames.

1. The lag is `first_CTC_onset - motion_onset`, clipped to the range 0 to 0.40 s.
2. Every anchor is shifted back by that lag.

EVIDENCE IN THE PROJECT: The measured lag

About **0.24 s** on the project's test clip. `MAX_LAG_CORRECTION = 0.40` seconds is the clip, chosen because "CTC never lags more than this in practice" - the comment is in the source.

8.4 Logits, softmax and log-probabilities

- **Logits** are the raw, unnormalised scores a network's final layer produces. They can be any real number.
- **Softmax** turns a vector of logits into a probability distribution: exponentiate each one, then divide by the sum. Larger logits get exponentially more probability.
- **Log-softmax** does the same but returns the logarithm, which is numerically safer and is what beam searches accumulate, because adding logs is multiplying probabilities.

EVIDENCE IN THE PROJECT: Where you see this

`app/timing.py` line 109: `logp = model.ctc.log_softmax(enc_feat.unsqueeze(0)).float()`.

And the decoder scores in the transcripts are log-probabilities, which is why they are negative and why less negative means more confident: `-0.36` is a confident decode and `-5.11` is not.

8.5 Beam search decoding

SIMPLE EXPLANATION: Greedy versus beam

Greedy decoding takes the single best next word at every step, and can paint itself into a corner - a good first word may have no good continuation. Beam search keeps the best *N* partial sentences alive at each step and only commits at the end. *N* is the beam width.

Technically, at each step the decoder expands every surviving hypothesis by every possible next token, scores all the results, and keeps the top *N* by accumulated log-probability. The final answer is the highest-scoring complete

hypothesis.

EVIDENCE IN THE PROJECT: The decoder configuration in this project

Frozen and taken unmodified from the official repository:

- beam size 40
- scorer weights: decoder 0.9, ctc 0.1, lm 0.0, length_bonus 0
- no external language model (lm_weight is 0.0)
- vocabulary 5049 tokens, SentencePiece unigram model

auto_avsr/spm/unigram/unigram5000.model

- start and end of sentence token: 5048

Number of hypotheses actually returned, across the seven recorded runs: 144, 144, 149, 152, 167, 172 and 215.

8.6 Tokens and SentencePiece

Language models do not work in characters or in whole words. They work in **tokens**: sub-word units learned from a corpus, so that common words are one token and rare words are assembled from pieces.

Auto-AVSR uses a **SentencePiece unigram** model with a 5000-token vocabulary. Word starts are marked with a special character.

EVIDENCE IN THE PROJECT: Read this straight out of your own output

For the clip that produced "HELLO EVERYONE HOW ARE YOU", the token identifiers are [5048, 2455, 1949, 2525, 768, 5034, 5048] and the token units are ['<eos>', 'HELLO', 'EVERYONE', 'HOW', 'ARE', 'YOU', '<eos>'] - five word-start tokens between two end-of-sentence markers.

For "KONA IS KONA" the tokens are ['<eos>', 'KO', 'NA', 'IS', 'KO', 'NA', '<eos>'] - here you can literally see sub-word splitting, because "KONA" is not a common enough word to have its own token. That transcript, incidentally, has a decoder score of -4.31 and is almost certainly wrong.

Chapter 9 — Generative models: VAE, diffusion, flow matching

This chapter explains how MOSS-SoundEffect actually generates audio. It is the most technically demanding chapter in the handbook and it is worth the effort, because it is where an examiner can most easily catch a student who has only used a model rather than understood it.

9.1 Latent space and the autoencoder

SIMPLE EXPLANATION: Why generate in a "latent space"

A second of 48 kHz audio is 48,000 numbers. Generating those directly, one at a time, is absurdly hard. So you first train a compressor that can squash audio into a much shorter sequence of numbers and expand it back with little loss. Then you generate the short sequence instead. It is the difference between painting every brick of a house and drawing a floor plan.

An **autoencoder** is two networks trained together: an **encoder** that maps input to a compact **latent** representation, and a **decoder** that maps the latent back to the input. Training minimises reconstruction error.

A **variational autoencoder (VAE)** additionally forces the latent space to be smoothly distributed, so that nearby latents decode to similar outputs and the space can be sampled meaningfully.

EVIDENCE IN THE PROJECT: The numbers in your own generation logs

MOSS uses a **DAC VAE** (Descript Audio Codec, Kumar et al., NeurIPS 2023). Read any file matching `Module3_Fresh/results/web_*.generation.json`:

- `"latent_shape": [1, 128, 1500]` - one item, 128 latent channels, 1500 time steps.
- `"num_samples": 1440000` - the waveform those 1500 steps decode to, which is 30 seconds

at 48 kHz.

- The compression factor is the VAE's `hop_length`, which is **960**: 1,440,000 samples

divided by 1500 latent steps equals 960 samples per step. The wrapper asserts this exact number (`assert int(vae.hop_length) == 960`).

- `"decoded_shape": [1, 1, 1440000]` then `"cropped_samples": 480000`, which is the

10 seconds actually kept.

So the model denoises 1500 latent steps and the decoder turns each one into 960 audio samples. That is a 960-fold reduction in what the generator has to produce.

9.2 Diffusion, in the way that actually makes sense

SIMPLE EXPLANATION: Diffusion as sculpture

Start with a block of pure noise. Ask a network: "if this were a slightly noisy version of a real sound, what would the noise be?" Subtract a bit of it. Ask again. Repeat fifty times. What is left is a sound. The network never has to invent a sound from nothing; it only ever has to answer the much easier question "what here is noise?".

Technically. A diffusion model is defined by two processes.

- The **forward process** takes real data and progressively adds Gaussian noise over many

steps until nothing but noise remains. This process is fixed, not learned.

- The **reverse process** is learned: a neural network is trained to estimate the noise

(or equivalently the direction back toward the data) at any noise level.

Generation runs the reverse process: sample pure noise, then iteratively denoise.

Steps. The number of reverse steps is a quality-versus-time dial. More steps means a finer-grained path back to the data.

EVIDENCE IN THE PROJECT: Your settings, and the experiment behind them

The validated settings are **50 denoising steps**, and this project actually ran the ablation rather than assuming.

From `paper/experiments/exp3_ablation.json`, on an impact sound:

Steps	Generation time	Quality score	Dynamic range	Detected transients
50 (production)	265.9 s	86.9	32.6 dB	7
35	200.4 s	86.3	31.3 dB	7
25	128.6 s	86.4	29.1 dB	7

Halving the steps halves the time and moves the score by 0.6 points, which is noise. The decision to keep 50 rests entirely on listening. That is worth saying out loud: it is an example of a metric being blind to a difference a human can hear.

9.3 Flow matching

Flow matching (Lipman et al., ICLR 2023) is a training objective closely related to diffusion, and it is what MOSS uses.

SIMPLE EXPLANATION: The difference in one sentence

Instead of learning to remove noise step by step along a wandering path, flow matching learns a *velocity field* that points straight from noise to data, so the model can be integrated along a much more direct route.

Technically. Flow matching defines a probability path from a simple distribution (Gaussian noise) to the data distribution and trains a network to regress the *vector field* that transports samples along that path. Generation is then numerically integrating an ordinary differential equation, typically with Euler steps.

Sigma shift. The path from noise to data is parameterised by a noise level, usually written sigma. The **sigma shift** parameter warps the schedule so that steps are spent where they matter most.

EVIDENCE IN THE PROJECT: In your code

`moss_generate.py` calls `pipe.scheduler.set_timesteps(A.steps, denoising_strength=1.0, shift=A.sigma_shift)` with a **FlowMatchScheduler**. The validated `sigma_shift` is 5, which is also the default read from `moss/checkpoints/MOSS-SoundEffect-v2.0/scheduler/scheduler_config.json`. `denoising_strength=1.0` means the process starts from pure noise, not from a partially noised real sample.

9.4 The Diffusion Transformer (DiT)

A **Diffusion Transformer** (Peebles and Xie, ICCV 2023) uses a Transformer, rather than the older U-Net, as the denoising network. The latent sequence is treated as a sequence of tokens, and the noise level and the text conditioning are injected into every block.

EVIDENCE IN THE PROJECT: The exact architecture of your denoiser

From `moss/checkpoints/MOSS-SoundEffect-v2.0/transformer/config.json`:

Field	Value	Meaning
<code>num_layers</code>	30	thirty Transformer blocks
<code>dim</code>	1536	model width
<code>num_heads</code>	12	attention heads per block
<code>ffn_dim</code>	8960	feed-forward inner width
<code>in_dim/out_dim</code>	128 / 128	latent channels in and out
<code>text_dim</code>	2048	width of the text conditioning vector
<code>freq_dim</code>	256	width of the timestep (noise-level) embedding
<code>patch_size</code>	[1]	one latent step per token
<code>vae_type</code>	"dac"	which decoder it was trained against

`model_index.json` records `"dit_variant": "1.3B"` and `"max_inference_seconds": 30`.

9.5 Classifier-free guidance (CFG)

SIMPLE EXPLANATION: What the CFG number does

Run the model twice: once told what you want, once told nothing (or told what you do not want). The difference between the two answers points in the direction of "more like what was asked for". Multiply that difference and add it back, and the output obeys the prompt harder. Turn it up too far and the output becomes exaggerated and unnatural.

Technically, the guided noise prediction is:

```
noise_pred = noise_negative + cfg_scale * ( noise_positive - noise_negative )
```

with `cfg_scale = 1` meaning no guidance and larger values meaning stronger adherence to the prompt.

EVIDENCE IN THE PROJECT: Read this literally out of your own source

`moss/scripts/moss_generate.py`, inside the denoising loop:

```
npos = model_fn_wan_video(dit=dit, ..., latents=latents, timestep=timestep, context=ctx_p)
nneg = model_fn_wan_video(dit=dit, ..., latents=latents, timestep=timestep, context=ctx_n)
noise_pred = nneg.float() + A.cfg * (npos.float() - nneg.float())
```

`ctx_p` is the encoded positive prompt and `ctx_n` the encoded negative prompt. The computation is deliberately done in float32 - the comment in the file header says so - even though the surrounding block runs under bfloat16 autocast.

The validated `cfg_scale` is **4.0** for MOSS. The Stable Audio backend uses 7.0, which is that model's own default.

Note the cost: **the model is evaluated twice per denoising step**, so 50 steps means 100 forward passes through a 1.4-billion-parameter Transformer. That is why generation takes about four and a half minutes.

9.6 Negative prompts

A negative prompt is a second text conditioning that the guidance formula pushes *away* from.

EVIDENCE IN THE PROJECT: The negative prompts in your project, and one that went wrong

The shared negative prompt in `prompt_map.py` is:

```
music, speech, talking, voice, singing, background ambience, room tone,  
environmental noise, crowd, traffic, cinematic sound design, electronic sounds,  
synthetic sounds, exaggerated impacts, reverb
```

But the drinking class carries an **empty** negative prompt, with its negations written inline in the positive text instead. The code comment says `negative="", # validated configuration: negations carried inline`. That is the exact configuration that produced the sound approved by listening.

And there is a recorded warning about pushing negatives too hard. A cup-pickup attempt grew the negative prompt from 17 terms to 24, and the output collapsed: peak fell from -36.0 dBFS to -61.7 dBFS, dynamic range to 1.06 dB, harmonic content to 95 percent. The report in `results/cup_pickup_moss_v2_report.md` records this as a **two-point observation, explicitly not an established relationship**, and says it was not acted upon. Reproduce that caution in viva; it is exactly the right scientific posture.

9.7 Seeds and determinism

A generative model needs random numbers. A **seed** fixes the random number generator, so the same seed with the same settings gives byte-identical output.

EVIDENCE IN THE PROJECT: Seeds in your project

The default seed is **42**. `moss_generate.py` draws noise with `pipe.generate_noise(shape, seed=A.seed, rand_device="cpu")` - deliberately on CPU, matching upstream, so the result does not depend on GPU-specific RNG behaviour.

The seed is part of the cache key, so changing it produces a different cached file rather than silently reusing the old one.

And the seed is the mechanism behind multi-candidate generation: `generate_best()` tries seeds 42, 43, 44 in turn. The cup-pickup class failed on 42 and 43 and passed on 44 with a score of 85.8. Same prompt, same model, same settings - a sampling failure, not a capability limit.

Chapter 10 — Vision-language and multimodal models

10.1 What "multimodal" means

A **modality** is a kind of input: pixels, audio, text. A **multimodal** model takes more than one.

A **vision-language model (VLM)** takes images or video plus text and produces text. It is built by attaching a vision encoder to a language model, so that image patches become tokens the language model can attend to alongside word tokens.

```
video frames --> vision encoder --> visual tokens --\
                                                    >--> language model --> text out
text prompt --> tokenizer      --> text tokens ---/
```

Figure 10.1 - The general shape of a vision-language model such as Qwen2.5-VL.

10.2 Why a VLM instead of an action classifier

This is the single most important model-choice question in Subsystem 2, and the project has measured evidence for the answer.

Approach	Vocabulary	What it returned on the test clip
VideoMAE (Kinetics fine-tune)	closed - a fixed list of Kinetics classes	"shredding paper", confidence 0.550
X-CLIP	closed - you supply candidate labels	"pouring liquid" at 0.21 to 0.38 in every window
Qwen2.5-VL-3B-Instruct	open - it writes free text	"pick up cup", "drink from cup", "place cup on table"

The failure of the first two is structural. Kinetics-style label sets are organised around whole-clip human activities: "playing tennis", "shredding paper". Foley needs *object-contact events*: a mug meeting a table. There simply is no such label.

EVIDENCE IN THE PROJECT: The decisive property, in your own code

03-FoleyCrafter-Test/action-recognition/action_recognition.py line 11: * Open vocabulary:
no candidate labels are ever shown to the model.

The prompt asks for ACTION: <short action phrase> and EVIDENCE: <brief visual evidence> and nothing else. Whatever the model writes is the label. That is why an unexpected action like "write on notebook" or "place bread in toaster" can be handled at all - and it is also why the labels are messy, which is the subject of Chapter 33.

10.3 How Qwen sees video here

Qwen2.5-VL can accept video, but this project deliberately does not hand it the whole clip. It hands it a series of short, overlapping windows.

Parameter	Value	Reason
Window length	2.0 s	Long enough to contain an action, short enough to be one action
Stride	1.0 s	Half the window, so consecutive windows overlap by 1 s
Frames per window	8	Uniformly sampled across the window
Frame size	448 x 252	Small enough to fit the memory budget

Parameter	Value	Reason
Precision	bfloat16	Halves the weight memory
Device	MPS	Apple Silicon
max_new_tokens	96	The answer is two short lines; more is wasted compute
Sampling	do_sample=False	Greedy decoding - deterministic, so the same video gives the same labels

KEY POINT: The design decision worth quoting

From the source header of `action_recognition.py`:

"Qwen is NEVER asked for timestamps. It answers only 'what is happening here?'. Timing comes from each window's known position on the timeline. Semantics from the VLM; timing from the windowing."

This is a genuinely good piece of system design. Language models are unreliable at producing numbers; they are much better at describing what they see. So the design asks the model only for the thing it is good at, and derives the numbers arithmetically from something already known.

PROFESSOR MAY ASK: Why 2-second windows with a 1-second stride?

Two reasons. A 2-second window is long enough to contain a complete short action but short enough that it usually contains only one. And a stride of half the window means every instant is seen by two windows, so an action that straddles a boundary is still described. The cost is that raw spans overlap by one stride, which is why the timeline has to be resolved afterwards - see Chapter 24.

Chapter 11 — Audio and video fundamentals

You cannot defend this project without these. Every one of them appears in your code or your results.

11.1 Digital audio

Term	Meaning	Value in this project
Sample	One measurement of air pressure at one instant	-
Sample rate	Samples per second	48,000 Hz throughout Subsystem 2; 24,000 Hz from Kokoro TTS; equal to the frame rate in Subsystem 3
Bit depth	Bits per sample	PCM 16-bit for every WAV this project writes
Channels	Mono is 1, stereo is 2	Mono everywhere. Foley for a single subject does not need a stereo image, and mono halves the data.
PCM	Pulse Code Modulation: raw uncompressed samples	<code>subtype="PCM_16"</code> in every <code>sf.write</code> call
Nyquist frequency	Half the sample rate; the highest representable frequency	24,000 Hz at 48 kHz. This is the entire limitation of Subsystem 3.

SIMPLE EXPLANATION: Why 48 kHz

Human hearing tops out around 20 kHz. To represent a frequency you need to sample at more than twice it - that is the sampling theorem - so you need at least 40,000 samples per second. 48 kHz is the professional standard with a little headroom. It is also simply what MOSS-SoundEffect produces natively, and it is the highest of any candidate model that was evaluated.

EVIDENCE IN THE PROJECT: Why 48 kHz specifically mattered for this material

From `results/text_to_audio_model_evaluation.md`: sipping, swallowing and wet mouth transients carry substantial energy above 8 kHz. A 16 kHz model - AudioLDM 2, FoleyCrafter, AudioGen - has a Nyquist limit of 8 kHz and cuts that band off entirely. That is the difference between a sip that sounds close-miked and one that sounds muffled. It was one of the four reasons MOSS was selected.

11.2 Decibels, dBFS, RMS, peak and crest factor

SIMPLE EXPLANATION: Why decibels

Loudness spans an enormous range - the quietest audible sound to the loudest is a factor of about a million in amplitude. Writing that as ordinary numbers is unusable, so audio uses a logarithmic scale. Every 6 dB is roughly a doubling of amplitude.

Term	Definition	Note
dBFS	Decibels relative to Full Scale. 0 dBFS is the loudest a digital sample can be	Always negative in a well-behaved file
Peak	The largest absolute sample value	-6.00 dBFS in the final mixes
RMS	Root Mean Square: square every sample, average, take the square root. A measure of average energy	-36.87 dBFS in the validated mix
Crest factor	Peak divided by RMS, in dB. How "spiky" the audio is	30.87 dB in the validated mix - high, which is correct for impulsive Foley

Term	Definition	Note
Active RMS	RMS computed over only the frames that contain signal	this project's level-setting metric

EVIDENCE IN THE PROJECT: Why active RMS and not plain RMS

A Foley clip is mostly silence with a short event in it. Whole-file RMS is dominated by the silence, so two clips with identical events but different amounts of silence would be levelled differently.

`active_rms()` in `backend/services/audio_processing.py` computes RMS in 1024-sample frames with 50 percent overlap, then keeps only frames at or above a threshold. The threshold is the higher of the 60th percentile and 40 dB below the loudest frame.

That second clause is a bug fix, and the comment says why: *"The percentile alone degenerates to 0 when a clip is mostly digital silence, which would make this identical to a whole-file RMS."*

11.3 Dynamic range

The difference between the loudest and quietest parts of a signal, in dB.

EVIDENCE IN THE PROJECT: How this project measures it, and the bug that forced the change

`foley_validation.py`:

```
active = rf[rf > max(rf.max(), 1e-12) * 10 ** (-70 / 20)]
db = 20 * np.log10(np.maximum(active, 1e-12) / max(rf.max(), 1e-12))
dyn = float(min(np.percentile(db, 95) - np.percentile(db, 5), 96.0))
```

Read the comment above it. Measuring dynamic range over *all* frames, including frames of exact digital silence, sends the 5th percentile to the numeric floor and returns values of **183 to 201 dB** - physically impossible for 16-bit audio, whose theoretical maximum is about 96 dB - and scores full marks. A file that was 90 percent silence with a few clicks would have passed as excellent.

The fix: restrict the measurement to frames above -70 dB relative to the frame maximum, and cap the result at 96 dB.

11.4 Spectrum, spectrogram, STFT

- The **Fourier transform** decomposes a signal into the frequencies it contains.
- The **Short-Time Fourier Transform (STFT)** slides a window along the signal and takes

a Fourier transform of each window, so you can see how the frequency content changes over time.

- A **spectrogram** is that result drawn as an image: time across, frequency up, intensity as colour.

- `n_fft` is the window length in samples and `hop_length` is how far the window moves each time.

EVIDENCE IN THE PROJECT: Where STFT appears

`foley_validation.py` uses `librosa.stft(y, n_fft=2048, hop_length=512)` for spectral flatness; `synchronization.py` uses `n_fft=2048, hop_length=256` in `band_ratio()`; Subsystem 3 uses `scipy.signal.stft` for spectral subtraction and renders a spectrogram PNG for every job.

11.5 Spectral flatness and harmonic ratio

These two metrics are the heart of the quality gate, so understand them properly.

Spectral flatness is the ratio of the geometric mean of the spectrum to its arithmetic mean. It is near 1 for white noise, which has energy everywhere, and near 0 for a pure tone, which has all its energy in one place.

Harmonic ratio, as this project computes it, is the fraction of total energy that survives harmonic-percussive separation:

```
harm = np.mean(librosa.effects.harmonic(y) ** 2) / (np.mean(y ** 2) + 1e-20)
```

`librosa.effects.harmonic` uses median filtering of the spectrogram to separate sustained, pitched content from transient, broadband content. A ratio near 0 means the signal is almost entirely percussive; near 1 means it is almost entirely tonal.

KEY POINT: Why this is the single most important metric in the project

Foley is inharmonic. A mug meeting a table is a broadband transient. A footstep is a broadband transient. If a generated "cup placement" comes back with a harmonic ratio of 0.997, it is a musical note, and no amount of good scoring on other axes makes that acceptable.

EVIDENCE IN THE PROJECT: The measurement that settled the model choice

Both generation backends were run through the identical pipeline:

Sound class	MOSS score / harmonic	Stable Audio score / harmonic
Walking	97.1 / 0.00	92.7 / 0.03
Drinking	70.9 / 0.06	75.6 / 0.09
Cup pickup	85.8 / 0.00	53.1 / 0.88
Cup placement	49.8 / 0.02	53.4 / 0.87

Stable Audio is comparable on walking and even scores slightly *higher* on drinking and cup placement. But on object contacts it produced musical tones. One output for "cup placed on a table" was a **346 Hz sine wave** with a harmonic ratio of 0.997 and a spectral flatness of 0.00000.

Across the whole 54-asset corpus the two backends have median quality scores of 54.5 and 53.2 - almost identical - while their median harmonic ratios are 0.040 and 0.898, a factor of 22 apart. **The aggregate score does not separate them. The harmonic ratio separates them decisively.**

11.6 Effective bits

A 16-bit sample can represent 65,536 levels. If the loudest sample in a file is very quiet, most of those levels are unused and the signal is being represented by only a handful of them - so the quantisation noise floor is relatively much louder.

```
effective_bits = 16 + log2(peak)
```

A peak of 1.0 gives 16 bits. A peak of -42 dBFS gives about 9 bits.

EVIDENCE IN THE PROJECT: The gate, and the degenerate file that motivated it

The gate rejects anything below 9.0 effective bits. The cup-pickup v2 file measured **5.8 effective bits**, with a peak of -61.7 dBFS, and contained only **40 distinct sample values** in the entire ten seconds, spanning -27 to +12 out of a possible plus or minus 32,768.

11.7 Envelope, onset, attack and the distinction that mattered

- The **amplitude envelope** is the outline of the waveform: how loud it is over time, ignoring the oscillation inside.
- **Onset detection** finds where events begin. The common approach, **onset strength**, measures spectral flux - how much the spectrum changed between frames - and picks the peaks.
- The **attack** is the leading edge of a transient: where the sound actually starts rising.

CAUTION: These are not the same thing, and confusing them cost this project 96 ms

An onset-strength peak marks where the spectrum is changing *fastest*, which is somewhere in the middle of the attack, not at its start. Measured on this project's own assets, the difference is **-96 ms to +250 ms**.

One footstep in the walking asset reports an onset-strength peak at 3.760 s. Its true attack is at 3.856 s. Aligning the strength peak to the visual event therefore put the audible transient 96 ms in the wrong place - an error that was **audible in the rendered output before it was found**.

How this project computes a true attack. `attack_times()` in `backend/services/synchronization.py`:

1. Compute the envelope as the magnitude of the analytic signal, via the Hilbert transform, smoothed with a 2 ms moving average.
1. Find prominent peaks of that envelope, at least 0.20 s apart.
2. For each peak, walk *backwards* until the envelope last fell below 20 percent of that peak's height. That backtracked point is the attack.

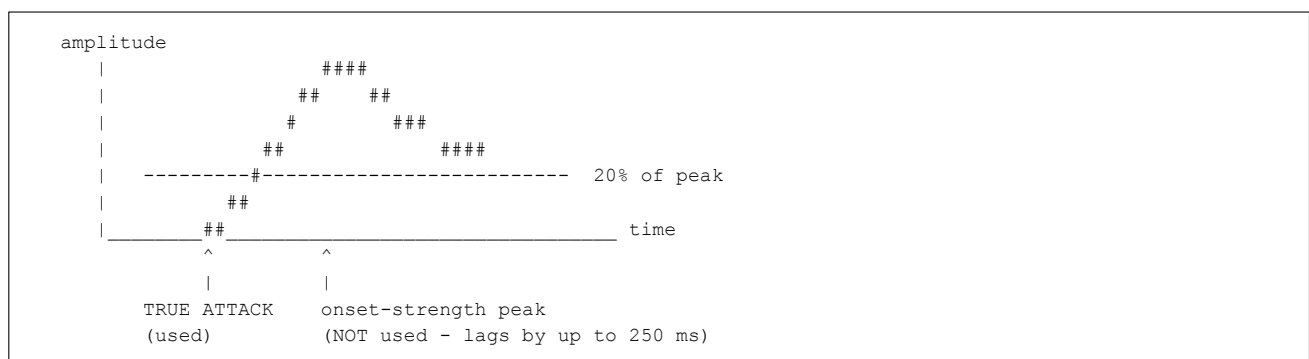


Figure 11.1 - Why the alignment anchor is the backtracked attack, not the onset-strength peak.

11.8 Digital video

Term	Meaning	Value here
Frame rate (fps)	Frames per second	24 fps for the Subsystem 2 reference clip; 25 fps forced for Subsystem 1; the <i>capture</i> rate is the sample rate in Subsystem 3
Frame interval	1 / fps	41.7 ms at 24 fps. Every synchronisation claim in this project is measured against this number.
Resolution	Pixels	1280 x 720 for the reference clip
Codec	How the picture is compressed	H.264 in, H.264 out (copied)

Term	Meaning	Value here
Container	The file wrapper	MP4
Stream copy	Passing a compressed stream through untouched	<code>-c:v copy</code> , used in every render
SDR / HDR	Standard or High Dynamic Range	Subsystem 1 rejects HDR outright
Colour transfer / primaries	How pixel values map to light	bt709 is SDR; ar1b-std-b67 is HLG HDR

EVIDENCE IN THE PROJECT: Why stream copy matters, and how it is proved

Every render in this project uses `-c:v copy`, so the picture is not re-encoded and no generation loss occurs. The QA gate asserts it: `"3_video_stream_untouched": {"pass": true, "detail": "240 frames, stream-copied"}` in `results/qa_polished.json`, together with a SHA-256 of the source video that is verified before and after every build.

11.9 AAC and why the audio duration does not exactly match

The output MP4 carries AAC at 192 kbit/s. AAC is a lossy codec that works in frames of 1024 samples, so an encoded track's duration is quantised to a whole number of those frames.

EVIDENCE IN THE PROJECT: The 16 ms you will be asked about

The QA record says: `"4_audio_duration_matches": {"pass": true, "detail": "audio 9.984s vs video 10.000s"}`. The 16 ms difference is AAC frame granularity, not a synchronisation error, and the automated check applies a 150 ms tolerance for exactly this reason.

If an examiner points at that number, the answer is: "that is codec frame quantisation. 1024 samples at 48 kHz is 21.3 ms, and the encoder cannot emit a partial frame. It does not shift any event within the track."

11.10 The mixing chain vocabulary

Term	What it does	Setting in this project
DC offset removal	Subtracts the mean so the waveform is centred on zero	per clip, in <code>mix()</code>
Zero-crossing snap	Moves a cut point to where the waveform crosses zero, so the edit does not click	nearest crossing within plus or minus 3 ms
Fade	Ramps amplitude in or out. A raised-cosine fade is smooth in slope, unlike a linear ramp	12 ms in and out
Normalisation	Scales the whole signal by one constant so its peak hits a target	linear, to -6 dBFS
Limiting	Reduces only the peaks that exceed a threshold	threshold -6 dBFS, ceiling -3 dBFS
Clipping	What happens when a sample exceeds full scale: it is flat-topped and audibly distorted	the mix is rejected if any sample reaches full scale
Bus	The summed output of all tracks	one mono bus at the video's duration

EVIDENCE IN THE PROJECT: The limiter never engages, and that is deliberate

`"max_gain_reduction_db": 0.0, "limiter_engaged": false` in every mix log. The limiter is protection against a future change producing an overshoot, not an effect. Because it did not engage, **no dynamic-range processing of any kind was applied**, and the relative levels between events are exactly what the per-class targets set. The crest factor of 30.87 dB confirms the transient structure survived.

PART 3

SUBSYSTEM 1: LIP READING AND SPEECH GENERATION

What lip reading is, how Auto-AVSR works, how the mouth is found and cropped, how the sentence is decoded, how word timings come out of the CTC head, how the speech is spoken on time, and what the measured results actually are.

Chapter 12 — What lip reading is, and the proof that it is really happening

12.1 The purpose of this subsystem

Purpose. Take a silent video of a person speaking and produce a video with an audio track of that speech, spoken by a synthetic voice, aligned to the speaker's lips.

Problem solved. Silent footage of a person talking contains what they said, but no one can read it back at speed. Visual speech recognition recovers the words; text-to-speech makes them audible.

Input. MP4, MOV or M4V, at most 200 MB. One front-facing speaker, mouth visible, English, ideally 2 to 8 seconds at 25 fps SDR.

Output. An MP4 with the original picture stream copied unchanged, carrying an AAC audio track at 160 kbit/s in which a synthetic voice speaks the recognised sentence, placed against onsets derived from the video itself. The recognised text is also returned as JSON.

12.2 What "lip reading" and "visual speech recognition" mean

SIMPLE EXPLANATION: Lip reading in one paragraph

When you speak, your lips, jaw and tongue move into specific shapes for specific sounds. Some sounds look completely different from each other, so you can tell them apart by sight. Others look identical - "p", "b" and "m" all look like a closed mouth opening - so vision alone cannot separate them. Lip reading therefore always involves guessing from context, which is why even expert human lip readers are far from perfect.

Technically, visual speech recognition (VSR) maps a sequence of images of a speaker's mouth to a sequence of words, using no audio at any stage. It is distinguished from **audio-visual speech recognition (AVSR)**, which uses both, and which is much easier because the audio carries most of the information.

Visemes and the fundamental ambiguity. A **phoneme** is a distinguishable unit of sound; a **viseme** is a distinguishable unit of *visible* mouth shape. Several phonemes share a viseme. The classic set is /p/, /b/, /m/ - all bilabial, all produced by closing the lips. Vision cannot separate them, because the distinguishing feature (voicing, and whether air goes through the nose) is invisible.

KEY POINT: The consequence you must be able to state

Lip reading is **fundamentally ambiguous**, not merely difficult. There exist different sentences that produce identical lip motion. A visual speech model therefore has to use its language model - its learned sense of what English sentences look like - to choose between candidates that are visually identical. This is why beam search with a decoder weight of 0.9 matters, and why the model can produce a fluent, confident, wrong sentence.

12.3 Why audio is unnecessary - and in fact forbidden - in this module

The entire claim of the project is that audio is reconstructed **from pixels**. If any audio leaked into the recognition path, the result would be worthless. So the design excludes audio three times over.

1. **At standardisation.** `app/video_processing.py` builds an FFmpeg command containing

```
-map 0:v:0 -an -sn -dn: take video stream zero, no audio, no subtitles, no data. It then re-probes the output and raises VideoError("Internal error: audio was not removed.") if an audio stream is still present.
```


1. At `decode.preprocess_video.py` calls

`torchvision.io.read_video(video_path, pts_unit="sec")[0]`. `read_video` returns a three-element tuple (video, audio, info); the [0] takes video only and the audio element is discarded on the same line. The source comment says exactly that.

1. Structurally, at the model. The model's frontend is a `Conv3dResNet` accepting

tensors of shape `(B, T, 1, 88, 88)`. There is no code path by which a one-dimensional waveform could enter it.

12.4 The complete pipeline

```

UPLOAD (mp4 / mov / m4v)
|
v
VALIDATE          ffmpeg: frames, fps, pix_fmt, colour transfer/primaries
|                reject: no frames, HDR, unknown fps
v
STANDARDISE       ffmpeg -map 0:v:0 -an -sn -dn
|                if already 25 fps + yuv420p -> -c:v copy (bit-exact)
|                else                                -> re-encode fps=25, yuv420p, bt709
v
DECODE            torchvision.io.read_video(...) [0]      (audio element discarded)
|
v
LANDMARKS         MediaPipe face detection, per frame
|
v
ALIGN + CROP       mean-face alignment, 96 x 96 mouth region of interest
|
v
TRANSFORM          /255 -> CenterCrop(88) -> greyscale -> normalise
|                result: tensor T x 1 x 88 x 88
v
AUTO-AVSR          Conv3dResNet frontend -> Conformer encoder -> beam search
|                with a Transformer decoder (0.9) + CTC (0.1)
+-----> PREDICTED SENTENCE
|
v
CTC FORCED ALIGN   forced_align(log-posteriors, decoded tokens)
|                -> one onset per token -> grouped into words
|                -> corrected for CTC lag using measured mouth motion
v
WORD ONSETS (40 ms resolution)
|
v
GENDER (optional) Levi and Hassner CNN over 12 sampled frames -> voice choice
|
v
TTS                Kokoro (primary) or Piper (fallback), phrase by phrase,
|                paced so each phrase fills its span
v
PLACE + MUX        phrases positioned on the canvas; ffmpeg -c:v copy + AAC 160k
|
v
FINAL MP4

```

Figure 12.1 - Subsystem 1 end to end. Every box corresponds to a named function in `02-Auto-AVSR-Test/app/`.

12.5 Video preprocessing: validation and standardisation

The single most consequential discovery in this subsystem is that **input format changes the answer**.

EVIDENCE IN THE PROJECT: The measured effect of input format

Format	Model-input tensor mean	Best decoder score
25 fps, SDR, BT.709	0.06	-0.36
30 fps, SDR, BT.709	0.57 to 0.61	-1.05 and -1.76
30 fps, HDR (HLG)	1.24 to 1.29	-4.31 and -5.11

Auto-AVSR is trained exclusively on 25 fps SDR material. HDR footage decoded without tone mapping shifts the pixel distribution far from the training regime, and the decoder's confidence falls by roughly an order of magnitude.

The backend therefore **standardises every upload**:

- If the source is already 25 fps and `yuv420p`, the video bitstream is **stream-copied** and only the audio is dropped, so the verified baseline path is preserved bit for bit.
- Otherwise it re-encodes with `-vf fps=25,format=yuv420p,libx264,CRF 16`, and explicit `bt709` colourspace, primaries and transfer.

KEY POINT: Why `fps=` and not `setpts=`

The `fps` filter performs frame-rate conversion by dropping or duplicating frames. It does **not** retime the video. That preserves the real-time speaking rate, so the model sees 25 frames per second *of speech*, which is what it was trained on. Retiming with `setpts` would slow the speech down and change the articulation rate the model sees.

The cost is stated honestly in the README: 30 to 25 fps conversion drops roughly one frame in six.

HDR is rejected, not converted. `is_hdr()` flags a stream when its colour transfer is `arib-std-b67` (HLG) or `smpte2084` (PQ), or its primaries are `bt2020`, or its pixel format contains a 10- or 12-bit marker. A correct HLG-to-SDR conversion needs FFmpeg's `zscale` filter, which requires `libzimg` and is absent from many builds. The design decision is explicit:

KEY POINT: The principle behind rejecting HDR

Silently degrading accuracy is worse than refusing the input. A user who is told "please export in SDR" can fix the problem. A user who is given a confidently wrong transcript cannot even tell that something went wrong.

12.6 Face detection and the mouth region of interest

This stage is taken **unmodified** from the official Auto-AVSR repository, and that is deliberate: the preprocessing geometry is part of the model. Change the crop and you change the input distribution.

Step	Implementation	What it produces
Landmark detection	<code>preparation/detectors/mediapipe/detect_or.py</code> - LandmarksDetector	Facial landmarks per frame, or None if no face was found
Alignment and crop	<code>preparation/detectors/mediapipe/video_process.py</code> - VideoProcess	Frames warped onto a mean face and cropped to a 96 x 96 mouth region

Step	Implementation	What it produces
Test transform	<code>datamodule/transforms.py - VideoTransform(subset="test")</code>	Divide by 255, centre-crop to 88 x 88, convert to greyscale, normalise

Mean-face alignment warps every frame so that a set of reference landmarks lands in the same place in every frame. This removes head translation, rotation and scale, so the model sees only the articulation, not the fact that the speaker moved.

Why 96 then 88. The 96 x 96 crop gives margin; the test-time centre crop to 88 x 88 removes it. That is the geometry the network was trained with.

EVIDENCE IN THE PROJECT: What the numbers look like on a real run

From `02-Auto-AVSR-Test/outputs/ong_speech_sdr_no_audio_run1.txt`:

```

decoded          : (89, 3840, 2160, 3) in 2.2s
landmarks        : 89/89 (100.0%) in 0.5s
mouth ROI        : (89, 96, 96, 3) in 0.1s
Final tensor shape : (89, 1, 88, 88)
Final tensor dtype  : torch.float32
min/max/mean/std   : -2.3141 / 2.6155 / 0.0591 / 0.9606
Input to frontend  : (1, 89, 1, 88, 88) (B,T,C,H,W after unsqueeze(0))
Shape check        : PASS

```

A 4K source, 89 frames, 100 percent landmark detection, reduced to an 89 by 88 by 88 greyscale tensor. Note the mean of 0.0591 - that is the "25 fps SDR" row of the format table above.

Failure handling. The official `LandmarksDetector` raises an `AssertionError` when no frame yields a face. `app/inference.py` catches that specific exception and converts it into a clean, user-facing `NoFaceDetected` error: *"No face could be detected in the video. Please upload a clip with a single, clearly visible, front-facing speaker."*

12.7 The audio-independence experiment

An examiner is entitled to be sceptical that a system claiming to read lips is not simply transcribing the audio. This project answered that with an experiment rather than an assertion.

Method. The audio stream was removed from a test recording with `ffmpeg -map 0:v:0 -c:v copy -an`, and the clip was re-run end to end.

Result.

Check	Outcome
Decoded video frames	bit-identical , maximum pixel difference 0 across 2.21 billion values
Model input tensor	bitwise identical
Predicted sentence	identical: HELLO EVERYONE HOW ARE YOU
Token identifiers	identical: [5048, 2455, 1949, 2525, 768, 5034, 5048]
Decoder score	identical: -0.3558

Both transcripts survive in the repository - `ong_speech_sdr_run1.txt` (with audio in the container) and `ong_speech_sdr_no_audio_run1.txt` (audio stripped) - and you can diff them.

REMEMBER THIS

This is the single strongest experimental result in Subsystem 1, and it is the answer to the most obvious hostile question. Memorise the phrase: *"bit-identical frames, maximum pixel difference zero across 2.21 billion values, identical token IDs, identical decoder score."*

Chapter 13 — The Auto-AVSR model

13.1 What it is

Property	Value
Name	Auto-AVSR (Ma et al., ICASSP 2023) - "Audio-Visual Speech Recognition with Automatic Labels"
Checkpoint used	vsr_trlrs2lrs3vox2avsp_base.pth, MD5 49f770f2c0d8b8d769347ee47ed1648f
Size on disk	955 MB
Parameters	250,383,410
Modality	video only (vsr = visual speech recognition)
Frontend	Conv3dResNet
Encoder	Conformer
Decoder	Transformer decoder, plus a CTC head
Vocabulary	5049 tokens, SentencePiece unigram
Device in this project	CPU only
Published accuracy	20.3 percent word error rate on LRS3
Source repository	github.com/mpc001/auto_avsr , pinned at commit 182b628
Licence	Apache 2.0 (weights are subject to the licences of the training corpora)

The checkpoint name decodes as: **visual speech recognition, trained on LRS2 + LRS3 + VoxCeleb2 + AVSpeech, base size.**

13.2 The architecture, layer by layer

```

INPUT      (B, T, 1, 88, 88)      B=1 batch, T frames, 1 grey channel, 88x88 mouth
|
v
Conv3dResNet FRONTEND
|   a 3-D convolution stem sees space AND time together, so it responds to
|   mouth MOVEMENT, not to a still shape; then a 2-D ResNet reduces each
|   frame to a feature vector
v
(T, feature_dim)
|
v
proj_encoder          linear projection into the encoder width
|
v
CONFORMER ENCODER     each block:  half feed-forward
|                               -> multi-head self-attention
|                               -> convolution module
|                               -> half feed-forward
|                               -> layer norm
v
ENCODER OUTPUT (T, 768)      one 768-dimensional vector per video frame
|
+-----+
|               |
v               v
TRANSFORMER DECODER      CTC HEAD
(autoregressive,         (one distribution over 5049 tokens

```

```

attends to encoder          + blank, per frame)
output, weight 0.9)         (weight 0.1 in the beam search;
|                             ALSO reused for word timing)
+-----+-----+
      v
    JOINT BEAM SEARCH, beam width 40
      v
    BEST HYPOTHESIS -> "HELLO EVERYONE HOW ARE YOU"

```

Figure 13.1 - Auto-AVSR. The encoder output shape (89, 768) is printed by the project's own transcripts.

13.3 Why each component exists

Component	Why it is there
Conv3d stem	Lip reading is about motion. A 2-D convolution on a single frame cannot see motion; a 3-D convolution over a short stack of frames can.
ResNet body	Residual connections let a deep convolutional stack train without the gradient vanishing.
Conformer encoder	Attention for long-range sentence structure, convolution for local articulator detail. Speech needs both.
Transformer decoder	Generates the output sequence one token at a time, attending to the whole encoder output. It also carries an implicit language model, which is what resolves visemic ambiguity.
CTC head	Provides a second, monotonic view of the alignment. In the beam search it stops the decoder from hallucinating fluent text that does not match the frames. In this project it is <i>also</i> the source of word timings.
Joint decoding, 0.9 / 0.1	The decoder is the more accurate scorer but can drift; CTC is strictly monotonic and anchors it.

PROFESSOR MAY ASK: Why is the CTC weight only 0.1 and not 0.5?

Those are the official repository's defaults and this project uses them unmodified, which matters because it means the reported behaviour is the published model's behaviour and not the result of my tuning. Conceptually, the attention decoder is the stronger sequence model, so it carries most of the weight; CTC's job is to keep the hypothesis monotonically consistent with the frames rather than to score the language.

13.4 Why this model was selected

It is visual-only. The published model zoo includes audio-visual checkpoints that would score far better - and would be cheating. `vsr_trlrs2lrs3vox2avsp_base` takes no audio input at all.

It is trained on the largest available corpus. LRS2, LRS3, VoxCeleb2 and AVSpeech, using the Auto-AVSR method of generating labels automatically for otherwise unlabelled audio-visual material.

It has a published, checkable number. 20.3 percent WER on LRS3.

It runs on this machine. 250 million parameters at float32 on CPU, loading in 0.5 to 0.7 seconds and running inference in 1.2 to 2.0 seconds per clip.

It has a CTC head. This was not the reason it was chosen, but it became essential: it is what makes word-level timing possible at all.

13.5 Alternatives considered

Alternative	Why not
AV-HuBERT (base_lrs3_433h)	Actually tried first - see Chapter 21. It requires fairseq, torch 1.13.1, numpy 1.23.5 and a 1.9 GB checkpoint; the stack is older and harder to keep working, and Auto-AVSR gave a cleaner path with a published visual-only checkpoint.
Audio-visual Auto-AVSR checkpoints	Would use audio, defeating the purpose.
LipNet	Trained on GRID, a tiny constrained-grammar corpus. It cannot do open-vocabulary sentences.
Training a model from scratch	Requires thousands of hours of aligned audio-visual data and GPU time that this project does not have.
Commercial or cloud APIs	The project constraint is local inference.

13.6 Why CPU and not MPS

This is a good question and the answer is specific, not "it was easier".

EVIDENCE IN THE PROJECT: The exact reason, from the README

`espnet/nets/ctc_prefix_score.py` branches on `x.is_cuda`. On an MPS tensor, `is_cuda` is `False`, so the code takes the CPU branch and assigns a CPU device to tensors that are actually on MPS. The beam search then crashes on a device mismatch.

`app/inference.py` enforces this in the constructor:

```
if device != "cpu":
    raise ValueError("This service is CPU-only by design.")
```

Conv3d support on the MPS backend was also found unreliable. The performance cost is acceptable: the measured real-time factor is 0.45x, meaning the system runs faster than real time even on CPU.

13.7 Loading the model once

Loading a 955 MB checkpoint takes 0.5 to 0.7 seconds, which would be a large fraction of every request if it were repeated.

```
class AutoAVSRLipReader:
    def load_model(self):
        if self._loaded:
            return self                # idempotent
        torch.set_grad_enabled(False)  # inference only, globally
        margs = argparse.Namespace()
        margs.modality = self.modality # "video" -> selects the video frontend
        margs.pretrained_model_path = self.checkpoint
        margs.ctc_weight = 0.1          # repository default
        self.modelmodule = ModelModule(margs) # OFFICIAL loader, not reimplemented
        self.modelmodule.eval()
        self.n_params = sum(p.numel() for p in self.modelmodule.model.parameters())
        self._loaded = True
        return self
```

`run_server.py` calls `reader.load_model()` **before** the Flask server starts accepting requests, so the first user does not pay the load cost. A `threading.Lock` guards decoding, because the model object is stateful during a beam search.

Chapter 14 — From encoder output to a sentence

14.1 The decode, executed inline and why

The official repository provides a one-line inference call. This project deliberately inlines the same five official operations so that the intermediate objects can be inspected and reported.

```
with self._lock:
    beam_search = get_beam_search_decoder(self.modelmodule.model,
                                          self.modelmodule.token_list)

    x = self.modelmodule.model.frontend(sample.unsqueeze(0))
    x = self.modelmodule.model.proj_encoder(x)
    enc_feat, _ = self.modelmodule.model.encoder(x, None)
    enc_feat = enc_feat.squeeze(0)
    nbest = beam_search(enc_feat)
    hyps = [h.asdict() for h in nbest[: min(len(nbest), 1)]]
    token_ids = [int(i) for i in hyps[0]["yseq"]]
    predicted_token_id = torch.tensor(list(map(int, hyps[0]["yseq"][1:])))
    prediction = self.modelmodule.text_transform.post_process(
        predicted_token_id.replace("<eos>", ""))
```

Line by line.

Line	What it does
<code>get_beam_search_decoder(...)</code>	Builds the official joint decoder with the official weights. Not reimplemented.
<code>model.frontend(sample.unsqueeze(0))</code>	<code>unsqueeze(0)</code> adds the batch dimension, giving (1, T, 1, 88, 88). The Conv3dResNet turns it into per-frame features.
<code>model.proj_encoder(x)</code>	Linear projection to the encoder's width.
<code>model.encoder(x, None)</code>	The Conformer. None is the mask - there is no padding because there is exactly one sequence. Returns (1, T, 768).
<code>enc_feat.squeeze(0)</code>	Drops the batch dimension, giving (T, 768).
<code>beam_search(enc_feat)</code>	Runs the beam search. Returns an n-best list, sorted best first.
<code>hyps[0]["yseq"]</code>	The winning token sequence, including the start-of-sentence token.
<code>[1:]</code>	Drops the start-of-sentence token before detokenising.
<code>text_transform.post_process(...)</code>	Official SentencePiece detokenisation.

KEY POINT: Why inline the official calls instead of calling the wrapper

Because the wrapper returns only a string. Inlining gives access to `enc_feat`, which is what the word-timing code needs, and to the hypothesis object, which carries the score breakdown that appears in the diagnostics. The computation is identical - same functions, same official beam configuration - which the source comment states explicitly.

14.2 Reading a decoder score

Scores are log-probabilities, so they are negative, and closer to zero means more confident.

Recording	Prediction	Best score	Decoder	CTC
ong_speech_sdr	HELLO EVERYONE HOW ARE YOU	-0.3558	-0.3813	-0.1256

Recording	Prediction	Best score	Decoder	CTC
ong_speech_sdr_no_audio	HELLO EVERYONE HOW ARE YOU	-0.3558	-0.3813	-0.1256
retry_01	AND TODAY IS A GOOD DAY	-1.0459	-0.9533	-1.8796
retry_02	AND TODAY IS BEAUTIFUL DAY	-1.7594	-1.6079	-3.1229
test_01	YOGA IS MY ONLY FULL DAY	-5.1119	-4.3538	-11.9344
test_02	KONA IS KONA	-4.3140	-3.5999	-10.7410
test_03	I AM WORKING ON MY FINAL YEAR PROJECT	-4.3932	-3.3951	-13.3759

Table 14.1 - All seven recorded runs, read from 02-Auto-AVSR-Test/outputs/.

CAUTION: What a score is and is not

A decoder score is the model's **confidence**, not its **accuracy**. A confident wrong answer is entirely possible. `test_03` scores -4.39, which is low confidence, yet the sentence "I AM WORKING ON MY FINAL YEAR PROJECT" is almost certainly what was said. And `test_02`'s "KONA IS KONA" - visible in the tokens as `KO`, `NA`, `IS`, `KO`, `NA` - is almost certainly wrong.

Never present decoder scores as accuracy. The handbook is explicit about this and so is the research paper.

Chapter 15 — Word timing from the CTC head

This is the most original piece of engineering in Subsystem 1.

15.1 The problem

Auto-AVSR returns a sentence with no timestamps. To lip-sync synthesised speech you need to know when each word starts. Options:

1. Spread the words evenly across the clip. Crude, and wrong whenever the speaker pauses.
2. Use a separate forced aligner on the synthesised audio. That aligns speech to speech,

not speech to *lips*, so it would not fix anything.

1. **Use the model's own CTC head.** It already emits per-frame token posteriors at 25

frames per second. Forced-aligning the decoded tokens against those posteriors gives every token a frame index derived from the video.

Option 3 is what the project does, and the design note in `app/timing.py` says why: *"timings come from the video itself and are frame-synchronous with the lips. Nothing here is estimated or invented."*

15.2 The algorithm

```
def word_timings(model, token_list, enc_feat, token_ids, n_frames, fps=25.0, sample=None):
    ids = [int(i) for i in token_ids if int(i) not in (model.eos, model.blank)]
    logp = model.ctc.log_softmax(enc_feat.unsqueeze(0)).float() # (1, T, V)
    targets = torch.tensor([ids], dtype=torch.int32)
    frames, _ = torchaudio.functional.forced_align(logp, targets, blank=model.blank)
    frames = frames[0].tolist()
    # collapse the frame-wise path into one onset per token occurrence
    onsets, prev = [], None
    for f, tok in enumerate(frames):
        if tok != prev:
            if tok != model.blank:
                onsets.append((tok, f))
            prev = tok
    # group sub-word tokens into words: a leading word-start marker begins a new word
    words = []
    for tok, f in onsets:
        unit = token_list[tok]
        if unit.startswith("\u2581") or not words: # U+2581 = word-start marker
            words.append({"word": unit.lstrip("\u2581"), "frame": f})
        else:
            words[-1]["word"] += unit
```

Step	What happens	Why
Strip eos and blank from the targets	Forced alignment needs the real symbol sequence only	those two are not emitted symbols
<code>ctc.log_softmax(enc_feat)</code>	Per-frame log-probabilities over 5049 tokens plus blank	this is the same encoder output used for decoding, so no extra compute
<code>forced_align(logp, targets, blank)</code>	Finds the most probable frame path that produces exactly this token sequence	this is where video timing enters
Collapse transitions	One onset per token occurrence	CTC repeats a symbol across frames

Step	What happens	Why
Group by the word-start marker	SentencePiece marks word starts; anything else is a continuation	turns tokens into words
<code>w["end"]</code> = next word's start	A word runs until the next one begins	CTC is peaky, so it gives onsets, not durations
Last word's end	<code>start + max(median duration, 0.25 s)</code> , capped at the video end	there is no "next word" to bound it

15.3 The lag correction

Already introduced in Section 8.3; here is the code and the reasoning together.

EVIDENCE IN THE PROJECT: The measurement, not an assumption

```
def motion_onset(sample, fps=25.0):
    a = sample.detach().float().squeeze(1).cpu().numpy() # (T, 88, 88)
    inner = a[:, 28:68, 20:76] # inner-mouth region
    motion = np.abs(np.diff(inner, axis=0)).mean(axis=(1, 2))
    norm = (motion - motion.min()) / (motion.max() - motion.min())
    thresh, run = 0.35, 0
    for i, v in enumerate(norm):
        run = run + 1 if v > thresh else 0
        if run >= 3:
            return max((i - run + 1) / fps, 0.0)
```

The crop `[:, 28:68, 20:76]` is the inner mouth inside the 88 by 88 tensor. The motion signal is the mean absolute inter-frame difference. The onset is the first frame from which normalised motion stays above 0.35 for **three consecutive frames** - the three-frame run is what stops a single noisy frame from triggering it.

The lag is then `words[0]["start"] - motion_onset`, clipped to `[0, 0.40]` seconds, and every anchor is shifted back by it. Measured value on the project's test clip: approximately **0.24 s**.

15.4 Pause detection, and why it comes from the video

`find_pauses()` finds stretches inside the speech where the mouth is essentially still: normalised motion below 0.18 for at least 0.20 seconds.

KEY POINT: Why pauses are read from the video rather than inferred

Two reasons, both stated in the source comments.

First, **anchor spacing cannot distinguish "a long word" from "a word plus a pause"**. If two onsets are 0.9 s apart, that could be a long word or a short word followed by silence. Only the mouth tells you which.

Second, **anything derived from a synthesis result is unstable**: Piper's duration predictor is stochastic, so grouping the sentence into phrases based on synthesised durations changed between runs and the phrasing kept flipping between one phrase and two. Deriving it from the video makes it deterministic.

Chapter 16 — Speech synthesis and placement

16.1 The TTS stack

Engine	Role	Why
Kokoro (kokoro-v1.0.onnx, 24 kHz)	primary	Judged markedly more natural on this material. Voices: af_heart (female), am_michael (male).
Piper (en_US-lessac-medium, en_US-ryan-medium)	automatic fallback	Offline, native arm64, runs in-process. -high variants supported via quality=high but were not judged better than medium here.
macOS say	last-resort fallback	So the pipeline still works if no voice model is installed at all.

Kokoro runs in a separate process, and this is forced. Kokoro needs `numpy>=2`; MediaPipe 0.10.21 declares `numpy<2`. They cannot share an interpreter. So Kokoro lives in `venv-tts` and is driven by `tts_worker.py` over a line-oriented JSON protocol on stdin and stdout:

```
Flask process (venv-autoavsr, numpy 1.26.4)
|
| {"text": ..., "voice": "af_heart", "speed": 1.0, "out": "/tmp/x.wav"}
v  (one JSON object per line, on stdin)
tts_worker.py (venv-tts, numpy >= 2)  <-- model loaded ONCE, process stays alive
|
| {"ok": true, "seconds": 1.83, "sample_rate": 24000}
v  (one JSON object per line, on stdout)
Flask process reads the WAV from disk
```

Figure 16.1 - Crossing an impossible dependency boundary with a subprocess and a pipe.

16.2 Choosing a voice: gender detection

There is no audio, so the voice has to be chosen from the picture.

`app/gender.py` samples 12 frames evenly across the clip, detects the face in each, classifies it with the **Levi and Hassner (2015)** gender CNN through `cv2.dnn`, and takes a confidence-weighted vote. Below a confidence of 0.60 the result is not trusted.

CAUTION: State the limitation before the examiner does

This classifier is roughly 85 to 90 percent accurate on clear frontal faces and reports only male or female. The API therefore accepts an explicit `voice=male|female` parameter that overrides it, the UI exposes that choice, and a low-confidence result falls back to the female voice rather than guessing. This is documented in the README as a limitation, not hidden.

16.3 Four lessons that shaped the placement code

Each of these was a real failure, found by listening, and each is recorded in the source.

Lesson 1 - do not cut per word. Synthesising each word separately and dropping it on its anchor produced **19 audio fragments separated by 40 to 80 ms gaps, with clipped phonemes**. Every word carried sentence-final intonation and sat in a hole of silence. The result was audibly choppy.

The fix: synthesise **whole phrases**, and split only where the video says the mouth was genuinely still. Inside a phrase the audio is never cut.

Lesson 2 - cap the pace, not the multiplier. Different voices speak at different intrinsic speeds; the male voice runs about 13 percent faster than the female one. A shared `length_scale` cap therefore left the faster voice finishing early.

The fix: cap an **absolute pace in seconds per character** (`MAX_SEC_PER_CHAR = 0.105`). Text length is backend-independent, unlike Piper's phoneme weights or Kokoro's solo durations. The code even records how each backend's duration responds to `length_scale`:

```
LS_RESPONSE = {"piper": 0.53, "kokoro": 1.15, "macos-say": 1.0}
```

Piper is markedly sublinear; Kokoro is roughly linear. That constant was found by measurement, and the code verifies the result by measuring the synthesised length and re-synthesising once if it undershoots.

Lesson 3 - do not trim quiet onsets with a fixed threshold. A fixed silence threshold cut into the /h/ of "hello", which ramps up gradually, so the word came out as "ello". The fix: the threshold is relative to the utterance peak, with a 45 ms margin.

Lesson 4 - do not warp individual words. Stretching each word onto its own anchor was implemented and then **reverted**. The comment explains why:

EVIDENCE IN THE PROJECT: Straight from `app/sync.py`

"The phrase is placed as one unbroken piece. Warping each word onto its own anchor was tried and reverted: the word boundaries here are ESTIMATED from phoneme counts, not measured, so stretching at them cuts mid-phoneme and short words hit the clamp and warble. Accurate per-word alignment needs real alignments, which this voice does not expose."

This is intellectually honest engineering: the feature was built, measured, judged worse, and removed, and the reason was written down.

16.4 The placement algorithm

For each phrase group:

1. Its span runs from the first word's anchor to either the next group's anchor minus

`PAUSE_KEEP` (0.10 s) or the end of speech.

1. Synthesise the phrase once and trim silence relative to the peak.
2. Compute a target duration: `min(span * 0.96, n_chars * 0.105)` - just under the span,

so the length-scale solve does not overshoot and force a second lossy time-stretch.

1. Solve for `length_scale` using the backend's measured response, synthesise again, and if it still undershoots by more than 7 percent, solve once more.

1. If it is still too long, apply a pitch-preserving `ffmpeg atempo` stretch, clamped to between 0.72x and 1.55x.

1. Apply a 6 ms fade at each phrase edge and add it into the canvas.

Finally the canvas is peak-limited to 0.99 and muxed:

```
ffmpeg -i standardised.mp4 -i speech.wav -map 0:v:0 -map 1:a:0 \
    -c:v copy -c:a aac -b:a 160k -shortest out.mp4
```


Chapter 17 — Datasets behind Subsystem 1

This project **did not train on these**, but the checkpoint it uses was trained on them, so you must be able to describe them.

Dataset	Content	Role in the checkpoint
LRS2-BB C	Thousands of spoken sentences from BBC television, with face tracks and transcripts, unconstrained vocabulary, real-world conditions	Training corpus
LRS3-TE D	Aligned face tracks and transcripts from TED and TEDx talks. The standard open benchmark for sentence-level lip reading	Training corpus and the benchmark on which the published 20.3 percent WER is reported
VoxCeleb2	Over a million utterances from thousands of speakers, collected from YouTube interviews. Originally built for speaker recognition, so it has no transcripts	Used as unlabelled data; Auto-AVSR's contribution is generating labels for it automatically with an ASR model
AVSpeech	A very large corpus of clips containing a single visible speaking face with clean speech	Same - automatically labelled

KEY POINT: What "Auto-AVSR" actually contributed as a method

The published contribution is not a new architecture; it is a **data** method. There is far more unlabelled audio-visual video in the world than transcribed video. Auto-AVSR runs an audio speech recogniser over unlabelled corpora to generate transcripts automatically, then trains the visual model on those. That is why the checkpoint name lists four corpora: two labelled, two automatically labelled.

17.1 Dataset characteristics that matter to your results

Property of the training data	Consequence for your recordings
25 frames per second	Your uploads are forced to 25 fps, and 30 fps material measurably degrades
SDR, BT.709, 8-bit	HDR is rejected outright
Mostly frontal, well-lit, single speaker	Side-on faces, poor lighting and multiple faces degrade or fail
English	The model has no other language
Broadcast and conference speech	Casual, fast or heavily accented speech is out of distribution

17.2 Dataset limitations and biases - state these honestly

- **Language bias.** English only. The model cannot read any other language.
- **Speaker bias.** LRS2 is British broadcast; LRS3 is TED speakers; VoxCeleb2 is

celebrity interviews. These are not a uniform sample of humanity, in accent, in demographics, or in speaking style.

- **Recording bias.** Professionally lit, professionally framed, front-facing.
- **Automatic labels carry ASR errors.** Two of the four corpora were labelled by an audio

speech recogniser, so any systematic ASR error becomes training signal.

- **Licence restrictions.** All four corpora restrict redistribution, which is why the checkpoint is not in the repository.

Chapter 18 — Subsystem 1: the web application

18.1 The API

Base URL `http://127.0.0.1:5001`.

Method	Path	Body	Returns
GET	/	-	the web interface
GET	/health	-	model and service status
POST	/predict	multipart, field <code>video</code>	recognised text and diagnostics
POST	/generate	multipart, field <code>video</code> , optional <code>`voice=auto`</code>	male\
GET	/generated/<name>	-	serves the produced video; <code>?download=1</code> to download

A real `/predict` response:

```
{
  "success": true,
  "prediction": "HELLO EVERYONE HOW ARE YOU",
  "frames": 89,
  "landmark_detection_rate": 100.0,
  "inference_time": 1.39,
  "total_time": 5.45,
  "modality": "video-only",
  "device": "cpu"
}
```

18.2 Error handling

Errors return `{"success": false, "error": "...", "code": "...}"` with an appropriate HTTP status, and never a stack trace.

Code	Status	Trigger
<code>missing_file</code>	400	no file, or an empty filename
<code>unsupported_format</code>	415	extension not in <code>.mp4 .mov .m4v</code> , or HDR input
<code>invalid_file</code>	400	empty file, or no video stream
<code>corrupt_video</code>	400	<code>ffprobe</code> failed, no frames, unknown frame rate, conversion failed
<code>no_face_detected</code>	400	the landmark detector found no face in any frame
<code>preprocessing_failed</code>	400	the crop failed, or the tensor shape is not <code>(T, 1, 88, 88)</code>
<code>no_speech_detected</code>	400	the model returned an empty string
<code>timing_failed</code>	400	word timings could not be derived
<code>file_too_large</code>	413	over 200 MB
<code>internal_error</code>	500	anything unhandled; logged server-side, never returned

18.3 Security details worth pointing out

Small, but they are exactly the kind of detail that impresses an examiner.

Uploaded filenames are never used as paths.

```
def safe_upload_name(original_name):
    ext = os.path.splitext(original_name or "")[1].lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise UnsupportedFormat(...)
    return f"{uuid.uuid4().hex}{ext}"
```

Only the extension is read from the client name, and only after it is checked against an allow-list. The stored path is always a generated UUID. This closes path traversal and overwrite attacks.

Served filenames are pattern-matched, not trusted.

```
if not re.fullmatch(r"speech_[0-9a-f]{32}\.mp4", name):
    abort(404)
```

Intermediates are always cleaned, in a `finally:` block, whether the request succeeded or failed.

18.4 The frontend

A single `index.html` of 232 lines, 335 lines of JavaScript and 459 lines of CSS. No framework and no build step.

Structure: a header with a live system-status pill; a hero title; a two-panel workspace with the upload dropzone and preview on the left and the result panel on the right. The result panel is a small state machine - empty, busy with a stage list, error, or result - and the result view shows the recognised text with a copy button, the final video with a download button, a word timeline, and a metrics list.

Chapter 19 — Subsystem 1: results, limitations and challenges

19.1 What was measured

Metric	Value	Source
Parameters	250,383,410	printed by the loader in all 7 transcripts
Vocabulary	5049 tokens	same
Beam size	40	same
Model load time	0.5 to 0.7 s	same
Inference time	1.2 to 2.0 s, mean 1.57 s over 7 runs	paper/experiments/exp6_latency.json
Total time per clip	2.5 to 6.0 s, mean 3.73 s	same
Real-time factor	0.45x - faster than real time, on CPU	derived from the same data
Landmark detection rate	100 percent on every recorded run	the transcripts
Clip lengths tested	59 to 98 frames	the transcripts
Best decoder score	-0.3558, on 25 fps SDR	ong_speech_sdr_run1.txt
Worst decoder score	-5.1119, on out-of-distribution material	test_01_run1.txt

19.2 What was verified qualitatively

Two transcriptions were spot-checked as correct:

- HELLO EVERYONE HOW ARE YOU at -0.36
- AND TODAY IS A GOOD DAY at -1.05

And the audio-independence experiment (Section 12.7) passed exactly.

19.3 The honest gap: there is no word error rate

CAUTION: Say this before you are asked

No word error rate is reported for Subsystem 1, because ground-truth transcripts for the test recordings were not retained. Recognition quality is characterised only by decoder scores and by two spot-checked correct transcriptions.

The 20.3 percent WER on LRS3 is a **published result for the checkpoint**, reported by its authors on their benchmark. It is **not** a measurement of this system on this material, and this handbook, the README and the research paper all say so in the same words.

If an examiner asks "how accurate is your lip reading?", the correct answer is: *"I cannot give you a word error rate, because I did not retain ground-truth transcripts for my own recordings. The published checkpoint reports 20.3 percent WER on LRS3. What I can show you is the confidence distribution across seven runs and two spot-checked correct transcriptions, and I can show you that the format standardisation improves confidence by about an order of magnitude."*

That answer is far stronger than a made-up number, and an examiner who is paying attention will recognise it as such.

19.4 Limitations

Limitation	Detail
CPU only	The MPS path crashes in <code>ctc_prefix_score.py</code> , which branches on <code>x.is_cuda</code>
HDR rejected, not converted	Correct HLG to SDR needs FFmpeg's <code>zscale</code> filter, absent from many builds
30 to 25 fps conversion drops frames	Roughly one frame in six, using <code>fps=</code> rather than retiming
Flask development server	Decoding is serialised by a lock; there is no production WSGI server
No authentication or rate limiting	Intended for local use
Sync quality is independent of recognition accuracy	If the model reads the wrong words, they will be lip-synced accurately and wrong
The voice is generic	It is a TTS voice, not the speaker's own
Gender detection is heuristic	85 to 90 percent on clear frontal faces, binary output, manual override provided
Word boundaries inside a phrase are estimated	From phoneme counts, not measured. Piper does not expose alignments for these models
English only	A property of the training corpora
Fundamentally ambiguous task	Different sentences can produce identical lip motion

19.5 Challenges faced, and how each was solved

Challenge	Cause	Solution	Result
Auto-AVSR's install instructions break on current releases	The README says <code>pip install torch torchvision torchaudio</code> with no version pins	Pin <code>torchvision==0.20.1</code> (<code>read_video</code> was removed in 0.26), <code>av==13.1.0</code> (PyAV 14 removed <code>av.AVError</code> , referenced in six places inside <code>read_video</code>), <code>numpy==1.26.4</code> and <code>mediapipe==0.10.21</code>	A reproducible environment, with the reason for every pin written into <code>requirements.txt</code>
MPS crashes the beam search	<code>ctc_prefix_score.py</code> branches on <code>x.is_cuda</code> , which is <code>False</code> for MPS tensors, so a CPU device is assigned to MPS tensors	Force CPU, and raise a clear error if anyone asks for another device	Works, at 0.45x real time
Kokoro and MediaPipe cannot coexist	Kokoro needs <code>numpy>=2</code> ; MediaPipe declares <code>numpy<2</code>	A second virtual environment plus a persistent worker process driven over <code>stdin</code> and <code>stdout</code>	Both work; the model loads once and stays alive
HDR input silently ruins accuracy	Untone-mapped HDR shifts the input tensor mean from about 0.06 to about 1.27	Detect HDR from colour transfer, primaries and pixel format, and reject with an explanatory message	Confidence stays in the good range or the user is told why it cannot
Speech lagged the lips	CTC peaks mark model confidence, which is later than the start of articulation	Measure mouth-motion onset directly and shift every anchor back by the difference	About 0.24 s of correction on the test clip
Per-word synthesis sounded choppy	19 fragments with 40 to 80 ms gaps and clipped phonemes	Synthesise whole phrases; split only at pauses measured from the video	Continuous, natural speech
Phrase grouping flipped between runs	It was derived from a synthesised duration, and Piper's duration predictor is stochastic	Derive grouping from video-measured pauses only	Deterministic

Challenge	Cause	Solution	Result
The male voice finished early	A shared <code>length_scale</code> cap, but voices differ in intrinsic speed by about 13 percent	Cap absolute pace in seconds per character, and measure each backend's <code>length_scale</code> response	Both voices land the same way
"hello" came out as "ello"	A fixed silence threshold cut into the gradual /h/ onset	Make the threshold relative to the utterance peak, with a 45 ms margin	The word survives
Per-word warping warbled	Word boundaries inside a phrase are estimated, not measured, so stretching cut mid-phoneme	Revert to unbroken phrase placement	Better audio, and the reason is documented in the source

Chapter 20 — Subsystem 1: important code, explained

20.1 run_server.py - the entry point

```
def main():
    print("Loading Auto-AVSR checkpoint (once)...")
    reader.load_model()
    info = reader.info()
    print(f"  model      : {info['model']} ({info['parameters'],} params)")
    print("Loading TTS voice (once)...")
    api.tts = TTSEngine()
    api.gender_detector = GenderDetector()
    print(f"Serving on http://{args.host}:{args.port}")
    app.run(host=args.host, port=args.port, debug=False, threaded=False)
```

Detail	Why it matters
<code>reader.load_model()</code> before <code>app.run(...)</code>	The 955 MB checkpoint is loaded once, at start-up, so no user pays for it
<code>TTSEngine()</code> and <code>GenderDetector()</code> also constructed once	Same reason
<code>threaded=False</code>	Deliberate. One decode at a time keeps CPU inference predictable and avoids two beam searches competing for cores
<code>debug=False</code>	The Werkzeug debugger executes arbitrary code from the browser; it must never be on

20.2 app/video_processing.py - standardize()

```
needs_fps = abs(info["fps"] - TARGET_FPS) > 0.01
needs_pix = (info.get("pix_fmt") or "") != "yuv420p"
reencoded = needs_fps or needs_pix

cmd = ["ffmpeg", "-hide_banner", "-loglevel", "error", "-y",
      "-i", src, "-map", "0:v:0", "-an", "-sn", "-dn"]

if reencoded:
    cmd += ["-vf", f"fps={TARGET_FPS},format=yuv420p",
          "-c:v", "libx264", "-preset", "fast", "-crf", "16",
          "-colorspace", "bt709", "-color_primaries", "bt709",
          "-color_trc", "bt709", "-pix_fmt", "yuv420p"]
else:
    cmd += ["-c:v", "copy"]
```

Flag	Meaning
<code>-map 0:v:0</code>	take video stream 0 from input 0
<code>-an -sn -dn</code>	no audio, no subtitles, no data streams
<code>-vf fps=25</code>	frame-rate conversion by dropping or duplicating; not retiming
<code>format=yuv420p</code>	8-bit 4:2:0, the format the model was trained on
<code>-crf 16</code>	near-visually-lossless x264 quality
<code>-colorspace/-color_primaries/-color_trc bt709</code>	tag the output as SDR so nothing downstream misinterprets it
<code>-c:v copy</code>	when nothing needs changing, do not re-encode at all

The function then re-probes the output and refuses to continue if audio survived. Belt and braces, and cheap.

20.3 app/inference.py - the shape contract

```
T, C, H, W = sample.shape
if (C, H, W) != (1, 88, 88):
    raise PreprocessingFailed(f"Unexpected model input shape {(T, C, H, W)}.")
```

Three numbers, one check, and it makes an entire class of silent failure impossible. If the preprocessing chain ever changes upstream, this fails loudly with a readable message rather than feeding a wrongly-shaped tensor into a 250-million-parameter network and producing confident nonsense.

KEY POINT: A general lesson worth stating in viva

Assert your interfaces. The cheapest bug to fix is the one that stops the program at the boundary where it happened, rather than the one that produces a plausible wrong answer three stages later.

20.4 app/tts.py - the Kokoro worker protocol

```
def start(self):
    if not (os.path.isfile(KOKORO_PY) and os.path.isfile(KOKORO_WORKER)
           and os.path.isfile(KOKORO_MODEL)):
        return False # missing -> fall back to Piper, no exception
    self.proc = subprocess.Popen(
        [KOKORO_PY, KOKORO_WORKER],
        stdin=subprocess.PIPE, stdout=subprocess.PIPE,
        stderr=subprocess.DEVNULL, text=True, bufsize=1, cwd=PROJECT_DIR)
    hello = json.loads(self.proc.stdout.readline() or "{}")
    if not hello.get("ok"):
        self.stop()
        return False
    self.voices = hello.get("voices", [])
    return True
```

Detail	Why
Returns False rather than raising	A missing optional voice must degrade to the fallback, not break the service
bufsize=1, text=True	Line buffering, so a single <code>readline()</code> gets a complete JSON object
A handshake line on start-up	Confirms the model actually loaded before any request depends on it
The process stays alive	The ONNX model is loaded once, so there is no per-request start-up cost
<code>stderr=subprocess.DEVNULL</code>	ONNX runtime chatter must not be mistaken for protocol output

Chapter 21 — The rejected first attempt: AV-HuBERT

Include this in a viva. Examiners like to see that you evaluated more than one option, and this one is fully documented in the repository.

21.1 What was tried

01-Lip-Reading/ contains a complete, working AV-HuBERT setup:

Item	Detail
Model	AV-HuBERT, checkpoint <code>base_lrs3_433h.pt</code> , 1.93 GB
Framework	fairseq, pinned to commit <code>afc77bdf</code>
Stack	torch 1.13.1, numpy 1.23.5, opencv-python 4.5.4.60, scikit-image 0.19.3
Landmark detector	dlib's 68-point <code>shape_predictor_68_face_landmarks.dat</code> plus a <code>20words_mean_face.npy</code> reference
Modality	forced to video only: <code>saved_cfg.task.modalities = ["video"]</code>
Device	CPU (CUDA_VISIBLE_DEVICES cleared)
Scripts written	<code>av_inference.py</code> , <code>test_lipreading.py</code> , <code>run_batch_test.py</code> , <code>diagnose_preprocessing.py</code> , <code>diagnose_nbest.py</code> , <code>ab_encoder_numeric.py</code> , <code>ab_prepare_for_inference.py</code> , <code>hdr_to_sdr.py</code> , <code>run_hdr_experiment.py</code> , <code>inspect_checkpoint.py</code>

21.2 What AV-HuBERT is

AV-HuBERT (Shi et al., ICLR 2022) learns audio-visual speech representations by **masked multimodal cluster prediction**: mask parts of the input, ask the model to predict cluster assignments that were themselves derived from the model's own earlier representations, and iterate. It is a self-supervised pre-training method, and its contribution is that it drastically reduces the amount of *labelled* data needed.

21.3 Why it was set aside

The repository shows what the friction was, and it was practical rather than scientific:

- **fairseq is a heavy, fragile dependency.** `av_inference.py` contains a comment

documenting a real workaround: `hubert_pretraining.py` chooses between flat and relative imports based on `len(sys.argv) == 1`, which breaks under a real argparse CLI, so the code has to temporarily replace `sys.argv` around the import.

- **The stack is old.** torch 1.13.1 and numpy 1.23.5 conflict with everything else in

the project.

- **Auto-AVSR offered a cleaner path:** an official visual-only checkpoint, a published benchmark number, a modern torch, and preprocessing that is part of the same repository.

- Both projects also had to solve the HDR problem, and `01-Lip-Reading/hdr_to_sdr.py` and

`run_hdr_experiment.py` are where that investigation started. The finding carried over into the delivered system.

PROFESSOR MAY ASK: Why did you abandon AV-HuBERT?

"Not on quality grounds - I did not run a head-to-head accuracy comparison, and I would not claim one. I set it aside for engineering reasons: it requires fairseq on torch 1.13.1 with numpy 1.23.5, and it needed an import workaround because the upstream code switches import style based on `sys.argv`. Auto-AVSR gave me an official visual-only checkpoint on a modern torch with the preprocessing in the same repository, so the integration risk was much lower. The AV-HuBERT work is still in `01-Lip-Reading/` if you want to see it."

PART 4

SUBSYSTEM 2: ACTION RECOGNITION AND SOUND GENERATION

The main web application. Nine stages from a silent upload to a video with synchronised Foley: what each stage does, why it exists, what it measured, and where it fails.

Chapter 22 — The nine-stage pipeline

22.1 Purpose and contract

Purpose. Take a silent video, recognise the physical actions in it, generate matching Foley audio, align each sound to the exact frame where the action is visible, mix it, and return a playable video with sound.

Input. MP4, MOV, AVI, M4V or MKV. At most 200 MB, between 0.4 and 60 seconds. A file that already has an audio track is accepted with a warning; that track is never decoded.

Output. An MP4 whose picture stream is copied unchanged and which carries AAC audio at 192 kbit/s, 48 kHz mono, generated from a 48 kHz mono PCM 16-bit master. The mixed WAV and a full JSON processing report are also downloadable.

Where it lives. Module3_Fresh/.

22.2 The nine stages

1	UPLOAD	streamed to disk in 1 MB chunks, size checked during the stream
2	VIDEO VALIDATION	ffprobe -> duration, resolution, fps, codec, stream inventory
		reject: no video stream, <=0.4 s, >60 s, >200 MB, bad extension
		warn: audio track present (never decoded), resolution < 64 px
	v	
3	ACTION RECOGNITION	subprocess -> venv-qwen -> Qwen2.5-VL-3B-Instruct
		2.0 s windows, 1.0 s stride, 8 frames each at 448x252, bfloat16, MPS
		writes a progress file per window, which the pipeline polls
	v	
4	ACTION TIMELINE	merge windows by action head -> midpoint boundary resolution
		-> a deterministic NON-OVERLAPPING timeline
	v	
5	FOLEY GENERATION	each distinct action -> a Foley class -> a text prompt
		subprocess -> venv-moss -> MOSS-SoundEffect v2.0
		up to 3 candidate seeds, stopping early once one scores >= 45
		content-addressed cache: identical request returns in seconds
	v	
6	FOLEY QUALITY VALIDATION	six gates measured on the RAW file, before any gain
		a class whose every candidate failed is left SILENT and reported
	v	
7	VISUAL SYNCHRONISATION	frames -> 320x180 grey at 24 fps -> band motion
		strategy per action: footstep hold contact continuous
		-> exact visual event instants
		-> clip selection -> alignment on TRUE ENVELOPE ATTACK
	v	
8	AUDIO MIXING	per clip: DC removal -> zero-crossing snap -> 12 ms fades
		-> active-RMS level -> -6 dBFS peak cap
		bus: sum -> normalise to -6 dBFS -> safety limiter
	v	
9	FINAL RENDERING	ffmpeg -map 0:v:0 -map 1:a:0 -c:v copy -c:a aac -b:a 192k
		-ar 48000 -movflags +faststart -shortest
	v	
	FINAL MP4 + mixed WAV + JSON report	

Figure 22.1 - The nine stages. Every one reports real state to the job store; nothing is simulated.

22.3 Stage-by-stage detail

Stage 1 - Upload. The file is streamed to `data/uploads/<uid><ext>` in 1 MB chunks with the size limit enforced *during* the stream, so an oversized file is rejected before it is fully written to disk.

Stage 2 - Validation.

Condition	Outcome
No video stream	rejected
Duration at or below 0.4 s, or above 60 s	rejected
Size above 200 MB	rejected
Extension not in <code>.mp4 .mov .avi .m4v .mkv</code>	rejected
Audio track present	accepted with a warning - it is never decoded
Either dimension below 64 px	accepted with a warning

Stage 3 - Action recognition. Covered in Chapter 23.

Stage 4 - Timeline. Covered in Chapter 24.

Stage 5 - Foley generation. Covered in Chapters 25 and 26.

Stage 6 - Quality validation. Covered in Chapter 27.

Stage 7 - Visual synchronisation. Covered in Chapter 29.

Stage 8 - Mixing. Covered in Chapter 31.

Stage 9 - Rendering. Covered in Chapter 31.

22.4 Why 60 seconds and not longer

`MAX_VIDEO_SECONDS = 60`, and the code comment gives the reason: `# Qwen windowing cost grows linearly.`

A 2-second window with a 1-second stride produces roughly one window per second of video, and each window costs about 5 seconds of inference. A 60-second video therefore costs about five minutes of action recognition alone, before any audio is generated. The cap is a deliberate product decision, and the error message tells the user exactly that:

"This video is 75.0 s long. The current limit is 60 s, because action recognition cost grows with duration."

KEY POINT: Error messages that explain the constraint

Every rejection in this system tells the user **why** the limit exists, not just that it exists. That is a small thing that examiners notice.

22.5 Expected processing time

Stage	Typical
Validation	under 1 s
Action recognition	about 5 s per window, so roughly 1 minute for a 10 s clip
Foley generation	about 4.7 minutes per candidate (then cached)
Foley quality validation	under 2 s
Visual sync, mixing and render	under 20 s

Stage	Typical
Whole job, fully cached	39.5 s median
Whole job, with generation	556.9 s median

Table 22.1 - Measured over 31 recorded generation runs and 32 completed job records. Source: `paper/experiments/exp6_latency.json`.

EVIDENCE IN THE PROJECT: Where the time actually goes

Diffusion consumes **94 percent** of generation time. Because the model denoises a fixed-length latent regardless of the requested duration, a short request costs the same as a long one. That is why the cache matters so much: 39.5 seconds against roughly nine minutes.

Chapter 23 — Module 2: action recognition with Qwen2.5-VL

23.1 What this stage does

Input. A silent video file.

Output. A JSON payload containing per-window predictions, merged spans, and a resolved non-overlapping timeline of actions with start and end times.

Model. Qwen/Qwen2.5-VL-3B-Instruct, bfloat16, on Apple MPS.

Where. backend/runners/run_module2.py, executed by the Python interpreter inside venv-qwen, which imports the validated implementation from 03-FoleyCrafter-Test/action-recognition/action_recognition.py.

23.2 The design decision that makes this work

KEY POINT: Semantics from the model, timing from the windowing

From the header of action_recognition.py:

"Qwen is NEVER asked for timestamps. It answers only 'what is happening here?'. Timing comes from each window's known position on the timeline."

Large language models are notoriously unreliable at producing numbers. They are much better at describing what they see. So the design asks the model only for the thing it is good at, and derives every number arithmetically from something already known - the position of the window on the timeline.

23.3 The prompt

```
You are analyzing a video for an action-recognition system.

Look only at the visual content.

Identify the main physical action occurring during this video segment.

Return:
ACTION: <short action phrase>
EVIDENCE: <brief visual evidence>

Do not infer sound.
Do not infer unseen events.
Do not invent actions that are not visually supported.
```

Line	Purpose
"Look only at the visual content"	Suppresses reasoning from context the model cannot see
"the main physical action"	One action per window, and a <i>physical</i> one - not a mood or a scene description
ACTION: / EVIDENCE:	A parseable two-line format
"Do not infer sound"	The model must not write "the cup clinks"; the sound is Subsystem 2's job, not the model's
"Do not infer unseen events"	Suppresses narrative completion
"Do not invent actions that are not visually supported"	Suppresses hallucination

CAUTION: The prompt does not fully work, and you should say so

Despite "Identify the main physical action", the model sometimes returns a **caption** rather than an action. On the coffee-stirring test video it produced `Stirring a cup of coffee` - a caption in gerund form with an article - alongside `stir coffee` and `stir the contents of the cup` for the same activity. Three labels, one action. That is documented as the top known limitation.

23.4 The windowing

```
WINDOW_S, STRIDE_S = 2.0, 1.0
FRAMES_PER_WINDOW, TW, TH = 8, 448, 252
MAX_NEW_TOKENS = 96

def plan_windows(duration):
    win = min(WINDOW_S, duration)
    span = duration - win
    if span <= 1e-6:
        return [(0.0, duration)]
    starts, s = [], 0.0
    while s < span - 1e-6:
        starts.append(round(s, 3)); s += STRIDE_S
    if not starts or abs(span - starts[-1]) > 0.25:
        starts.append(round(span, 3)) # snap the final window to the end
    return [(st, round(st + win, 3)) for st in starts]
```

Detail	Reason
<code>win = min(WINDOW_S, duration)</code>	A clip shorter than the window still produces one window
The final snap to span	Otherwise the last fraction of a second would never be seen
<code>abs(span - starts[-1]) > 0.25</code>	Do not add a final window that duplicates the previous one

For the 10.005-second reference clip this produces **9 windows**: 0-2, 1-3, 2-4, 3-5, 4-6, 5-7, 6-8, 7-9, 8-10.

23.5 Frame extraction - video stream only

```
def extract_video_frames(path, w, h):
    """VIDEO STREAM ONLY: `--map 0:v:0`. The audio stream is never decoded."""
    out = subprocess.run(["ffmpeg", "-v", "error", "-i", path, "-map", "0:v:0",
        "-vf", f"scale={w}:{h}", "-f", "image2pipe", "-pix_fmt", "rgb24",
        "-vcodec", "rawvideo", "-"], capture_output=True).stdout
    fb = w*h*3
    n = len(out)//fb
    return np.frombuffer(out, dtype=np.uint8)[:n*fb].reshape(n, h, w, 3)
```

FFmpeg writes raw RGB frames to a pipe; NumPy reinterprets the byte buffer as an array of shape `(n, 252, 448, 3)`. There is no temporary image file and no image decoding library. `--map 0:v:0` means the audio is never even sent down the pipe.

For each window, 8 frames are selected uniformly:

```
sel = np.flatnonzero((ftimes >= s) & (ftimes < e))
pick = sel[np.linspace(0, len(sel) - 1, AR.FRAMES_PER_WINDOW, dtype=int)]
```

23.6 Running the model

```
model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
    MODEL_ID, torch_dtype=torch.bfloat16, device_map=None).to("mps").eval()
...
msgs = [{"role": "user", "content": [{"type": "video", "video": imgs},
                                     {"type": "text", "text": AR.PROMPT}]]
text = proc.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True)
inputs = proc(text=[text], videos=[imgs], return_tensors="pt").to("mps")
with torch.inference_mode():
    out = model.generate(*inputs, max_new_tokens=AR.MAX_NEW_TOKENS,
                        do_sample=False, temperature=None, top_p=None, top_k=None)
gen = out[:, inputs.input_ids.shape[1]:]
raw = proc.batch_decode(gen, skip_special_tokens=True)[0].strip()
```

Detail	Why
<code>torch_dtype=torch.bfloat16</code>	Halves weight memory
<code>device_map=None then .to("mps")</code>	Explicit placement; "auto" would fall back to CPU
<code>.eval()</code>	Disables dropout and other training-time behaviour
<code>do_sample=False</code>	Greedy decoding. The same video always gives the same labels. Determinism matters because the labels feed a cache key
<code>temperature=None, top_p=None, top_k=None</code>	Explicitly cleared so no sampling parameter is silently applied
<code>out[:, inputs.input_ids.shape[1]:]</code>	Slices off the prompt tokens, keeping only what was generated
Model loaded once , reused for every window	Loading takes 17.2 s; doing it per window would dominate
<code>gc.collect(); torch.mps.empty_cache()</code> after every window	Keeps peak memory flat across a long clip

23.7 The memory guard

```
vm = psutil.virtual_memory()
if vm.available / 1e9 < a.min_avail_gb:
    raise MemoryError(f"available RAM {vm.available/1e9:.2f} GB below "
                     f"{a.min_avail_gb} GB guard at window {i}")
```

`MIN_AVAILABLE_GB = 1.5`. If free memory drops below that, the runner aborts cleanly with a specific message rather than letting the machine swap itself to a halt. The caller converts it into a readable user-facing error:

"Action recognition stopped because the machine ran low on memory. Close other applications and try again."

EVIDENCE IN THE PROJECT: This guard has actually fired

Three of the nine failed jobs in `data/jobs/` failed with exactly that message: `78f5e52e8980, 8391f3f5b3c6` and `98333984b713`. The guard is not theoretical.

The standalone implementation goes further: a background thread samples memory every 50 ms, and a `MemoryGuard(StoppingCriteria)` is passed to `model.generate()` so that generation can be stopped **mid-token** if memory is breached.

23.8 Parsing the response

```
def parse_response(text):
    m = re.search(r"ACTION\s*:\s*(.+)", text, re.I)
    if m: action = m.group(1).splitlines()[0].strip().strip('.')
    m = re.search(r"EVIDENCE\s*:\s*(.+)", text, re.I | re.S)
    if m: evidence = " ".join(m.group(1).split()).strip().strip('')
    if not action:
        # fallback: first non-empty line
        for ln in text.splitlines():
            if ln.strip(): action = ln.strip().strip('.'); break
    return action, evidence
```

Case-insensitive, tolerant of extra whitespace, takes only the first line of `ACTION:`, and falls back to the first non-empty line if the model ignored the format entirely.

23.9 Merging windows into spans

Because the stride is half the window, consecutive windows overlap. Merging is done by **action head**, not by a confidence threshold.

```
def action_head(phrase):
    """Primary '-ing' verb = the action head. Falls back to first content word."""
    cw = content_words(phrase)
    for t in cw:
        if t.endswith("ing") and len(t) > 4:
            return t
    return cw[0] if cw else ""

def same_action(a, b):
    ha, hb = action_head(a), action_head(b)
    if ha and hb:
        return ha == hb
    A, B = set(content_words(a)), set(content_words(b))
    return len(A & B) / min(len(A), len(B)) >= 0.5
```

Consecutive windows whose heads match are merged into one span. The representative label is the most common variant, tie-broken by first occurrence.

KEY POINT: Why head matching rather than a confidence threshold

The source comment says it: *"Merging uses ACTION-HEAD matching (the '-ing' verb), not numeric confidence thresholds, so it stays transparent and explainable."*

A threshold is a magic number that has to be tuned and cannot be explained. Head matching is a rule you can state in one sentence and check by eye. When a merge looks wrong, you can see exactly why.

23.10 The reference clip, end to end

RAW WINDOW PREDICTIONS		MERGED SPANS (overlapping)
0.00 - 2.00	stand	0.00 - 2.00 stand
1.00 - 3.00	walk around table	1.00 - 3.00 walk around table
2.00 - 4.00	pick up the cup	2.00 - 6.00 pick up cup
3.00 - 5.00	pick up cup	5.00 - 9.00 drink from cup
4.00 - 6.00	pick up cup	8.00 - 10.00 place cup on table
5.00 - 7.00	drink from cup	
6.00 - 8.00	drink from cup	note: 'pick up cup' 2.0-6.0 and
7.00 - 9.00	drink from cup	'drink from cup' 5.0-9.0 OVERLAP
8.00 - 10.00	place cup on table	by exactly one stride

Figure 23.1 - Nine windows become five spans. Source: 03-FoleyCrafter-Test/action-recognition/results/module2_action_timeline.txt.

23.11 Recorded performance

Metric	Value
Model load	17.22 s
Frame extraction	0.14 s
Total inference, 9 windows	47.10 s
Average per window	5.23 s
Total wall time	68.07 s
Peak memory used	12.00 GB
Minimum available memory	2.35 GB
Swap growth	+1.36 GB (5.13 baseline to 6.49 peak)
Guard breach	none

Table 23.1 - From `module2_action_segments.json`.

Chapter 24 — Timeline resolution

24.1 The problem: overlapping spans are ambiguous

The merged spans overlap by one stride. On the reference clip, `pick up cup` runs 2.0-6.0 s and `drink from cup` runs 5.0-9.0 s, so the second between 5.0 and 6.0 belongs to both.

KEY POINT: Why that is fatal for audio placement

An interval that belongs to two actions gives no answer to the question "which sound plays here?". Sound has to be placed somewhere definite. Ambiguity is acceptable in a label; it is not acceptable in a mix.

24.2 Midpoint boundary resolution

```
def resolve_boundaries(segments):
    """Midpoint boundary resolution. Returns new list; inputs are not mutated."""
    segs = [dict(s) for s in sorted(segments, key=lambda s: (s["start"], s["end"]))]
    adjustments = []
    for i in range(len(segs) - 1):
        cur, nxt = segs[i], segs[i + 1]
        if cur["end"] > nxt["start"] + EPS:
            # overlap
            mid = round((cur["end"] + nxt["start"]) / 2.0, 3)
            adjustments.append({...})
            cur["end"] = mid
            nxt["start"] = mid
    return segs, adjustments
```

Sort chronologically, walk left to right, and where two consecutive spans overlap, set the shared boundary to the midpoint of the overlap.

Property	Why it matters
Deterministic	Strictly left to right, so each boundary is decided once
Non-destructive	Inputs are copied, not mutated. The raw window predictions and merged spans are preserved as an audit record
Never invents or deletes	The number of segments is preserved, and the code asserts it
Every adjustment is logged	The pair of actions, the overlap, its duration, the chosen boundary, and both original values

24.3 Suspect flagging

A segment supported by only one window is marked `suspect` rather than `deleted`.

```
MIN_SUPPORT_CONFIRMED = 2      # >=2 supporting windows -> "confirmed"

if support < MIN_SUPPORT_CONFIRMED:
    reasons.append(f"single_window_support ({support} window)")
if i == 0:
    reasons.append("first_segment (window sees pre-action framing)")
if i == n - 1:
    reasons.append("last_segment (window sees post-action framing)")
status = "confirmed" if support >= MIN_SUPPORT_CONFIRMED else "suspect"
```

KEY POINT: The policy, quoted from the source

"Conservative - never deletes. A segment is marked 'suspect' when its evidence is thin. It is ALWAYS preserved, with the reasons recorded, so a human can review. No confidence threshold ever removes an action."

The first and last segments are always flagged, because the first window sees pre-action framing and the last window sees post-action framing. That is a systematic property of sliding windows, not a property of the video, and naming it is good engineering.

In the user interface, `confirmed` surfaces as **High** confidence and `suspect` as **Medium**.

24.4 The resolved timeline for the reference clip

Action	Start	End	Duration	Support	Status	UI confidence
stand	0.00	1.50	1.50 s	1 window	suspect	Medium
walk around table	1.50	2.50	1.00 s	1 window	suspect	Medium
pick up cup	2.50	5.50	3.00 s	3 windows	confirmed	High
drink from cup	5.50	8.50	3.00 s	3 windows	confirmed	High
place cup on table	8.50	10.00	1.50 s	1 window	suspect	Medium

Table 24.1 - The validated timeline. This is the array Module 3 consumes.

24.5 Validation

```
def validate(resolved):
    checks = {}
    checks["chronological"] = all(r[i]["start"] <= r[i+1]["start"] + EPS ...)
    checks["non_overlapping"] = all(r[i]["end"] <= r[i+1]["start"] + EPS ...)
    checks["no_negative_duration"] = all(s["end"] >= s["start"] - EPS for s in resolved)
    checks["count_preserved"] = None # filled by the caller
    return checks
```

The recorded result for the reference clip is all six checks passing, including `raw_windows_unmodified` and `merged_segments_unmodified` verified by SHA-256 fingerprint.

24.6 The bug that this stage caused, and how it was fixed

CAUTION: A real bug worth telling an examiner about

`resolve_boundaries` returns a **tuple** (`segments`, `adjustments`). An early version of `run_module2.py` assigned that tuple to a single variable, so `resolved` became a tuple rather than a list, and every downstream consumer broke on the live path with `AttributeError: 'list' object has no attribute 'get'`.

You can still see the failure recorded in the job store: job 3805a3d282d4 in `data/jobs/` failed at the `action_recognition` stage with exactly that message.

The fix is now defended by a comment in the source:

```
# resolve_boundaries returns (segments, adjustments) - unpack exactly as the
# validated resolve_segments.main() does.
resolved_raw, adjustments = RS.resolve_boundaries(merged)
```

This is a good story to tell, because it shows a real integration bug found on a real run and fixed with a comment that stops it recurring.

24.7 Which array the pipeline uses, and why

The Module 2 payload contains two action arrays. `pipeline.py` picks deliberately:

```
resolved = m2.get("resolved_actions") or m2.get("actions") or []
```

`resolved_actions` first. The raw overlapping `actions` array is a fallback that should never be needed, and it is retained only as an audit record.

Chapter 25 — From an action phrase to a Foley prompt

25.1 What this layer does

Module 2 returns free text: walk around table, drink from cup, place bread in toaster, a dog barking. This layer must turn any such phrase into four things:

1. a **generation prompt** for the audio model,
2. a **negative prompt**,
3. a **synchronisation strategy** - how to find the audible instant in the video,
4. a **level target** and a **frame region** and a **selection rule**.

That bundle is the `FoleySpec` dataclass in `backend/services/prompt_map.py`.

```
@dataclass
class FoleySpec:
    key: str
    label: str
    prompt: str
    negative: str = ""
    strategy: str = "contact"      # footstep | hold | contact | continuous | none
    region: str = "table"         # feet | head | table | full
    selection: str = "event"      # steps | wet_segment | event | slice
    generic: bool = False        # fallback class: used only if nothing specific matches
    target_rms_dbfs: float = -34.0
    match: list[str] = field(default_factory=list)
```

25.2 The resolution tiers

```
action phrase (free text from Qwen)
|
v
TIER 1  SPECIFIC CURATED CLASSES      17 hand-tuned classes, each keyword names
|      e.g. "place cup" -> cup_placement    its object. Longest keyword wins.
|
v (no match)
TIER 2  GENERIC CURATED CLASSES      object_placement, object_pickup
|      e.g. "put on table"                tried ONLY after specific ones
|
v (no match)
TIER 3  DELIBERATE SILENCES           standing, reaching, holding, looking,
|      -> return (None, reason)           waiting, idle, moving hand, unknown
|
v (no match)
TIER 4  VERB + OBJECT FALLBACK        an action verb plus any object noun
|      e.g. "place bread in toaster" -> object_placement
|
v (no match)
TIER 5  OPEN-VOCABULARY SYNTHESIS    prompt_synthesis.synthesise() writes a
|      e.g. "kicking a football"         prompt and picks a strategy from the
|      -> impact_football                verb's ARCHETYPE. Never returns None.
```

Figure 25.1 - Five tiers. The ordering is load-bearing; Section 25.5 explains why.

25.3 The 17 curated classes

Key	Label	Strategy	Region	Selection	Target RMS
walking	Walking	footstep	feet	steps	-34.0 dBFS
running	Running	footstep	feet	steps	-32.0 dBFS
drinking	Drinking	hold	head	wet_segment	-38.0 dBFS
cup_placement	Cup placement	contact	table	event	-32.0 dBFS
cup_pickup	Cup pickup	contact	table	event	-32.0 dBFS
stirring	Stirring	continuous	table	slice	-34.0 dBFS
spoon_placement	Spoon placement	contact	table	event	-34.0 dBFS
spoon_pickup	Spoon pickup	contact	table	event	-34.0 dBFS
object_placemen t	Object placement (generic)	contact	table	event	-33.0 dBFS
object_pickup	Object pickup (generic)	contact	table	event	-33.0 dBFS
button_press	Button / lever press	contact	table	event	-33.0 dBFS
door_opening	Door opening	contact	full	event	-32.0 dBFS
door_closing	Door closing	contact	full	event	-32.0 dBFS
sitting	Sitting down	contact	full	event	-36.0 dBFS
clapping	Clapping	footstep	full	steps	-30.0 dBFS
typing	Typing	continuous	full	slice	-34.0 dBFS
pouring	Pouring liquid	continuous	table	slice	-34.0 dBFS

Table 25.1 - Counted directly from ACTION_PROMPT_MAP in prompt_map.py.

CAUTION: A documentation discrepancy you should know about

The project's README, docs/pipeline.md and the research paper all say **16** curated classes. The code now contains **17**: `button_press` was added later and the prose was not updated.

If an examiner has read your report and asks about 16, the right answer is: "the written documentation says 16 and the code now has 17 - `button_press` was added after the report was written. Let me show you ACTION_PROMPT_MAP." That is a far better answer than either repeating a stale number or pretending you did not notice.

25.4 An example prompt, in full

close-up realistic Foley recording of natural human footsteps walking on a hard wooden floor, clearly audible alternating left and right footsteps with realistic heel and toe impacts, natural walking rhythm and slight variation between steps, subtle shoe contact and floor resonance, isolated dry Foley recording, no speech, no music, no ambience, no cinematic sound design

Notice the structure, which is consistent across every class:

Prompt element	Purpose
"close-up realistic Foley recording of"	Sets the recording perspective. Foley is close-miked, not room-miked
"natural human footsteps walking on a hard wooden floor"	The literal event and the material
"clearly audible alternating left and right footsteps"	Asks for the temporal structure - a sequence, not one thud

Prompt element	Purpose
"realistic heel and toe impacts"	Asks for the internal structure of each event
"natural walking rhythm and slight variation between steps"	Asks against mechanical repetition
"isolated dry Foley recording"	No reverb, no room
"no speech, no music, no ambience"	Inline negations, in addition to the negative prompt

25.5 Keyword matching, and the bug that shaped it

```
def _match(pool):
    best, best_len = None, 0
    for spec in pool:
        for kw in spec.match:
            kw_words = [w for w in _tokens(kw) if w not in fillers]
            hit = all(any(kwd == w or stem(kwd) == stem(w) or w.startswith(kwd)
                        for w in stems) for kwd in kw_words)
            if hit and len(kw) > best_len:
                best, best_len = spec, len(kw)
    return best

specific = [s for s in ACTION_PROMPT_MAP.values() if not s.generic]
generic = [s for s in ACTION_PROMPT_MAP.values() if s.generic]
best = _match(specific) or _match(generic)
```

Matching is **token-based**, not substring-based: a keyword matches when *all* of its words appear in the phrase, after filler words are stripped and both sides are stemmed. So `open door` still matches `opening the door`.

CAUTION: The mis-mapping bug, and the two rules that fixed it

The original keyword lists contained bare verbs: `place`, `pick`. The result was that **"place spoon on table" matched the CUP class and produced ceramic-mug Foley for a metal spoon**.

Two rules fixed it, and both are in the source as comments:

Rule 1 - every keyword names its object. There is no bare `place` or `pick` anywhere in `ACTION_PROMPT_MAP`. The comment above `cup_placement.match` says: *"NOTE: no bare 'place'/'put' keyword. A bare verb matched 'place spoon on table' to this class and produced ceramic-mug Foley for a metal spoon. Every keyword names the object."*

Rule 2 - specificity beats keyword length. Specific classes are searched first, and generic ones only if nothing specific matched. Without this, the generic keyword `place on table` (14 characters) would beat the specific `place cup` (9 characters) purely on length.

The test suite defends both rules permanently:

```
@test("object-specific classes beat the generic fallback (specificity over length)")
def _():
    for phrase, key in {"place cup on table": "cup_placement",
                       "place spoon on table": "spoon_placement",
                       "pick up the cup": "cup_pickup",
                       "pick up the spoon": "spoon_pickup"}.items():
        sp, _ = resolve(phrase)
        assert sp and sp.key == key
```

25.6 Deliberate silences

```
SILENT_ACTIONS: dict[str, str] = {
    "standing": "Standing still produces no Foley event.",
    "looking": "No physical contact event.",
    "waiting": "No physical contact event.",
    "idle": "No physical contact event.",
    "move hand": "Hand movement alone produces no audible Foley.",
    "reach": "Reaching toward an object produces no audible contact.",
    "hold": "Holding an object still produces no Foley event.",
    "unknown": "Action not recognised confidently enough to select a Foley class.",
    ...
}
```

KEY POINT: Silent is not the same as unsupported

These actions return `(None, reason)` with a *specific* reason. That distinction matters in the interface: "standing produces no Foley event" is a correct outcome, whereas "no Foley class is defined for this action" would be a gap in the system. The test suite asserts that the two never get confused:

```
assert "no foley class is defined" not in r.lower(), \
    f"'{phrase}' should be a known silent action, not an unknown one: {r}"
```

25.7 The verb fallback, and the hole it closed

Rule 1 above - every keyword names its object - created a new problem. The phrase `place bread in toaster` is unmistakably a placement, but it contains none of the literal surfaces the generic keywords ask for: no "table", no "down", no "object". It therefore resolved to nothing, and **the whole job failed before the sound model was ever called**.

EVIDENCE IN THE PROJECT: The failure is recorded in the job store

Four failed jobs in `data/jobs/` carry the message: *"Action recognition completed, but no supported Foley action was found. The original video can still be exported without generated audio."* - jobs 929eebd61570, 9db3e502d3f3, 6a0be3048f3d and bc05ab49d718.

The fix is a final verb-plus-object tier:

```
_PLACE_VERBS = {"place", "put", "set", "drop", "insert", "load", "lower", "deposit",
                "stack", "rest", "slide"}
_PICKUP_VERBS = {"pick", "lift", "take", "grab", "remove", "pull", "retrieve", "raise"}
_PRESS_VERBS = {"press", "push", "click", "flip", "toggle", "switch", "tap", "punch"}

obj = [w for w in words
        if w not in _ALL_ACTION_VERBS and stem(w) not in _ALL_ACTION_VERBS]
if obj:
    for verbs, key in ((_PLACE_VERBS, "object_placement"),
                      (_PICKUP_VERBS, "object_pickup"),
                      (_PRESS_VERBS, "button_press")):
        if stems & verbs:
            return ACTION_PROMPT_MAP[key], None
```

An action verb plus **any object noun that is not itself a verb** resolves to the matching generic class. The object check matters: without it, "pressing" would count as its own object and a bare verb would resolve.

KEY POINT: Why this tier runs LAST

The source comment is explicit: *"It runs LAST, so specific classes and deliberate silences still win, which is what keeps the original bug fixed."*

If the verb fallback ran first, `place spoon on table` would resolve to `object_placement` instead of `spoon_placement`, and the mis-mapping bug would be back in a different form.

25.8 Open-vocabulary prompt synthesis

The final tier removes the ceiling entirely. `backend/services/prompt_synthesis.py` writes a prompt for **any** phrase.

The insight is that the four synchronisation properties follow from the **archetype of the verb**, not from the object:

KEY POINT: The idea in one sentence

"'Kick', 'slam' and 'drop' are all single transients regardless of what is kicked, slammed or dropped."

Archetype	Strategy	Region	Selection	Level	Example verbs
locomotion	footstep	feet	steps	-34.0	walk, run, jog, march, climb, stomp
impact	contact	full	event	-31.0	kick, hit, slam, drop, throw, chop, clap
placement	contact	table	event	-33.0	place, put, set, insert, stack, lay
pickup	contact	table	event	-33.0	pick, lift, take, grab, retrieve
mechanism	contact	full	event	-32.0	open, close, press, flip, lock, zip, crank
friction	continuous	full	slice	-34.0	stir, rub, sweep, type, write, ride, fold
liquid	continuous	table	slice	-34.0	pour, splash, drip, fill, rinse, spray
oral	hold	head	wet_segment	-38.0	drink, sip, eat, chew, swallow, slurp
ambient	continuous	full	slice	-36.0	bark, chirp, rain, crackle, hum, buzz

Table 25.2 - The nine archetypes, from `ARCHETYPE_SYNC` in `prompt_synthesis.py`.

Region hints override the archetype default. Certain object nouns pull the motion measurement to a specific band whatever the verb suggests: `football`, `shoe`, `pedal`, `stair` force the feet band; `mouth`, `mug`, `straw`, `sandwich` force the head band; `table`, `desk`, `keyboard`, `chopping` force the table band.

The unrecognised-verb default is deliberate:

KEY POINT: A quietly excellent design decision

"An unrecognised verb defaults to a gentle continuous texture rather than a sharp transient, because a misplaced texture is far less jarring than a misplaced impact, and we are guessing by definition."

When you do not know, fail toward the least damaging option. A continuous texture placed slightly wrong sounds like ambience. An impact placed slightly wrong sounds broken.

Stemming is done by generating candidates, not by one rule:

```
def _stem_candidates(w):
    """A single strip rule cannot cover English -ing: 'tapping' needs the doubled
    consonant removed, 'writing' needs an 'e' restored, 'kicking' needs neither.
    Guessing one rule mis-stems the others ('opening' -> 'ope'), so instead we
    generate all the candidates and let the verb lexicon decide which is real."""
```

Cache keys are archetype plus object, not the raw phrase:

```
slug = re.sub(r"^[a-z]+", "_", f"{archetype}_{obj or 'generic'}").strip("_")
```

So kick the ball and kicking a football both become `impact_football` and share one generation, instead of paying twice.

EVIDENCE IN THE PROJECT: Open-vocabulary classes actually produced in real jobs

Read the filenames in `data/generated/` and `results/`: `impact_football`, `impact_vegetables`, `friction_notebook`, `ambient_toast_bread`, `ambient_dog`. None of those has a curated class; all of them were sounded by synthesis. The corresponding job records are in `data/jobs/`, including a toaster video whose four actions produced `object_placement`, `button_press` and `ambient_toast_bread`.

Chapter 26 — MOSS-SoundEffect v2.0

26.1 The model

Property	Value
Repository	OpenMOSS-Team/MOSS-SoundEffect-v2.0
Model revision	e35df4d82f8e87fcd5d14e5d100e349c0c3c076d
Source repository	github.com/OpenMOSS/MOSS-TTS, commit 58b20a0
Licence	Apache-2.0 - commercial use permitted
Checkpoint size	11.23 GB (10 GB on disk as installed)
Conditioning	text only - it accepts no video, image or audio conditioning
Output	48 kHz mono, up to 30 seconds
Architecture	Diffusion Transformer + flow matching, Qwen3 text encoder, DAC VAE

26.2 The three components

Component	Parameters	Role	Resident size (bfloat16)
Qwen3 text encoder	1,720.57 M	Encodes the prompt to a (1, 512, 2048) context tensor	3.44 GB
Diffusion Transformer (DiT)	1,416.05 M	The denoiser, trained with a flow-matching objective	2.85 GB
DAC VAE	371.59 M	Decodes the latent to a 48 kHz waveform	0.74 GB
Total	3,508.21 M		7.03 GB if co-resident

26.3 Duration handling, and the parameter that was added

The pipeline denoises a fixed-size latent and crops the result.

EVIDENCE IN THE PROJECT: The exact numbers, from your own generation records

- `denoised_seconds_internally: 30`
- `latent_shape: [1, 128, 1500]`
- `num_samples: 1440000 (30 s at 48 kHz)`
- `decoded_shape: [1, 1, 1440000]`
- `cropped_samples: 480000 (10 s)`

Duration is communicated to the model **as text**: the string " `duration: 10.0s`" is appended to the prompt, matching the training-time convention. You can see both forms in every generation record: `prompt` and `prompt_sent_to_model`.

The consequence is that a short request costs the same as a long one. That is why `--full-seconds` was later exposed as a command-line parameter, so the denoised window could be shortened for speed experiments. The default stays 30, and Chapter 33 explains in detail why the shortcut was rejected.

26.4 Phased inference: the memory problem and its solution

The problem. Loading all three components in their as-shipped precision requires about **10.59 GB of resident weights**, because the pipeline's `torch_dtype` argument is honoured only by the text encoder - the DiT and the VAE load in float32. On a 17.18 GB machine that produced approximately **+9.8 GB of swap growth during loading alone**.

The solution. Split `MossSoundEffectPipeline.__call__` into three phases, each of which loads its component, uses it, and releases it before the next one loads.

```

PHASE 1  Qwen3 text encoder  (3.44 GB)
|       encode positive prompt -> ctx_posi  (1, 512, 2048)
|       encode negative prompt -> ctx_neg  (1, 512, 2048)
|       move both to CPU, delete the encoder, gc, torch.mps.empty_cache()
|       ASSERT live_instances(Qwen3TextEncoder)["Qwen3TextEncoder"] == 0
v
PHASE 2  MOSS DiT  (2.85 GB)
|       50 Euler steps of flow matching, CFG 4.0
|       each step: TWO forward passes (positive and negative context)
|       -> latent (1, 128, 1500)
|       move latent to CPU, delete the DiT, sweep
|       ASSERT live_instances(WanAudioModel)["WanAudioModel"] == 0
v
PHASE 3  DAC VAE  (0.74 GB)
|       decode latent -> waveform (1, 1, 1440000)
|       crop to 480000 samples, write PCM_16 WAV at 48 kHz
v
DONE     peak resident ~12 GB instead of ~17 GB; swap growth +0.00 to +0.01 GB

```

Figure 26.1 - Phased inference. The residency assertions are real `assert` statements in `moss_generate.py`, not comments.

EVIDENCE IN THE PROJECT: The measured effect

Metric	All-resident (as shipped)	Phased + bfloat16
Swap growth	+9.80 GB	+0.01 GB
Peak RAM	-	11.68 GB
Minimum available RAM	-	1.85 GB
Memory guard breach	-	none

Across 31 recorded generations the guard was never breached, and in the worst run available memory fell to **1.58 GB against a 1.50 GB threshold** - a margin of 80 MB. Without phase separation the run would simply fail.

How residency is proved rather than assumed:

```

def live_instances(*classes) -> dict:
    """Count reachable instances - residency proof after deletion."""
    return {c.__name__: sum(1 for o in gc.get_objects() if type(o) is c) for c in classes}

```

This walks the garbage collector's object graph and counts live instances of the class. `assert resid["Qwen3TextEncoder"] == 0` then *proves* the encoder is gone, rather than hoping that `del` was enough.

26.5 The two MPS compatibility fixes

Both live in wrappers. **No file inside the MOSS repository is modified**, and the build asserts `git status --porcelain` returns empty inside `moss/MOSS-TTS`.

Fix 1: the float64 timestep embedding.

`sinusoidal_embedding_1d` computes in `float64`. MPS has no `float64` support and raises:

```
TypeError: Cannot convert a MPS Tensor to float64 dtype as the MPS framework
doesn't support float64. Please use float32 instead.
```

The shim runs the identical `float64` arithmetic on CPU and moves the result back:

```
def sinusoidal_embedding_1d_cpu_f64(dim, position):
    device, out_dtype = position.device, position.dtype
    # two steps: move to CPU first, THEN cast - a fused .to("cpu", float64)
    # still attempts the float64 conversion on the MPS device and raises.
    pos = position.detach().cpu().to(torch.float64)
    freqs = torch.pow(torch.tensor(10000.0, dtype=torch.float64),
                      -torch.arange(dim // 2, dtype=torch.float64).div(dim // 2))
    sinusoid = torch.outer(pos, freqs)
    x = torch.cat([torch.cos(sinusoid), torch.sin(sinusoid)], dim=1)
    return x.to(device=device, dtype=out_dtype)
```

Note the two-step move, which is itself a documented trap: a fused `.to("cpu", torch.float64)` still attempts the `float64` conversion *on the MPS device* and raises.

The shim is patched into three modules at runtime and **verified numerically**:

```
"mps_compat": {"sinusoidal_embedding_1d": {
    "max_abs_diff_vs_upstream_cpu": 0.0,
    "exact_match": true,
    "shape": [1, 256]},
    "patched_modules": [
        "moss_soundeffect_v2.diffsynth.models.wan_video_dit",
        "moss_soundeffect_v2.diffsynth.models.wan_audio_dit",
        "moss_soundeffect_v2.diffsynth.pipelines.wan_audio"]}
```

That block appears in every single generation record. The fix is not merely applied; it is proved bit-exact against upstream on every run.

Fix 2: the complex rotary buffers.

The DiT carries three `complex128` RoPE tables. MPS has no `ComplexDouble` support, and a blanket `.to(dtype=bfloat16)` would discard the imaginary part entirely. The wrapper downcasts them to `complex64` and casts **parameters only**:

```
def cast_params_only(module, device, dtype):
    converted = []
    if device == "mps":
        for bname, buf in list(module.named_buffers()):
            if buf.dtype == torch.complex128:
                owner, parts = module, bname.split(".")
                for part in parts[:-1]:
                    owner = getattr(owner, part)
                setattr(owner, parts[-1], buf.to(torch.complex64))
                converted.append(bname)
    module.to(device=device)
    for p in module.parameters():
        p.data = p.data.to(dtype)
    module._complex128_downcast = converted
    return module
```

The downcast is **lossless in use**, and the reason is precise: `rope_apply` only reads `freqs.real` and `freqs.imag` and casts both to `float32`, so `complex64` reproduces them bit for bit. The generation record carries the proof:

```
"phase2_rope": {"downcast_complex128_to_complex64":
    ["freqs_cis_0", "freqs_cis_1", "freqs_cis_2"],
    "lossless": "rope_apply reads .real/.imag at float32"}
```

And the wrapper asserts the result:

```
assert d["buffer_dtypes"] == ["torch.complex64"], \
    f"DiT RoPE buffers unexpected: {d['buffer_dtypes']}"
```

PROFESSOR MAY ASK: Why did you not just fork the model and patch it?

"Because then I could not prove my results come from the published model. Every fix lives in a wrapper outside the repository, and the build asserts that `git status --porcelain` inside `moos/MOSS-TTS` returns empty. It also means an upstream update does not conflict with my changes."

26.6 The validated settings

Setting	Value	Notes
Seed	42	Base seed; candidates use 42, 43, 44
Inference steps	50	Ablation in Chapter 33
CFG scale	4.0	Stable Audio uses 7.0, its own default
Sigma shift	5.0	Read from the checkpoint's <code>scheduler_config.json</code>
Denoised window	30 s	Ablation in Chapter 33 - do not shorten
Output duration	10.0 s	Cropped from the 30 s decode
Sample rate	48,000 Hz	Native
Channels	1 (mono)	Native
Precision	bfloat16 parameters	float16 explicitly forbidden
TORCHDYNAMO_DISABLE	"1"	The repository's install docs are CUDA-first and use <code>torch.compile</code> , which has no MPS path

The wrapper refuses to run with the wrong precision:

```
FORBIDDEN_DTYPES = (torch.float16,)
...
bad = [str(d) for d in (pdt | bdt) if d in FORBIDDEN_DTYPES]
assert not bad, f"{name}: forbidden dtype {bad}"
```

And the generation script asserts its whole environment before doing any work:

```
assert not OUT_WAV.exists(), f"refusing to overwrite {OUT_WAV}"
assert torch.backends.mps.is_available(), "MPS unavailable"
assert not torch.cuda.is_available(), "CUDA unexpectedly present"
assert M.DTYPE is torch.bfloat16
```

26.7 Recorded generation performance

Metric	Value
Phase 1, text encoding	4.7 s
Phase 2, diffusion, 50 steps	264.4 s (94 percent of the total)

Metric	Value
Phase 3, decode	8.6 s
Total per asset	280.9 s, about 4.7 minutes
Peak resident memory	12.11 GB average, 12.51 GB worst
Minimum available memory	2.35 GB average, 1.58 GB worst
Swap growth	0.00 to 0.01 GB
Guard breaches across 31 runs	zero

Table 26.1 - Means over 31 recorded runs at production settings. Source: `paper/experiments/exp6_latency.json`.

26.8 The content-addressed cache

Generation is the dominant cost, so nothing is ever generated twice.

```
def cache_key(spec: FoleySpec, settings: dict) -> str:
    payload = {"backend": _backend_name(settings),
               "action": spec.key, "prompt": spec.prompt, "negative": spec.negative,
               "seed": int(settings["seed"]), "steps": int(settings["steps"]),
               "cfg": round(float(settings["cfg_scale"]), 4),
               "sigma": round(float(settings["sigma_shift"]), 4),
               "seconds": round(float(settings["duration"]), 4),
               "sr": int(settings["sample_rate"]), "model": str(settings["model"])}
    return hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()[:16]
```

The file lands at `data/generated/<backend>_<action_key>_<16 hex chars>.wav`.

CAUTION: The type-sensitivity bug

Read the docstring:

"Numeric settings are normalised before hashing. Without this, a client sending `duration: 10 (int)` and a default of `10.0 (float)` serialise differently and produce different keys for byte-identical audio, causing needless regeneration."

`json.dumps({"seconds": 10})` gives `{"seconds": 10}` and `json.dumps({"seconds": 10.0})` gives `{"seconds": 10.0}` - different strings, different SHA-256, different cache entry, and a wasted 4.7 minutes.

Every numeric is coerced with `int()` or `round(float(), 4)` before hashing.

The backend name is part of the key and the filename, so the two Foley models can never collide, and the test suite asserts that too.

26.9 Multi-candidate generation

```
def generate_best(spec, settings, max_candidates=3, on_progress=None, timeout_s=3600):
    attempts, best = [], None
    base = int(settings["seed"])
    for i in range(max(1, max_candidates)):
        st = {**settings, "seed": base + i}
        path, cached = generate(spec, st, timeout_s=timeout_s)
        v = FV.validate(path, spec.target_rms_dbfs, int(st["sample_rate"]))
        rec = {"candidate": i+1, "seed": st["seed"], "path": str(path),
              "cached": cached, "ok": v.ok, "score": v.score,
              "reason": v.reason, "failures": v.failures, "metrics": v.metrics.dict()}
        attempts.append(rec)
        if v.ok and (best is None or v.score > best["score"]):
            best = rec
        if v.ok and v.score >= FV.GOOD_ENOUGH_SCORE:
```

```

        break
    return best, attempts
# cost control: stop as soon as one is good enough

```

Property	Detail
Why it exists	MOSS occasionally collapses to degenerate output for a given seed. That is a sampling failure, not a capability limit , so trying another seed is the cheapest available remedy
Cost control	Candidates are generated one at a time, and the loop stops as soon as one scores at least <code>GOOD_ENOUGH_SCORE = 45</code> . A class that works on the first seed costs exactly one generation
Selection	The best-scoring passing candidate wins, not merely the first one that passed
Caching	Each candidate is cached under its own key, so nothing is ever generated twice
Full audit	Every attempt - seed, verdict, score, reason and all metrics - is recorded in the job report

Chapter 27 — The Foley quality gate

27.1 Why generated audio cannot be trusted

SIMPLE EXPLANATION: The problem in one paragraph

Sometimes the sound model returns a file that is technically valid - correct sample rate, correct length, no errors - but contains essentially nothing: a near-silent, near-constant hum. If you then run your normal "make this the right loudness" step, it faithfully tries to raise that nothing to target volume, applies forty-something decibels of gain, and what you hear is amplified digital noise. The system has to notice that the file is empty *before* it tries to make it louder.

Technically, an active-RMS leveller has no concept of "there is no signal here". It computes the gain that would bring the measured level to the target and applies it. If the raw material is quantisation noise, the output is amplified quantisation noise, which is audible hiss.

KEY POINT: The design principle

Measure the asset **raw** - before any gain, before any normalisation - and refuse it if the measurement says it is unusable. Never repair; refuse.

27.2 The six gates

backend/services/foley_validation.py.

#	Gate	Rejects when	Threshold constant
1	Effective bits	below 9.0, meaning a peak below about -42 dBFS	MIN_EFFECTIVE_BITS = 9.0
2	Dynamic range	below 6 dB, measured over signal-bearing frames only	MIN_DYNAMIC_RANGE_DB = 6.0
3	Sustained tone	harmonic ratio above 0.80 and dynamic range below 10 dB	TONAL_HARMONIC_RATIO = 0.80, TONAL_MAX_DYNAMIC_DB = 10.0
4	Pure tone	harmonic ratio above 0.90 and spectral flatness below 0.01	PURE_TONE_HARMONIC_RATIO = 0.90, PURE_TONE_MAX_FLATNESS = 0.01
5	Automatic gain	required make-up gain above +25 dB	MAX_AUTO_GAIN_DB = 25.0
6	Integrity	NaN or Inf, wrong sample rate, or duration below 0.05 s	MIN_DURATION_S = 0.05

Table 27.1 - The six gates. All six are evaluated; a file can fail several, and every failure is reported.

27.3 Why the gates are multi-criteria

EVIDENCE IN THE PROJECT: The observation that settled it

One cup-pickup candidate had **63.7 percent of its energy in the ceramic band** - more than the candidate eventually chosen - but only **1.3 dB of dynamic range**. It was continuous hiss, not a contact sound.

Judged on ceramic content alone, it would have won.

That is the whole argument for multi-criteria gating in one example: any single metric can be gamed by a degenerate signal that happens to score well on that axis.

27.4 Why gate 4 exists separately from gate 3

Gate 3 requires a tone to be **both** harmonic and dynamically flat. A musical tone with a natural envelope therefore slipped through.

EVIDENCE IN THE PROJECT: The 346 Hz sine wave

The Stable Audio backend produced an output for "cup placed on a table" that was a **pure 346 Hz sine wave** with a harmonic ratio of **0.997** and a spectral flatness of **0.00000** - and **32 dB of dynamic range**, because it had a natural attack and decay.

Gate 3 did not fire, because the dynamic range was not below 10 dB.

The source comment records the reasoning:

"A second, independent tonality test. The pair above only fires when a tone is ALSO flat in level, so a musical tone with a natural envelope slipped through... Foley is inharmonic; a near-pure tone is disqualifying on its own."

27.5 The quality score

Passing candidates are ranked 0 to 100 by four independent components.

```
def quality_score(m: FoleyMetrics) -> float:
    """Four independent components, so no single metric dominates:
        dynamic range (40) - Foley is impulsive; flat material scores nothing
        signal level (25) - headroom above the quantisation floor
        gain headroom (20) - the less make-up gain needed, the less noise is lifted
        non-tonality (15) - Foley is inharmonic; a tone is not a contact sound
    """
    def band(v, lo, hi, pts):
        return float(np.clip((v - lo) / (hi - lo), 0.0, 1.0) * pts)
    return round(
        band(m.dynamic_range_db, 0.0, 40.0, 40.0)
        + band(m.effective_bits, 6.0, 14.0, 25.0)
        + band(MAX_AUTO_GAIN_DB - abs(m.required_gain_db), 0.0, MAX_AUTO_GAIN_DB, 20.0)
        + band(0.5 - min(m.harmonic_ratio, 0.5), 0.0, 0.5, 15.0), 1)
```

The score only ever ranks candidates that have already **passed** all six gates. It is a tie-breaker, not a gate.

GOOD_ENOUGH_SCORE = 45.0 is the early-stop threshold for candidate generation.

27.6 What happens to a rejected asset

```
generated WAV
|
v
measure RAW (no gain applied)
|
+---+-----+
|                                     |
PASS                                FAIL
|                                     |
v                                     v
score 0-100                          * never reaches the mixer
|                                     * interval marked no_usable_foley
v                                     * measured values + reason recorded in the report
eligible for the mix                  * file KEPT ON DISK for diagnostics
                                     * processing CONTINUES for every other action
                                     * if EVERY asset fails, the video is still produced
                                     with a silent track
```

Figure 27.1 - One unusable asset never fails the job.

27.7 The second, independent limit in the mixer

The gate is not the only defence. The mixer holds its own hard limit:

```
need_db = 20 * np.log10(max(g, 1e-12))
if need_db > MAX_AUTO_GAIN_DB:
    log["rejected"].append({... "stage": "mixer_gain_limit", ...})
    continue
```

KEY POINT: Refuse rather than clamp

The comment explains why this is a refusal and not a clamp: *"A clip needing more than MAX_AUTO_GAIN_DB has too little signal to level; applying the gain would amplify quantisation noise into audible hiss. Refuse it rather than clamping - clamping still admits noise."*

Clamping the gain at +25 dB would still put a +25 dB amplified noise floor into the mix. The only correct response is to leave the interval silent.

27.8 What the gate actually did, across the whole corpus

All 54 assets generated during development were re-measured with the production validator.

Statistic	Value
Assets measured	54
Rejected	16 (29.6 percent)
Rejected assets that would have needed more than +25 dB	11
Median required gain among those 11	+37.0 dB
Maximum required gain	+42.1 dB
Assets with a harmonic ratio above 0.80	10
MOSS assets	46, of which 12 rejected (26 percent)
Stable Audio assets	8, of which 4 rejected (50 percent)
Median quality score, MOSS	54.5
Median quality score, Stable Audio	53.2
Median harmonic ratio, MOSS	0.040
Median harmonic ratio, Stable Audio	0.898

Table 27.2 - Recomputed from `paper/experiments/exp2_gate.json`.

REMEMBER THIS

29.6 percent of everything generated was rejected. Applying +42 dB to a file whose peak is -62 dBFS does not produce a quiet contact sound; it produces amplified quantisation noise. That single sentence justifies the entire gate.

KEY POINT: The deepest result in the project

The two backends have **almost identical median scores** - 54.5 and 53.2 - while their median harmonic ratios differ by a **factor of 22**. The aggregate score does not separate them. The harmonic ratio separates them decisively.

That is the empirical case for multi-criteria gating: a single scalar, however carefully weighted, can be blind to the failure that matters.

Two assets at harmonic ratios of 0.880 and 0.868 **pass**, which shows the gate has a real boundary rather than rejecting everything remotely tonal.

Chapter 28 — Case study: the cup-pickup failure

An examiner will ask you about something that went wrong. This is the best answer you have, because it is fully documented, fully measured, and it ends in a genuinely interesting engineering decision.

28.1 Why cup pickup was attempted at all

The reference video shows a person walking to a table, picking up a cup, drinking twice, and putting the cup down. Module 2 labels `pick up cup` across 2.5 to 5.5 seconds - three seconds, the joint-longest action in the clip. Leaving it silent leaves a visible hole.

28.2 Attempt 1 - MOSS v1

Verdict: UNCERTAIN, leaning FAIL. Not approved.

Measurement	Value	Comparison
Peak	-36.0 dBFS	28.7 dB below the approved walking asset
RMS	-65.1 dBFS	-
Effective bits	10.0 of 16	walking: 14.8
Energy in 1-5 kHz (the ceramic band)	1.68 percent	drinking, same mug: 28.96 percent
Energy in 1-5 kHz, walking asset	1.63 percent	a wooden floor, containing no ceramic at all
Dynamic range	15.6 dB	-
Harmonic ratio	0.1635	walking: 0.0041
Events detected	27, mostly at -56 to -70 dBFS	only about 7 reached -35 to -44 dBFS

KEY POINT: The killer comparison

The cup pickup had no more ceramic-band content than the footsteps did. 1.68 percent against 1.63 percent, where the footsteps are a shoe on a wooden floor.

And the same model, prompted for drinking from the same ceramic mug, produced 28.96 percent. So the model *can* produce strong ceramic character. It simply did not here.

The report is also notable for correcting itself:

EVIDENCE IN THE PROJECT: A self-correction, recorded in the report

"My initial pass flagged per-event 'dominant frequencies' of 0/23/47 Hz and I suspected subsonic rumble. That was a measurement artefact - those were low STFT bins in a magnitude-weighted readout. The correct figures: only 13.0 percent of energy is below 20 Hz and 17.4 percent below 40 Hz, so 87 percent is audible, and high-passing at 40 Hz barely moves anything (peak -36.0 to -35.8 dBFS). This is not a rumble file."

Being able to point at a place where you found and corrected your own error is worth more in a viva than any result.

28.3 Attempt 2 - MOSS v2, with a longer negative prompt

Verdict: FAIL. 8 of 10 quality gates failed.

Measurement	v2	v1	drinking (approved)	walking (approved)
Peak dBFS	-61.7	-36.0	-23.5	-7.3
RMS dBFS	-68.7	-65.1	-55.1	-35.1
Effective bits	5.8	10.0	12.1	14.8
Peak in LSB (out of 32768)	27	520	2192	14144
Energy 1-5 kHz	1.50 %	1.68 %	28.96 %	1.63 %
Energy below 200 Hz	96.5 %	58.3 %	8.7 %	55.8 %
Dynamic range	1.1 dB	15.6 dB	29.2 dB	50.7 dB
Harmonic ratio	0.9501	0.1635	-	0.0041

Table 28.1 - From `results/cup_pickup_moss_v2_report.md`.

Why this is degenerate output rather than quiet Foley:

- Only **40 distinct sample values** in the entire ten-second file, spanning -27 to +12 out of plus or minus 32,768.
- Per-second RMS is flat: `0.000372, 0.000369, 0.000363, 0.000364, 0.000363, 0.000361, 0.000362, 0.000365, 0.000368, 0.000363` - **under 3 percent variation across all ten seconds**.
- Dynamic range 1.06 dB. Foley is impulsive; the approved walking asset measures 50.7 dB.
- Harmonic ratio 0.9501 - that is a *tone*, not an impact.
- It would need **+54.4 dB** of gain to match the walking asset.

EVIDENCE IN THE PROJECT: Honest reporting of a detector artefact

The report says: *"The 83 'events' my detector reported are all at -66.3 to -66.8 dBFS - an identical level, because it is picking ripples in a steady signal rather than discrete events. That count is an artefact and should not be read as structure."*

The analysis tool produced a number, and the report explains why that number is meaningless rather than quoting it as a result.

The hypothesis that was raised and deliberately not acted upon:

The negative prompt grew from 17 terms in v1 to 24 in v2. Output level tracked negative prompt length: 17 terms gave -36.0 dBFS, 24 terms gave -61.7 dBFS. With CFG 4.0 pushing away from 24 concepts simultaneously, the sampler may have been steered into a degenerate low-energy solution.

KEY POINT: The scientific posture to copy

"That is a pattern across two data points, not a finding. I am not acting on it, and I have not regenerated."

Two points do not establish a relationship. Recording the observation and explicitly refusing to act on it is exactly right, and it is a sentence worth reproducing in a viva.

28.4 Attempt 3 - extraction from an existing asset

Could the cup-pickup sound be cut out of the approved drinking asset, which contains real ceramic handling?

Verdict: NOT VIABLE. The single qualifying transient is a lip-contact tink, not a table-contact-and-lift sequence. Documented in `results/cup_pickup_extraction_inspection.md`.

28.5 The decision in the standalone build: silence

For the validated standalone Module 3 build, the interval 2.5 to 5.5 seconds contains **no audio at all**, and this is recorded explicitly:

```
UNAVAILABLE_FOLEY = {
  "pick up cup": "No approved Foley. Two MOSS generations (UNCERTAIN, FAIL) and an "
                "extraction study from the drinking asset (NOT VIABLE) were rejected. "
                "Left intentionally silent; not fabricated.",
}
```

An automated check asserts that no audio was written there: `"12_pickup_still_unavailable": {"pass": true, "detail": "silent, documented"}`.

KEY POINT: Why silence is better than a substitute

Three reasons, and you should be able to give all three.

Honesty. The system's claim is that it generates sound for what it sees. Substituting a library sound, or a sound from a different action, breaks that claim silently.

Diagnosis. A silent interval that is reported tells the user, and the developer, exactly where the system failed and why. A substituted sound hides the failure.

Perception. A wrong sound is worse than no sound. Silence where a viewer expects a faint noise is barely noticeable. A ceramic clink at the wrong moment, or a hiss instead of a contact, is immediately and obviously wrong.

28.6 The rescue in the web pipeline

The story does not end there, and the ending is the interesting part.

The web pipeline generates up to three candidates with successive seeds. On the same prompt, same model and same settings:

Seed	Verdict	Score	Peak	Dynamic range	Effective bits	Harmonic ratio	Gain needed
42	FAIL	0.0	-62.01 dBFS	1.13 dB	5.70	0.9452	+37.02 dB
43	FAIL	0.0	-18.55 dBFS	1.27 dB	12.92	0.0433	+28.90 dB
44	PASS	85.8	-8.34 dBFS	27.97 dB	14.61	0.0005	+2.72 dB

Table 28.2 - Read from `data/jobs/aa4b2f4e6049/report.json`, the current end-to-end run on the reference clip.

The failure reasons are recorded in full:

- Seed 42: `"effective bits 5.7 below 9.0 (peak -62.0 dBFS, almost no signal); dynamic range 1.1 dB below 6.0 dB (flat, not impulsive); harmonic ratio 0.95 above 0.8 with only 1.1 dB dynamic range (a sustained tone); harmonic ratio 0.95 with spectral flatness 0.0072, a near-pure musical tone, not a physical contact sound; would need +37.0 dB of make-up gain, above the 25.0 dB safety limit."`
- Seed 43: `"dynamic range 1.3 dB below 6.0 dB (flat, not impulsive); would need +28.9 dB`

of make-up gain, above the 25.0 dB safety limit."*

- Seed 44: *"passed all quality gates."*

REMEMBER THIS

Seed 44 scored **85.8 out of 100** with a harmonic ratio of **0.0005** and needed only **+2.72 dB** of gain. Same prompt, same model, same settings.

This was a sampling failure, not a capability limit. That single sentence is the whole justification for multi-candidate generation, and it is the payoff of the entire cup- pickup story.

28.7 How to tell this story in a viva

> "The hardest sound class was picking up a ceramic mug. My first two attempts both > failed, and I measured why rather than guessing. The second one was mathematically > near-empty: 40 distinct sample values in ten seconds, 1.06 dB of dynamic range, and 95 > percent harmonic content, which is a tone rather than an impact. For reference, the > approved drinking sound - recorded from the same ceramic mug by the same model - has > 28.96 percent of its energy in the 1 to 5 kHz ceramic band, and neither pickup attempt > exceeded 1.68 percent, which is indistinguishable from my footsteps asset, and that > contains no ceramic at all. > > So in the standalone build I left that interval silent and documented it, rather than > substituting something. An automated check asserts that no audio is written there. > > But the interesting part is what happened next. When I added multi-candidate generation > to the web pipeline, the same class failed on seeds 42 and 43 and passed on seed 44 with > a quality score of 85.8 and a harmonic ratio of 0.0005. Same prompt, same model, same > settings. It was a sampling failure, not a capability limit - and that is exactly why > the retry loop exists, and why refusing bad output is more useful than trying to repair > it."

Chapter 29 — Visual event localisation

This is the core technical contribution of the project.

29.1 The problem restated

SIMPLE EXPLANATION: Why a label is not a cue

An action recogniser tells you "walking, 1.5 to 2.5 seconds". But you do not hear walking for a second; you hear four separate footsteps, each at one instant. If you stretch a walking sound across the label you get audio that is present and obviously wrong, because the steps you hear do not coincide with the feet you see.

Two measured failure modes:

(a) The audible event is not at the interval start. A `place cup on table` interval spanning 8.5 to 10.0 seconds contains 1.5 seconds of arm movement and one instant of contact. Playing a contact sound at 8.5 s puts it approximately **1.3 seconds** before the mug touches the table.

(b) The label may not describe the whole interval it covers. Module 2 labels 0.0 to 1.5 seconds as `stand`, and flags it `suspect`. Frame measurement contradicts the label:

Interval	Module 2 label	Mean lower-body motion
0.0 - 1.5 s	stand	1.708
1.5 - 2.5 s	walk around table	1.711
2.5 - 5.5 s	pick up cup	0.954
5.5 - 8.5 s	drink from cup	0.145
8.5 - 10.0 s	place cup on table	0.446

Lower-body motion during the "stand" label is **statistically indistinguishable** from the labelled walk interval, and falls by 44 percent only when the subject stops at the table. The subject is walking from about 0.2 seconds, and **two of the four visible foot plants fall inside the "stand" label**.

KEY POINT: What the system does about it, and what it deliberately does not do

It widens the *search span* for footstep audio to 0.0 - 2.50 seconds. It does **not** modify the Module 2 timeline. The label stays wrong and is reported as wrong; only the region in which footstep audio may be placed is widened. That separation - fix the placement, do not silently rewrite the recognition output - is deliberate.

29.2 Frame motion in region bands

```
def load_frames(video: Path, fps: float = 24.0) -> np.ndarray:
    """Decode to (T, H, W) uint8 greyscale."""
    cmd = ["ffmpeg", "-v", "error", "-i", str(video),
           "-vf", f"fps={fps},scale={W}:{H}", "-pix_fmt", "gray",
           "-f", "rawvideo", "-"]
    raw = subprocess.run(cmd, capture_output=True, check=True).stdout
    n = len(raw) // (W * H)
    return np.frombuffer(raw[:n * W * H], np.uint8).reshape(n, H, W)

def motion(frames, y0=0.0, y1=1.0):
    """Per-frame mean |difference| within a horizontal band. Length T (first = 0)."""
    a, b = int(y0 * frames.shape[1]), int(y1 * frames.shape[1])
    band = frames[:, a:b, :].astype(np.float32)
```



```
d = np.abs(np.diff(band, axis=0)).mean(axis=(1, 2))
return np.concatenate([[0.0], d])
```

$w, h = 320, 180$, decoded at 24 fps. A 240-frame clip is 240 x 180 x 320 bytes, about 14 MB, which fits comfortably in memory. No OpenCV, no image decoding library, no temporary files.

The motion signal is the **mean absolute inter-frame difference** inside a horizontal band:

Band	Frame-height fraction	Used for
feet	0.62 - 1.00	foot plants
head	0.00 - 0.50	sip holds, mug at face height
table	0.40 - 0.85	mug-table contact
full	0.00 - 1.00	doors, sitting, whole-body events

KEY POINT: Why the band restriction is what makes this work

Measuring motion over the whole frame mixes a footstep into arm and head movement. The feet band responds almost exclusively to gait.

The exact fractions were chosen empirically. `visual_events.py` records that across three prominence thresholds the 0.62-1.00 band recovered **all four** foot plants, whereas a taller 0.55-1.00 band recovered only two at the default threshold.

29.3 The four detection strategies

The rule depends on the **physics of the action**, not on the object.

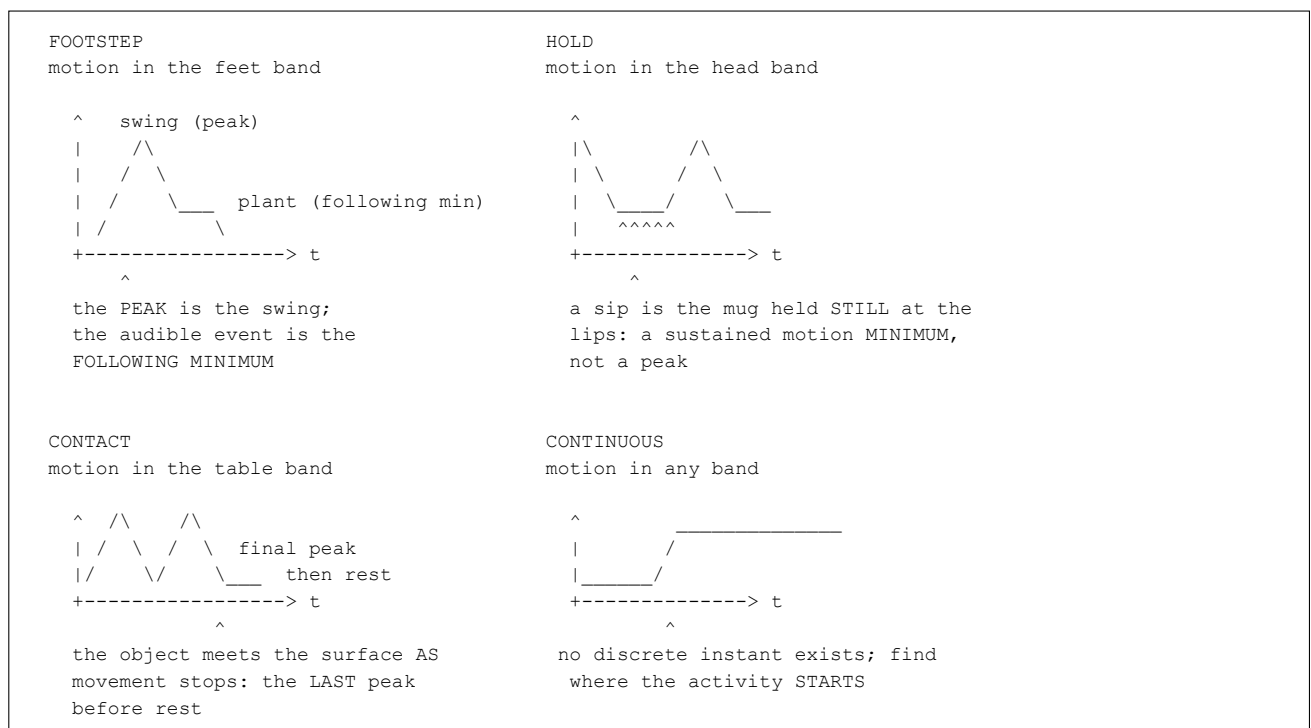


Figure 29.1 - Four strategies, four different physical stories.

Footstep.

```

prom = max(0.15 * (seg.max() - seg.min()), 0.15 * seg.std())
pk, _ = find_peaks(seg, prominence=prom, distance=max(1, int(0.25 * fps)))
for p in pk:
    j = p
    while j + 1 < len(seg) and seg[j + 1] <= seg[j]:
        j += 1
    tc = float(t[wdx[j]])
    if sa <= tc < sb:
        ev.append(VisualEvent(action, "foot_contact", tc, "high",
                               "motion peak resolved to following minimum (plant)"))

```

Prominence, not a plain threshold. A prominence-based peak finder measures how far a peak rises above the surrounding terrain. A simple height threshold admits low-amplitude ripples between real steps as false positives.

Hold.

```

low = np.percentile(s2, 40)
run, holds = [], []
for k, v in enumerate(s2):
    if v <= low:
        run.append(k)
    else:
        if len(run) >= max(2, int(0.15 * fps)):
            holds.append(run[len(run) // 2])
            run = []

```

Find runs of at least 0.15 s where motion sits below the 40th percentile, and take the **middle** of each run.

EVIDENCE IN THE PROJECT: Why holds are minima, and the measurement that proves it

Drinking is by a wide margin the least visually active interval in the reference clip: mean motion 0.277 overall, 0.145 in the feet band. That is *why* a sip is detected as a minimum. If you looked for peaks in the drinking interval you would find the raise and the lower - the movements either side of the sip - not the sip itself.

Contact.

```

thr = s2.mean() + 0.3 * s2.std()
pk, _ = find_peaks(s2, height=thr, distance=max(1, int(0.12 * fps)))
if len(pk):
    p = pk[-1] if "place" in action or spec.key.endswith("placement") else pk[0]
    j = p
    while j + 1 < len(s2) and s2[j + 1] <= s2[j]:
        j += 1
    ev.append(VisualEvent(action, "contact", float(t[idx[j]]), "medium",
                          "motion peak resolved to rest = contact"))
else:
    k = int(np.argmax(s2))
    ev.append(VisualEvent(action, "contact", float(t[idx[k]]), "low",
                          "no clear peak; motion minimum used"))

```

Note the asymmetry: for a **placement** the *last* peak is taken (`pk[-1]`), because the object meets the surface at the end of the gesture; for a **pickup** the *first* (`pk[0]`), because contact happens at the start. And when no clear peak exists, the code falls back to the motion minimum and **labels the event** low confidence.

Continuous.

```

idx = np.flatnonzero((t >= a - 0.40) & (t < b))
thr = float(np.median(m) + 0.5 * m.std())
rise = np.flatnonzero(s2 > thr)
if len(rise):
    tc = float(t[idx[rise[0]]])

```

```
ev.append(VisualEvent(action, "activity_start", tc, "medium", ...))
```

KEY POINT: Even a continuous action has a start

The comment says: *"A continuous action still has a START. Placing audio at the label edge is exactly the 'trust the boundary' mistake this project avoids: the label is a coarse span, the motion tells us when the activity actually begins."*

This was a bug fix. An earlier version placed continuous audio at the label edge.

29.4 Confidence is not uniform, and it is reported

Event kind	Confidence	Why
foot_contact	high	Prominent, well-separated motion features
hold	medium	Derived from a motion minimum, which is less sharply defined
contact (peak found)	medium	Derived from a final motion peak
contact (no peak)	low	Fallback to the motion minimum
activity_start (rise found)	medium	First frame above median plus half a standard deviation
activity_start (no rise)	low	Fallback to the interval start

Every event also carries a `basis` string explaining how it was derived, and both surface in the API and the user interface.

29.5 The wide search for footsteps

```
WIDE_SEARCH_STRATEGIES = {"footstep"}
...
search = (0.0, a["end"]) if spec.strategy in WIDE_SEARCH_STRATEGIES else None
```

Footsteps are searched from the start of the video to the end of the labelled interval, because *"footsteps commonly begin while a previous label is still active"*. Every other strategy searches only inside its own interval.

29.6 The results on the reference clip

Action	Strategy	Events found
walk around table	footstep	four foot plants at 0.458, 1.083, 1.667, 2.208 s
pick up cup	contact	one contact at 2.625 s
drink from cup	hold	two sip holds at 6.625, 7.792 s
place cup on table	contact	one contact at 9.833 s

The foot plants are spaced 0.625, 0.584 and 0.541 seconds apart - a natural, slightly decelerating gait.

EVIDENCE IN THE PROJECT: These exact numbers are asserted in the test suite

```
@test("visual detection reproduces the validated foot plants")
def _():
    ev = detect_events(MO, "walk around table", s, 1.5, 2.5, (0.0, 2.5))
    got = [round(e.t_s, 3) for e in ev]
    assert got == [0.458, 1.083, 1.667, 2.208], got
```

The generalised service is not merely believed to reproduce the validated reference implementation; it is **asserted** to, on every test run.

Chapter 30 — Temporal alignment

30.1 Segment selection: cutting a 10-second asset down

The generated asset is 10 seconds. A single cup contact needs about 400 milliseconds of it. Four rules exist, one per selection value.

Selection	Method	Used by
steps	A continuous slice containing a run of footsteps whose spacing best matches the filmed gait	walking, running, clapping, locomotion
wet_segment	Isolated segments dominated by 200 Hz to 1 kHz energy - mouth and liquid content - one per detected hold	drinking, oral archetype
event	The single contact-plus-resonance window with the highest peak and the cleanest cut edges	all contact classes
slice	A continuous slice spanning the interval, starting at the asset's own first transient	stirring, typing, pouring, friction, liquid, ambient

event **selection, scored:**

```
score = 20*np.log10(max(peak, 1e-12)) - 20*np.log10(max(edge / max(peak,1e-12), 1e-6))
```

Reward a loud peak; penalise loud edges. A window whose boundaries fall in the middle of the sound would click when it is cut, so quiet edges are worth as much as a loud centre.

wet_segment **selection:**

```
wet = band_ratio(seg, sr, 200, 1000)
if wet >= min_wet:                                # min_wet = 0.45
    cands.append((float(np.abs(seg).max()), wet, t0 - half_s, t0 + half_s))
```

Segments must have at least 45 percent of their energy between 200 Hz and 1 kHz, which is where mouth and liquid content lives. If nothing qualifies, the code falls back to the loudest transients - a graceful degradation rather than a failure.

steps **selection - matching the gait:**

```
if n >= 2 and len(steps) >= n:
    vis = np.diff(contacts)                        # the filmed inter-step intervals
    best, best_err = 0, None
    for k in range(len(steps) - n + 1):            # every consecutive run of n steps
        err = float(np.sum((np.diff(steps[k:k+n]) - vis) ** 2))
        if best_err is None or err < best_err:
            best, best_err = k, err
    run = steps[best:best+n]
```

Every consecutive run of n steps in the generated asset is scored on how closely its internal spacing matches the n visible intervals, by sum of squared differences. The best-matching run is selected and translated so its first step lands on the first visible plant.

30.2 The alignment rule

KEY POINT: The single most important sentence in the project

A clip is positioned so that its TRUE ENVELOPE ATTACK coincides with the visual event. Clips are shifted, never time-stretched.

Both halves matter.

Why true attack and not onset strength. Covered in Section 11.7. Onset-strength peaks lead or lag the real transient by -96 to +250 ms. One asset footstep reports a strength peak at 3.760 s whose true attack is at 3.856 s, so aligning the strength peak misplaced the audible sound by exactly 96 ms - and it was audible before it was found.

Why shift and never stretch.

KEY POINT: The reasoning, from the source

"Time-stretching alters the generated audio's character. Where the generated cadence differs from the filmed one, the residual is absorbed and reported rather than corrected."

Time-stretching a footstep changes what the footstep sounds like. The project's position is that it is better to have a slightly late footstep that sounds like a footstep than a perfectly placed one that sounds like a processed artefact - and, crucially, to **report** the residual rather than hide it.

The test suite defends this:

```
@test("alignment never time-stretches: output length equals source length")
```

30.3 The alignment plan

Each placement entry records everything needed to audit it:

Field	Meaning
asset_start_s, asset_end_s	Which part of the generated file is used
video_start_s	Where it is placed on the video timeline
aligned_to_s	The visual event it was aligned to
alignment_kind	foot_contact, hold, contact or activity_start
clip_onset_offset_s	How far into the selected clip the attack is
visible_events_s	All visible events for a footstep run
per_event_error_ms	The residual at every contact, in milliseconds
strategy	A human-readable sentence describing what was done

30.4 Why residual error accumulates for rhythmic actions

This is the sharpest analytical result in the project, and it comes directly from the shift-only policy.

EVIDENCE IN THE PROJECT: The cadence measurement

Source	Mean inter-step interval	Standard deviation	Difference from filmed
Filmed gait	0.583 s	0.034 s	-
MOSS walking asset	0.641 s	0.042 s	9.9 percent slower
Stable Audio walking asset	0.407 s	0.026 s	30.2 percent faster

Because clips are shifted and never stretched, a cadence mismatch **accumulates** across successive contacts.

- The MOSS asset's residuals stay within 0 to -67.6 ms across four plants.
- The Stable Audio asset's residual grows monotonically to **-462.4 ms** by the fourth

plant.

And that growth is **predictable from the assets alone**: three step intervals at a 0.176 s deficit accumulate to 0.528 s, and -462.4 ms was observed after the matcher selected the best-fitting run.

KEY POINT: What this result actually argues

It is the clearest argument for the *limits* of the shift-only policy. Shifting preserves the character of the generated audio but cannot correct a cadence mismatch, so alignment quality for rhythmic actions depends on the generator producing a plausible tempo.

It also gives an **objective, timing-based reason to prefer MOSS** for this material, entirely independent of the harmonic-ratio argument. Two independent lines of evidence pointing the same way is much stronger than either alone.

30.5 Merging consecutive intervals - and why only some of them

Module 2 emits one span per window, so a single continuous activity arrives as several adjacent labels. Placing the same short source segment repeatedly is audible as an obvious loop.

```
merged_actions, MERGE_GAP = [], 0.15
for a in actions:
    sp, _ = resolve(a["action"])
    key = sp.key if sp else None
    # Only CONTINUOUS activities merge. Discrete contact events must not: two
    # separate spoon placements are two separate sounds, and merging them would
    # discard one of the visual events.
    mergeable = sp is not None and sp.strategy == "continuous"
    if (merged_actions and key is not None and mergeable
        and merged_actions[-1]["_key"] == key
        and a["start"] - merged_actions[-1]["end"] <= MERGE_GAP):
        merged_actions[-1]["end"] = a["end"]
        merged_actions[-1]["_merged"].append(a["action"])
    else:
        merged_actions.append(**a, "_key": key, "_merged": [a["action"]])
```

EVIDENCE IN THE PROJECT: Merging discrete events was tried and reverted

The `HANDOFF.md` bug list records it: *"Consecutive same-class intervals each replayed the SAME 1 s source segment (audible loop). Continuous activities are now merged into one span; discrete contacts are NOT merged (that lost a visual event when tried)."*

And you can see the merge working in a real job report: the coffee-stirring video's three separate stirring labels - stir the contents of the cup, Stirring a cup of coffee and stir coffee - were merged into one 3.0-second span from 3.5 to 6.5 seconds. The merge is logged in the report under `merged_intervals`, so it is auditable.

Chapter 31 — Mixing and rendering

31.1 The per-clip chain

Applied to every clip, in this order:

Step	Setting	Purpose
Zero-crossing snap	nearest crossing within plus or minus 3 ms	Cut where the waveform is at zero, so the edit does not click
DC removal	per-clip mean subtraction	Removes offsets of order 1e-4
Fades	12 ms raised cosine, in and out	Continuous in slope, unlike a linear ramp
Level	active RMS against the class target	See Section 31.2
Peak cap	-6 dBFS	An outlier guard only

Zero-crossing snap first, then DC removal, then fades - and the snap offset is recorded per clip so the placement can be corrected by the same amount:

```
start = int(round((p["video_start_s"] + snap_ms / 1000) * sr))
```

Measured snap corrections in the validated build were between **-0.021 ms** and **+0.167 ms**.

31.2 Why levels differ by class

KEY POINT: The acoustic argument

"A mug meeting a table is percussive, footsteps are mid-ground, a sip is intimate. Normalising all three to equal loudness would not correspond to any real recording position."

Equal-loudness normalisation would place the sip at the loudness of a footstep, which does not correspond to any microphone anyone could actually put anywhere.

Class	Target active RMS
Clapping	-30.0 dBFS (loudest)
Running, cup placement, cup pickup, doors	-32.0 dBFS
Object placement, object pickup, button press	-33.0 dBFS
Walking, stirring, typing, pouring, spoon handling	-34.0 dBFS
Sitting down	-36.0 dBFS
Drinking	-38.0 dBFS (quietest)

31.3 The peak cap, and the bug it caused

```
CLIP_PEAK_CEILING_DBFS = -6.0
```

CAUTION: A subtle bug worth understanding

The cap was originally -12 dBFS. The comment in `config.py` explains what went wrong:

"Outlier guard only. At -12 dBFS this cap was binding on most clips, which made it - not the active-RMS targets - the thing setting relative level, flattening the dynamics between events. Raised so that per-class RMS balancing actually governs."

A safety limit that binds on the common case has stopped being a safety limit and has become the primary control - and in this case it silently defeated the entire per-class level design. The symptom was not an error; it was that all the events sounded equally loud.

General lesson: a guard that fires often is not a guard. Check how often your limits actually bind.

31.4 End-of-video handling

```
MIN_KEPT_FRACTION = 0.45
if kept < MIN_KEPT_FRACTION * len(clip):
    log["rejected"].append({..., "stage": "end_of_video_truncation",
        "reason": (f"only {kept/sr*1000:.0f} ms of a {len(clip)/sr*1000:.0f} ms "
            f"sound fits before the video ends; a fragment that short "
            f"reads as a click, so it was omitted"))
    continue
clip = rcos_fade(clip[:kept], sr, C.FADE_MS)
```

If at least 45 percent of a clip fits, it is truncated with a fade and the truncation is logged. Below that, it is **omitted entirely**, because a sliver of a contact sound reads as a click rather than as the event.

The video timeline length is never extended. In the validated build the placement clip would have run to 10.098 s, past the end of the 10.005-second video, so its tail was truncated by **93.1 ms** with a fade.

31.5 The bus

```
sum -> linear normalisation to -6 dBFS -> safety limiter (threshold -6, ceiling -3)
```

```
def soft_limit(x, thresh_db, ceiling_db):
    t, c = 10 ** (thresh_db / 20), 10 ** (ceiling_db / 20)
    a = np.abs(x); over = a > t
    if not over.any():
        return x, 0.0
    y = x.copy(); room = c - t
    y[over] = np.sign(x[over]) * (t + room * np.tanh((a[over] - t) / max(room, 1e-9)))
    gr = 20 * np.log10(np.max(a[over]) / max(np.max(np.abs(y[over])), 1e-12))
    return y, float(gr)
```

A tanh soft knee, and the gain reduction is always measured and reported.

EVIDENCE IN THE PROJECT: The limiter has never engaged

"max_gain_reduction_db": 0.0, "limiter_engaged": false in every recorded mix.

Because it did not engage, **no dynamic-range processing of any kind was applied**, and the inter-clip balance is exactly what the per-class RMS targets set. The crest factor of 30.87 dB confirms the transient structure is intact.

The limiter exists as protection against a future change producing an overshoot. Reporting that it did nothing is more informative than reporting that it worked.

31.6 Mix validation

```
if not np.isfinite(bus).all():
    raise ValueError("mix contains NaN or Inf")
if np.abs(bus).max() >= 1.0:
    raise ValueError("mix clips")
```

The mix is rejected outright if it contains a non-finite sample or clips. There is no "clamp and continue" path.

31.7 The final render

```
ffmpeg -y -v error -i source.mp4 -i mixed.wav \
-map 0:v:0 -map 1:a:0 \
-c:v copy \
-c:a aac -b:a 192k -ar 48000 \
-movflags +faststart -shortest out.mp4
```

Flag	Meaning
-map 0:v:0	video stream 0 from input 0 (the source video)
-map 1:a:0	audio stream 0 from input 1 (the mixed WAV)
-c:v copy	the picture is stream-copied: not re-encoded, no quality loss, bit-identical to the source
-c:a aac -b:a 192k	AAC at 192 kbit/s
-ar 48000	48 kHz output
-movflags +faststart	Moves the MP4 index to the front so the file streams in a browser without a full download
-shortest	Stop at the shorter of the two inputs, so the audio can never extend the video

The render function then re-probes the output and returns a full description - codecs, durations, frame count, resolution, sample rate, channels, byte size - which becomes part of the job report.

Chapter 32 — Model selection: everything that was rejected

Model choice was made **empirically on the target hardware**, not from published benchmarks, because the binding constraints - 17 GB of unified memory, MPS rather than CUDA, no cloud inference - are not the constraints those benchmarks were produced under.

32.1 Action recognition

Model	Type	Result	Why rejected
VideoMAE (Kinetics fine-tune, 86.5 M params)	closed-vocabulary video classifier	"shredding paper" 0.550; no relevant label in the top ten	Kinetics labels describe whole-clip activities, not object-contact events
X-CLIP	open-set but requires candidate labels	"pouring liquid" 0.21-0.38 in every window, collapsing to one segment	No temporal discrimination; still a closed label set
Qwen2.5-VL-3B-Instruct	vision-language model	free-text action phrases per window	Selected

KEY POINT: State the failure as structural, not incidental

"The failure is not that those models are bad. It is that the label set is organised around the wrong unit. Kinetics-style labels describe activities; Foley needs events. There is no Kinetics class for 'a mug is set on a table'."

32.2 Sound generation

Model	Output	Measured failure mode	Licence
Stable Audio Open 1.0	44.1 kHz stereo, 4.5 s	96.2 percent silence, 2 clicks of 0.04 s	Stability Community, non-commercial
Stable Audio Open Small	44.1 kHz stereo, 3.0 s	95.6 percent silence	non-commercial
AudioLDM 2	16 kHz mono, 10 s	91.9 percent silence, 30 ms events	-
FoleyCrafter	16 kHz mono, 3.0 s	Continuous noise bed: 25 "events" at 8.6 Hz, 0 percent below -20 dB	non-commercial
MMAudio v1 / v2	44.1 kHz mono	86 / 96 percent silence, sound placed on the wrong action	non-commercial
AudioGen medium	16 kHz mono	Not tried - ranked second	CC-BY-NC-4.0
MOSS-SoundEffect v2.0	48 kHz mono, up to 30 s	Selected	Apache-2.0

Table 32.1 - From results/text_to_audio_model_evaluation.md

The four reasons MOSS was selected:

1. **48 kHz output** - the highest of any candidate. It matters specifically for this

material: wet mouth transients and ceramic contacts carry substantial energy above 8 kHz, which every 16 kHz model cuts off entirely.

1. **Duration control up to 30 s** - which directly addresses the "silence plus clicks"

failure signature that sank the three text-to-audio attempts.

1. An explicitly documented human-action Foley category - its documentation groups

sounds into natural environments, urban environments, animals and creatures, and human actions, with worked Foley examples. None of the other candidates advertise that category.

1. Apache-2.0 - the only candidate that is not non-commercial.

Honest risks that were recorded before installing it:

- Python 3.12 and torch 2.9 required, where the existing environments were 3.10 with torch

2.7.1.

- The repository's install docs are CUDA-first, with `cu128` wheels and `torch.compile`;

MPS is supported in the API but the compile path is not. The documented escape hatch is `TORCHDYNAMO_DISABLE=1`, set from the start.

- A recent model with less community MPS validation than AudioGen.
- A genuine 6 to 8 GB download, with nothing cached.

Why TangoFlux and EzAudio were ranked below it:

- TangoFlux** reuses the Stable Audio Open VAE, and Stable Audio Open had already failed

this exact task twice with the clicks-in-silence signature. Sharing the decoder lineage makes it a poor bet against *this specific* failure. Its licence is also research-only.

- EzAudio** has the cleanest licence (MIT) but thin published specifications and 24 kHz

output.

Why AudioGen was ranked second and not first:

Its single strongest argument was that it is **architecturally unlike everything that had failed**: an autoregressive transformer over EnCodec tokens, where all five failures were latent diffusion or flow matching. After five same-family failures, that diversification had real value. It was held back by 16 kHz mono output, a CC-BY-NC licence, an `xformers` dependency, and autoregressive slowness.

32.3 The direct A/B: MOSS against Stable Audio

Stable Audio Open was **retained as a switchable backend** (`FOLEY_BACKEND=stable_audio`) and run through the identical pipeline, which permits a controlled comparison.

Class	MOSS score	MOSS harmonic	Stable Audio score	Stable Audio harmonic
Walking	97.1	0.00	92.7	0.03
Drinking	70.9	0.06	75.6	0.09
Cup pickup	85.8	0.00	53.1	0.88
Cup placement	49.8	0.02	53.4	0.87

Stable Audio is comparable on walking and scores *higher* on drinking and cup placement. On object contacts it produced musical tones. It is also faster - about 66 seconds against about 4 minutes per asset.

REMEMBER THIS

Two independent measurements point the same way, and neither is the aggregate score: **harmonic ratio** (0.00-0.02 against 0.87-0.88 on object contacts) and **cadence** (9.9 percent slow against 30.2 percent fast, producing -67.6 ms against -462.4 ms of accumulated residual). The aggregate score is blind to both.

32.4 The runner for the alternative backend

`backend/runners/run_stable_audio.py` is worth reading for its MPS notes:

Setting	Value	Why
<code>apg_scale</code>	0.0	The library default of 1.0 takes an adaptive-projected-guidance path that computes in float64, which MPS does not support
Precision	float32 throughout	MPS lacks the float64 paths the library would otherwise use, and float16 produced non-finite values in earlier testing
<code>sample_size</code>	a latent-token budget, not the model's native 475	The native size does not fit in this machine's RAM
Sampler	<code>dpmpp-3m-sde</code> , sigma 0.3 to 500.0	The configuration proven in the earlier experiment
Output	44.1 kHz stereo, converted by the caller	So the runner stays minimal and the rest of the system is backend-agnostic

Chapter 33 — Subsystem 2: results

33.1 Functional verification

Suite	Tests	Result
backend/tests/test_suite.py	42	all pass
backend/tests/test_foley_validation.py	22	all pass
backend/tests/e2e_gate.py	end-to-end on the reference clip	passes
Total	64 automated tests	all pass

CAUTION: Another documentation discrepancy

The README and HANDOFF.md say 59 tests (36 + 22). The suites now contain **64** (42 + 22); tests were added when open-vocabulary synthesis was introduced. Both suites were run while writing this handbook and both pass in full.

The end-to-end gate reports:

```
[gate] completed in 5s
[gate] every asset was reused from cache (no regeneration): True
[gate] VALIDATION VERDICTS
  Walking      PASS    peak    -6.7  dyn  43.7  bits  14.9  harm  0.00  gain  -3.6
  Cup pickup   PASS    peak    -8.3  dyn  28.0  bits  14.6  harm  0.00  gain  +2.7
  Drinking     PASS    peak   -23.5  dyn  29.2  bits  12.1  harm  0.06  gain  +13.2
  Cup placement PASS    peak   -30.5  dyn  17.5  bits  10.9  harm  0.02  gain  +21.9
[gate] counts: actions_detected 5, sounds_generated 4, sounds_rejected 0,
               placements 5, unsupported_actions 1
[gate] intervals left silent: stand - Standing still produces no Foley event.
[gate] no rejected asset reached the mix: OK
```

33.2 Synchronisation accuracy - the headline result

Measured on the **rendered audio** by detecting envelope attacks in the final WAV and comparing against the visual event timestamps. Not asserted from the plan.

Action	Visual event	Rendered attack	Error
Walk, plant 1	0.458 s	0.458 s	-0 ms
Walk, plant 2	1.083 s	1.063 s	-20 ms
Walk, plant 3	1.667 s	1.675 s	+8 ms
Walk, plant 4	2.208 s	2.221 s	+13 ms
Drink, hold 1	6.625 s	6.638 s	+13 ms
Drink, hold 2	7.792 s	7.803 s	+11 ms
Place cup	9.833 s	9.833 s	-0 ms

Table 33.1 - The validated standalone build. Source: results/qa_polished.json.

Worst error: 20 ms. One frame at 24 fps is 41.7 ms. Every event is inside half a frame.

KEY POINT: Why "measured on the rendered audio" is the important phrase

It is a stronger measurement than reading the alignment plan, because it includes any error introduced by segment selection, fades and mixing. A plan can say the right thing and the render can still be wrong.

CAUTION: State the scope honestly, before you are asked

This is **seven events on one clip, evaluated on the build that was tuned against it**. It demonstrates that the method can achieve sub-frame placement. It does **not** demonstrate that it does so in general, and the research paper says so in the same words.

33.3 The aggregate picture across all recorded jobs

The 20 ms figure is the best case. Across **all 32 recorded job runs**, using both generation backends:

Statistic	Value
Jobs analysed	31
Events with a recorded residual	45
Distinct clips	2
Median absolute error	4.7 ms
Mean absolute error	56.2 ms
90th percentile	150.6 ms
Worst	462.4 ms
Within half a frame (20.8 ms)	66.7 percent

Table 33.2 - From `paper/experiments/exp1_sync.json`.

EVIDENCE IN THE PROJECT: What the 462 ms outlier is, and why quoting it is a strength

It is the **Stable Audio** walking asset, whose cadence is 30.2 percent faster than the filmed gait. Under the shift-only policy that mismatch accumulates monotonically across four plants. It is a fully explained result, not an unexplained failure.

The honest summary is: *"median 4.7 ms, two thirds of events inside half a frame, and one explained outlier at 462 ms caused by a cadence mismatch in the alternative backend."* That is a much more credible answer than "20 ms" alone.

CAUTION: A further limitation on this evidence

Only **footstep** placements record a per-event residual. Hold and contact placements record the aligned instant but not a residual, so the multi-clip evidence is thinner than "32 job runs" suggests. The paper states this explicitly.

33.4 The current end-to-end result on the reference clip

From `data/jobs/aa4b2f4e6049/report.json`, the most recent full run:

Metric	Value
Actions detected	5
Sounds generated	4
Sounds rejected	0
Placements written to the mix	5

Metric	Value
Intervals left silent	1 (stand)
Worst alignment error	67.6 ms
Mix peak	-6.00 dBFS
Mix RMS	-38.72 dBFS
Crest factor	32.72 dB
Clipped samples	0
Limiter gain reduction	0.00 dB
Output	h264 video stream-copied, 240 frames, AAC 48 kHz mono

Track	Placed at	Gain applied	Raw active RMS	Target
walk around table	0.158 - 2.500 s	-3.46 dB	-30.54 dBFS	-34.0 dBFS
pick up cup	2.575 - 2.975 s	-5.91 dB	-26.09 dBFS	-32.0 dBFS
drink from cup (1)	6.342 - 7.043 s	+11.62 dB	-49.62 dBFS	-38.0 dBFS
drink from cup (2)	7.453 - 8.153 s	+8.75 dB	-46.75 dBFS	-38.0 dBFS
place cup on table	9.787 - 10.005 s	+12.47 dB	-44.47 dBFS	-32.0 dBFS

CAUTION: Two builds, two numbers - do not confuse them

The **validated standalone build** (`scripts/run_module3.py`, using hand-approved assets) reports a worst error of **20.3 ms**.

The **current web pipeline** on the same clip reports **67.6 ms**, because it selects a different step run from a freshly generated walking asset and now also sounds the cup pickup, which the standalone build left silent.

Both are real. Quote whichever you are asked about, and be ready to explain the difference.

33.5 The 19-check quality gate on the validated build

#	Check	Result	Measured
1	Final MP4 opens	PASS	2 streams
2	Video duration preserved	PASS	10.005 s to 10.000 s
3	Video stream untouched	PASS	240 frames, stream-copied
4	Audio duration matches video	PASS	9.984 s against 10.000 s (AAC frame granularity)
5	Sample rate	PASS	48,000 Hz
6	Channel count	PASS	1 (mono)
7	No clipping	PASS	peak -6.00 dBFS, 0 samples at full scale
8	No NaN or Inf	PASS	all samples finite
9a	Walking synchronisation	PASS	-0 / -20 / +8 / +13 ms
9b	Drinking synchronisation	PASS	+13 / +11 ms
9c	Placement synchronisation	PASS	-0 ms
10	No edit-boundary discontinuities	PASS	all 8 clip boundaries clean
11	No bleed into silent intervals	PASS	zero overlap with the pick-up interval
12	Cup pickup documented unavailable	PASS	silent, documented

#	Check	Result	Measured
13	Not over-compressed	PASS	limiter gain reduction 0.00 dB
14	Healthy crest factor	PASS	30.87 dB
15	Original video unchanged	PASS	SHA-256 a620ee58 . . .
16	Locked assets unchanged	PASS	2 of 2 verified
17	Earlier outputs not overwritten	PASS	first-pass mix and MP4 both present

Table 33.3 - 19 checks across 17 numbered categories; the synchronisation check is evaluated separately for walking, drinking and placement.

33.6 The two ablations, and what they prove

Ablation A - denoising steps.

Setting	Generation time	Score	Dynamic range	Detected attacks
50 steps (production)	265.9 s	86.9	32.6 dB	7
35 steps	200.4 s	86.3	31.3 dB	7
25 steps	128.6 s	86.4	29.1 dB	7

Halving the steps halves the time and moves the score by 0.6 points, which is within noise. The transient count is unchanged. **The gate is effectively blind to this change**, and the decision to keep 50 rests entirely on listening.

Ablation B - the denoised latent length. This is the important one.

Setting	Generation time	Score	Dynamic range	Effective bits	Attacks	Inter-step std dev
30 s latent (production)	242 s	97.1	43.7 dB	14.89	16	0.042 s
10 s latent	88 s	86.0	61.1 dB	16.0	8	0.244 s
Filmed gait, for reference	-	-	-	-	4	0.034 s

REMEMBER THIS

Cutting the denoised window from 30 s to 10 s makes generation **2.8 times faster** and **raises** both the measured dynamic range (43.7 to 61.1 dB) and the effective bits (14.89 to 16.0) - **both of which the quality score rewards**.

Yet the number of detectable transients **falls from 16 to 8**, and the standard deviation of inter-step intervals **rises from 0.042 s to 0.244 s, a factor of 5.8**. The production asset is a regular gait; the ablated asset is a sparse, irregular sequence.

The scalar improved while the audio got worse.

And the effect on the hardest class is decisive:

EVIDENCE IN THE PROJECT: The shortcut would have made a class permanently silent

With the 30-second latent, one of three cup-pickup seeds passes the gate (seed 44, score 85.8 - see Chapter 28).

With the 10-second latent, **none** of the three passes. Peaks sit at -33 dBFS and all three would need over +40 dB of gain.

The speed-up would have made the hardest class permanently silent, and the aggregate quality score would have gone **up**.

KEY POINT: The general lesson, worth quoting verbatim in a viva

"An aggregate score computed from marginal statistics can improve while the temporal structure that makes audio usable is destroyed. A structural measure - how many transients are present and how regularly they are spaced - captures what the scalar misses, and is cheap to compute."

That is also why `config.py` carries this comment on the MOSS defaults:

"50 steps and a 30 s latent. Both were reduced for speed (25 steps, 10 s latent) and reverted: the output was audibly worse. The measured scores barely moved, so the quality gate did NOT catch it - listening did. Do not trade these for speed again without an A/B listen."

33.7 Output sparsity - a property, not a defect

For discrete object interactions, roughly **18 percent** of a 10-second timeline carries audio.

KEY POINT: How to answer "why is your output so quiet and empty?"

"Because that is what the scene is. These are discrete contact events, not a continuous soundtrack. A mug being picked up and set down produces two short sounds separated by several seconds of nothing. The silence between events is correct, not missing - and a system that filled it would be adding sound that no physical event produced."

PART 5

SUBSYSTEM 3: ACOUSTIC EYE

The visual microphone. What sound-induced vibration is, how phase-based analysis recovers it, what was adapted and what was fixed, and exactly how far the sampling theorem lets it go.

Chapter 34 — The visual microphone principle

34.1 What this subsystem does

Purpose. Take a video and attempt to reconstruct the sound that was present while it was recorded, using the tiny sound-induced vibrations visible on objects in the frame.

Input. MP4, AVI, MOV, MKV or WEBM, up to 250 MB by upload, or any size by local path.

Output. A 16-bit PCM WAV whose sample rate equals the video's capture frame rate, plus a waveform PNG, a spectrogram PNG, an optional denoised version, and a plain-English description of what the recovered signal contains.

Where. `Acoustic eye/acoustic-eye/`.

There is no machine learning in this subsystem at all. It is classical signal processing, and saying so clearly is a strength.

34.2 The physics

SIMPLE EXPLANATION: Why sound is visible

Sound is a pressure wave. When it hits an object, the object moves. It moves by a fraction of the width of a human hair - far too little to see - but a camera does not need to see it, it only needs to *measure* it. If the camera takes pictures fast enough, the tiny back-and-forth movement of, say, a crisp packet becomes a signal, and that signal is the sound that pushed it.

Technically. Davis et al. (SIGGRAPH 2014) showed that sound-induced vibrations of everyday objects can be recovered from high-speed video, reconstructing intelligible speech and music from footage of a chip bag and a potted plant. Displacements of the order of a **thousandth of a pixel** are recoverable, because the measurement aggregates evidence over the whole image rather than tracking any single point.

34.3 The sampling theorem, which bounds everything

KEY POINT: The single most important fact about this subsystem

The visual microphone produces **one audio sample per video frame**. Therefore:

output sample rate = video frame rate, and the highest recoverable frequency = fps / 2.

Camera frame rate	Nyquist limit	What is recoverable
30 fps	15 Hz	below human hearing (which starts around 20 Hz)
60 fps	30 Hz	a low rumble; no speech, no music
240 fps	120 Hz	the very lowest voiced fundamentals only
2,000 fps	1,000 Hz	some speech intelligibility
20,000 fps	10,000 Hz	full-band audio

Speech occupies roughly 100 to 8,000 Hz. Music extends far higher. **Neither is recoverable from ordinary 30 or 60 fps footage, and no amount of processing changes that. It is the sampling theorem, not a defect in the implementation.**

That is why the original paper used cameras running at thousands of frames per second, and it is why the project's own README opens with a scientific-scope warning rather than a feature list.

34.4 The algorithm

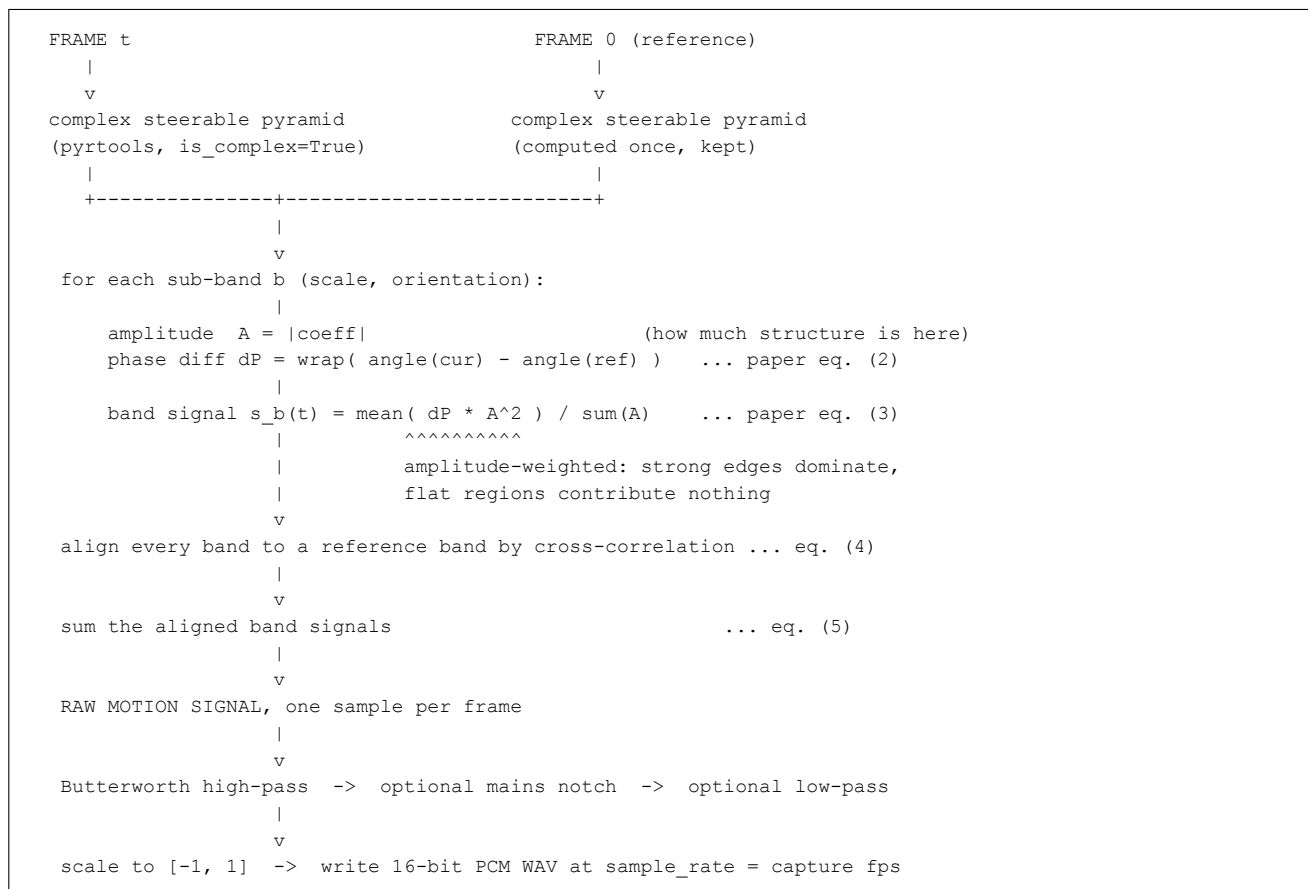


Figure 34.1 - The phase-based visual microphone, with the paper's equation numbers.

34.5 Why phase, and not simply pixel brightness

SIMPLE EXPLANATION: The intuition

If you watch a single pixel and the object moves slightly, the brightness of that pixel changes - but so does the brightness if a cloud passes, or the camera's automatic exposure twitches, or the sensor is noisy. Brightness is a poor motion detector.

Local *phase* is different. If you decompose the image into oriented wave-like patterns at several scales, then a small shift of the image shifts the *phase* of those patterns by a predictable amount, while leaving their *amplitude* alone. Phase measures displacement directly, and it is far less confused by brightness change.

Technically, this follows the phase-based motion analysis of Wadhwa et al. (SIGGRAPH 2013). The image is decomposed with a **complex steerable pyramid** (Simoncelli and Freeman, 1995): a bank of filters at several **scales** and several **orientations**, whose outputs are complex numbers. The argument of each coefficient is a local phase, and small translations of the underlying structure appear as changes in that phase.

Amplitude weighting matters: a region with strong oriented structure - an edge - gives a reliable phase measurement, while a flat region gives noise. Weighting by amplitude squared and dividing by the total amplitude means edges dominate and flat regions contribute almost nothing.

Band alignment matters because different scales and orientations respond to the same physical motion with different delays. Cross-correlating each band against a reference band and rolling it into place before summing is equation (4) of the paper.

34.6 The core code

```
def align_vectors(v1, v2):
    """Circularly shift v1 so it best lines up with v2 (paper eq. 4)."""
    v1 = np.nan_to_num(np.asarray(v1, dtype=np.float64))
    v2 = np.nan_to_num(np.asarray(v2, dtype=np.float64))
    acorb = np.convolve(v1, np.flip(v2))
    maxind = int(np.argmax(acorb))
    shift = v2.size - maxind
    return np.roll(v1, shift)
```

Convolution with the time-reversed reference is cross-correlation; the lag that maximises it is the alignment offset.

```
def _accumulate(coeffs):
    for band in band_keys:
        cur, ref = coeffs[band], first_coeffs[band]
        amp = np.abs(cur) # eq. (3)
        dphase = (np.mod(math.pi + np.angle(cur) - np.angle(ref), 2*math.pi)
                  - math.pi) # eq. (2), wrapped
        weighted = dphase * amp * amp
        total_amp = float(np.sum(amp))
        if total_amp <= 0.0 or not np.isfinite(total_amp):
            signals[band].append(0.0) # silent-region guard
        else:
            signals[band].append(float(np.mean(weighted)) / total_amp)
```

The `np.mod(pi + x, 2*pi) - pi` construction wraps the phase difference into the range minus pi to plus pi, which is essential: phase is circular, and an unwrapped difference of "plus 6.2 radians" is really "minus 0.08 radians".

Chapter 35 — Implementation, and the eleven fixes

35.1 What was adapted, and from where

KEY POINT: Attribution, stated plainly

The Visual Microphone algorithm is **not this project's invention**. It is Davis et al., ACM Transactions on Graphics (SIGGRAPH) 33(4), 2014.

The Python implementation adapted here is `visual-mic-master` by Antonio Musolino and Davide Sforza, MIT licence, copyright 2020, itself based on the original MATLAB code by the paper authors.

The MIT licence text is retained in `LICENSE` with the original copyright line, and every adapted source file carries an attribution header. The paper and `pyrtools` are cited in the README, the licence file, and the site's About page.

The separation of contributions is stated in the README as:

```
Existing research (Davis et al., SIGGRAPH 2014)
+
Adapted implementation (visual-mic-master, MIT - Musolino & Sforza)
+
Our Acoustic Eye web application (FastAPI backend, REST API, HTML/CSS/JS UI)
+
Our integration / robustness fixes / testing / visualisation / packaging
```

35.2 The eleven documented fixes

This table is the original engineering contribution of Subsystem 3, and it is worth knowing several entries by heart.

#	Reference behaviour	The problem it caused	The fix
1	<code>nframes = int(video.get(CAP_PROP_FRAME_COUNT))</code> , then <code>sound = np.zeros(nframes)</code>	Container metadata is often wrong - variable frame rate, dropped frames, truncated files - and a length mismatch breaks the final <code>sound += sig_aligned.broadcast</code>	Never allocate from metadata. Count the frames that can actually be decoded; output length is <code>len(signals[ref_band])</code> ; every band is length-harmonised before summation
2	Per-frame decode, greyscale and normalise happen inside the algorithm loop with no error handling	One unreadable frame throws inside <code>cvtColor</code> or <code>resize</code> on <code>None</code> and aborts the whole run	<code>video_reader.iter_gray_norm_frames</code> isolates decoding, skips bad frames, and yields only usable ones
3	<code>get_scaled_sound</code> divides by <code>max - min</code> unconditionally	A silent or constant signal gives divide-by-zero, producing <code>inf</code> and <code>nan</code> in the WAV	Guard: if the range is at or below <code>1e-12</code> , return zeros
4	<code>total_amp = np.sum(amp)</code> used as a divisor with no check	A fully dark or zero-contrast band gives divide-by-zero and <code>nan</code> propagates into the sum	If <code>total_amp <= 0</code> , contribute 0.0 for that frame and band
5	Spectral subtraction reconstructs the STFT as <code>st_mags * (1j * st_angles)</code>	That is not a phasor. A phasor is <code>magnitude * exp(i*theta)</code> . The reference multiplies magnitude by an imaginary number, which scrambles the phase and distorts the result	Correct reconstruction <code>mags * np.exp(1j * angles)</code> , with the ISTFT output trimmed or padded back to the input length

#	Reference behaviour	The problem it caused	The fix
6	<code>downsample_factor=0.1</code> hard-wired at the call site	On small frames, 0.1x collapses the image and the pyramid fails	<code>downsample</code> is configurable and auto-relaxed so the shorter side stays at or above 24 px, with a processing note when this happens
7	<code>import pyrtools</code> at module top	A missing dependency gives the user a raw <code>ImportError</code> traceback	Guarded import, a typed <code>PyrtoolsUnavailableError</code> , and <code>/health</code> plus a UI banner explaining the fix
8	Output written with <code>scipy.io.wavfile</code> as 64-bit float WAV	64-bit float WAV is not reliably playable in browsers	Written with <code>soundfile</code> as 16-bit PCM
9	Spectrogram window NFFT defaults to 256	Fails or warns on very short recovered signals	NFFT chosen as the largest power of two at or below the signal length, between 32 and 1024
10	Alignment reference hard-coded to band key (0, 0)	<code>KeyError</code> for pyramid configurations without that key	Use (0, 0) when present, else the first oriented band, else the first key
11	Blocking command-line tool, <code>plt.show()</code>	Not usable from a web server	Non-blocking pipeline with a background worker, staged progress, and Matplotlib in Agg mode

Table 35.1 - The fixes, from `Acoustic eye/acoustic-eye/README.md`.

KEY POINT: Fix 5 is the one to memorise

`mags * (1j * angles)` versus `mags * exp(1j * angles)`.

The first multiplies the magnitude by an imaginary scalar - it does not encode an angle at all, and it destroys the phase relationship the inverse transform needs. The second is the correct polar-to-rectangular conversion.

It is a one-character-class difference with a completely different meaning, and finding it in someone else's published code is exactly the kind of thing an examiner will be impressed by.

35.3 The three parameters that matter for high-speed footage

These are this project's additions to the configuration, and each carries a reason in the source.

`capture_fps` - the true capture frame rate, used when the container metadata is wrong.

```
#: TRUE capture frame rate, used when the container's metadata is wrong.
#: High-speed cameras routinely store a *playback* rate (e.g. 30 or 60 fps)
#: in the AVI/MP4 header while having actually recorded at several kHz.
#: Because the Visual Microphone emits one audio sample per frame, this
#: number IS the output sample rate -- getting it wrong makes the result
#: useless. ``None`` means "trust the container".
capture_fps: Optional[float] = None
```

KEY POINT: Why this parameter is essential and not a convenience

A 20,000 fps camera very often writes "30 fps" into the file header, because that is the rate at which the footage is meant to be *played back*. If you trust that, you write a 20,000-sample signal into a WAV tagged as 30 Hz, and the result is a five-hundred-times slowed-down rumble that sounds like nothing. The recovered content is all there; the label is wrong.

`mains_notch_hz` - remove the mains-frequency comb.

```
def notch_mains(sound, sample_rate, base_hz=60.0, quality=35.0):
    """Remove base_hz and every harmonic of it with narrow IIR notches.

    High-speed photography needs very bright continuous light, and mains-powered
    lamps pulse at twice the supply frequency. That flicker is a genuine
```

```
brightness oscillation, so the Visual Microphone recovers it faithfully --
it lands in the output as an extremely strong 100/120 Hz tone plus
harmonics that can sit 100s of times above the acoustic signal and mask it
completely. Notching the comb is what makes the recording audible.
"""
```

Zero-phase `filtfilt` is used deliberately, so the notches introduce no group delay that would smear transients.

KEY POINT: Why this is a genuinely interesting problem

The flicker is not an artefact of the algorithm. Mains-powered lighting really does pulse at twice the supply frequency, and the visual microphone really does measure it - the algorithm is working perfectly and faithfully recovering a real oscillation that happens not to be the one you wanted. High-speed capture needs very bright continuous light, which makes the problem unavoidable rather than incidental.

`low_pass_hz` - remove broadband phase noise.

```
#: Low-pass cutoff in Hz applied to the recovered signal. The phase
#: estimate's noise is broadband, so on kHz-rate captures most output
#: energy can be hiss well above the real content.
```

35.4 The eight pipeline stages

```
validate -> read_frames -> extract_phase -> reconstruct
        -> filter -> generate_audio -> visualize -> analyze
```

Each stage reports `running`, `done` or `error` with a fraction, which the frontend polls.

35.5 The API

Method	Path	Purpose
GET	/	serves the frontend
GET	/health	pyrtools availability, limits, defaults, local-ingest configuration
POST	/upload	stores and validates a video, returns a job id and real frame counts
POST	/process	starts a background job with optional parameter overrides
POST	/process-local	reads a segment of a file already on the server, with no upload and no size limit
GET	/status/{job_id}	status, per-stage progress, error, result
GET	/result/{filename}	streams a produced WAV or PNG, path-traversal protected

KEY POINT: Why `/process-local` exists

A high-speed clip is enormous. One second at 20,000 fps is 20,000 frames. Uploading such a file through a browser is impractical, so the endpoint reads a *segment* of a file already on the server's disk.

It is also the one place where a security boundary matters, and it is handled: the path must resolve **inside** one of `LOCAL_PATH_ALLOWED_ROOTS`, which blocks `..` escapes and arbitrary file reads, the feature can be disabled entirely with `allow_local_path_ingest: false`, and the segment length is capped.

35.6 The "audio in text" report

Because the recovered signal is usually not intelligible, the system describes it in words. `text_report.analyze_signal()` produces:

`duration_seconds`, `dominant_frequency_hz`, `spectral_centroid_hz`, `rms`, `peak`, `crest_factor_db`, `band_energy_percent`, `bursts[]` with timestamps, and a plain-English summary.

Optional offline speech-to-text via `faster-whisper` exists but is **off by default** and is a no-op unless the package is installed. It is only meaningful for multi-kHz captures.

35.7 Cost

Cost is approximately `frames × pyramid_size`, because a complex steerable pyramid is built for **every frame**. Measured throughput on this machine:

Frame size	Throughput
64 x 64	1284 frames/s
128 x 128	441 frames/s
256 x 256	103 frames/s

Table 35.2 - From `paper/experiments/exp5_vm2.json`. Cost scales with pixel count.

RAM scales with per-frame size, not clip length, because only small per-band scalars are retained.

`MAX_PROCESS_FRAMES = 150_000`, and the comment explains the number: *"High-speed captures need a lot of headroom: at 20 000 fps a single second of footage is 20 000 frames. Small ROIs (e.g. 192x192) build a steerable pyramid in ~2 ms, so ~150 000 frames is a few minutes of CPU time."*

Chapter 36 — Subsystem 3: results

36.1 The controlled characterisation

Because there was no positive result on ordinary footage - for the reason given in Section 34.3 - the implementation was validated against a **synthetic stimulus with known ground truth**: a band-limited noise texture translated sinusoidally by a known sub-pixel amplitude at a known frequency, rendered at several frame rates.

CAUTION: What this experiment is and is not

It is an **optical** test, not an acoustic capture. It establishes that the implementation correctly recovers a known sub-pixel oscillation. It does not establish that it recovers sound from a real scene.

Result A - frequency recovery and aliasing.

A 10 Hz oscillation of 0.30 px amplitude is recovered with **zero frequency error** at every frame rate from 30 to 480 fps, carrying 65 to 66 percent of the spectral energy above 0.5 Hz.

At 120 fps (Nyquist 60 Hz):

True frequency	Recovered	Behaviour
5 Hz	5 Hz	exact
10 Hz	10 Hz	exact
20 Hz	20 Hz	exact
40 Hz	40 Hz	exact
55 Hz	55 Hz	exact - just below Nyquist
70 Hz	50 Hz	aliased : folds about Nyquist exactly as theory predicts
100 Hz	20 Hz	aliased : $120 - 100 = 20$

Result B - the sub-pixel detection floor.

Displacement	Dominant frequency recovered	Share of energy	Detected
0.100 px	10.0 Hz	65.7 percent	yes
0.050 px	10.0 Hz	61.8 percent	yes
0.030 px	10.0 Hz	58.5 percent	yes
0.020 px	10.0 Hz	48.2 percent	yes
0.015 px	0.67 Hz	~0	no
0.010 px	0.67 Hz	~0	no
0.005 px	0.67 Hz	~0	no

Table 36.1 - From paper/experiments/exp5_vm2.json. Recovery is exact down to 0.020 px and fails completely by 0.015 px.

REMEMBER THIS

Two clean results to quote: **exact recovery down to a displacement of 0.02 pixels**, and **aliasing about Nyquist exactly as theory predicts**. Together they establish that the implementation is correct and that its limit on ordinary footage is the sampling theorem, not a defect.

36.2 Ordinary footage: what actually happens

The one recorded job on 60 fps footage produced a signal whose dominant component is at **0.06 Hz**. That is camera drift, not sound. At 60 fps the recoverable band ends at 30 Hz, which excludes speech and essentially all musical content.

KEY POINT: The honest description

"Subsystem 3 is a demonstrator of the principle, subject to a stated frame-rate requirement. On 30 or 60 fps footage it recovers camera drift and low-frequency motion, not sound, and that is the sampling theorem rather than a bug."

36.3 The high-speed demonstration

There is one further result in the repository that post-dates the research paper, and it should be described carefully and precisely.

EVIDENCE IN THE PROJECT: What exists on disk

The folder `Acoustic_eye/recovered/` contains three WAV files and two PNG figures, produced on 2 September at 20:42 to 20:56:

- `Mary_Had-app-output.wav` - the raw pipeline output
- `Mary_Had-RECOVERED-cleaned.wav` - after mains notching
- `Mary_Had-RECOVERED-FINAL.wav` - the final version
- `waveform-final.png` and `spectrogram-final.png`

Measured properties of all three, verified while writing this handbook:

Property	Value
Sample rate	20,000 Hz
Duration	5.000 s (100,001 samples)
Channels	mono
Nyquist limit	10,000 Hz
Dominant frequencies, raw output	75.0 Hz, then 20.8 / 38.6 / 34.6 Hz
Dominant frequencies, cleaned	75.0 Hz, then 305.6 Hz, 137.2 Hz
Dominant frequencies, final	305.6 Hz , then 467.8 Hz, 226.2 Hz

A 20,000 Hz sample rate means the `capture_fps` override was set to 20,000: this is a **20,000 frames-per-second capture**, consistent with the MIT Visual Microphone dataset clip in which "Mary Had a Little Lamb" is played to a bag of crisps.

The progression across the three files is exactly what the processing chain predicts. The raw output is dominated by 75 Hz - low-frequency lighting and mechanical content. After notching, 305.6 Hz emerges. In the final version 305.6 Hz, 467.8 Hz and 226.2 Hz dominate, which is musical-band content in the register of a simple melody, not drift.

CAUTION: How to describe this result, and its limits

Do say: "I ran the pipeline on a 20,000 frames-per-second clip through the `/process-local` endpoint, with the capture-rate override set to 20,000 and the mains notch enabled, and recovered a 5-second, 20 kHz signal whose dominant energy sits at 305.6, 467.8 and 226.2 Hz - musical-band content rather than drift. Both the `capture_fps` override and the mains notch were essential; without them the output is dominated by 75 Hz lighting content."

Also say, unprompted: "Three caveats. First, the source clip is from the original authors' published dataset - it is their high-speed capture, not mine, because I had no high-speed camera. Second, the source file is not retained in the repository; the copy in the folder is a failed download. Third, my research paper's limitations section was written before this run and still says no real sound has been recovered, so the paper and the repository disagree and the repository is the newer evidence."

Do not say: "my system recovers speech from video." It does not, and it cannot at ordinary frame rates.

36.4 Testing

`Acoustic eye/acoustic-eye/tests/` contains **37 pytest tests** across four files:

File	Tests	Covers
<code>test_video.py</code>	11	reader, validation, real frame counting
<code>test_api.py</code>	11	FastAPI endpoint smoke tests
<code>test_processing.py</code>	10	signal helpers and the full pipeline
<code>test_text_report.py</code>	5	the signal-to-text description

`conftest.py` builds synthetic tiny-video fixtures, so the tests do not depend on any particular media file being present.

36.5 Limitations, stated as the project states them

- **Camera frame rate is the ceiling.** One audio sample per frame.
- **Nyquist.** 30 fps gives 15 Hz; 60 fps gives 30 Hz; 240 fps gives 120 Hz. Speech and music are not recoverable from normal footage.
- **You need visible, sound-induced vibration.** A rigid wall, a distant object, or a quiet room produces essentially nothing.
- **Object and material matter.** Light, high-contrast, resonant surfaces - crisp packets, foil, paper, thin plastic, leaves, a water surface - work. Heavy, rigid ones do not.
- **Lighting and noise.** Mains flicker beating with the shutter, rolling-shutter artefacts, sensor noise, compression blocking, and any camera motion all inject noise or destroy the signal. A tripod and bright steady light are required.
- **Processing cost.** One complex steerable pyramid per frame.
- **Single-process job store.** Jobs live in memory; restarting the server forgets them.
- **It does not magically recover arbitrary audio from any video.**

PART 6

THE WEB APPLICATIONS

How the three interfaces are built: the FastAPI job model, the React client, the polling contract, and exactly what happens between pressing a button and getting a file back.

Chapter 37 — Subsystem 2: the backend

37.1 Three processes, isolated by design

Process	Runtime	Responsibility
Frontend	Node during development, then a browser	UI, upload, polling, presentation
Backend	Python 3.12 in <code>venv-moss</code>	orchestration, synchronisation, mixing, rendering, REST
Model runners	<code>venv-qwen (3.10)</code> and <code>venv-moss (3.12)</code>	Module 2 and Module 3 inference

Models are invoked as **subprocesses**, never imported into the API process. Three consequences follow, all deliberate:

KEY POINT: The three reasons for subprocess isolation

- 1. Memory isolation.** Qwen2.5-VL peaks near 12 GB and MOSS near 12.1 GB on a 17.18 GB machine. Neither can leak into the long-lived API process, and **process exit returns memory to the operating system unconditionally** - which is more reliable than garbage collection plus `torch.mps.empty_cache()`.
- 2. Dependency isolation.** Module 2 needs Python 3.10 with torch 2.13; MOSS needs Python 3.12 with torch 2.9.1. These cannot coexist in one interpreter.
- 3. Failure isolation.** A model crash returns a non-zero exit code, which is converted into a readable message. It cannot take the server down.

37.2 The layers

<code>api/routes.py</code>	HTTP surface; validation and error mapping only
<code>core/jobs.py</code>	job store, state machine, background worker, disk persistence
<code>core/config.py</code>	paths, interpreters, defaults, limits
<code>services/</code>	one module per pipeline concern
<code>runners/</code>	scripts executed inside each model environment

`routes.py` contains no algorithm. It validates input, calls a service, and maps exceptions to HTTP status codes. Every route is under 20 lines.

37.3 The job model

A job is a serialisable dataclass persisted to `data/jobs/<id>.json` **after every transition**, so state survives a reload and is inspectable after the fact.

```
created -> queued -> running -> completed
                \-> failed
```

Nine stages, each pending | active | done | skipped | failed.

```
STAGES = [
    ("upload", "Video uploaded"),
    ("validation", "Video validation"),
    ("action_recognition", "Action recognition"),
    ("timeline", "Action timeline generation"),
    ("foley_generation", "Foley generation"),
    ("foley_validation", "Foley quality validation"),
    ("visual_sync", "Visual synchronization"),
    ("audio_mixing", "Audio mixing"),
    ("rendering", "Final video rendering"),
```

```
]

```

The store is thread-safe with an `RLock`, and every mutation persists to disk:

```
def stage(self, jid, stage, state, progress=None, **extra):
    with self._lock:
        job = self._jobs.get(jid)
        job.stages[stage] = state
        if state == "active":
            job.current_stage = stage
        if progress is not None:
            job.progress = round(max(job.progress, min(100.0, progress)), 1)
        for k, v in extra.items():
            setattr(job, k, v)
        job.updated_at = _now()
    self._persist(job)

```

Note `max(job.progress, ...)`: **progress never goes backwards**, even if a stage reports a lower number than a previous one.

37.4 The background worker

```
def run(self, jid, fn):
    job = self.get(jid)
    if not job or job.status == "running":
        return # idempotent: no double-start
    self.update(jid, status="running", started_at=_now(), progress=2.0)

    def _target():
        try:
            fn(self.get(jid), self)
            j = self.get(jid)
            if j and j.status == "running":
                self.update(jid, status="completed", progress=100.0,
                           finished_at=_now(), current_stage="done")
        except Exception as exc:
            traceback.print_exc() # full detail to the backend log
            msg = str(exc) or exc.__class__.__name__
            self.fail(jid, msg, stage=(self.get(jid).current_stage if ... else None))

    t = threading.Thread(target=_target, daemon=True, name=f"job-{jid}")
    self._threads[jid] = t
    t.start()

```

Detail	Why
Early return if already running	Pressing Generate twice cannot start two pipelines
<code>daemon=True</code>	The thread does not prevent the server from shutting down
Named <code>job-<id></code>	Threads are identifiable in a stack dump
<code>traceback.print_exc()</code> then a plain message	Full detail to the log, a readable sentence to the user. Stack traces are never returned to the client
Status only set to completed if still running	A job that failed or was cancelled inside <code>fn</code> is not overwritten

37.5 Real progress, not a fake bar

KEY POINT: The claim, and how it is kept

"Progress is a real function of stage completion. No progress value is fabricated on the client."

Within action recognition, the runner writes a progress file after every window:

```
emit(a.progress, stage="recognition", pct=12 + int(70 * i / len(wins)),
     detail=f"window {i}/{len(wins)}: {act}")
```

and the pipeline polls it once a second from a daemon thread:

```
def _poll():
    while True:
        j = store.get(jid)
        if not j or j.stages.get("action_recognition") != "active":
            return
        d = json.loads(prog.read_text())
        store.stage(jid, "action_recognition", "active", 10 + 0.35 * float(d.get("pct", 0)))
        time.sleep(1.0)
```

So the percentage reflects **windows actually processed**, communicated across a process boundary by a file. The poll thread exits as soon as the stage is no longer active.

37.6 The endpoints

Method	Path	Purpose
GET	/api/health	component availability, backends, stages, defaults, limits
GET	/api/actions/supported	the curated Foley classes and the vocabulary mode
POST	/api/upload	multipart upload, creates a job
POST	/api/demo	creates a job against the bundled validated clip
POST	/api/process/{job_id}	starts background processing, optional settings body
GET	/api/status/{job_id}	poll this every 1 to 2 seconds
GET	/api/actions/{job_id}	timeline, visual events, silent intervals
GET	/api/result/{job_id}	full result; 409 if the job is not completed
GET	/api/preview/{job_id}	the uploaded source video
GET	/api/video/{job_id}	the final video, inline
GET	/api/audio/{job_id}	the mixed audio, 48 kHz mono WAV
GET	/api/download/{job_id}	the final video as final_silent_to_audio.mp4
GET	/api/report/{job_id}	the full processing report as JSON
GET	/api/jobs?limit=20	recent jobs

37.7 Streaming upload with the limit enforced during the stream

```
with open(dest, "wb") as out:
    while chunk := await file.read(1 << 20):          # 1 MB chunks
        size += len(chunk)
        if size > limit:
            out.close(); dest.unlink(missing_ok=True)
```

```
raise HTTPException(413, f"File exceeds the {C.MAX_UPLOAD_MB} MB limit.")
out.write(chunk)
```

The file is never fully loaded into memory, and an oversized upload is aborted and deleted **mid-stream** rather than after it has all arrived.

37.8 Status codes

Code	Meaning
200	success
400	invalid input - unsupported format, undecodable, too long
404	unknown job, or artefact not ready
409	job is not in a state that allows the request
413	upload exceeds the size limit
500	unexpected server error; details logged, never returned

409 for "result requested but the job is still running" is a genuinely correct choice - the request is well-formed but conflicts with the resource's current state - and the test suite asserts it.

37.9 Failure messages

Failure	What the user sees
Not a video	"This file could not be read as a video. It may be corrupted or in an unsupported format."
Too long	"This video is 75.0 s long. The current limit is 60 s, because action recognition cost grows with duration."
Model environment missing	"The action-recognition environment is unavailable on this machine."
Out of memory	"Action recognition stopped because the machine ran low on memory. Close other applications and try again."
No visual event locatable	"Foley was generated, but no visual event could be located to synchronise it to. The original video can still be exported without generated audio."
Render failure	"Final video rendering failed."

KEY POINT: Every message names the cause and, where possible, the remedy

Compare "Error 500" with "the machine ran low on memory, close other applications and try again". The second one is actionable. Every failure sets the job to `failed`, marks the stage, and stores a readable message; full tracebacks go to the backend log only.

Chapter 38 — Subsystem 2: the React frontend

38.1 The stack

Technology	Version	Role
React	18.3.1	UI components and state
Vite	6	Dev server and build; proxies /api to the backend
TypeScript	5.6.3	Strict mode, noUnusedLocals, noUnusedParameters
Tailwind CSS	3.4.17	Utility-first styling

Total application source: **933 lines** across 11 files.

38.2 React concepts you need to be able to explain

SIMPLE EXPLANATION: React in one paragraph

A React application is built from **components**: functions that take some data and return a description of what should appear on screen. When the data changes, React works out the smallest set of changes to the real page and applies them. You never write "find this element and change its text"; you write "given this data, the screen looks like this", and React does the rest.

Concept	Meaning	Example in this project
Component	A function returning UI	Uploader, Pipeline, Results, ActionTimeline
Props	Data passed <i>into</i> a component from its parent	<code><Pipeline status={status} stages={stages} generated={...} /></code>
State	Data a component owns and can change	<code>const [phase, setPhase] = useState<Phase>('idle')</code>
Hook	A function starting with <code>use</code> that lets a component use React features	<code>useState, useEffect, useCallback, useRef</code>
<code>useState</code>	Declares a piece of state and a setter	<code>const [jobId, setJobId] = useState<string> \</code>
<code>useEffect</code>	Runs a side effect after render, re-running when its dependencies change	<code>fetching /api/health once on mount</code>
<code>useCallback</code>	Memoises a function so it is not recreated every render	<code>onFile, onDemo, onGenerate</code>
<code>useRef</code>	Holds a mutable value that does not trigger a re-render when changed	<code>the polling setInterval handle</code>
Custom hook	Your own function that composes the built-in hooks	<code>useJob(jobId)</code>

38.3 The application state machine

```
type Phase = 'idle' | 'ready' | 'processing' | 'done' | 'error'
```

```

idle  --upload or demo-->  ready  --Generate-->  processing  --result-->  done
  ^                               |                               |                               |
  |                               v                               v                               |

```

```
+----- reset() ----- error <----- +-----
```

Figure 38.1 - Five phases. Every screen the user can see corresponds to exactly one.

```
{phase === 'idle' && <Uploader ... />}
{phase === 'ready' && video && jobId && (<><VideoPreview ... /><AdvancedSettings ... /></>)}
{(phase === 'processing' || phase === 'error') && status && (
  <><Pipeline ... />{actions.length > 0 && <ActionTimeline ... /></>)}
{phase === 'done' && result && (<><Results ... /><ActionTimeline ... /></>)}
```

A single variable decides what is on screen. There is no way for two panels to be visible at once, and no way to reach an undefined screen.

38.4 The `useJob` polling hook

```
export function useJob(jobId: string | null) {
  const [status, setStatus] = useState<JobStatus | null>(null)
  const [result, setResult] = useState<ResultPayload | null>(null)
  const [actions, setActions] = useState<ActionRow[]>([])
  const timer = useRef<number | null>(null)
  const seenTimeline = useRef(false)

  const stop = useCallback(() => {
    if (timer.current) { clearInterval(timer.current); timer.current = null }
  }, [])

  useEffect(() => {
    if (!jobId) return
    seenTimeline.current = false
    const tick = async () => {
      try {
        const s = await api.status(jobId)
        setStatus(s)
        // pull the timeline as soon as Module 2 finishes, so the user sees it
        // while Foley generation is still running
        if (!seenTimeline.current && s.stages?.timeline === 'done') {
          seenTimeline.current = true
          const a = await api.actions(jobId)
          setActions(a.actions); setEvents(a.visual_events); setUnsupported(a.unsupported)
        }
        if (s.status === 'completed') {
          stop()
          const [r, a] = await Promise.all([api.result(jobId), api.actions(jobId)])
          setResult(r); setActions(a.actions); ...
        } else if (s.status === 'failed') {
          stop(); setError(s.errors?.[0] ?? 'Processing failed.')
        }
      } catch (e) { stop(); setError((e as Error).message) }
    }
    tick() // fire immediately, do not wait 1.5 s
    timer.current = window.setInterval(tick, 1500)
    return stop // cleanup on unmount or jobId change
  }, [jobId, stop])

  return { status, result, actions, events, unsupported, error }
}
```

Detail	Why
Header comment: <i>"Polls real backend job state. Progress is never synthesised on the client."</i>	The claim is enforced at the top of the file that would be the natural place to cheat
<code>tick()</code> before <code>setInterval</code>	The first poll is immediate, so the UI updates within milliseconds rather than after 1.5 seconds

Detail	Why
<code>seenTimeline</code> is a <code>useRef</code> , not state	Changing it must not trigger a re-render; it is bookkeeping, not display data
Fetching the timeline as soon as <code>stages.timeline === 'done'</code>	The user sees the detected actions while the four-minute Foley generation is still running. This is the single best UX decision in the application
<code>Promise.all</code> at completion	Two requests in parallel rather than in series
<code>return stop</code> from <code>useEffect</code>	React calls this on unmount or when <code>jobId</code> changes, so the interval can never leak
<code>stop()</code> on any error	A broken backend does not produce an infinite loop of failing requests
Polling, not WebSockets	Simpler, no connection state to manage, and a 1.5 s latency is irrelevant for a four-minute job

38.5 The API client

```

async function j<T>(r: Response): Promise<T> {
  if (!r.ok) {
    let msg = `Request failed (${r.status})`
    try { const d = await r.json(); msg = d.detail ?? d.message ?? msg } catch { }
    throw new Error(msg)
  }
  return r.json() as Promise<T>
}

```

One generic response handler. It extracts FastAPI's `detail` field so the backend's carefully written error messages reach the user, and falls back to a status-code message if the body is not JSON.

Upload uses `XMLHttpRequest`, not `fetch`, and that is deliberate:

```

upload: (file: File, onProgress?: (pct: number) => void) =>
  new Promise((resolve, reject) => {
    const fd = new FormData(); fd.append('file', file)
    const xhr = new XMLHttpRequest()
    xhr.open('POST', `${BASE}/upload`)
    xhr.upload.onprogress = e => {
      if (e.lengthComputable && onProgress) onProgress((e.loaded / e.total) * 100)
    }
    ...
  })

```

KEY POINT: Why `XMLHttpRequest` in 2026

`fetch` has no upload-progress event. `XMLHttpRequest` has `xhr.upload.onprogress`. For a 200 MB video on a slow connection, an upload progress bar is not decoration - it is the difference between "this is working" and "this has frozen".

Using the older API deliberately, for a capability the newer one lacks, is a good answer to "why did you not use `fetch`?"

38.6 TypeScript interfaces as a contract

`src/types/index.ts` mirrors every backend response shape. With `strict: true`, any mismatch between what the API returns and what a component consumes is a **compile error**, not a runtime crash.

```
export interface JobStatus {
  job_id: string
  status: 'created' | 'queued' | 'running' | 'completed' | 'failed'
  progress: number
  current_stage: string
  stages: Record<string, string>
  errors: string[]
  warnings: string[]
  counts: Record<string, number>
  generated_audio: GeneratedSound[]
  updated_at: string
}
```

The `status` field is a **union of string literals**, so if `(s.status === 'complete')` - a typo - fails to compile.

38.7 The action timeline visualisation

The most informative component. It renders a table and a proportional bar chart, and overlays the visual events as white markers:

```
{events.map((e, i) => (
  <div key={i} title={` ${e.kind} @ ${e.t_s.toFixed(3)}s`}
    className="absolute top-0 h-full w-px ..."
    style={{ left: `${(e.t_s / d) * 100}%` }}>
    </div>))}
```

with the caption:

"White markers are visual events - the exact frames where sound is anchored, detected independently of the action-label boundaries."

KEY POINT: Why this one component is worth demonstrating live

It makes the project's central idea **visible**. The coloured blocks are what the action recogniser said. The markers are where the sound actually goes. A viewer can see with their own eyes that the markers do not sit at the block edges - which is the entire argument of the project, shown rather than asserted.

38.8 Honest reporting in the interface

The `Results` component does not hide failures:

- Generated sounds show `best of N` when several candidates were tried, with a tooltip

explaining that the best was selected by quality score.

- The quality score is shown as `quality 86/100`.
- Rejected classes are listed with the message `"No usable Foley generated - Reason:`

`generated audio failed quality validation. The interval was intentionally left silent."`

- A "Show quality measurements" toggle reveals the raw numbers: peak dBFS, dynamic range, effective bits, harmonic ratio and gain needed.

- Silent intervals are listed with their reason.

KEY POINT: An interface that admits failure

Most student projects hide errors. This one has a dedicated panel for them, with the measurements attached. If an examiner asks "what happens when it fails?", the answer is "let me show you - it tells the user exactly what failed and by how much."

Chapter 39 — The complete request flow

This chapter answers, in one place, the question *"what happens internally when a user uploads a video and presses Generate?"*.

39.1 The diagram

```

USER
  | drags a file onto the drop zone
  v
Uploader.tsx -> onFile(f)
  |
  v
api.upload(file, onProgress)      XMLHttpRequest, so upload progress is reported
  | POST /api/upload  multipart/form-data, field "file"
  v
routes.py::upload()
  | 1. check the extension against ALLOWED_SUFFIX
  | 2. stream to data/uploads/<uid><ext> in 1 MB chunks, aborting over 200 MB
  | 3. VS.probe(dest)      -> ffprobe: duration, w, h, fps, frames, codec, has_audio
  | 4. VS.validate(info)   -> raises VideoError, or returns warnings
  | 5. STORE.create(...)   -> a Job with stages{} all "pending", upload="done"
  | 6. persist to data/jobs/<id>.json
  v
{ job_id, video, warnings, original_filename }
  |
  v
App.tsx: setJobId, setVideo, setWarnings, setPhase('ready')
  |
  v
VideoPreview shows the clip, its metadata, and whether an audio track was found
AdvancedSettings lets the user change seed, steps, cfg, sigma, duration, candidates
  |
  | USER presses "Generate Sound"
  v
api.process(jobId, settings)
  | POST /api/process/{job_id}  body = settings
  v
routes.py::process()
  | merge allowed settings keys over the job's existing settings
  | STORE.update(status="queued", errors=[])
  | STORE.run(job_id, run_pipeline)  -> starts a daemon thread, returns immediately
  v
{ job_id, status: "queued" }          <- the HTTP request ends here
  |
  v
App.tsx: setPhase('processing')  -> useJob(jobId) begins polling every 1.5 s

```

39.2 What the background thread does

```

run_pipeline(job, store)          [ thread "job-<id>" ]
  |
  |-- stage validation  active 4%
  |   VS.probe / VS.validate again on the stored file
  |-- stage validation  done 8%
  |
  |-- stage action_recognition  active 10%
  |   if demo: load the stored Module 2 JSON
  |   else:    start a poll thread on m2_progress.json
  |             AR.run(video, m2_json, progress_file=prog)
  |             |
  |             +--> subprocess: venv-gwen/bin/python run_module2.py
  |                   AR.probe / AR.extract_video_frames (-map 0:v:0)
  |                   AR.plan_windows(duration)

```



```

|         load Qwen2.5-VL-3B-Instruct ONCE on MPS, bfloat16
|         for each window: memory guard -> 8 frames -> generate
|                             -> parse ACTION/EVIDENCE -> emit progress
|
|         AR.merge(windows)
|         RS.resolve_boundaries(merged) -> (segments, adjustments)
|         RS.flag_suspects / RS.validate
|         write module2.json
|-- stage action_recognition done 45%
|
|-- stage timeline active 46%
|     resolved = m2["resolved_actions"]         <- NOT the raw overlapping array
|     actions = [{action, start, end, status, confidence}, ...]
|-- stage timeline done 50%                     <- THE UI FETCHES THE TIMELINE HERE
|
|-- stage foley_generation active 51%
|     for each action: spec, reason = resolve(action)
|                     spec is None -> unsupported.append({action, start, end, reason})
|                     else -> needed[spec.key] = spec
|
|     for each distinct spec:
|         SG.generate_best(spec, settings, max_candidates=3, on_progress=...)
|         for seed in (42, 43, 44):
|             cached_path exists? -> reuse, was_cached=True
|             else subprocess: venv-moss/bin/python moss_generate.py
|                 PHASE 1 Qwen3 text encoder -> ctx_posi, ctx_neg -> free
|                 PHASE 2 DiT, 50 steps, CFG 4 -> latent (1,128,1500) -> free
|                 PHASE 3 DAC VAE decode -> crop to 10 s -> write PCM_16 WAV
|                 FV.validate(path, target_rms, sample_rate) -> six gates + score
|                 if ok and score >= 45: break
|-- stage foley_generation done 72%
|
|-- stage foley_validation active 73%
|     record every attempt; pick the best passing candidate per class
|     a class with no passing candidate -> its intervals become no_usable_foley
|-- stage foley_validation done 76%
|
|-- stage visual_sync active 77%
|     SY.analyse_video(video) ffmpeg -> 320x180 grey at 24 fps -> 4 band motion signals
|     merge consecutive CONTINUOUS intervals of the same class (gap <= 0.15 s)
|     for each merged action with a usable asset:
|         SY.detect_events(...) -> footstep | hold | contact | continuous
|         SY.plan_action(...) -> select a segment, align on TRUE ATTACK
|-- stage visual_sync done 85%
|
|-- stage audio_mixing active 86%
|     AP.mix(placements, duration, out_wav, 48000)
|     per clip: zero-cross snap -> DC removal -> 12 ms fades
|               -> active-RMS gain (refuse above +25 dB) -> -6 dBFS peak cap
|               -> truncate or omit at the end of the video
|     bus: sum -> normalise to -6 dBFS -> safety limiter -> reject NaN/clipping
|-- stage audio_mixing done 92%
|
|-- stage rendering active 93%
|     VR.mux(video, out_wav, out_mp4, 48000)
|     ffmpeg -map 0:v:0 -map 1:a:0 -c:v copy -c:a aac -b:a 192k -ar 48000
|             -movflags +faststart -shortest
|     re-probe the output and return codecs, durations, frames, size
|-- stage rendering done 99%
|
|-- build the report: settings, video, validations, module2, visual_events,
|                     merged_intervals, foley, unsupported, placements, mix,
|                     render, sync{worst_error_ms}, counts{...}
|     write data/jobs/<id>/report.json and STORE.update(report=report)
v
the worker sets status="completed", progress=100

```

39.3 What the browser does while that happens

```

every 1.5 s: GET /api/status/{job_id}
            -> { status, progress, current_stage, stages{}, generated_audio[] }
            -> Pipeline re-renders: tick, spinner or empty circle per stage

once stages.timeline == "done":
    GET /api/actions/{job_id}
    -> { actions[], visual_events[], unsupported[] }
    -> ActionTimeline appears WHILE Foley generation is still running

once status == "completed":
    clearInterval
    GET /api/result/{job_id} and GET /api/actions/{job_id} in parallel
    -> Results renders the video player, the counts, the sync figure,
        the mix measurements, the generated sounds with their scores,
        and the intervals left silent
    -> setPhase('done')

if status == "failed":
    clearInterval; show errors[0]; setPhase('error')

```

39.4 Every arrow, explained

Arrow	Mechanism
Browser to backend	HTTP over localhost. In development the Vite dev server proxies /api to <code>http://127.0.0.1:8000</code> , so the browser sees one origin and there is no CORS problem
Route to service	A direct Python function call. Routes contain no algorithm
Backend to Module 2	<code>subprocess.run</code> with a different Python interpreter, a JSON file for output, and a JSON file for progress
Module 2 back to the pipeline	A written <code>module2.json</code> , plus <code>stdout</code> for a success line and <code>stderr</code> for an error line
Progress across the process boundary	<code>m2_progress.json</code> , written by the child once per window, polled by a thread in the parent once a second
Backend to MOSS	<code>subprocess.run</code> with the <code>venv-moss</code> interpreter, writing a WAV to the cache path
Between pipeline stages	Plain Python objects in one function's local scope
Pipeline to the browser	The job store, persisted to disk, read by the polling endpoints
Backend to disk	<code>data/uploads</code> , <code>data/generated</code> (the cache), <code>data/outputs</code> , <code>data/jobs</code>
Backend to the user's file	<code>FileResponse</code> with a filename so the browser downloads rather than navigates

PART 7

CODE ARCHITECTURE AND INTEGRATION

Who calls whom, what every file is for, how three incompatible environments coexist, and what happens to the video file at each stage.

Chapter 40 — Who calls whom

40.1 Subsystem 2 call graph

```

    uvicorn
    |
    v
    backend/main.py          creates FastAPI(), adds CORS, includes the router
    |
    v
    backend/api/routes.py    imports: core.config, core.jobs, services.video_service,
                             services.prompt_map, services.pipeline
    |
    +--> core/jobs.py         STORE.create / update / stage / fail / run
    |
    |       +--> core/config.py      JOBS, UPLOADS, GENERATED, OUTPUTS paths
    |       +--> threading.Thread    the background worker
    |
    +--> services/video_service.py  probe(), validate()  -> ffmpegprobe
    |
    +--> services/pipeline.py       run_pipeline(job, store)
    |
    |       +--> services/video_service.py  probe / validate
    |
    |       +--> services/action_recognition.py run()
    |               +--> subprocess -> venv-qwen -> backend/runners/run_module2.py
    |               |
    |               |       +--> 03-FoleyCrafter-Test/action-recognition/
    |               |       |       action_recognition.py  (probe, frames,
    |               |       |       windows, prompt, parse, merge)
    |               |       |       resolve_segments.py    (boundaries,
    |               |       |       suspects, validate)
    |               |
    |       +--> services/prompt_map.py      resolve(action_phrase)
    |               +--> services/prompt_synthesis.py synthesise()  [open vocabulary]
    |
    |       +--> services/sound_generation.py generate_best()
    |               +--> services/foley_validation.py validate()
    |               +--> subprocess -> venv-moss -> moss/scripts/moss_generate.py
    |               |
    |               |       +--> moss/scripts/moss_phased.py  loaders
    |               |       +--> moss/scripts/mps_compat.py   the shim
    |               |       +--> moss/MOSS-TTS/    (imported, NEVER modified)
    |
    |       +--> services/synchronization.py analyse_video / detect_events / plan_action
    |               +--> scripts/visual_events.py  load_frames, motion, _smooth
    |
    |       +--> services/audio_processing.py  mix()
    |               +--> services/foley_validation.py  MAX_AUTO_GAIN_DB
    |
    +--> services/video_render.py           mux()  -> ffmpeg
  
```

Figure 40.1 - The complete call graph. Note that `pipeline.py` is the only module that knows the order of the stages.

40.2 Subsystem 1 call graph

```

    run_server.py
    |
    +--> app/api.py          Flask app, routes
    |
    |       +--> app/inference.py      AutoAVSRLipReader
    |               +--> preprocess_video.py      decode, landmarks, crop, transform
    |               |
    |               |       +--> auto_avsr/preparation/detectors/mediapipe/detector.py
    |               |       +--> auto_avsr/preparation/detectors/mediapipe/video_process.py
    |               |       +--> auto_avsr/datamodule/transforms.py
  
```

```

|         |         +--> auto_avsr/lightning.py      ModelModule, get_beam_search_decoder
|         |         +--> app/timing.py              word_timings, motion_onset, find_pauses
|         |
|         |         +--> app/video_processing.py     probe, validate, standardize, safe_upload_name
|         |         +--> app/gender.py              GenderDetector -> cv2.dnn
|         |         +--> app/tts.py                 TTSEngine, KokoroWorker
|         |         +--> subprocess -> venv-tts -> tts_worker.py (Kokoro ONNX)
|         |         +--> app/sync.py                build_track, mux -> ffmpeg

```

Figure 40.2 - Subsystem 1.

40.3 Subsystem 3 call graph

```

run.py / uvicorn
|
+--> backend/main.py          FastAPI app; also serves frontend/index.html
|
+--> backend/api/routes.py    /health /upload /process /process-local
|                             /status /result + the in-memory job store
|
+--> backend/utils/file_handler.py  safe names, size caps, TTL
|
+--> backend/processing/pipeline.py  run_pipeline
|
|
+--> processing/video_reader.py      open, validate, iterate
|
+--> processing/visual_microphone.py sound_from_frames
|
+--> pytools SteerablePyramidFreq
|
+--> processing/signal_processing.py highpass, notch_mains,
|                                     lowpass, spectral_subtraction
|
+--> processing/audio_writer.py      WAV + waveform + spectrogram
|
+--> processing/text_report.py       analyze_signal, transcribe
+--> backend/config.py              every tunable, plus config.json overrides

```

Figure 40.3 - Subsystem 3.

40.4 Import discipline

Two patterns recur and are worth naming.

`sys.path` **insertion at the top of each backend module:**

```

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from core import config as C

```

This lets `backend/` be importable both as a package under `uvicorn` and as a script under the test runner, without a package install step.

Local imports to break cycles:

```

def resolve(action_phrase):
    ...
    from .prompt_synthesis import synthesise      # local import avoids a cycle
    return synthesise(action_phrase), None

```

`prompt_map` needs `prompt_synthesis`, and `prompt_synthesis` needs `FoleySpec` from `prompt_map`. Importing inside the function breaks the cycle at module-load time.

Chapter 41 — File-by-file reference

41.1 Subsystem 2 backend

File	Lines	Purpose	Key functions	Input	Output	Connected to
backend/main.py	23	FastAPI application object	root()	-	-	api/routes.py
backend/api/routes.py	201	Every HTTP endpoint	health, upload, demo, process, status, actions, result, video, audio, download, report, jobs	HTTP	JSON or files	jobs, video_service, prompt_map, pipeline
backend/core/config.py	111	Paths, interpreters, backends, defaults, limits	backend(name)	-	constants	everything
backend/core/jobs.py	169	Job store, state machine, worker	JobStore.create/get/update/stage/fail/run	-	Job records	config, pipeline
backend/services/video_service.py	87	Probe and validate	probe, validate, ensure_ffmpeg	a file path	VideoInfo, warnings	ffprobe
backend/services/action_recognition.py	45	Module 2 wrapper	run, load_existing	video path	Module 2 JSON	subprocess to venv-qwen
backend/services/prompt_map.py	345	17 curated Foley classes	resolve, supported_actions, vocabulary_mode	action phrase	FoleySpec or a reason	prompt_synthesis
backend/services/prompt_synthesis.py	239	Open-vocabulary prompt writer	classify, synthesise	action phrase	FoleySpec	prompt_map
backend/services/sound_generation.py	161	Generation, cache, candidates	cache_key, cached_path, generate, generate_best	FoleySpec, settings	WAV path, attempts	foley_validation, subprocess
backend/services/foley_validation.py	185	The six gates and the score	measure, quality_score, validate	WAV path	FoleyVerdict	numpy, librosa, soundfile
backend/services/synchronization.py	308	Events and alignment	analyse_video, detect_events, plan_action, attack_times, select_event_clip, select_wet_segments	video, spec, asset	placements	scripts/visual_events.py
backend/services/audio_processing.py	181	Mixing	mix, active_rms, soft_limit, rcos_fade, snap_zero	placements	mixed WAV, mix log	config, foley_validation
backend/services/video_render.py	36	Final mux	mux	video, WAV	MP4, render info	ffmpeg
backend/services/pipeline.py	252	Stage orchestration	run_pipeline	Job, JobStore	the report	every service

File	Lines	Purpose	Key functions	Input	Output	Connected to
backend/runners/run_module2.py	130	Runs inside venv-qwen	main	--video --out --progress	Module 2 JSON	the validated Module 2 source
backend/runners/run_stable_audio.py	118	Runs inside venv-stable-audio	main	prompt, settings	44.1 kHz stereo WAV	stable_audio_tools

41.2 Subsystem 2 frontend

File	Lines	Purpose
src/main.tsx	7	Mounts <App /> in StrictMode
src/App.tsx	179	Phase state machine, health check, handlers, layout
src/api/client.ts	59	Typed API client; XHR upload with progress
src/hooks/useJob.ts	50	Polling hook
src/types/index.ts	57	Every response interface
src/components/Uploader.tsx	75	Drag and drop, file picker, demo button
src/components/VideoPreview.tsx	68	Preview, metadata, warnings, Generate
src/components/Pipeline.tsx	87	Nine-stage progress list
src/components/ActionTimeline.tsx	116	Action table, proportional bar chart, event markers
src/components/Results.tsx	148	Player, counts, quality detail, downloads
src/components/AdvancedSettings.tsx	58	Seed, steps, CFG, sigma, duration, candidates

41.3 The MOSS wrapper and the standalone scripts

File	Lines	Purpose
moss/scripts/moss_generate.py	274	The phased generation driver; writes the WAV and a full JSON record
moss/scripts/moss_phased.py	238	Component loaders, MemoryTracker, cast_params_only, live_instances
moss/scripts/mps_compat.py	81	The float64-to-CPU shim, with numerical verification
scripts/visual_events.py	187	Frame decode, band motion, smoothing - the validated primitives
scripts/m3_config.py	58	Paths, approved assets, level targets, walking search span
scripts/sync_actions.py	232	The validated standalone synchronisation planner
scripts/audio_mixer.py, polish_mix.py	128, 187	First-pass and polished mixes
scripts/analyze_sync.py, qa_polished.py	126, 110	The 19-check quality gates
scripts/run_module3.py	41	Orchestrates the eleven standalone build steps

41.4 Subsystem 1 and 3

File	Lines	Purpose
02-Auto-AVSR-Test/app/api.py	267	Flask routes and error handling
.../app/inference.py	206	AutoAVSRLipReader
.../app/video_processing.py	256	Validation and standardisation
.../app/timing.py	193	CTC forced alignment, motion onset, pauses
.../app/tts.py	360	Voice selection, Kokoro worker, Piper fallback
.../app/sync.py	305	Phrase grouping, pacing, placement, mux
.../app/gender.py	128	Gender classification from the face
.../preprocess_video.py	99	The official Auto-AVSR preprocessing chain
.../tts_worker.py	72	Kokoro worker, runs in venv-tts
Acoustic eye/.../visual_microphone.py	264	The phase-based core
.../signal_processing.py	166	High-pass, mains notch, low-pass, spectral subtraction
.../video_reader.py	335	Robust decode and real frame counting
.../pipeline.py	356	Eight-stage orchestration
.../api/routes.py	425	Endpoints and the job store
.../config.py	234	Every tunable, with a rationale comment

Chapter 42 — Environments and process isolation

42.1 The five environments

Environment	Python	Key packages	Used for
Module3_Fresh/moss/venv-moss	3.12.13	torch 2.9.1, torchaudio 2.9.1, numpy 1.26.4, transformers 4.57.1, diffusers 0.37.1, descript-audiotools 0.7.2, soundfile, librosa, scipy, fastapi, uvicorn	The backend and MOSS generation
03-FoleyCrafter-Test/action-recognition/qwen/venv-qwen	3.10	torch 2.13, transformers, psutil, Pillow	Qwen2.5-VL action recognition
.../stable-audio/venv-stable-audio	3.10	stable_audio_tools, torch	The alternative Foley backend
02-Auto-AVSR-Test/venv-autoavsr	3.11	torch 2.5.1, torchvision 0.20.1, torchaudio 2.5.1, numpy 1.26.4, av 13.1.0, mediapipe 0.10.21, pytorch-lightning 2.4.0, sentencepiece, flask 3.0.3, piper-tts	Lip reading
02-Auto-AVSR-Test/venv-tts	3.11	kokoro-onnx (needs numpy>=2)	Text to speech only

Two further environments exist for models that were evaluated and rejected: `venv-foley`, `venv-audioldm2` and `venv-mmaudio`.

KEY POINT: Why this is not over-engineering

It is forced. `numpy==1.26.4` and `numpy>=2` cannot coexist. `torch 2.5.1` and `torch 2.9.1` cannot coexist. Python 3.10, 3.11 and 3.12 cannot coexist in one interpreter. The alternative to five environments is not one environment; it is a system that does not run.

42.2 How a subprocess is invoked

```
cmd = [str(C.PY_QWEN), str(C.RUNNERS / "run_module2.py"),
      "--video", str(video), "--out", str(out_json),
      "--min-avail-gb", str(C.MIN_AVAILABLE_GB)]
if progress_file:
    cmd += ["--progress", str(progress_file)]
p = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout_s,
                  env=C.ENV_NO_PYC, cwd=str(C.MODULE3))
```

`C.PY_QWEN` is an **absolute path to another interpreter**. The parent never imports the child's dependencies.

```
ENV_NO_PYC = {**os.environ, "PYTHONDONTWRITEBYTECODE": "1",
              "PYTHONUNBUFFERED": "1", "TORCHDYNAMO_DISABLE": "1"}
```

Variable	Why
<code>PYTHONDONTWRITEBYTECODE=1</code>	Do not write <code>.pyc</code> files into the protected model repositories
<code>PYTHONUNBUFFERED=1</code>	Progress output arrives immediately rather than sitting in a buffer
<code>TORCHDYNAMO_DISABLE=1</code>	<code>torch.compile</code> has no MPS path

42.3 Structured error propagation

The child prints JSON on failure:

```
except Exception as exc:
    print(json.dumps({"ok": False, "error": f"{type(exc).__name__}: {exc}"}),
          file=sys.stderr)
    sys.exit(1)
```

The parent parses it and translates it into something a user can act on:

```
if p.returncode != 0:
    detail = (p.stderr or "").strip().splitlines()
    msg = detail[-1] if detail else "unknown error"
    try:
        msg = json.loads(msg).get("error", msg)
    except Exception:
        pass
    if "MemoryError" in msg or "available RAM" in msg:
        raise ActionRecognitionError(
            "Action recognition stopped because the machine ran low on memory. "
            "Close other applications and try again.")
    raise ActionRecognitionError(f"Action recognition failed: {msg}")
```

KEY POINT: The pattern to describe in a viva

Structured error propagation across a process boundary: the child emits machine-readable JSON on stderr, the parent parses it, recognises specific conditions, and turns them into domain-specific exceptions with human-readable messages. If parsing fails, it degrades gracefully to the last line of stderr rather than crashing.

42.4 Protecting validated work

HANDOFF.md records a set of hard constraints:

- moss/MOSS-TTS/ must return an empty git status --porcelain.
- Two approved audio assets are **filesystem write-protected** at r--r--r--, with their

SHA-256 hashes recorded in results/APPROVED_ASSETS.lock. Write attempts are refused by the operating system, and **this was tested rather than assumed**.

- The source video's SHA-256 is verified before and after every build.
- The validated environments may be run but never modified, and no packages may be installed into them.

KEY POINT: Why an examiner should care about this

It is reproducibility engineering. If your assets can silently change, none of your measurements mean anything. Hashing them, write-protecting them, and asserting the hashes in an automated check is how you make a result stand up months later.

Chapter 43 — Data flow: what happens to the video file

43.1 Subsystem 2, stage by stage

```

user's file          an MP4 on the user's machine
|
| multipart HTTP upload, 1 MB chunks
v
data/uploads/<uid>.mp4          bytes on disk, unmodified
|
| ffprobe (metadata only, no decode)
v
VideoInfo{duration_s, width, height, fps, frames, codec, has_audio, size_bytes}
|
| ffmpeg -map 0:v:0 -vf scale=448:252 -pix_fmt rgb24 -f image2pipe
v
numpy uint8 array (N, 252, 448, 3)          RGB frames in memory, NO AUDIO
|
| 8 frames per 2 s window
v
PIL images -> Qwen processor -> input_ids + pixel tensors on MPS
|
| model.generate(...), greedy
v
raw text  "ACTION: pick up cup\nEVIDENCE: a hand grasps a mug"
|
| parse_response -> action_head -> merge -> resolve_boundaries -> flag_suspects
v
module2.json {windows[], actions[], resolved_actions[], boundary_resolution{}}
|
| pipeline: resolved_actions -> actions[{action,start,end,status,confidence}]
v
ACTION TIMELINE
|
| prompt_map.resolve(phrase)  -> FoleySpec(prompt, negative, strategy,
|                                region, selection, target_rms)
v
a text prompt + " duration: 10.0s"
|
| Qwen3 text encoder
v
context tensors (1, 512, 2048) x2 (positive and negative)
|
| DiT, 50 flow-matching steps, CFG 4.0
v
latent (1, 128, 1500)
|
| DAC VAE decode
v
waveform (1, 1, 1440000)  = 30 s at 48 kHz
|
| crop
v
data/generated/moss_<key>_<hash>.wav          10 s, 48 kHz, mono, PCM_16
|
| foley_validation.measure (RAW - no gain)
v
FoleyMetrics{peak_dbfs, active_rms_dbfs, dynamic_range_db, effective_bits,
              spectral_flatness, harmonic_ratio, required_gain_db, silence_pct}
|
| six gates -> pass or fail; passing candidates scored 0-100
v
a validated asset (or an interval marked no_usable_foley)
|
|                               ...meanwhile, from the same source video...
|               ffmpeg -vf fps=24,scale=320:180 -pix_fmt gray -f rawvideo
|
|                               v

```

```

|          numpy uint8 (240, 180, 320) greyscale frames
|
|          | mean |inter-frame difference| per band
|          v
|          motion signals: feet, head, table, full
|          | strategy-specific detection
|          v
|          VisualEvent{action, kind, t_s, confidence, basis}
v          |
placements[] <----- plan_action(spec, asset, events, interval, ...)
{asset, asset_start_s, asset_end_s, video_start_s, aligned_to_s,
  alignment_kind, per_event_error_ms, target_rms_dbfs, strategy}
|
| mix(): snap -> DC -> fades -> gain -> cap -> sum -> normalise -> limit
v
data/outputs/<job>_audio.wav          48 kHz mono PCM_16, peak -6 dBFS
|
| ffmpeg -c:v copy -c:a aac -b:a 192k
v
data/outputs/<job>_final.mp4          picture bit-identical, AAC audio added
|
v
the user's download                  final_silent_to_audio.mp4

```

Figure 43.1 - The complete data flow. Every named type in this diagram is a real dataclass or a real file on disk.

43.2 Important data formats

Format	Where	Detail
MP4 / H.264	input and output	The picture stream is byte-identical between them
Raw RGB24 over a pipe	Qwen frame extraction	N x 252 x 448 x 3 uint8, no image files
Raw grey8 over a pipe	motion analysis	240 x 180 x 320 uint8
PCM_16 WAV	every generated and mixed asset	48 kHz mono
AAC 192 kbit/s	the output audio track	48 kHz mono
JSON	Module 2 output, job records, reports, generation records	Human-readable, greppable, diffable
SHA-256, first 16 hex characters	the Foley cache key	Content-addressed

43.3 Where things are written

Directory	Contents	Lifetime
data/uploads/	uploaded videos, named by UUID	until manually cleaned
data/jobs/<id>.json	the job record, rewritten after every transition	permanent
data/jobs/<id>/module2.json	the full Module 2 payload	permanent
data/jobs/<id>/m2_progress.json	per-window progress	overwritten during the run
data/jobs/<id>/report.json	the complete processing report	permanent
data/generated/	the Foley cache, content-addressed	permanent, reused across jobs
data/outputs/	mixed WAV and final MP4 per job	permanent

Directory	Contents	Lifetime
results/web_*_generation .json	one record per MOSS run: config, phases, memory, output	permanent

KEY POINT: Why the job record is written after every transition

Because it makes the system inspectable after the fact. There are 122 job records and 32 full reports on disk, and every claim in this handbook about what the pipeline actually did was read out of them. A system that only reports to a browser leaves no evidence.

PART 8

IMPORTANT CODE, LINE BY LINE

Fifteen pieces of code you should be able to open, read aloud and explain. For each: what it does, why every line is there, what goes in, what comes out, and where it sits in the pipeline.

Chapter 44 — Imports: what they are and why they are there

Before the code itself, here is what every significant import actually provides. An examiner asking "what is FastAPI?" is asking whether you understand your own dependencies.

44.1 The backend imports

```
from fastapi import APIRouter, File, HTTPException, UploadFile, Body
from fastapi.responses import FileResponse, JSONResponse
```

Import	What it is	Why it is here
fastapi	A Python web framework for building APIs. It reads your function's type annotations and uses them to validate requests, generate responses, and build interactive documentation	It is the whole HTTP layer of Subsystem 2
APIRouter	Groups related endpoints so they can be mounted under a common prefix	<code>router = APIRouter(prefix="/api")</code> - every route is automatically under <code>/api</code>
UploadFile, File	FastAPI's streaming file-upload type and its parameter marker	<code>file: UploadFile = File(...).UploadFile</code> streams from a spooled temporary file rather than loading the whole upload into memory
HTTPException	Raise it and FastAPI turns it into an HTTP error response	<code>raise HTTPException(413, "File exceeds the 200 MB limit.")</code>
Body	Marks a parameter as coming from the JSON request body	<code>`settings: dict`</code>
FileResponse	Streams a file from disk, with the right content type and an optional download filename	serving the final MP4, the WAV, and the preview
JSONResponse	Returns a raw dictionary as JSON without re-validating it	returning the processing report

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
                  allow_methods=["*"], allow_headers=["*"])
```

CORS is Cross-Origin Resource Sharing. A browser refuses to let JavaScript served from one origin call a different origin unless the second one explicitly allows it. In development the frontend runs on port 5173 and the backend on port 8000 - different origins - so the middleware is needed.

CAUTION: Say this before the examiner does

`allow_origins=["*"]` allows **any** website to call this API. That is acceptable for a local demonstrator and would be unacceptable in production, where it should be a specific list of origins. In development it is also partly redundant, because the Vite dev server proxies `/api` to the backend so the browser sees a single origin.

```
import numpy as np
import soundfile as sf
import librosa
from scipy.signal import find_peaks, hilbert
```

Import	What it is	Used for
numpy	The array library that all Python numerical work is built on	Every audio buffer, every motion signal, every measurement

Import	What it is	Used for
soundfile	Reads and writes audio files through libsndfile	<code>sf.read</code> , <code>sf.write(..., subtype="PCM_16")</code> . Chosen over <code>scipy.io.wavfile</code> because it writes 16-bit PCM reliably
librosa	A music and audio analysis library	<code>librosa.feature.rms</code> , <code>librosa.stft</code> , <code>librosa.feature.spectral_flatness</code> , <code>librosa.effects.harmonic</code>
<code>scipy.signal.find_peaks</code>	Finds local maxima with constraints on height, prominence and spacing	Foot plants, contacts, transients
<code>scipy.signal.hilbert</code>	The analytic signal, whose magnitude is the amplitude envelope	<code>envelope()</code> - the basis of true attack detection
<code>scipy.signal.butter</code> , <code>iirnotch</code> , <code>filtfilt</code> , <code>sosfilt</code>	Filter design and application	Subsystem 3's high-pass, mains notch and low-pass

```
import subprocess, hashlib, json, threading, uuid, gc
from dataclasses import dataclass, field, asdict
from pathlib import Path
```

Import	Why
subprocess	Runs the model runners in their own interpreters. <code>subprocess.run(..., capture_output=True, timeout=...)</code>
hashlib	SHA-256 for the content-addressed cache key and for asset integrity
json	Every inter-process message, every job record, every report
threading	The background worker, the progress poller, and the <code>RLock</code> on the job store
uuid	Unguessable identifiers for jobs and uploaded files
gc	<code>gc.collect()</code> after freeing a model; <code>gc.get_objects()</code> to <i>prove</i> it was freed
dataclass	Declarative records with automatic <code>__init__</code> and <code>asdict()</code> for JSON
<code>pathlib.Path</code>	Path arithmetic that works the same on every platform. <code>Path(__file__).resolve().parents[2]</code>

```
import torch
from transformers import AutoProcessor, Qwen2_5_VLForConditionalGeneration
```

Import	What it is
torch	PyTorch: tensors, automatic differentiation, and the MPS backend
transformers	Hugging Face's library of pre-trained model implementations and loaders
AutoProcessor	Loads the correct tokenizer and image processor for a given model id
<code>Qwen2_5_VLForConditionalGeneration</code>	The Qwen2.5-VL model class with a language-modelling head, so it can generate text

Chapter 45 — Snippet 1: the application entry point

File: Module3_Fresh/backend/main.py

```
"""ACTION RECOGNITION AND SOUND GENERATION - FastAPI backend."""
from __future__ import annotations
import sys
from pathlib import Path
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

sys.path.insert(0, str(Path(__file__).resolve().parent))
from api.routes import router

app = FastAPI(title="Action Recognition and Sound Generation",
              description="Transform silent videos into synchronized sound",
              version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
                  allow_methods=["*"], allow_headers=["*"])
app.include_router(router)

@app.get("/")
def root():
    return {"name": "ACTION RECOGNITION AND SOUND GENERATION",
            "subtitle": "Transform silent videos into synchronized sound",
            "docs": "/docs", "health": "/api/health"}
```

Line	Explanation
<code>from __future__ import annotations</code>	Makes all type annotations lazily evaluated strings, so <code>`dict`</code>
<code>sys.path.insert(0, ...parent)</code>	Adds <code>backend/</code> to the import path, so <code>from api.routes import router</code> works whether <code>uvicorn</code> is started from the project root or elsewhere
<code>FastAPI(title=..., description=..., version=...)</code>	These three strings become the interactive documentation at <code>/docs</code> , generated for free
<code>add_middleware(CORSMiddleware, ...)</code>	Wraps every request and response with the CORS headers
<code>include_router(router)</code>	Mounts all the <code>/api</code> endpoints defined in <code>routes.py</code>
<code>@app.get("/")</code>	A friendly root that tells a visitor where the docs and the health check are

Input: none - this is a module that `uvicorn` imports. **Output:** the `app` object. **Where it fits:** the top of everything. `uvicorn backend.main:app` starts here.

Chapter 46 — Snippet 2: the upload endpoint

File: backend/api/routes.py

```
@router.post("/upload")
async def upload(file: UploadFile = File(...)):
    suffix = Path(file.filename or "").suffix.lower()
    if suffix not in C.ALLOWED_SUFFIX:
        raise HTTPException(400, f"Unsupported format '{suffix or 'unknown'}'. "
                                f"Supported: {' '.join(sorted(C.ALLOWED_SUFFIX))}.")

    uid = uuid.uuid4().hex[:12]
    dest = C.UPLOADS / f"{uid}{suffix}"
    size = 0
    limit = C.MAX_UPLOAD_MB * 1024 * 1024
    with open(dest, "wb") as out:
        while chunk := await file.read(1 << 20):
            size += len(chunk)
            if size > limit:
                out.close(); dest.unlink(missing_ok=True)
                raise HTTPException(413, f"File exceeds the {C.MAX_UPLOAD_MB} MB limit.")
            out.write(chunk)

    try:
        info = VS.probe(dest)
        warnings = VS.validate(info)
    except VS.VideoError as e:
        dest.unlink(missing_ok=True)
        raise HTTPException(400, str(e))
    job = STORE.create(video_path=str(dest), video_info=info.dict(),
                      warnings=warnings, settings=dict(C.DEFAULTS))
    return {"job_id": job.id, "video": info.dict(), "warnings": warnings,
            "original_filename": file.filename}
```

Line	Explanation
<code>async def</code>	FastAPI runs this on the event loop, so the server stays responsive while a large upload streams in
<code>Path(file.filename or "").suffix.lower()</code>	<code>or ""</code> guards against a missing filename; only the extension is taken from the client
the extension check	Fails fast, before writing a single byte, with a message that lists what <i>is</i> supported
<code>uuid.uuid4().hex[:12]</code>	An unguessable identifier. The client's filename is never used as a path, which closes path traversal and overwrite attacks
<code>while chunk := await file.read(1 << 20)</code>	The walrus operator assigns and tests in one expression. <code>1 << 20</code> is 1,048,576 - one megabyte
the size check inside the loop	An oversized file is rejected mid-stream. Checking afterwards would mean writing 2 GB to disk before refusing it
<code>dest.unlink(missing_ok=True)</code>	Delete the partial file. <code>missing_ok=True</code> means no error if it is already gone
<code>VS.probe</code> then <code>VS.validate</code>	Metadata first, then policy. Probe raises for unreadable files; validate raises for policy violations and returns non-fatal warnings
<code>dest.unlink</code> in the <code>except</code>	A file that fails validation does not linger on disk
<code>STORE.create(...)</code>	Creates the job with all stages pending and upload already done, and persists it
the returned dict	FastAPI serialises it to JSON automatically

Input: a multipart HTTP request with a field named `file`. **Output:** `{job_id, video, warnings, original_filename}`, or 400 / 413. **Where it fits:** stage 1 and stage 2 of the pipeline.

Chapter 47 — Snippet 3: starting the job

File: backend/api/routes.py

```
@router.post("/process/{job_id}")
def process(job_id: str, settings: dict | None = Body(default=None)):
    job = _job_or_404(job_id)
    if job.status == "running":
        return {"job_id": job.id, "status": "running", "note": "Already processing."}
    if settings:
        allowed = {"seed", "steps", "cfg_scale", "sigma_shift", "duration",
                  "sample_rate", "max_candidates", "backend"}
        merged = {(job.settings or C.DEFAULTS),
                  **{k: v for k, v in settings.items() if k in allowed}}
        STORE.update(job_id, settings=merged)
    STORE.update(job_id, status="queued", errors=[])
    STORE.run(job_id, run_pipeline)
    return {"job_id": job.id, "status": "queued"}
```

Line	Explanation
<code>_job_or_404(job_id)</code>	A helper that raises <code>HTTPException(404, "Job not found.")</code> . Every job endpoint uses it, so the behaviour is identical everywhere
the running early return	Idempotent. Pressing Generate twice returns a note rather than starting a second pipeline
the allowed set	An allow-list . Any key the client sends that is not in this set is silently dropped, so a malicious or mistaken client cannot inject arbitrary configuration
<code>{**defaults, **filtered}</code>	Dictionary merge: defaults first, then the filtered overrides, so anything not supplied keeps its default
<code>errors=[]</code>	Clears errors from a previous failed attempt, so a retry starts clean
<code>STORE.run(job_id, run_pipeline)</code>	Starts a daemon thread and returns immediately
returning "queued"	The HTTP request is over in milliseconds; the client now polls

Input: a job id in the path, an optional JSON settings body. **Output:** `{job_id, status}`. **Where it fits:** the transition from "ready" to "processing".

Chapter 48 — Snippet 4: the pipeline orchestrator

File: backend/services/pipeline.py

```
def run_pipeline(job: Job, store: JobStore) -> None:
    jid = job.id
    settings = {**C.DEFAULTS, **(job.settings or {})}
    jdir = C.JOBS / jid
    jdir.mkdir(parents=True, exist_ok=True)
    video = Path(job.video_path)

    # ----- 2. validation
    store.stage(jid, "validation", "active", 4)
    info = VS.probe(video)
    warnings = VS.validate(info)
    store.stage(jid, "validation", "done", 8,
                video_info=info.dict(), warnings=warnings)
```

Line	Explanation
<code>settings = {**C.DEFAULTS, **(job.settings or {})}</code>	Defaults, then the job's overrides. Even a job created before a new default existed gets a complete settings dictionary
<code>jdir.mkdir(parents=True, exist_ok=True)</code>	A per-job directory for <code>module2.json</code> , <code>m2_progress.json</code> and <code>report.json</code> . <code>exist_ok</code> makes it safe to re-run
<code>store.stage(jid, "validation", "active", 4)</code>	Marks the stage active and sets progress to 4 percent, then persists. The browser can see this within 1.5 seconds
Validating again, after the upload already validated	Deliberate. The file could have changed, and the pipeline must not assume anything about how the job was created - <code>/api/demo</code> creates a job without going through <code>/api/upload</code>

The stage percentages are hard-coded and monotonic:

Stage	Enters at	Leaves at
validation	4	8
action_recognition	10	45 (with 10 to 45 driven by the progress file)
timeline	46	50
foley_generation	51	72 (subdivided per class and per candidate)
foley_validation	73	76
visual_sync	77	85
audio_mixing	86	92
rendering	93	99
the worker sets	-	100

The Foley sub-progress is genuinely computed:

```
base_pct = 51 + 21 * (i - 1) / max(1, len(needed))

def _prog(n, total, seed, _b=base_pct, _k=spec.label):
    store.stage(jid, "foley_generation", "active",
                _b + 21 / max(1, len(needed)) * (n - 1) / max(1, total),
                current_detail=f"{_k}: candidate {n}/{total} (seed {seed})")
```

KEY POINT: Two Python details worth noticing

`max(1, len(needed))` prevents division by zero when nothing needs generating.

`_b=base_pct, _k=spec.label` are **default-argument captures**. Without them, the closure would capture the loop variables by reference and every callback would report the *last* class's values. This is the classic Python late-binding closure bug, and it has been avoided deliberately.

Chapter 49 — Snippet 5: resolving an action to a Foley class

File: backend/services/prompt_map.py

```
def resolve(action_phrase: str) -> tuple[Optional[FoleySpec], Optional[str]]:
    p = (action_phrase or "").strip().lower()
    if not p:
        return None, "Empty action label."

    fillers = {"a", "an", "the", "his", "her", "their", "its", "of", "to", "on", "in",
               "at", "with", "from", "into", "onto", "is", "are", "and", "person",
               "man", "woman", "someone"}
    words = [w for w in _tokens(p) if w not in fillers]
    stem = lambda w: w[:-4] if w.endswith("ping") or w.endswith("ting") else (
        w[:-3] if w.endswith("ing") and len(w) > 5 else w.rstrip("s"))
    stems = {stem(w) for w in words} | set(words)

    specific = [s for s in ACTION_PROMPT_MAP.values() if not s.generic]
    generic = [s for s in ACTION_PROMPT_MAP.values() if s.generic]
    best = _match(specific) or _match(generic)
    if best:
        return best, None

    for kw, reason in SILENT_ACTIONS.items():
        if kw in p:
            return None, reason

    obj = [w for w in words
           if w not in _ALL_ACTION_VERBS and stem(w) not in _ALL_ACTION_VERBS]
    if obj:
        for verbs, key in ((_PLACE_VERBS, "object_placement"),
                           (_PICKUP_VERBS, "object_pickup"),
                           (_PRESS_VERBS, "button_press")):
            if stems & verbs:
                return ACTION_PROMPT_MAP[key], None

    from .prompt_synthesis import synthesise
    return synthesise(action_phrase), None
```

Line	Explanation
the return type <code>tuple[Optional[FoleySpec], Optional[str]]</code>	Either (spec, None) or (None, reason). The caller can always distinguish "here is your sound" from "here is why there is no sound"
<code>(action_phrase or "").strip().lower()</code>	Handles None, whitespace and case in one expression
<code>fillers</code>	Grammatical words carry no action information and would confuse keyword matching
the stem lambda	Handles -ping, -ting, -ing and plural -s. Crude, but it is only a first pass; <code>prompt_synthesis</code> has a much better stemmer
<code>'stems = {stem(w) ...} \</code>	<code>set(words)'</code>
<code>specific before generic</code>	The specificity rule. Without this, the longer generic keyword wins and "place cup" gets generic object Foley
<code>SILENT_ACTIONS</code> after matching	A deliberate silence must not shadow a real class
the <code>obj</code> filter	An action verb needs an object that is not itself a verb, or "pressing" would count as its own object
<code>stems & verbs</code>	Python set intersection: does the phrase contain any of these verbs?
the local import of <code>synthesise</code>	Breaks the import cycle between the two modules

Line	Explanation
falls through to synthesis	The function never returns "unsupported". Either a class, a deliberate silence, or a synthesised spec

Input: a free-text action phrase. **Output:** `(FoleySpec, None)` or `(None, reason)`. **Where it fits:** stage 5, and again in stage 7 when placements are planned.

Chapter 50 — Snippet 6: the cache key

File: backend/services/sound_generation.py

```
def cache_key(spec: FoleySpec, settings: dict) -> str:
    payload = {"backend": _backend_name(settings),
               "action": spec.key, "prompt": spec.prompt, "negative": spec.negative,
               "seed": int(settings["seed"]), "steps": int(settings["steps"]),
               "cfg": round(float(settings["cfg_scale"]), 4),
               "sigma": round(float(settings["sigma_shift"]), 4),
               "seconds": round(float(settings["duration"]), 4),
               "sr": int(settings["sample_rate"]), "model": str(settings["model"])}
    return hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()[:16]

def cached_path(spec: FoleySpec, settings: dict) -> Path:
    return C.GENERATED / f"{_backend_name(settings)}_{spec.key}_{cache_key(spec, settings)}.wav"
```

Line	Explanation
Every field that affects the audio is in the payload	If it changes the audio, it must change the key. If it does not, it must not
backend in the payload and in the filename	The two Foley models can never collide, and you can see which model produced a file by looking at its name
int(...) and round(float(...), 4)	The type-normalisation fix. 10 and 10.0 serialise differently in JSON, so without this a client sending an integer would miss a cache entry created from a float default and pay 4.7 minutes for byte-identical audio
sort_keys=True	Python dictionaries preserve insertion order, so without this the same content could produce different JSON and therefore different hashes
.encode()	hashlib needs bytes, not a string
[:16]	64 hex characters is unwieldy in a filename. 16 hex characters is 64 bits, which is far beyond any collision risk for a few dozen files

Input: a `FoleySpec` and a settings dictionary. **Output:** a 16-character hexadecimal string. **Where it fits:** stage 5, before any generation is attempted.

Chapter 51 — Snippet 7: the quality gate

File: backend/services/foley_validation.py

```
def validate(path: Path, target_rms_dbfs: float,
            expect_sr: int = C.DEFAULTS["sample_rate"]) -> FoleyVerdict:
    m = measure(path, target_rms_dbfs)
    f: list[str] = []

    if not m.finite:
        f.append("audio contains NaN or Inf")
    if m.duration_s < MIN_DURATION_S:
        f.append(f"duration {m.duration_s:.3f}s below {MIN_DURATION_S}s")
    if m.sample_rate != expect_sr:
        f.append(f"sample rate {m.sample_rate} Hz, expected {expect_sr} Hz")
    if m.effective_bits < MIN_EFFECTIVE_BITS:
        f.append(f"effective bits {m.effective_bits:.1f} below {MIN_EFFECTIVE_BITS} "
                f"(peak {m.peak_dbfs:.1f} dBFS - almost no signal)")
    if m.dynamic_range_db < MIN_DYNAMIC_RANGE_DB:
        f.append(f"dynamic range {m.dynamic_range_db:.1f} dB below "
                f"{MIN_DYNAMIC_RANGE_DB} dB (flat, not impulsive)")
    if m.harmonic_ratio > TONAL_HARMONIC_RATIO and m.dynamic_range_db < TONAL_MAX_DYNAMIC_DB:
        f.append(f"harmonic ratio {m.harmonic_ratio:.2f} above {TONAL_HARMONIC_RATIO} "
                f"with only {m.dynamic_range_db:.1f} dB dynamic range (a sustained tone)")
    if (m.harmonic_ratio > PURE_TONE_HARMONIC_RATIO
        and m.spectral_flatness < PURE_TONE_MAX_FLATNESS):
        f.append(f"harmonic ratio {m.harmonic_ratio:.2f} with spectral flatness "
                f"{m.spectral_flatness:.4f} - a near-pure musical tone, not a "
                f"physical contact sound")
    if m.required_gain_db > MAX_AUTO_GAIN_DB:
        f.append(f>would need {m.required_gain_db:.1f} dB of make-up gain, above the "
                f"{MAX_AUTO_GAIN_DB} dB safety limit (would amplify noise)")

    ok = not f
    v = FoleyVerdict(ok=ok, metrics=m, failures=f,
                    reason="; ".join(f) if f else "passed all quality gates",
                    user_reason="" if ok else "generated audio failed quality validation")
    v.score = quality_score(m) if ok else 0.0
    return v
```

Design choice	Why
<code>measure()</code> is a separate function from <code>validate()</code>	Measurement has no policy in it, so the same measurements can be reported for a rejected asset. The user interface shows them
Every gate appends rather than returning early	A file that fails four gates reports all four. Early return would report only the first, and the user would fix one problem at a time
Every message includes the measured value and the threshold	"effective bits 5.7 below 9.0 (peak -62.0 dBFS - almost no signal)" tells you what happened, by how much, and what it means
Every message includes a short interpretation in brackets	"flat, not impulsive", "would amplify noise", "a sustained tone"
<code>reason</code> for the log, <code>user_reason</code> for the interface	Different audiences need different detail
<code>v.score = quality_score(m) if ok else 0.0</code>	A failing asset has no meaningful score, so it gets 0 rather than a misleading number

Input: a WAV path, the class's target level, and the expected sample rate. **Output:** a `FoleyVerdict` with `ok`, `score`, `failures`, `reason` and full `metrics`. **Where it fits:** inside `generate_best()`, immediately after every generation.

Chapter 52 — Snippet 8: true attack detection

File: backend/services/synchronization.py

```
def envelope(y: np.ndarray, sr: int) -> np.ndarray:
    e = np.abs(hilbert(y))
    k = max(1, int(0.002 * sr))
    return np.convolve(e, np.ones(k) / k, mode="same")

def attack_times(y, sr, min_gap_s=0.20, gate_db=-30.0) -> np.ndarray:
    """True transient attack times from the amplitude envelope."""
    env = envelope(y, sr)
    prom = max(0.08 * env.max(), 3.0 * np.percentile(env, 25))
    pk, _ = find_peaks(env, prominence=prom, distance=int(min_gap_s * sr))
    if not len(pk):
        return np.array([])
    pk = pk[env[pk] >= env[pk].max() * 10 ** (gate_db / 20)]
    out = []
    for i in pk:
        thr, j, lo = 0.20 * env[i], i, max(0, i - int(0.30 * sr))
        while j > lo and env[j] > thr:
            j -= 1
        out.append(j / sr)
    out = np.array(sorted(set(round(x, 4) for x in out)))
    ded = [out[0]]
    for x in out[1:]:
        if x - ded[-1] >= min_gap_s:
            ded.append(x)
    return np.array(ded)
```

Line	Explanation
<code>np.abs(hilbert(y))</code>	The Hilbert transform gives the analytic signal; its magnitude is the amplitude envelope , the outline of the waveform ignoring the oscillation inside
<code>k = int(0.002 * sr)</code> then <code>np.convolve(..., np.ones(k)/k)</code>	A 2 ms moving average. Long enough to smooth the audio-rate ripple, short enough to preserve a transient
<code>prom = max(0.08 * env.max(), 3.0 * np.percentile(env, 25))</code>	Prominence, taken as the larger of 8 percent of the envelope maximum and three times the 25th percentile. The first term adapts to loud material; the second adapts to the noise floor
<code>find_peaks(..., prominence=prom, distance=...)</code>	Prominence, not height. A plain height threshold admits ripples on the shoulder of a big peak; prominence measures how far a peak rises above the surrounding terrain
<code>distance=int(min_gap_s * sr)</code>	Two footsteps cannot be 20 ms apart
<code>pk = pk[env[pk] >= env[pk].max() * 10**(gate_db/20)]</code>	Discard peaks more than 30 dB below the loudest peak. $10^{**}(\text{dB}/20)$ converts decibels to a linear ratio
the backtracking while loop	This is the true attack. Walk backwards from the peak until the envelope last fell below 20 percent of the peak height. That point is where the transient began
<code>lo = max(0, i - int(0.30 * sr))</code>	Do not walk back more than 300 ms, in case the envelope never drops
<code>sorted(set(round(x, 4) ...))</code>	Two peaks can backtrack to the same attack; deduplicate at 0.1 ms resolution
the final ded loop	Enforce the minimum gap again, since backtracking can bring two attacks closer together

Input: an audio buffer and its sample rate. **Output:** an array of attack times in seconds. **Where it fits:** stage 7, in `plan_action`, and again in the quality gate to count transients.

REMEMBER THIS

This function is the fix for the 96 ms error. If you remember one code snippet from the whole project, make it the backtracking loop: *"walk backwards from the envelope peak until it last fell below 20 percent of that peak - that is where the sound actually starts."*

Chapter 53 — Snippet 9: footstep detection

File: backend/services/synchronization.py, inside detect_events

```
if spec.strategy == "footstep":
    prom = max(0.15 * (seg.max() - seg.min()), 0.15 * seg.std())
    pk, _ = find_peaks(seg, prominence=prom, distance=max(1, int(0.25 * fps)))
    for p in pk:
        j = p
        while j + 1 < len(seg) and seg[j + 1] <= seg[j]:
            j += 1
        tc = float(t[widx[j]])
        if sa <= tc < sb:
            ev.append(VisualEvent(action, "foot_contact", tc, "high",
                                "motion peak resolved to following minimum (plant)"))
```

Line	Explanation
seg	The motion signal restricted to the feet band and to the search window
prom = max(0.15 * range, 0.15 * std)	Adaptive prominence: 15 percent of the signal's range or 15 percent of its standard deviation, whichever is larger
distance=int(0.25 * fps)	Steps cannot be closer than a quarter of a second
the while loop	The physical insight. The motion <i>peak</i> is the leg swinging. The <i>following minimum</i> is the foot planted and still. The plant is what you hear
if sa <= tc < sb	The peak may be found outside the search span because the window is widened by a margin, but only plants inside the span are kept
"high" confidence	Foot plants are prominent, well-separated features, unlike holds and contacts
the basis string	Every event explains how it was derived, and that string reaches the user interface

Input: the band motion signal, the interval, and an optional wider search span. **Output:** a list of `VisualEvent` records.

Where it fits: stage 7.

Chapter 54 — Snippet 10: matching the generated cadence to the filmed gait

File: backend/services/synchronization.py, inside plan_action

```
if spec.selection == "steps" and events:
    steps = attack_times(y, sr, min_gap_s=0.25, gate_db=-30.0)
    contacts = sorted(e.t_s for e in events)
    n = len(contacts)
    if n >= 2 and len(steps) >= n:
        vis = np.diff(contacts)
        best, best_err = 0, None
        for k in range(len(steps) - n + 1):
            err = float(np.sum((np.diff(steps[k:k + n]) - vis) ** 2))
            if best_err is None or err < best_err:
                best, best_err = k, err
        run = steps[best:best + n]
    else:
        run = steps[:1]
    anchor = float(run[0])
    offset = contacts[0] - anchor
    src_start = max(0.0, anchor - 0.30, lo_bound - offset)
    src_end = min(len(y) / sr, float(run[-1]) + 0.35, hi_bound - offset)
    out.append({"asset_start_s": round(src_start, 4), "asset_end_s": round(src_end, 4),
               "video_start_s": round(src_start + offset, 4),
               "aligned_to_s": round(contacts[0], 4),
               "per_event_error_ms": [round(1000 * (float(r) + offset - c), 1)
                                     for r, c in zip(run, contacts)],
               ...})
```

Line	Explanation
steps = attack_times(...)	All the transients in the generated 10-second walking asset - typically 16 of them
contacts	The visible foot plants, typically 4
vis = np.diff(contacts)	The filmed inter-step intervals, e.g. [0.625, 0.584, 0.541]
the for k loop	Slide a window of n consecutive generated steps across the asset
np.sum((np.diff(steps[k:k+n]) - vis) ** 2)	Sum of squared differences between the generated spacing and the filmed spacing. This is the matching criterion
run = steps[best:best+n]	The consecutive run whose internal rhythm best matches the filmed gait
offset = contacts[0] - anchor	The single translation that puts the run's first step on the first visible plant
src_start = max(0.0, anchor - 0.30, lo_bound - offset)	Start the slice 300 ms before the first step, so the clip does not begin abruptly - but never before the asset starts, and never so early that it would be placed before the allowed region
src_end = min(len(y)/sr, run[-1] + 0.35, hi_bound - offset)	End 350 ms after the last step, for the decay - subject to the same two limits
per_event_error_ms	The residual at every contact. This is what makes the cadence analysis of Chapter 30 possible, and it is why the honest aggregate numbers exist

KEY POINT: Why the residual is recorded rather than corrected

Because the alternative is time-stretching, which changes what the footstep sounds like. The system's position is that a slightly late footstep that sounds like a footstep is better than a perfectly placed artefact - and that the error should be **reported**, so that anyone can see how large it is.

Input: the asset, its detected transients, and the visible contacts. **Output:** one placement entry with a per-event error list. **Where it fits:** stage 7.

Chapter 55 — Snippet 11: the mixer

File: backend/services/audio_processing.py

```
for p in placements:
    y, sr = sf.read(p["asset"])
    y = y.astype(np.float64)
    if sr != sample_rate:
        raise ValueError(f"asset sample-rate {sr} != {sample_rate}")
    srch = int(0.003 * sr)
    a0 = snap_zero(y, int(p["asset_start_s"] * sr), srch)
    a1 = snap_zero(y, int(p["asset_end_s"] * sr), srch)
    if a1 <= a0:
        continue
    clip = y[a0:a1].copy()
    snap_ms = round((a0 - int(p["asset_start_s"] * sr)) / sr * 1000, 3)
    dc = float(clip.mean()); clip -= dc
    clip = rcos_fade(clip, sr, C.FADE_MS)

    raw_peak, raw_arms = float(np.abs(clip).max()), active_rms(clip)
    g = 10 ** (p["target_rms_dbfs"] / 20) / max(raw_arms, 1e-12)

    need_db = 20 * np.log10(max(g, 1e-12))
    if need_db > MAX_AUTO_GAIN_DB:
        log["rejected"].append({... "stage": "mixer_gain_limit" ...})
        continue

    cap = 10 ** (C.CLIP_PEAK_CEILING_DBFS / 20) / max(raw_peak, 1e-12)
    capped = g > cap
    g = min(g, cap); clip *= g

    start = int(round((p["video_start_s"] + snap_ms / 1000) * sr))
    ...
    bus[start:start + len(clip)] += clip
```

Line	Explanation
<code>y.astype(np.float64)</code>	All processing in double precision, so repeated gain changes do not accumulate rounding error
the sample-rate check	A hard failure, not a silent resample. Every asset in the pipeline is already 48 kHz
<code>snap_zero(y, index, srch)</code>	Move the cut point to the nearest zero crossing within plus or minus 3 ms, so the edit does not click
<code>snap_ms</code>	The snap offset is recorded and later added to the placement , so moving the cut does not move the sound
<code>dc = clip.mean(); clip -= dc</code>	Remove any DC offset. Offsets of order 1e-4 were observed
<code>rcos_fade(clip, sr, 12.0)</code>	A raised-cosine fade is continuous in slope; a linear ramp has a corner that is faintly audible
<code>g = 10** (target/20) / raw_arms</code>	The linear gain that would bring the measured active RMS to the class target
<code>need_db > MAX_AUTO_GAIN_DB</code>	Refuse, do not clamp. Clamping still admits amplified noise
<code>cap = 10** (-6/20) / raw_peak</code>	The gain that would put the peak at -6 dBFS
<code>g = min(g, cap)</code>	Take the smaller of the two, and record whether the cap engaged
<code>start = ... + snap_ms/1000</code>	The snap correction applied
<code>bus[start:start+len(clip)] += clip</code>	Addition, not assignment. Overlapping sounds sum, which is what mixing means

Every clip logs 18 fields: the action, the asset name, its position, its source range, the snap offset, the DC removed, the raw peak and active RMS, the target, the gain applied, whether the cap engaged, the output peak, the fades and any truncation.

Input: the placement list, the video duration, an output path. **Output:** a 48 kHz mono PCM_16 WAV, and a mix log.

Where it fits: stage 8.

Chapter 56 — Snippet 12: the final render

File: backend/services/video_render.py

```
def mux(video: Path, audio: Path, out_mp4: Path, sample_rate: int = 48000) -> dict:
    out_mp4.parent.mkdir(parents=True, exist_ok=True)
    cmd = ["ffmpeg", "-y", "-v", "error",
           "-i", str(video), "-i", str(audio),
           "-map", "0:v:0", "-map", "1:a:0",
           "-c:v", "copy",
           "-c:a", "aac", "-b:a", "192k", "-ar", str(sample_rate),
           "-movflags", "+faststart", "-shortest", str(out_mp4)]
    p = subprocess.run(cmd, capture_output=True, text=True)
    if p.returncode != 0 or not out_mp4.is_file():
        raise RenderError(f"Final video rendering failed: {(p.stderr or '').strip()[:300]}")
    probe = subprocess.run(["ffprobe", "-v", "error", "-show_streams", "-show_format",
                           "-of", "json", str(out_mp4)], capture_output=True, text=True)
    d = json.loads(probe.stdout)
    v = next(s for s in d["streams"] if s["codec_type"] == "video")
    a = next(s for s in d["streams"] if s["codec_type"] == "audio")
    return {"path": str(out_mp4), "duration_s": float(d["format"]["duration"]),
           "video_codec": v["codec_name"], "audio_codec": a["codec_name"],
           "frames": int(v.get("nb_frames", 0)),
           "resolution": f"{v['width']}x{v['height']}",
           "audio_sample_rate": int(a["sample_rate"]),
           "audio_channels": int(a["channels"]),
           "video_stream_copied": True,
           "bytes": out_mp4.stat().st_size}
```

Flag	Explanation
-y	Overwrite the output without asking. There is no interactive terminal
-v error	Only print errors, so stdout is not full of progress noise
two -i inputs	Input 0 is the video, input 1 is the mixed audio
-map 0:v:0 -map 1:a:0	Explicit stream selection. Take the video from input 0 and the audio from input 1, and nothing else - so any audio in the source is not carried over
-c:v copy	The picture is stream-copied: not decoded, not re-encoded. Bit-identical output, no generation loss, and far faster
-c:a aac -b:a 192k	AAC at 192 kbit/s, which browsers can play natively in MP4
-ar 48000	48 kHz, matching the master
-movflags +faststart	Move the MP4 index to the front of the file so a browser can begin playback before the whole file has downloaded
-shortest	Stop at the shorter input, so the audio can never make the video longer
the re-probe	Verify what was produced rather than assuming. The returned dictionary becomes part of the job report, and the quality gate checks it

Input: the source video, the mixed WAV, an output path. **Output:** an MP4, and a dictionary describing it. **Where it fits:** stage 9.

Chapter 57 — Snippet 13: the MOSS denoising loop

File: moss/scripts/moss_generate.py

```
pipe.scheduler.set_timesteps(A.steps, denoising_strength=1.0, shift=A.sigma_shift)
num_channels, num_samples = pipe.check_resize_num_channels_num_samples(
    M.NUM_CHANNELS, sr * full_seconds)
shape = (1, pipe.audio_latent_dim, num_samples // pipe.num_samples_division_factor)
latents = pipe.generate_noise(shape, seed=A.seed, rand_device="cpu")

ctx_p = ctx_posi_c.to(M.DEVICE, M.DTYPE)
ctx_n = ctx_neg_a_c.to(M.DEVICE, M.DTYPE)
with torch.no_grad(), torch.autocast(M.DEVICE, dtype=M.DTYPE):
    for i, ts in enumerate(pipe.scheduler.timesteps):
        timestep = ts.unsqueeze(0).to(device=M.DEVICE)
        npos = model_fn_wan_video(dit=dit, motion_controller=None, vace=None,
                                latents=latents, timestep=timestep, context=ctx_p)

        npos = npos.clone()
        nneg = model_fn_wan_video(dit=dit, motion_controller=None, vace=None,
                                latents=latents, timestep=timestep, context=ctx_n)

        noise_pred = nneg.float() + A.cfg * (npos.float() - nneg.float())
        latents = pipe.scheduler.step(noise_pred, pipe.scheduler.timesteps[i], latents)
    assert torch.isfinite(latents).all(), "latent contains NaN/Inf"
```

Line	Explanation
set_timesteps(steps, denoising_strength=1.0, shift=sigma_shift)	Builds the flow-matching schedule. denoising_strength=1.0 means start from pure noise
shape = (1, 128, num_samples // 960)	One item, 128 latent channels, and one latent step per 960 audio samples. 30 s at 48 kHz gives 1,440,000 / 960 = 1500 steps
generate_noise(shape, seed, rand_device="cpu")	Noise is drawn on CPU with a seeded generator , matching upstream, so results do not depend on GPU-specific random number generation
torch.no_grad()	No gradients: this is inference
torch.autocast(device, bfloat16)	The forward pass computes in bfloat16
two model_fn_wan_video calls per step	Once with the positive context, once with the negative. This is why 50 steps means 100 forward passes through a 1.4-billion-parameter Transformer, and why generation takes 4.7 minutes
npos = npos.clone()	Defensive: the second call may reuse a buffer, and without the clone the first result could be overwritten
.float() in the CFG formula	Guidance is computed in float32 even though the surrounding block is bfloat16, because the difference npos - nneg can be small and precision matters there
scheduler.step(...)	Takes one Euler step along the flow field
assert torch.isfinite(latents).all()	Catch a numerical blow-up here, rather than writing a WAV full of NaN

Input: encoded prompts, a seed and sampler settings. **Output:** a latent tensor of shape (1, 128, 1500). **Where it fits:** phase 2 of the three-phase generation.

Chapter 58 — Snippet 14: the frontend polling hook

Covered in full in Chapter 38.4. The three lines worth being able to point at:

```
tick() // fire immediately, do not wait 1.5 s
timer.current = window.setInterval(tick, 1500)
return stop // React calls this on unmount
```

and

```
if (!seenTimeline.current && s.stages?.timeline === 'done') {
  seenTimeline.current = true
  const a = await api.actions(jobId) // show the timeline DURING generation
  ...
}
```

Input: a job id, or null. **Output:** {status, result, actions, events, unsupported, error}. **Where it fits:** between `App.tsx` and every display component.

Chapter 59 — Snippet 15: CTC forced alignment

File: 02-Auto-AVSR-Test/app/timing.py

```
logp = model.ctc.log_softmax(enc_feat.unsqueeze(0)).float()
targets = torch.tensor([ids], dtype=torch.int32)
frames, _ = torchaudio.functional.forced_align(logp, targets, blank=model.blank)
frames = frames[0].tolist()

onsets, prev = [], None
for f, tok in enumerate(frames):
    if tok != prev:
        if tok != model.blank:
            onsets.append((tok, f))
        prev = tok

words = []
for tok, f in onsets:
    unit = token_list[tok]
    if unit.startswith("\u2581") or not words: # U+2581 = word-start marker
        words.append({"word": unit.lstrip("\u2581"), "frame": f})
    else:
        words[-1]["word"] += unit
```

Line	Explanation
<code>model.ctc.log_softmax(enc_feat.unsqueeze(0)).float()</code>	Per-frame log-probabilities over the vocabulary plus blank. <code>unsqueeze(0)</code> adds the batch dimension the function expects
<code>targets</code>	The already-decoded token sequence, with end-of-sentence and blank removed
<code>forced_align(logp, targets, blank)</code>	Finds the most probable frame-by-frame path that produces exactly this token sequence. This is where video timing enters the system
the transition loop	CTC repeats a symbol across several frames; a new onset is recorded only when the symbol changes and is not blank
the word-start marker check	SentencePiece marks word starts with a special character; anything else is a continuation of the current word
<code>not words</code>	The first token always begins a word, even if the marker is missing

Input: the encoder output, the decoded tokens, the frame count. **Output:** `[{word, start, end}]` in seconds, plus speech start, speech end and pauses. **Where it fits:** immediately after decoding in `inference.py`, using the *same* encoder output - so it costs no extra forward pass.

PART 9

MODELS, DATASETS, ACCURACY AND TESTING

A single reference chapter for every model, a single reference chapter for every dataset, and an honest account of what "accuracy" does and does not mean in this project.

Chapter 60 — Every model, in one place

60.1 Models actually used in the delivered system

Model 1: Auto-AVSR (visual speech recognition)

Field	Value
Full name	Auto-AVSR: Audio-Visual Speech Recognition with Automatic Labels (Ma et al., ICASSP 2023)
Checkpoint	vsr_trlrs2lrs3vox2avsp_base.pth, MD5 49f770f2c0d8b8d769347ee47ed1648f
Purpose	Read lip motion and produce an English sentence
Input	Tensor (B, T, 1, 88, 88) - greyscale mouth crops, one per video frame at 25 fps
Output	A token sequence, detokenised to a sentence, plus per-frame CTC posteriors
Architecture	Conv3dResNet frontend, Conformer encoder, Transformer decoder plus a CTC head
Parameters	250,383,410
Vocabulary	5049 SentencePiece unigram tokens
Training data	LRS2, LRS3, VoxCeleb2, AVSpeech
Pre-training	Yes - this is a published pre-trained checkpoint
Fine-tuning in this project	None
Inference here	CPU, float32, beam width 40, weights decoder 0.9 / CTC 0.1
Published accuracy	20.3 percent word error rate on LRS3 (their number, on their benchmark)
Why selected	Visual-only checkpoint; largest training corpus; published benchmark; runs on CPU; has a CTC head that makes word timing possible
Advantages	Open-vocabulary sentences; strong published result; official preprocessing in the same repository
Disadvantages	English only; requires 25 fps SDR input; CPU only here because the MPS path crashes; the task is fundamentally ambiguous
Alternatives	AV-HuBERT (tried first, set aside for dependency reasons); audio-visual checkpoints (would defeat the purpose); LipNet (constrained grammar only)
Compute needed	955 MB checkpoint; 0.5 to 0.7 s to load; 1.2 to 2.0 s per clip on CPU

Model 2: Qwen2.5-VL-3B-Instruct (action recognition)

Field	Value
Full name	Qwen2.5-VL, 3-billion-parameter instruction-tuned variant (Bai et al., 2025)
Purpose	Describe the physical action in a 2-second video window, in free text
Input	8 frames at 448 x 252 plus a text prompt
Output	Two lines: ACTION: <phrase> and EVIDENCE: <phrase>
Architecture	Vision-language Transformer: a vision encoder feeding a language model
Parameters	about 3 billion
Training data	Not specified in the project implementation - it is a published checkpoint
Fine-tuning in this project	None

Field	Value
Inference here	Apple MPS, bfloat16, greedy decoding, max_new_tokens=96
Accuracy measured here	No accuracy metric exists. Its output is qualitatively assessed; see Chapter 62
Why selected	Free-text output means the vocabulary is open, which is what Foley needs; a 3-billion-parameter model in bfloat16 fits the memory budget
Advantages	No fixed label set; describes object interactions; deterministic under greedy decoding
Disadvantages	The weakest component in the system. It misses events, emits captions instead of actions, and labels the same activity several different ways
Alternatives	VideoMAE and X-CLIP, both measured and rejected (Chapter 32)
Compute needed	About 7 GB of checkpoint; peaks at 12.0 GB resident; 5.23 s per window

Model 3: MOSS-SoundEffect v2.0 (Foley generation)

Field	Value
Full name	MOSS-SoundEffect v2.0, OpenMOSS Team
Repository	OpenMOSS-Team/MOSS-SoundEffect-v2.0, revision e35df4d8 . . .
Purpose	Generate Foley audio from a text description
Input	A text prompt plus an optional negative prompt. No video, image or audio conditioning
Output	48 kHz mono audio, up to 30 seconds
Architecture	Diffusion Transformer (30 layers, dim 1536, 12 heads, ffn 8960) trained with flow matching; Qwen3 text encoder (hidden size 2048); DAC variational autoencoder
Parameters	3,508.21 M total: DiT 1,416.05 M, text encoder 1,720.57 M, VAE 371.59 M
Training data	Not specified in the project implementation
Fine-tuning in this project	None
Inference here	Apple MPS, bfloat16 parameters, 50 flow-matching steps, CFG 4.0, sigma shift 5.0, seed 42
Licence	Apache-2.0
Why selected	48 kHz output; duration control to 30 s; an explicit human-action Foley category; the only commercially licensed candidate
Advantages	The highest sample rate of any candidate; strong inharmonic output for object contacts (harmonic ratio 0.00 to 0.02)
Disadvantages	Slow - 4.7 minutes per asset, because it denoises a fixed 30-second latent regardless of the requested duration; occasionally collapses to degenerate output for a given seed
Alternatives	Stable Audio Open (retained as a switchable backend), AudioLDM 2, FoleyCrafter, MMAudio, AudioGen, TangoFlux, EzAudio
Compute needed	11.23 GB checkpoint; peak 12.11 GB resident with phase separation; would need about 17 GB without it

Model 4: Kokoro (text to speech)

Field	Value
Purpose	Speak the recognised sentence
Input	Text, a voice name, a speed multiplier
Output	24 kHz mono audio
Architecture	Not specified in the project implementation; shipped as an ONNX graph
Voices used	af_heart (female), am_michael (male)

Field	Value
Why selected	Judged markedly more natural than Piper on this material
Disadvantages	Needs <code>numpy>=2</code> , which forces a separate virtual environment and a worker process
Fallbacks	Piper (<code>en_US-lessac-medium</code> , <code>en_US-ryan-medium</code>), then macOS <code>say</code>

Model 5: Levi and Hassner gender CNN

Field	Value
Purpose	Choose a male or female TTS voice from the speaker's face
Input	A 227 x 227 face crop
Output	<code>male</code> or <code>female</code> with a confidence
Architecture	A small convolutional network, loaded through <code>cv2.dnn.readNet</code>
Accuracy	Roughly 85 to 90 percent on clear frontal faces (a published characterisation, not measured here)
Mitigations	A manual override in the API and UI; below 0.60 confidence it falls back to the female voice rather than guessing

60.2 Models evaluated and rejected

Model	Purpose	Parameters	Why rejected
VideoMAE (Kinetics)	Action recognition	86,534,800	"shredding paper" 0.550; closed vocabulary organised around activities, not events
X-CLIP	Action recognition	not recorded	"pouring liquid" 0.21-0.38 in every window; no temporal discrimination
Stable Audio Open 1.0	Foley	not recorded	96.2 percent silence at short durations; produces musical tones for object contacts (harmonic 0.87-0.88). Retained as a switchable backend
Stable Audio Open Small	Foley	not recorded	95.6 percent silence
AudioLDM 2	Foley	not recorded	91.9 percent silence; 16 kHz cuts off the band that matters
FoleyCrafter	Video to audio	not recorded	Continuous noise bed rather than events; 16 kHz
MMAudio v1 / v2	Video to audio	157 M (published)	86 and 96 percent silence; sound placed on the wrong action
AudioGen medium	Foley	1.5 B	Not installed. Ranked second: architecturally different from all five failures, but 16 kHz mono, CC-BY-NC, and an <code>xformers</code> dependency
TangoFlux	Foley	515 M	Not installed. Reuses the Stable Audio Open VAE, which had already failed this exact task twice; research-only licence
EzAudio	Foley	about 1 B	Not installed. Thin published specifications; 24 kHz
AV-HuBERT	Lip reading	not recorded	Set aside for dependency reasons: <code>fairseq</code> on <code>torch 1.13.1</code> , <code>numpy 1.23.5</code> , plus an import workaround

CAUTION: Where a number does not exist, say so

Several parameter counts and training-data descriptions are **not recorded in the project files**. This handbook writes "not specified in the project implementation" rather than filling the gap from memory. Do the same in a viva: "I did not record that, and I would rather not guess" is a perfectly good answer.

Chapter 61 — Every dataset, in one place

61.1 The honest summary first

REMEMBER THIS

This project trained on no dataset. It has no training set, no validation set and no test set of its own. Every model is used with published pre-trained weights.

The datasets below are the corpora the **checkpoints** were trained on. Their statistics are **dataset information published by their authors**, not experiments run in this project.

61.2 Datasets behind the lip-reading checkpoint

Dataset	What it contains	Data type	Labels	Role
LRS2-BB C	Thousands of spoken sentences from BBC television, with face tracks	Audio-visual video	Human transcripts	Training
LRS3-TE D	Aligned face tracks and transcripts from TED and TEDx talks	Audio-visual video	Human transcripts	Training and the benchmark for the published 20.3 percent WER
VoxCeleb2	Over a million utterances from thousands of speakers, from YouTube interviews	Audio-visual video	None originally - built for speaker recognition	Training, with labels generated automatically by an ASR model
AVSpeech	A very large corpus of clips with a single visible speaking face and clean speech	Audio-visual video	None originally	Training, automatically labelled

Video and audio characteristics that matter: 25 frames per second, SDR BT.709 8-bit, mostly frontal and well-lit, single speaker, English, broadcast or conference speech.

How this project uses them: it does not. It uses the checkpoint they produced. But the characteristics above are exactly why the backend forces 25 fps SDR and rejects HDR.

Limitations and biases:

- **Language.** English only.
- **Speaker.** British broadcast, TED speakers, and celebrity interviews are not a uniform sample of humanity in accent, demographics or speaking style.
- **Recording.** Professionally lit, professionally framed, front-facing.
- **Label quality.** Two of the four corpora were labelled by an audio speech recogniser, so systematic ASR errors become training signal.
- **Licensing.** All four restrict redistribution, which is why the checkpoint is not in the repository.

61.3 Datasets behind the other models

Model	Training data	What the project knows
Qwen2.5-VL-3B-Instruct	Not specified in the project implementation	It is a published instruction-tuned vision-language checkpoint
MOSS-SoundEffect v2.0	Not specified in the project implementation	Its documentation describes categories - natural environments, urban environments, animals and creatures, human actions - which is why it was chosen
Kokoro	Not specified in the project implementation	-
Levi and Hassner gender CNN	The Adience benchmark (published)	Roughly 85 to 90 percent on clear frontal faces
VideoMAE checkpoint (rejected)	Kinetics	A label set organised around whole-clip human activities

61.4 The project's own evaluation material

What this project *does* have is a small set of recordings it evaluated on. Calling this a "test set" would be an overstatement, and the handbook does not.

Material	Contents	Used for
Module3_Fresh/input/test_video.mp4	10.005 s, 1280 x 720, 24 fps, h264. A person walks to a table, picks up a cup, drinks twice, and puts the cup down. SHA-256 a620ee5820ab9dfc...	The reference clip for every Subsystem 2 measurement
A coffee-stirring video	Spoon, cup, stirring, placements	The video that exposed Module 2's weaknesses
A toaster video	Bread placement, lever press, toasting	The video that exposed the "no supported Foley action" failure
Seven lip-reading recordings	59 to 98 frames each, various formats	Every Subsystem 1 measurement
A 20,000 fps high-speed clip	From the original Visual Microphone authors' published dataset	The Subsystem 3 demonstration
Synthetic stimuli	Band-limited noise translated by a known sub-pixel amplitude	The Subsystem 3 characterisation

CAUTION: What is missing, and say it plainly

There is **no held-out labelled evaluation set**. No ground-truth action timeline, no ground-truth transcripts, no ground-truth audio. Every quantitative result in this project is either a **measurement of a signal property** (dynamic range, harmonic ratio, alignment residual) or a **published number from a model's authors**. Neither is an accuracy figure for this system.

61.5 Dataset vocabulary, defined

Term	Meaning
Training set	Examples the model learns from
Validation set	Held-out examples used to tune choices <i>during</i> training
Test set	Held-out examples used once, to report a final number
Ground truth	The correct answer for an example
Label	The category or text attached to an example

Term	Meaning
Annotation	The human act of producing labels
Augmentation	Artificially varying training data - crops, noise, speed changes - to improve robustness
Overfitting	Memorising the training data and failing on new data
Underfitting	Being too simple to capture the pattern
Generalisation	Performing well on data never seen in training
Class imbalance	Some labels being far more common than others, which biases a classifier
Distribution shift	Test data differing systematically from training data - the relevant one here : 30 fps HDR footage is a distribution shift away from 25 fps SDR training data

Chapter 62 — Accuracy and performance, carefully separated

KEY POINT: The most important framing in the whole handbook

"Accuracy" means four different things in this project, and mixing them up is the fastest way to lose credibility in a viva. Separate them explicitly:

1. Model accuracy - how often a model gets the right answer. Requires ground truth. **2. Synchronisation accuracy** - how close a sound is to its visual event, in milliseconds. Requires no ground truth, only measurement. **3. Audio quality** - measurable signal properties, plus a listening judgement. **4. Application performance** - latency, memory, throughput, reliability.

A website does not have "model accuracy". Saying it does is a category error.

62.1 Model accuracy: what exists and what does not

Model	Accuracy measured in this project	What exists instead
Auto-AVSR	None. Ground-truth transcripts were not retained	Decoder scores across 7 runs, 2 spot-checked correct transcripts, and a published 20.3 percent WER on LRS3 by the model's authors
Qwen2.5-VL	None. No ground-truth action timeline exists	Qualitative failure analysis: it missed a cup placement, emitted one action under three labels, and returned a caption rather than an action
MOSS-SoundEffect	Not applicable - generation has no "correct answer"	A six-gate pass/fail rate of 70.4 percent across 54 assets, and a 0-100 quality score
Levi and Hassner gender CNN	None	A published characterisation of 85 to 90 percent on clear frontal faces

PROFESSOR MAY ASK: How accurate is your system?

"That depends which part, and I want to be precise about it rather than give you one number that means nothing.

For lip reading I cannot give you a word error rate, because I did not retain ground-truth transcripts. The published checkpoint reports 20.3 percent WER on LRS3, but that is their measurement on their benchmark.

For action recognition I have no accuracy metric either, and I know it is the weakest component - I can give you specific documented failures.

What I *can* measure without ground truth is synchronisation, and that is the number I would stand behind: worst-case 20 milliseconds on the reference clip measured on the rendered audio, with a median of 4.7 milliseconds across all recorded runs."

62.2 Synchronisation accuracy

Measurement	Value	Scope
Worst error, validated build, reference clip	20.3 ms	7 events, 1 clip
One frame at 24 fps	41.7 ms	for comparison
Fraction of those 7 events inside half a frame	100 percent	
Median absolute error across all recorded runs	4.7 ms	45 events, 31 jobs, 2 clips
Mean absolute error	56.2 ms	same

Measurement	Value	Scope
90th percentile	150.6 ms	same
Worst	462.4 ms	the Stable Audio walking asset, cadence mismatch
Within half a frame	66.7 percent	same
Worst error, current web pipeline, reference clip	67.6 ms	different step run selected

How it was measured. Envelope attacks are detected in the final rendered WAV and compared against the visual event timestamps. This is stronger than reading the alignment plan, because it includes any error introduced by segment selection, fades and mixing.

62.3 Audio quality

Measurement	Value
Assets generated during development	54
Rejected by the six gates	16, or 29.6 percent
Rejected that would have needed more than +25 dB of gain	11 (median +37.0 dB, maximum +42.1 dB)
MOSS rejection rate	12 of 46, 26 percent
Stable Audio rejection rate	4 of 8, 50 percent
Median quality score, MOSS / Stable Audio	54.5 / 53.2
Median harmonic ratio, MOSS / Stable Audio	0.040 / 0.898
Final mix peak	-6.00 dBFS
Final mix RMS	-36.87 dBFS
Final mix crest factor	30.87 dB
Clipped samples	0
Limiter gain reduction	0.00 dB - never engaged
Listening assessment	Single assessor, the developer

62.4 Application performance

Quantity	Measured
Subsystem 1, model load	0.5 to 0.7 s
Subsystem 1, inference	1.57 s mean over 7 runs
Subsystem 1, total per clip	3.73 s mean
Subsystem 1, real-time factor	0.45x - faster than real time, on CPU
Subsystem 2, action recognition	5.23 s per window, 68.07 s total for a 10 s clip
Subsystem 2, Foley generation	280.9 s per asset (94 percent of it diffusion)
Subsystem 2, whole job fully cached	39.5 s median
Subsystem 2, whole job with generation	556.9 s median
Subsystem 2, peak memory	12.11 GB mean, 12.51 GB worst, of 17.18 GB
Subsystem 2, minimum available memory	2.35 GB mean, 1.58 GB worst against a 1.50 GB guard
Subsystem 2, swap growth	0.00 to 0.01 GB
Subsystem 2, guard breaches in 31 runs	zero

Quantity	Measured
Subsystem 3, throughput	1284 / 441 / 103 frames per second at 64 / 128 / 256 pixels square

62.5 Reliability

Quantity	Value
Job records on disk	122
Completed jobs	33
Failed jobs	9
Failure causes	4 "no supported Foley action" (since fixed by the verb fallback and open vocabulary), 3 out-of-memory, 1 tuple-unpacking bug (fixed), 1 model load failure
Automated tests, Subsystem 2	64, all passing
Automated tests, Subsystem 3	37
Quality-gate checks on the validated build	19, all passing

Chapter 63 — Testing

63.1 Subsystem 2 test suites

`backend/tests/test_suite.py` - 42 tests in nine groups

Group	Tests	What is covered
1. Video validation	5	probe reads the demo video; validate warns about an existing audio track; rejects an unsupported extension, an over-long video, and a non-video file
2. Action timeline parsing	2	the resolved timeline is 5 non-overlapping actions; the raw array does overlap, confirming why resolved is used
3. Prompt generation	11	known phrasings map to the right class; specificity beats length; the generic fallback; stirring phrasings; silent actions are silent rather than unknown; the verb fallback; the fallback never overrides a specific class; a bare verb never inherits an object-specific class; open vocabulary produces a usable spec for 15 arbitrary phrases; every archetype maps to a real motion band; synthesis is deterministic
4. Foley cache keying	1	the key is stable, settings-sensitive, and namespaced per backend
5. Audio segment extraction	4	<code>attack_times</code> finds synthetic transients; <code>first_attack</code> returns the leading edge, not the peak; <code>select_event_clip</code> returns a short window round a real transient; <code>select_wet_segments</code> returns the requested count
6. Synchronisation	4	detection reproduces the validated foot plants, sip holds and cup contact exactly ; walking alignment matches the reference within 1 ms; alignment never time-stretches
7. Audio mixing	5	a valid non-clipping 48 kHz mono WAV; a clip crossing the end is truncated, never extends the timeline; a fragment is omitted rather than clipped to a click; the limiter stays disengaged; the helper functions behave
8. FFmpeg rendering	1	mux copies the video stream and attaches 48 kHz mono audio
9. API endpoints	7	health reports the nine stages in order; supported actions; demo creates a real job; unknown ids return 404 without a stack trace; result on an unfinished job returns 409; upload rejects a bad extension and an undecodable file

`backend/tests/test_foley_validation.py` - 22 tests covering the six gates individually, the score components, and candidate selection - including *"a failing first candidate triggers retries with new seeds"*, *"the best-scoring candidate wins, not merely the first passing one"*, *"when every candidate fails, no asset is selected"*, and *"each candidate uses a distinct cache key so nothing is regenerated twice"*.

`backend/tests/e2e_gate.py` runs the whole pipeline on the reference clip from cached assets in about 5 seconds and asserts that no rejected asset reached the mix.

63.2 The four most valuable tests, and why

KEY POINT: These four are worth naming in a viva

1. **"visual detection reproduces the validated foot plants"** - asserts `[0.458, 1.083, 1.667, 2.208]` exactly. It means the generalised service is *proved* to reproduce the validated reference implementation, not merely believed to.

2. **"raw actions array overlaps, confirming why resolved is used"** - a test that documents a *design decision* by asserting the property that motivated it. If someone later "fixes" the overlap upstream, this test tells them the resolution step may no longer be needed.

3. **"the verb fallback never overrides a specific class or a silent action"** - a regression test guarding the mis-mapping bug. Its comment says: *"If it ever ran first, 'place spoon on table' would inherit ceramic-mug Foley again - the original mis-mapping bug."*

4. **"every archetype maps onto a real motion band and a real selection path"** - an integration guard between two modules that never call each other directly. Its comment: *"prompt_synthesis invents specs, synchronization consumes them. A region that is not a motion band would KeyError at sync time, on a live job."*

63.3 A test that documents a limitation

```
@test("stirring phrasings resolve to the stirring class")
def _():
    # NOTE: "mixing the drink" is deliberately excluded - it contains the noun
    # "drink", which makes it genuinely ambiguous. Documented as a limitation.
```

The test excludes a case that does not work, **and says why**, rather than silently passing over it.

63.4 Running them

```
cd Module3_Fresh
moss/venv-moss/bin/python backend/tests/test_suite.py          # 42
moss/venv-moss/bin/python backend/tests/test_foley_validation.py # 22
moss/venv-moss/bin/python backend/tests/e2e_gate.py           # end-to-end, ~5 s
```

All were executed while this handbook was written. Result: **42 passed, 0 failed** and **22 passed, 0 failed**, and the end-to-end gate reported OK.

63.5 Subsystem 3 tests

37 pytest tests across four files, using synthetic tiny-video fixtures built in `confest.py` so the suite does not depend on any particular media file.

63.6 What is not tested

Be honest about this.

- **Subsystem 1 has no automated test suite.** It has seven recorded inference transcripts, which are evidence but not tests.
- There is no test that runs the real models end to end from a cold cache, because that would take about nine minutes per run.
- There are no browser or user-interface tests.
- There is no load or concurrency testing, because the system is explicitly single-job.

PART 10

CHALLENGES, LIMITATIONS AND FUTURE WORK

Every real engineering problem this project hit, what caused it, what was done, and what it achieved; then an honest list of what the system still cannot do.

Chapter 64 — Challenges: cause, solution, result

Every entry in this chapter is supported by a comment in the source, a record in the job store, or a written report. None of them is invented for the sake of having a challenges section.

64.1 Model and environment challenges

Challenge	Cause	Solution	Result
Three dependency stacks cannot coexist	<code>numpy<2</code> for MediaPipe against <code>numpy>=2</code> for Kokoro; <code>torch 2.5.1</code> against 2.9.1 against 2.13; Python 3.10, 3.11 and 3.12	Five isolated virtual environments; models invoked as subprocesses; a persistent worker process for Kokoro driven over <code>stdin</code> and <code>stdout</code>	All three subsystems run on one machine
Two models each peak near 12 GB on a 17 GB machine	Qwen2.5-VL and MOSS both need most of the machine	Never load them together; subprocess exit returns memory to the operating system unconditionally	Peak 12.11 GB mean; zero guard breaches in 31 runs
MOSS needs about 10.59 GB of resident weights	The upstream pipeline forwards <code>torch_dtype</code> only to the text encoder, so the DiT and VAE load in <code>float32</code>	Three memory-separated phases with <code>assert live_instances(...) == 0</code> between them, plus a parameters-only <code>bfloat16</code> cast in a wrapper	Swap growth fell from +9.80 GB to +0.01 GB
MPS has no <code>float64</code>	<code>sinusoidal_embedding_1d</code> computes in <code>float64</code> and raises a <code>TypeError</code>	Run the identical <code>float64</code> arithmetic on CPU and move the result back; patch three modules at runtime	Verified bit-exact against upstream: <code>max_abs_diff 0.0</code> , <code>exact_match true</code>
A blanket <code>bfloat16</code> cast destroys rotary encoding	The DiT carries three <code>complex128</code> RoPE buffers, and casting complex to real discards the imaginary part	Cast parameters only ; downcast <code>complex128</code> buffers to <code>complex64</code> , which is lossless because <code>rope_apply</code> reads <code>.real</code> and <code>.imag</code> at <code>float32</code>	Asserted: <code>buffer_dtypes == ["torch.complex64"]</code>
Auto-AVSR's beam search crashes on MPS	<code>ctc_prefix_score.py</code> branches on <code>x.is_cuda</code> , which is <code>False</code> for MPS tensors, so a CPU device is assigned to MPS tensors	Force CPU and raise a clear error if any other device is requested	Works at 0.45x real time
<code>torch.compile</code> has no MPS path	MOSS's install docs are CUDA-first and use Triton and TorchDynamo	<code>TORCHDYNAMO_DISABLE=1</code> set in the subprocess environment from the start	No compile attempts
Stable Audio's default guidance needs <code>float64</code>	<code>apg_scale=1.0</code> takes an adaptive-projected-guidance path that computes in <code>float64</code>	Set <code>apg_scale=0.0</code> for vanilla CFG; <code>float32</code> throughout, because <code>float16</code> produced non-finite values	The alternative backend runs on MPS
Auto-AVSR's install instructions break	The upstream README says <code>pip install torch torchvision torchaudio</code> with no pins	Pin <code>torchvision==0.20.1</code> (<code>read_video</code> removed in 0.26), <code>av==13.1.0</code> (PyAV 14 removed <code>av.AVError</code>), <code>numpy==1.26.4</code> , <code>mediapipe==0.10.21</code> , with the reason for each written into <code>requirements.txt</code>	A reproducible environment

64.2 Recognition challenges

Challenge	Cause	Solution	Result
Closed-vocabulary recognisers return nonsense	Kinetics labels describe whole-clip activities, not object-contact events	Switch to a vision-language model with free-text output	Real action phrases: "pick up cup", "drink from cup"

Challenge	Cause	Solution	Result
A VLM asked for timestamps is unreliable	Language models are poor at producing numbers	Ask only "what is happening here?"; derive timing from the window's known position	"Semantics from the VLM; timing from the windowing"
Merged spans overlap by one stride	The stride is half the window	Midpoint boundary resolution, deterministic and left-to-right, preserving the raw arrays as an audit record	A non-overlapping timeline with all six validation checks passing
Thin-evidence segments	Some spans are supported by a single window	Flag as suspect and never delete ; surface as Medium confidence	No action is ever silently removed
<code>resolve_boundaries</code> returns a tuple	It returns (<code>segments</code> , <code>adjustments</code>) and an early version assigned the tuple to one variable	Unpack explicitly, with a comment naming the trap	Job 3805a3d282d4 records the original failure
HDR input silently ruins accuracy	Untone-mapped HDR shifts the model-input tensor mean from about 0.06 to about 1.27	Detect HDR from colour transfer, primaries and pixel format; reject with an explanation rather than converting badly	Confidence stays in the good range, or the user is told why it cannot
30 fps material degrades accuracy	The model is trained on 25 fps	Standardise every upload with <code>fps=25</code> , using frame-rate conversion and not retiming, so the speaking rate is preserved	The verified baseline path is stream-copied bit for bit when already correct

64.3 Generation challenges

Challenge	Cause	Solution	Result
Five text-to-audio models produced silence and clicks	Every run asked for 2.0 to 4.5 s from models trained at 10 to 47 s	Generate at the model's native duration and crop afterwards	The final system generates 10 s and uses only what it needs
MOSS occasionally collapses for a given seed	A sampling failure, not a capability limit	Generate up to three candidates with successive seeds, stopping early at score 45	Cup pickup failed on seeds 42 and 43 and passed on 44 with 85.8
Degenerate output is amplified into hiss	An active-RMS leveller faithfully raises an empty file to target, applying tens of dB	Measure every asset raw against six gates before it can enter the mix; refuse rather than repair	29.6 percent of 54 assets rejected; 11 would have needed more than +25 dB
A single metric can be gamed	One candidate had 63.7 percent ceramic-band energy but 1.3 dB of dynamic range - it was hiss	Six independent gates, no single one decisive	The hiss candidate is rejected
A musical tone with a natural envelope passed	Gate 3 requires a tone to be both harmonic and dynamically flat	Add an independent pure-tone gate on harmonic ratio plus spectral flatness	The 346 Hz sine wave is now rejected
Dynamic range measured 183 to 201 dB	Frames of exact digital silence send the 5th percentile to the numeric floor	Measure over frames above -70 dB relative to the maximum; cap at 96 dB	A 90-percent-silent file no longer scores full marks
<code>active_rms</code> collapsed to whole-file RMS	The 60th-percentile gate degenerates to 0 when a clip is mostly digital silence	Take the higher of the 60th percentile and 40 dB below the loudest frame	Level setting works on sparse clips
Identical audio was regenerated	<code>duration: 10</code> and <code>10.0</code> serialise differently, giving different cache keys	Normalise every numeric with <code>int()</code> or <code>round(float(), 4)</code> before hashing	4.7 minutes saved per avoided regeneration

Challenge	Cause	Solution	Result
"place spoon on table" got ceramic-mug Foley	Bare <code>place</code> and <code>pick</code> keywords matched any object	Every keyword names its object; specific classes are tried before generic ones	A permanent regression test guards it
"place bread in toaster" resolved to nothing, failing the job	The precision fix left a hole: generic keywords demanded literal surfaces	A verb-plus-object fallback tier, running last	Four recorded job failures are now impossible
Any unlisted action was silent	The curated class list was a closed set	Open-vocabulary prompt synthesis from nine verb archetypes	<code>impact_football</code> , <code>friction_notebook</code> , <code>ambient_toast_bread</code> all sounded
A longer negative prompt collapsed the output	Possibly CFG pushing away from 24 concepts at once	Recorded as a two-point observation, explicitly not a finding , and not acted upon	The correct scientific posture, documented

64.4 Synchronisation and mixing challenges

Challenge	Cause	Solution	Result
Sound placed at label boundaries is visibly wrong	An action interval is a span; a Foley event is an instant	Independent frame-level motion analysis with four physics-based detection strategies	Worst error 20 ms, below half a frame
A 96 ms audible misalignment	Onset- strength peaks were used as anchors; they lead or lag the true attack by -96 to +250 ms	Align on the true envelope attack : back-track from the envelope maximum to where it last rose through 20 percent	The error disappeared
Footsteps begin before the "walk" label	Module 2 labels 0.0 to 1.5 s as <code>stand</code> , but feet-band motion there is 1.708 against 1.711 in the labelled walk	Widen the search span for footsteps to 0.0 to 2.50 s; do not modify the timeline	All four foot plants are found; the label stays reported as wrong
Residual error accumulates across footsteps	Shift-only placement cannot correct a cadence mismatch	Match the best-fitting run of generated steps to the filmed gait, and report the residual per event	MOSS 0 to -67.6 ms; Stable Audio -462.4 ms, fully explained
Consecutive same-class intervals looped audibly	Each replayed the same 1-second source segment	Merge consecutive continuous activities; deliberately do not merge discrete contacts	Three stirring labels became one 3-second span; merging contacts lost a visual event when tried
Continuous audio started at the label edge	The "trust the boundary" mistake, in a different place	Detect an <code>activity_start</code> from the first frame whose motion exceeds median plus half a standard deviation	Audio starts when the activity does
All events sounded equally loud	The -12 dBFS peak cap was binding on most clips, so the cap rather than the per-class targets set relative level	Raise the cap to -6 dBFS so it is an outlier guard again	Inter-event dynamics restored; crest factor 30.87 dB
A clip crossing the end of the video	The event happens too near the end	Truncate with a fade if at least 45 percent fits; otherwise omit, because a sliver reads as a click	Timeline length always preserved
Edit points clicked	Cuts landed mid-waveform	Snap to the nearest zero crossing within plus or minus 3 ms, and correct the placement by the snap offset	All 8 clip boundaries verified clean

64.5 Application challenges

Challenge	Cause	Solution	Result
A four-minute request cannot be an HTTP request	Foley generation dominates	A background worker thread with a persisted job record and a polling client	The browser stays responsive
Progress across a process boundary	The model runs in a different interpreter	The child writes <code>m2_progress.json</code> per window; a parent thread polls it once a second	Real progress, not a fake bar
The user waits four minutes with nothing to see	Generation is the long pole	Fetch and display the action timeline as soon as the <code>timeline</code> stage completes	The user sees detected actions during generation
A model crash could take down the server	Models are large and fragile	Subprocess isolation; a non-zero exit code becomes a readable message	The API survives
Stack traces leaking to the client	Unhandled exceptions	<code>traceback.print_exc()</code> to the log; a plain sentence to the user; a catch-all handler in Flask and explicit mapping in FastAPI	Tests assert that no response contains "Traceback"
No usable Foley for any action failed the whole job	The pipeline raised rather than degrading	Treat it as a legitimate partial result: mark the stage <code>skipped</code> , produce the video with a silent track	<i>"Failing the job here used to strand the user with nothing to export"</i>
A stale server serving old code	uvicorn holds imported modules in memory	Documented in <code>HANDOFF.md</code> as a hard rule: restart the backend after any code change	It had already caused one confusing debugging session

Chapter 65 — Limitations

State these before you are asked. Every one of them is in the project's own documentation.

65.1 The limitation that matters most

CAUTION: Action recognition is the binding constraint

Foley can only be as good as the timeline it is given.

On a coffee-stirring test video, Qwen2.5-VL **missed a cup placement entirely**, emitted one stirring action under **three different labels**, and returned a **caption** ("Stirring a cup of coffee") rather than an action.

On the reference clip it labels 0.0 to 1.5 s as `stand` when feet-band motion there is 1.708 against 1.711 in the labelled walk - the subject is walking from about 0.2 s, and two of the four foot plants fall inside the wrong label.

The pipeline compensates by widening the footstep search span, but **the label itself is wrong**, and no amount of downstream engineering fixes a timeline that omits an event.

65.2 Subsystem 1 limitations

- CPU only; the MPS path crashes in `ctc_prefix_score.py`.
- HDR is rejected rather than converted, because a correct HLG-to-SDR chain needs FFmpeg's

`zscale` filter, which many builds lack.

- 30 to 25 fps conversion drops roughly one frame in six.
- Flask development server; decoding is serialised by a lock.
- No authentication and no rate limiting.
- **Sync quality is independent of recognition accuracy**: wrong words are lip-synced accurately and wrongly.
- The voice is a generic TTS voice, not the speaker's own.
- Gender detection is a heuristic, binary, and roughly 85 to 90 percent accurate.
- Word boundaries *inside* a phrase are estimated from phoneme counts, not measured.
- English only.
- **No word error rate is reported**, because ground-truth transcripts were not retained.
- The task is fundamentally ambiguous: different sentences produce identical lip motion.

65.3 Subsystem 2 limitations

- **Apple Silicon only**. The pipeline depends on MPS; there is no CUDA or CPU path.
- Foley generation is slow, and a short request costs the same as a long one because the model denoises a fixed-length latent.
- **Output is sparse by nature**: roughly 18 percent of a 10-second timeline carries audio for discrete object interactions. The silence between events is correct.
- Sound quality varies by class. Walking and drinking were validated by listening; quiet

ceramic contacts are the hardest class.

- **Visual localisation is motion-based**, not pose estimation or object tracking. It has been validated on limited footage; other camera angles and subjects are unverified.
- **The synchronisation result is seven events on one clip**, on the build tuned against it. Across all recorded runs the median is 4.7 ms but the worst is 462.4 ms.
- Only **footstep** placements record a per-event residual, so the multi-clip evidence is thinner than "32 job runs" suggests.
- Video length is capped at 60 seconds because recognition cost grows linearly.
- **Single-machine, single-job**: no authentication, no queue, no multi-user isolation.
- `allow_origins=["*"]` would be unacceptable in production.
- Jobs run in-process; a server restart loses running jobs, though completed records persist to disk.

65.4 Subsystem 3 limitations

- **Camera frame rate is the ceiling**: one audio sample per frame, so the highest recoverable frequency is half the frame rate. Speech and music are not recoverable from 30 or 60 fps footage. This is the sampling theorem, not a defect.
- Visible, sound-induced vibration is required. A rigid wall or a quiet room gives nothing.
- Material matters: light, high-contrast, resonant surfaces work; heavy rigid ones do not.
- Mains flicker, rolling shutter, sensor noise, compression blocking and camera motion all inject noise or destroy the signal.
- Cost is one complex steerable pyramid per frame.
- The job store is in memory; restarting the server forgets running jobs.
- **The high-speed demonstration used the original authors' published clip**, not footage captured for this project, because no high-speed camera was available. The source file is not retained in the repository.
- The synthetic characterisation is an **optical** test, not an acoustic capture.

65.5 Cross-cutting limitations

- **No model was trained or fine-tuned**. Every model is used with published weights.
- **No held-out labelled evaluation set exists** for any subsystem.
- **Listening evaluation is single-assessor** - the developer. Given that Chapter 33 demonstrates objective metrics failing to capture perceptible degradation, this is a substantive limitation rather than a formality.
- **No comparative benchmark** against Diff-Foley, MMAudio or FoleyCrafter on a standard dataset with standard metrics. The rejections recorded here are measurements on this project's own material and hardware and should not be read as general statements about those systems.

- The project's written documentation has drifted from the code in two places: the class count (16 in prose, 17 in code) and the test count (59 in prose, 64 in code).

65.6 Ethical and privacy considerations

An examiner may well ask, and having thought about it is worth marks.

Concern	Position
Fabricated evidence	This system generates <i>plausible</i> audio, not <i>recovered</i> audio. Reconstructed speech from lip reading must never be treated as a transcript of what was actually said. Lip reading is fundamentally ambiguous, and a confident wrong sentence is possible
Surveillance	Lip reading applied to surveillance footage raises obvious privacy concerns. The system runs entirely locally, which is a mitigation, but the capability itself is dual-use
Consent	Any deployment on footage of identifiable people needs their consent
Bias	The lip-reading checkpoint's training corpora are English, largely professionally recorded, and not demographically uniform. Performance on under-represented accents and appearances is unknown and likely worse
Misinformation	Adding convincing synthetic audio to real video is a deepfake-adjacent capability. Outputs should be labelled as synthetic
Mitigation actually implemented	The system reports every uncertainty: suspect labels, low-confidence visual events, rejected assets, and intervals left silent. It never fabricates audio to fill a gap it could not fill honestly

Chapter 66 — Future scope

Separate what exists from what does not. Never present future work as if it were done.

66.1 Currently implemented

Capability	Status
Lip reading to timed synthetic speech	Working, verified on 7 recordings
Action recognition to synchronised Foley	Working end to end, 64 automated tests pass
Visual microphone	Working; characterised on synthetic stimuli; demonstrated on one high-speed clip
Visual-event synchronisation with true attack alignment	Working, 20 ms worst case on the reference clip
Six-gate quality validation with candidate retry	Working, 29.6 percent rejection rate over 54 assets
Open-vocabulary Foley	Working, demonstrated on football, vegetables, notebook, toaster and dog
Content-addressed cache	Working, 39.5 s cached against 556.9 s uncached
Three web interfaces with real progress reporting	Working
Process and memory isolation across five environments	Working, zero guard breaches in 31 runs

66.2 Future work, in priority order

1. Improve the action timeline. This bounds everything downstream in Subsystem 2.

Two concrete directions:

- **Prompt engineering** for Qwen, to elicit action phrases rather than captions. The current prompt already says "Identify the main physical action", and it still returns "Stirring a cup of coffee".

- **Label normalisation**: merge synonymous predictions across windows, strip caption phrasing, and deduplicate. Three labels for one stirring action is a solvable problem.

2. A formal listening evaluation. Multiple assessors, randomised presentation, a pairwise design. Chapter 33 shows that objective scores are insensitive to audible degradation, which makes a perceptual study **the only way** to validate the gate's thresholds.

3. Relax the shift-only policy for rhythmic actions. Chapter 30 shows that residual accumulation is driven by cadence mismatch. Selecting among several generated candidates **by cadence fit**, rather than by quality score alone, would address it without time-stretching. This is a small, well-defined change with a clear motivation.

4. Evaluate the visual microphone on genuine high-speed footage captured for the project. The synthetic characterisation establishes the implementation is correct; a 240 fps or faster capture of a loud source next to a light, resonant object is the natural test.

5. A comparative benchmark. Diff-Foley, MMAudio and FoleyCrafter on VGGSound or a comparable dataset, reporting Frechet Audio Distance alongside onset accuracy, would place the alignment method in context.

6. Object-aware visual localisation. Motion in a fixed horizontal band is a crude proxy. Pose estimation or object tracking would generalise across camera angles and subjects, which is currently unverified.

- 7. Word error rate for Subsystem 1.** Record ground-truth transcripts for a set of test recordings and report a real number.
- 8. Broader deployment.** A CUDA path, a job queue, authentication, multi-user isolation, and containerisation. All are straightforward engineering, and none of them is the interesting part.
- 9. More Foley classes and better prompts.** The curated set is 17; open vocabulary covers the rest but is not hand-tuned by ear. Promoting frequently-synthesised classes to curated ones, after listening, is incremental and cheap.
- 10. Real-time processing.** Currently out of reach: about 4.7 minutes per generated asset and 5.2 seconds per recognition window. A faster generator, aggressive caching, or a smaller model would be required, and the ablation in Chapter 33 shows that naive speed-ups degrade quality in ways the metrics do not catch.

PROFESSOR MAY ASK: Can this work in real time?

"Not as it stands, and I can tell you exactly why. Action recognition is 5.2 seconds per 2-second window and Foley generation is 4.7 minutes per asset, of which 94 percent is diffusion. Caching brings a repeated job down to about 40 seconds, but that is reuse, not speed.

What is closer to real time is the *placement* half - the visual analysis, alignment, mixing and render together take under 20 seconds for a 10-second clip. So if the sounds were pre-generated into a library, the synchronisation could run near real time. The generation is the bottleneck, not the method.

I also tried the obvious speed-up - a shorter denoised latent, 2.8 times faster - and reverted it, because it halved the number of audible transients while *raising* the measured quality score."

PART 11

VIVA QUESTION BANK

Fifteen levels, from "what is your project" to "why did you choose this and not that". Every answer has a one-line version, a full version, and the project-specific evidence that turns it into proof.

Chapter 67 — Level 1: basic project questions

Q1. What is your project?

Short. A system that generates the audio for a silent video using only the pixels.

Detailed. It reconstructs audio along three independent paths: visual speech recognition with timed synthesis, action recognition with synchronised Foley generation, and phase-based recovery of surface vibration. All three run locally on one laptop.

In our project. The three paths live in `02-Auto-AVSR-Test/`, `Module3_Fresh/` and `Acoustic_eye/`, and the research paper in `paper/` covers all three.

Q2. What problem does it solve?

Short. A great deal of recorded video has no usable audio, but the picture still constrains what the audio was.

Detailed. Surveillance cameras often record picture only; archive footage may have lost its sound; phone recordings are ruined by wind or a muted microphone. In each case lips move in a way that limits what could have been said, a mug meeting a table implies a specific contact sound at a specific instant, and surfaces vibrate in response to the sound field.

In our project. The three subsystems attack those three kinds of information respectively.

Q3. What is the input and what is the output?

Short. In: a silent video file. Out: the same video with a generated audio track.

Detailed. Subsystem 2 accepts MP4, MOV, AVI, M4V or MKV, at most 200 MB and between 0.4 and 60 seconds, and returns an MP4 whose picture stream is bit-identical to the input and which carries AAC audio at 192 kbit/s, 48 kHz mono. Subsystem 1 accepts MP4, MOV or M4V and returns the recognised sentence plus a video with timed speech.

In our project. The limits are constants in `backend/core/config.py`: `MAX_UPLOAD_MB = 200`, `MAX_VIDEO_SECONDS = 60`, `ALLOWED_SUFFIX = {".mp4", ".mov", ".avi", ".m4v", ".mkv"}`.

Q4. Why does it need three separate systems?

Short. Three kinds of information, and three dependency stacks that cannot coexist.

Detailed. Speech is carried by articulator motion, Foley by contact events, and vibration by sub-pixel displacement; no single model reads all three. And Kokoro needs `numpy>=2` while MediaPipe declares `numpy<2`, so they physically cannot share an interpreter.

In our project. Five virtual environments across Python 3.10, 3.11 and 3.12.

Q5. What is Foley?

Short. Everyday sound effects created to match action on screen.

Detailed. Named after Jack Foley. In film, footsteps, cloth movement and object handling are almost always re-performed in a studio rather than captured on set, because production audio is dominated by dialogue.

In our project. The Foley is *generated* by a model from a text description, not performed or sampled from a library.

Q6. Which is the main part of your project?

Short. Subsystem 2 - action recognition and Foley generation.

Detailed. It is where the original engineering is. The models are off-the-shelf, but the placement, the validation, the level policy, the open vocabulary, the cache, the memory-phased inference wrapper and the nine-stage pipeline are the project's own work.

In our project. `Module3_Fresh/` is 16 GB and contains 64 automated tests.

Q7. What is your contribution?

Short. Placement and validation.

Detailed. Anyone can call a generative model. Making the sound land on the right frame

- by measuring visual events and aligning on true envelope attacks - and refusing to ship

sound that measurement says is unusable, is the work.

In our project. Worst-case 20 ms alignment on the reference clip, and 29.6 percent of 54 generated assets rejected by the quality gate.

Q8. Did you build this alone?

Short. Yes, and I used published pre-trained models rather than training my own.

Detailed. Auto-AVSR, Qwen2.5-VL, MOSS-SoundEffect and Kokoro are all published checkpoints. The Visual Microphone algorithm is Davis et al. 2014, adapted from an MIT-licensed reference implementation with attribution retained.

In our project. The attribution is in `Acoustic_eye/acoustic-eye/README.md` section 9 and in every adapted source file's header.

Chapter 68 — Level 2: module and pipeline questions

Q9. Walk me through the pipeline.

Short. Nine stages: upload, validation, action recognition, timeline, Foley generation, quality validation, visual synchronisation, mixing, rendering.

Detailed. See Chapter 22 and Figure 22.1.

In our project. The stage list is literally a constant in `backend/core/jobs.py`, and `/api/health` returns it so the frontend renders the same nine names.

Q10. What happens when a user uploads a video?

Short. The extension is checked, the file is streamed to disk in 1 MB chunks with the size limit enforced during the stream, `ffprobe` reads its metadata, validation raises or returns warnings, and a job record is created and persisted.

Detailed. See Chapter 46.

In our project. `routes.py::upload()`. The client's filename is never used as a path; the stored name is a UUID.

Q11. What happens internally after pressing Generate?

Short. A background thread runs the nine stages while the browser polls status every 1.5 seconds.

Detailed. See Chapter 39, which traces every arrow.

In our project. The HTTP request returns `{"status": "queued"}` in milliseconds; `STORE.run(job_id, run_pipeline)` starts a daemon thread.

Q12. Why nine stages and not one function?

Short. So that progress is real, failures are localised, and each concern is testable on its own.

Detailed. Every stage reports its state to a job store that is persisted after every transition, so the browser sees genuine progress and a failure names the stage it happened in. Each stage is a separate service module with its own tests.

In our project. 122 job records on disk, each showing exactly which stage a job reached.

Q13. Why is `resolved_actions` used and not `actions`?

Short. Because the raw array overlaps and an overlapping interval is ambiguous for audio placement.

Detailed. The stride is half the window, so merged spans overlap by one stride. On the reference clip `pick up cup` runs 2.0-6.0 s and `drink from cup` runs 5.0-9.0 s. An instant belonging to two actions gives no answer to "which sound plays here?".

In our project. `pipeline.py: resolved = m2.get("resolved_actions") or m2.get("actions") or []`, and a test asserts that the raw array really does overlap.

Q14. What is "demo mode"?

Short. It runs the same pipeline on the bundled validated clip, reusing the stored Module 2 timeline for that specific video.

Detailed. Foley generation, quality validation, synchronisation, mixing and rendering all execute for real; only action recognition is short-circuited, because it takes about a minute and its answer for that clip is already recorded.

In our project. `routes.py::demo()` sets `is_demo=True`, and `pipeline.py` branches on it to call `AR.load_existing(C.DEMO_MODULE2)`.

Q15. How do the modules communicate?

Short. Subprocesses with JSON files and JSON on standard streams.

Detailed. The backend never imports a model. It runs another Python interpreter with `subprocess.run`, passes paths as arguments, and reads a JSON file the child writes. Progress crosses the boundary through a small JSON file the child rewrites per window and the parent polls once a second. Errors cross as JSON on `stderr`.

In our project. `run_module2.py` writes `module2.json` and `m2_progress.json`; `moss_generate.py` writes a WAV and a full JSON generation record.

Chapter 69 — Level 3: machine-learning questions

Q16. What is machine learning?

Short. Programs whose behaviour comes from parameters fitted to data rather than from rules a programmer wrote.

Detailed. Deep learning is the subset using many-layered neural networks, so early layers learn simple features and later layers learn compositions of them.

In our project. Every model here is deep learning. Subsystem 3 contains **no machine learning at all** - it is classical signal processing.

Q17. Did you train any model?

Short. No. Every model uses published pre-trained weights, unmodified.

Detailed. Training a 3-billion-parameter vision-language model requires hardware and data this project does not have. The engineering is in the pipeline, the placement and the validation.

In our project. There is no optimiser, no loss function, no backward pass and no dataset loader anywhere in the delivered code. `app/inference.py` calls `torch.set_grad_enabled(False)` before anything else.

Q18. What is the difference between training and inference?

Short. Training adjusts parameters using labelled data; inference runs a trained model forward.

Detailed. Training needs gradients, a loss function and an optimiser, and is expensive. Inference needs none of those.

In our project. Inference only: `torch.inference_mode()` in `run_module2.py`, `torch.no_grad()` in `moss_generate.py`.

Q19. What is overfitting? Is your system overfitted?

Short. Overfitting is memorising the training data and failing on new data. The term does not apply here, because I did not train anything.

Detailed. What *is* relevant is generalisation. Every clip this project processes is data no model ever saw.

In our project. The honest caveat is that the 20 ms synchronisation figure is measured on one clip using the build tuned against it, so I quote it as a demonstration that sub-frame placement is achievable, not as a general accuracy figure.

Q20. What is a parameter?

Short. One of the adjustable numbers inside a neural network.

Detailed. Training chooses their values; inference uses them. Parameter count determines memory: a 3.5-billion-parameter model is 7 GB at bfloat16 and 14 GB at float32.

In our project. Auto-AVSR 250,383,410; Qwen2.5-VL about 3 billion; MOSS 3,508.21 M across three components loaded one at a time.

Q21. What is bfloat16 and why do you use it?

Short. A 16-bit float with float32's exponent range but less precision. It halves memory.

Detailed. Neural network inference cares far more about dynamic range than about the last decimal places, so the trade is usually free.

In our project. It was not free by default: MOSS's upstream pipeline forwards `torch_dtype` only to the text encoder, so the DiT and VAE load in float32 - 10.59 GB of weights, producing +9.8 GB of swap. The wrapper casts parameters itself, and swap growth fell to +0.01 GB.

Q22. What is a "seed"?

Short. A number that fixes the random number generator so a run is reproducible.

Detailed. Generative models sample from a distribution; the seed determines which sample you get. Same seed and settings, byte-identical output.

In our project. Default 42, part of the cache key, and the mechanism behind multi-candidate generation: cup pickup failed on 42 and 43 and passed on 44.

Chapter 70 — Level 4: deep-learning questions

Q23. What is a CNN?

Short. A network built from small filters slid across an image, so the same pattern detector works everywhere.

Detailed. Convolution produces feature maps; stride and pooling downsample; stacking builds a hierarchy from edges to parts to objects. A `Conv3d` slides over space *and time*, so it responds to motion.

In our project. Auto-AVSR's frontend is a `Conv3dResNet` taking `(B, T, 1, 88, 88)`; the gender classifier is a small CNN; the DAC decoder inside MOSS is convolutional.

Q24. What is an RNN, an LSTM, a GRU?

Short. Networks that read a sequence step by step while keeping a running memory. LSTMs and GRUs add gates so that memory survives long sequences.

Detailed. A plain RNN suffers vanishing gradients. An LSTM adds a cell state updated additively, controlled by forget, input and output gates. A GRU simplifies this to two gates and no separate cell state.

In our project. None of them appear. Every sequence model here is a Transformer or a Conformer. Understand them because you will be asked; do not claim you use one.

Q25. What is a Transformer? What is attention?

Short. A Transformer processes a whole sequence at once using attention, which lets every position look at every other and weigh what matters.

Detailed. Each position produces a query, a key and a value. Dot products of queries with keys, scaled and softmaxed, become weights on the values. Multi-head attention runs several of these in parallel. Because positions are independent, the whole sequence computes in parallel, unlike an RNN.

In our project. Auto-AVSR's encoder is a Conformer and its decoder a Transformer; Qwen2.5-VL is a Transformer; MOSS's denoiser is a 30-layer Diffusion Transformer with 12 heads and dimension 1536; MOSS's text encoder is Qwen3.

Q26. What is a Conformer and why is it right for lip reading?

Short. A Transformer block with a convolution module inside it.

Detailed. Attention captures long-range sentence structure; convolution captures local articulator detail. Speech needs both.

In our project. `auto_avsr/espnet/nets/pytorch_backend/encoder/conformer_encoder.py`.

Q27. What is positional encoding? What is RoPE?

Short. Attention has no notion of order, so position must be added. RoPE adds it by rotating the query and key vectors by an angle proportional to position.

Detailed. Because the dot product of two rotated vectors depends on the difference of their angles, RoPE makes attention naturally *relative* rather than absolute.

In our project. MOSS's DiT carries three `complex128` RoPE tables. A blanket `bfloat16` cast would discard their imaginary part and destroy rotary encoding, so the wrapper casts parameters only and downcasts those buffers to `complex64` - which is lossless because `rope_apply` reads `.real` and `.imag` at `float32`.

Q28. What is softmax? What are logits?

Short. Logits are raw unnormalised scores. Softmax turns them into probabilities that sum to one.

Detailed. Log-softmax returns the logarithm, which is numerically safer and lets a beam search add scores instead of multiplying probabilities.

In our project. `model.ctc.log_softmax(enc_feat.unsqueeze(0))` in `app/timing.py`. The decoder scores in the transcripts are log-probabilities, which is why they are negative.

Q29. What is a VAE?

Short. An encoder that compresses data to a compact latent and a decoder that expands it back, with the latent space shaped so it can be sampled meaningfully.

Detailed. Generative models work in latent space because generating 48,000 numbers per second directly is intractable.

In our project. MOSS uses a DAC VAE with `hop_length = 960`, so 1500 latent steps decode to 1,440,000 audio samples - a 960-fold reduction in what the generator must produce. The wrapper asserts that exact number.

Q30. What is diffusion?

Short. Start from pure noise and repeatedly ask a network "what here is noise?", subtracting a little each time.

Detailed. A fixed forward process adds Gaussian noise to data; a learned reverse process removes it. Generation is running the reverse process from noise.

In our project. 50 steps, and the ablation shows halving them moves the quality score by 0.6 points - within noise - so the decision to keep 50 rests on listening.

Q31. What is flow matching, and how is it different from diffusion?

Short. Instead of learning to remove noise along a wandering path, it learns a velocity field pointing straight from noise to data.

Detailed. Flow matching regresses the vector field that transports samples along a probability path; generation integrates that field, typically with Euler steps.

In our project. MOSS uses a `FlowMatchScheduler` with `sigma_shift = 5`, read from the checkpoint's own `scheduler_config.json`.

Q32. What is classifier-free guidance?

Short. Run the model with and without the prompt, and push the answer in the direction of the difference.

Detailed. `noise_pred = negative + cfg * (positive - negative)`. A scale of 1 means no guidance; larger values obey the prompt harder but can become exaggerated.

In our project. CFG 4.0. Note the cost: the model is evaluated **twice per step**, so 50 steps means 100 forward passes through a 1.4-billion-parameter Transformer - which is why generation takes 4.7 minutes.

Chapter 71 — Level 5: model questions

Q33. Which models does your system use?

Short. Auto-AVSR for lip reading, Qwen2.5-VL-3B-Instruct for action recognition, MOSS-SoundEffect v2.0 for Foley, Kokoro for speech, and a small CNN for gender.

Detailed. See Chapter 60 for the full specification of each.

In our project. They never run at the same time; each is a subprocess in its own environment.

Q34. Why Qwen2.5-VL and not an action classifier?

Short. Because it writes free text, so the vocabulary is not fixed in advance.

Detailed. Kinetics-style label sets describe whole-clip activities; Foley needs object-contact events, and there is no Kinetics class for "a mug is set on a table".

In our project. VideoMAE returned "shredding paper" at 0.550 with no relevant label in the top ten; X-CLIP returned "pouring liquid" at 0.21-0.38 in every window. Both records are in `03-FoleyCrafter-Test/action-recognition/results/`.

Q35. Why MOSS-SoundEffect and not Stable Audio?

Short. Because Stable Audio produced musical tones for object contacts, and Foley must be inharmonic.

Detailed. Four reasons for MOSS: 48 kHz output, duration control to 30 s, an explicit human-action Foley category, and an Apache-2.0 licence where the alternatives are non-commercial.

In our project. Two independent measurements agree. Harmonic ratio on object contacts: MOSS 0.00 to 0.02, Stable Audio 0.87 to 0.88 - one output was a pure 346 Hz sine wave with spectral flatness 0.00000. And cadence: the MOSS walking asset is 9.9 percent slow giving -67.6 ms of accumulated residual, where Stable Audio is 30.2 percent fast giving -462.4 ms. Crucially, their **median quality scores are nearly identical**, 54.5 against 53.2.

Q36. Why did you keep Stable Audio in the code if you rejected it?

Short. So the two can be compared on the identical pipeline without changing code.

Detailed. `FOLEY_BACKEND=stable_audio` switches it. Keeping the loser runnable is what made the controlled comparison possible.

In our project. The backend name is part of the cache key and the filename, so the two models can never collide, and a test asserts it.

Q37. How many parameters does MOSS have?

Short. 3,508.21 million in total, across three components loaded one at a time.

Detailed. DiT 1,416.05 M, Qwen3 text encoder 1,720.57 M, DAC VAE 371.59 M. The DiT is 30 layers, dimension 1536, 12 heads, feed-forward 8960.

In our project. Those architecture numbers come straight from `mooss/checkpoints/MOSS-SoundEffect-v2.0/transformer/config.json`.

Q38. What data was MOSS trained on?

Short. Not specified in the project implementation.

Detailed. Its documentation describes categories - natural environments, urban environments, animals and creatures, human actions - which is why it was chosen, but the corpus itself is not recorded here.

In our project. I would rather say "I did not record that" than guess.

Q39. How do you load the model?

Short. In three memory-separated phases, so the three components are never co-resident.

Detailed. Phase 1 loads the text encoder, encodes both prompts, moves them to CPU and frees the encoder. Phase 2 loads the DiT, denoises, moves the latent to CPU and frees it. Phase 3 loads the VAE, decodes, crops and writes.

In our project. Residency is *proved*, not *hoped*: `live_instances()` walks `gc.get_objects()` and the code asserts `resid["Qwen3TextEncoder"] == 0` and `resid["WanAudioModel"] == 0` between phases.

Chapter 72 — Level 6: dataset questions

Q40. What dataset did you use?

Short. None for training. My models are pre-trained.

Detailed. The lip-reading checkpoint was trained on LRS2, LRS3, VoxCeleb2 and AVSpeech by its authors. Qwen2.5-VL's and MOSS's corpora are not specified in the project files.

In our project. My evaluation material is a small set of recordings: one 10-second reference clip held under a recorded SHA-256, a coffee-stirring video, a toaster video, seven lip-reading recordings, and synthetic stimuli for the visual microphone.

Q41. What are LRS2 and LRS3?

Short. Lip Reading Sentences 2 and 3: audio-visual corpora of spoken sentences with face tracks and transcripts, from BBC television and from TED talks respectively.

Detailed. LRS3-TED is the standard open benchmark for sentence-level lip reading, and the published 20.3 percent WER for this checkpoint is on it.

In our project. They matter because their characteristics - 25 fps, SDR, frontal, English - are exactly why the backend forces 25 fps SDR and rejects HDR.

Q42. What did the Auto-AVSR paper actually contribute?

Short. A data method, not an architecture.

Detailed. There is far more unlabelled audio-visual video than transcribed video, so Auto-AVSR runs an audio speech recogniser over unlabelled corpora to generate transcripts automatically, then trains the visual model on those.

In our project. That is why the checkpoint name lists four corpora: two labelled and two automatically labelled.

Q43. What biases are in your datasets?

Short. English only, professionally recorded, and not demographically uniform.

Detailed. LRS2 is British broadcast, LRS3 is TED speakers, VoxCeleb2 is celebrity interviews. Accent, demographics and speaking style are all skewed. Two of the four corpora carry automatic labels, so systematic ASR errors become training signal.

In our project. Performance on under-represented accents and appearances is unknown and likely worse. I state that rather than assuming it generalises.

Q44. Why do you not have a test set?

Short. Because building one requires ground truth I did not create.

Detailed. A lip-reading test set needs verified transcripts; an action test set needs a hand-annotated timeline; a Foley test set needs reference audio. I did not produce any of those, so I report signal measurements and published numbers instead, and label them as such.

In our project. This is why no word error rate is reported, and I would rather say that than invent one.

Chapter 73 — Level 7: backend questions

Q45. Why FastAPI?

Short. Because the work is long-running and needs a job model, and FastAPI gives typed requests, background execution and free interactive documentation.

Detailed. Foley generation takes about 4.7 minutes; an HTTP request cannot stay open that long, so the work runs in a background thread and the client polls.

In our project. `/docs` is generated automatically from the type annotations, and Subsystem 1 deliberately uses Flask instead because its work completes in seconds.

Q46. How does the background job work?

Short. A daemon thread runs the pipeline; a thread-safe store records every transition and persists it to disk.

Detailed. `STORE.run` refuses to start a second thread for a running job, wraps the pipeline in a try/except, prints the traceback to the log and stores a readable message on failure, and only marks the job completed if it is still running.

In our project. 122 job records on disk, each a complete audit trail.

Q47. Why polling and not WebSockets?

Short. Simplicity, with no meaningful cost.

Detailed. A WebSocket adds connection state, reconnection logic and a second protocol. For a four-minute job, 1.5 seconds of latency is irrelevant.

In our project. `useJob` polls `/api/status/{id}` every 1500 ms and stops on completion, failure or unmount.

Q48. How is progress calculated? Is it real?

Short. Real. Each stage sets a percentage when it starts and finishes, and action recognition reports per-window progress across the process boundary.

Detailed. The runner writes `m2_progress.json` after every window; a thread in the parent polls it once a second and maps it into the 10 to 45 percent band.

In our project. The comment at the top of `useJob.ts` says: *"Polls real backend job state. Progress is never synthesised on the client."*

Q49. How do you stop stack traces reaching the user?

Short. Full detail to the log, a readable sentence to the client.

Detailed. The worker catches every exception, prints the traceback server-side, and stores `str(exc)`. Domain errors are converted to specific messages. FastAPI's `HTTPException` carries only the message.

In our project. Two tests assert that responses contain no "Traceback".

Q50. What happens if the machine runs out of memory?

Short. The runner aborts cleanly at a 1.5 GB threshold and the user gets a specific message.

Detailed. `psutil.virtual_memory().available` is checked before every window and sampled every 50 ms by a background tracker in the MOSS wrapper.

In our project. Three recorded job failures carry exactly the message *"Action recognition stopped because the machine ran low on memory."* The guard is not theoretical.

Q51. Why subprocesses instead of importing the models?

Short. Memory, dependencies and failure isolation.

Detailed. Process exit returns memory to the operating system unconditionally, which is more reliable than garbage collection plus `torch.mps.empty_cache()`. The environments are mutually incompatible. And a model crash returns an exit code rather than taking the server down.

In our project. Qwen peaks near 12 GB and MOSS near 12.1 GB on a 17.18 GB machine.

Chapter 74 — Level 8: frontend questions

Q52. Why React for one interface and plain JavaScript for the others?

Short. Because only one of them has real client state.

Detailed. Subsystem 2 has a phase machine, a polled job, a timeline that appears mid-run, and a results view. React models that directly. Subsystems 1 and 3 have a form, a progress list and a result - about 335 lines of JavaScript - and adding a framework would be worse.

In our project. `App.tsx` is 179 lines and the whole frontend is 933 lines.

Q53. What is state, and what is a hook?

Short. State is data a component owns and can change; a hook is a function starting with `use` that lets a component use a React feature.

Detailed. `useState` declares state; `useEffect` runs side effects after render; `useCallback` memoises a function; `useRef` holds a mutable value that does not trigger a re-render.

In our project. `useJob` is a custom hook composing `useState`, `useEffect`, `useCallback` and `useRef`.

Q54. Why is `seenTimeline` a ref and not state?

Short. Because changing it must not cause a re-render - it is bookkeeping, not display data.

Detailed. If it were state, every poll would re-render the whole tree for a value the user never sees.

In our project. `const seenTimeline = useRef(false).`

Q55. Why `XMLHttpRequest` for upload and `fetch` everywhere else?

Short. Because `fetch` has no upload-progress event.

Detailed. `xhr.upload.onprogress` gives byte-level progress. For a 200 MB video that is the difference between "working" and "frozen".

In our project. `api/client.ts` uses XHR only for upload and `fetch` for everything else.

Q56. What does TypeScript buy you here?

Short. The API contract is checked at compile time.

Detailed. With `strict: true`, any mismatch between what the backend returns and what a component consumes is a compile error. `status` is a union of string literals, so a typo like `'complete'` fails to build.

In our project. `src/types/index.ts` mirrors every response shape; `npm run build` runs `tsc -b` before Vite.

Q57. What is the most informative thing in your interface?

Short. The action timeline with the visual-event markers.

Detailed. Coloured blocks are what the recogniser said; white markers are where the sound is actually anchored. A viewer can see that the markers do not sit at the block edges, which is the whole argument of the project, shown rather than asserted.

In our project. `ActionTimeline.tsx`, with the caption `White markers are visual events`

- the exact frames where sound is anchored, detected independently of the action-label

boundaries."*

Chapter 75 — Level 9: API and integration questions

Q58. What is a REST API?

Short. An HTTP interface where each URL identifies a resource and the method says what to do with it.

Detailed. `GET` reads, `POST` creates or triggers, and the status code communicates the outcome. The payload is JSON.

In our project. `GET /api/status/{job_id}` reads a job; `POST /api/upload` creates one; `POST /api/process/{job_id}` triggers work.

Q59. Why does `/api/result` return 409 and not 404?

Short. Because the job exists; it just is not finished.

Detailed. 404 means "no such thing". 409 Conflict means "the request is well-formed but conflicts with the current state of the resource", which is exactly right.

In our project. `raise HTTPException(409, f"Job is '{j.status}', not completed.")`, and a test asserts it.

Q60. What is CORS and why is it in your code?

Short. A browser security rule that stops JavaScript from one origin calling another unless the second one allows it.

Detailed. In development the frontend is on port 5173 and the backend on 8000 - different origins - so the middleware is needed.

In our project. `allow_origins=["*"]` allows any site to call the API. Acceptable for a local demonstrator, unacceptable in production. It is also partly redundant, because the Vite dev server proxies `/api` so the browser sees one origin.

Q61. How does the frontend know about the nine stages?

Short. It asks the backend.

Detailed. `/api/health` returns `stages: [{key, label}, ...]`, and the `Pipeline` component renders whatever it is given.

In our project. Adding a stage means editing one list in `core/jobs.py`; the interface follows automatically.

Q62. How do you prevent someone uploading a huge file and filling your disk?

Short. The size limit is enforced during the stream, not after it.

Detailed. The upload is read in 1 MB chunks; a running total is compared to the limit each chunk; on exceeding it the partial file is closed, deleted, and 413 is returned.

In our project. `MAX_UPLOAD_MB = 200`.

Q63. Can two users use it at once?

Short. Not safely. It is explicitly a single-machine, single-job demonstrator.

Detailed. Jobs run in-process with no queue, no authentication and no multi-user isolation. Two concurrent jobs would compete for memory that only fits one model.

In our project. Stated as a limitation in the README, `HANDOFF.md` and the paper.

Chapter 76 — Level 10: audio and video processing questions

Q64. What is a sample rate, and why 48 kHz?

Short. Samples per second. 48 kHz is the professional standard and is what MOSS produces natively.

Detailed. The sampling theorem says you must sample at more than twice the highest frequency, and human hearing reaches about 20 kHz.

In our project. 48 kHz specifically mattered: wet mouth transients carry substantial energy above 8 kHz, which every 16 kHz candidate model cut off entirely.

Q65. What is dBFS? What is RMS? What is crest factor?

Short. dBFS is decibels relative to digital full scale. RMS is average energy. Crest factor is peak divided by RMS, in dB.

Detailed. A high crest factor means impulsive, transient-rich material; a low one means compressed or continuous material.

In our project. Final mix: peak -6.00 dBFS, RMS -36.87 dBFS, crest 30.87 dB. That high crest factor is evidence the transient structure survived mixing.

Q66. What is "active RMS" and why not ordinary RMS?

Short. RMS over only the frames that contain signal.

Detailed. A Foley clip is mostly silence with a short event, so whole-file RMS is dominated by the silence and two clips with identical events but different silence would be levelled differently.

In our project. Frames are gated at the higher of the 60th percentile and 40 dB below the loudest frame - the second clause added because the percentile alone degenerates to zero on a mostly-silent clip.

Q67. What is spectral flatness? What is harmonic ratio?

Short. Flatness is geometric mean over arithmetic mean of the spectrum - near 1 for noise, near 0 for a tone. Harmonic ratio is the fraction of energy that survives harmonic-percussive separation.

Detailed. Both measure tonality from different directions, which is why the gate uses them together.

In our project. This is the most important pair of metrics in the project. Foley is inharmonic. A "cup placement" that comes back at harmonic ratio 0.997 with flatness 0.00000 is a 346 Hz sine wave, and the pure-tone gate exists precisely to reject it.

Q68. What is the difference between an onset and an attack?

Short. An onset-strength peak is where the spectrum changes fastest; the attack is where the sound actually begins rising.

Detailed. They differ by -96 to +250 ms on this project's own material.

In our project. Using strength peaks caused a real, audible 96 ms misalignment. The fix is `attack_times()`: back-track from the envelope maximum to where it last rose through 20 percent of that maximum.

Q69. Why do you never time-stretch?

Short. Because stretching changes what the sound is.

Detailed. A time-stretched footstep no longer sounds like a footstep. The project's position is that a slightly late footstep that sounds right is better than a perfectly placed artefact - and that the residual should be reported rather than hidden.

In our project. A test asserts that output length equals source length. And the cost of the policy is measured: cadence mismatch accumulates to -67.6 ms with MOSS and -462.4 ms with Stable Audio.

Q70. What is a zero-crossing snap and why do you need it?

Short. Moving a cut point to where the waveform crosses zero, so the edit does not click.

Detailed. Cutting mid-waveform creates a step discontinuity, which is broadband and audible as a click.

In our project. Nearest crossing within plus or minus 3 ms, and the snap offset is **recorded and added back to the placement** so moving the cut does not move the sound. Measured corrections: -0.021 to +0.167 ms. All 8 clip boundaries verified clean.

Q71. Why a raised-cosine fade and not a linear one?

Short. Because it is continuous in slope.

Detailed. A linear ramp has a corner at each end, which is a discontinuity in the first derivative and is faintly audible. A raised cosine has none.

In our project. 12 ms in and out, `rcos_fade` in `audio_processing.py`.

Q72. What is `-c:v copy` and why does it matter?

Short. Stream-copying the picture: not decoding, not re-encoding.

Detailed. The output picture is bit-identical to the input, there is no generation loss, and it is far faster than re-encoding.

In our project. The quality gate asserts it, alongside a SHA-256 of the source video verified before and after every build.

Q73. Why is your output audio 9.984 s when the video is 10.000 s?

Short. AAC frame granularity.

Detailed. AAC encodes in frames of 1024 samples - 21.3 ms at 48 kHz - and cannot emit a partial frame. It does not shift any event within the track.

In our project. The automated duration check applies a 150 ms tolerance for exactly this reason.

Chapter 77 — Level 11: synchronisation questions

Q74. How do you find the exact timestamp for a sound?

Short. Frame-level motion analysis in a region band, with a detection rule chosen by the physics of the action.

Detailed. Frames are decoded to 320 x 180 greyscale at 24 fps; motion is the mean absolute inter-frame difference within a horizontal band; then footstep, hold, contact or continuous logic locates the instant.

In our project. Four bands: feet 0.62 to 1.00, head 0.00 to 0.50, table 0.40 to 0.85, full 0.00 to 1.00.

Q75. Why is a footstep the *minimum after the peak*?

Short. Because the peak is the leg swinging and the minimum is the foot planted.

Detailed. You hear the plant, not the swing. The code finds a prominent peak then walks forward to the following local minimum.

In our project. Four plants at 0.458, 1.083, 1.667 and 2.208 s, asserted exactly in the test suite.

Q76. Why is a sip a motion *minimum*?

Short. Because a sip is the mug held still at the lips.

Detailed. The raise and the lower are the movements; the sip is the stillness between them. Looking for peaks would find the wrong things.

In our project. Drinking is by a wide margin the least visually active interval: mean motion 0.277 overall and 0.145 in the feet band. The measurement is what justified the rule.

Q77. Why not just use the action label boundaries?

Short. Because a label is a span and a Foley event is an instant.

Detailed. Playing a contact sound at the start of an 8.5 to 10.0 s "place cup" interval puts it about 1.3 seconds before the mug touches the table.

In our project. And worse: Module 2 labels 0.0 to 1.5 s as `stand` when feet-band motion there is 1.708 against 1.711 in the labelled walk. Two of the four foot plants fall inside the wrong label.

Q78. How accurate is your synchronisation?

Short. Worst 20 ms on the reference clip, below half a frame at 24 fps.

Detailed. Measured on the **rendered audio**, by detecting envelope attacks in the final WAV and comparing against the visual events - not asserted from the plan.

In our project. Across all 31 recorded jobs and 45 events: median 4.7 ms, 66.7 percent inside half a frame, worst 462.4 ms. That worst case is the Stable Audio walking asset, whose cadence is 30.2 percent fast, and it is fully explained by the shift-only policy.

Q79. Why does the error grow across successive footsteps?

Short. Because clips are shifted, never stretched, so a cadence mismatch accumulates.

Detailed. The filmed gait has a mean inter-step interval of 0.583 s. The MOSS asset generates at 0.641 s, 9.9 percent slow. Stable Audio generates at 0.407 s, 30.2 percent fast, and its residual grows monotonically to -462.4 ms by the fourth plant.

In our project. That growth is predictable from the assets alone: three intervals at a 0.176 s deficit accumulate to 0.528 s.

Q80. What if no visual event can be found?

Short. The system says so and produces the video with a silent track.

Detailed. `contact` detection falls back to the motion minimum and labels the event `low confidence`; `continuous` falls back to the interval start. If no placement can be made at all, the job still completes.

In our project. *"Foley was generated, but no visual event could be located to synchronise it to. The original video can still be exported without generated audio."*

Chapter 78 — Level 12: architecture questions

Q81. Draw your architecture.

Use Figure 1.1 for the three subsystems, Figure 22.1 for the nine stages, and Figure 40.1 for the call graph. If you can only draw one, draw the nine stages.

Q82. Why is `pipeline.py` separate from the services?

Short. So that exactly one module knows the order of the stages.

Detailed. Each service does one thing and knows nothing about the others. The pipeline composes them and reports progress. That makes each service independently testable.

In our project. 252 lines of orchestration against 1,500 lines of services.

Q83. Why does `routes.py` contain no algorithm?

Short. So the HTTP layer can be changed without touching the logic, and the logic can be tested without HTTP.

Detailed. Every route validates input, calls a service, and maps exceptions to status codes. All are under 20 lines.

In our project. The test suite exercises the services directly *and* the routes through FastAPI's `TestClient`.

Q84. How would you add a new sound class?

Short. Add one entry to `ACTION_PROMPT_MAP`. No other file changes.

Detailed. The entry supplies the key, label, prompt, negative prompt, strategy, region, selection rule, level target and keywords. Everything downstream is parameterised by those fields.

In our project. The source docstring says exactly that: *"Adding a curated action means adding one entry here - no other file changes."*

Q85. How would you add a fourth subsystem?

Short. A new folder, a new environment, and a new runner invoked as a subprocess.

Detailed. The pattern is already established three times: an isolated environment, a runner script with a command-line interface, JSON in and JSON out, and structured error propagation on `stderr`.

Q86. What is the single most important design decision?

Short. Aligning audio to measured visual events instead of to action-label boundaries.

Detailed. Everything else follows from it: the region bands, the four detection strategies, true attack alignment, the shift-only policy, and the per-event residual reporting.

In our project. `HANDOFF.md` lists it first: *"This is the core contribution of the project."*

Chapter 79 — Level 13: difficult technical questions

Q87. Your quality gate has six thresholds. How did you choose them, and are they not arbitrary?

Short. Two were set by specific observed failures; the rest are physically motivated.

Detailed. Effective bits at 9.0 corresponds to a peak of about -42 dBFS, below which 16-bit quantisation noise is no longer negligible. Dynamic range at 6 dB is the point below which material is not impulsive. The +25 dB gain limit is a judgement about how much noise amplification is acceptable.

The two empirical ones are honest: the pure-tone gate exists because a 346 Hz sine wave with 32 dB of dynamic range passed the combined tonality test, and the dynamic-range measurement was restricted to signal-bearing frames because including digital silence returned 183 to 201 dB.

In our project. I would not claim the thresholds are optimal. I would claim they are *motivated*, *documented*, and *validated by an ablation over all 54 assets*, and that two assets at harmonic ratio 0.880 and 0.868 pass, which shows the boundary is real rather than rejecting everything tonal.

Q88. Your quality score did not catch a degradation you could hear. Does that not invalidate the score?

Short. It limits it, and I report that as a finding rather than hiding it.

Detailed. Cutting the denoised latent from 30 s to 10 s *raised* dynamic range from 43.7 to 61.1 dB and effective bits from 14.89 to 16.0 - both rewarded by the score - while halving the detectable transients from 16 to 8 and raising inter-step irregularity by a factor of 5.8.

In our project. The conclusion is that an aggregate score computed from marginal statistics can improve while temporal structure is destroyed. A structural measure - transient count and spacing regularity - catches what the scalar misses and is cheap to compute. That is one of the paper's stated contributions, not a defect I discovered late.

Q89. You say the gate rejected 29.6 percent. Is that not a very high failure rate?

Short. Yes, and that is the point.

Detailed. Eleven of the sixteen rejections would have needed more than +25 dB of automatic gain, with a median of +37.0 dB and a maximum of +42.1 dB. Applying +42 dB to a file whose peak is -62 dBFS produces amplified quantisation noise, not a quiet sound.

In our project. The alternative to a 29.6 percent rejection rate is not a better system; it is a system that ships hiss.

Q90. Why is your median synchronisation error 4.7 ms but your worst 462 ms?

Short. One asset, from the alternative backend, with a badly mismatched cadence.

Detailed. Under a shift-only policy, a tempo mismatch accumulates linearly across successive contacts. The MOSS asset is 9.9 percent slow and stays within 67.6 ms; the Stable Audio asset is 30.2 percent fast and reaches 462.4 ms by the fourth plant.

In our project. It is a fully explained outlier, and it is also an independent, timing-based argument for the model choice.

Q91. Only footsteps record a per-event residual. Is your evidence not thinner than it looks?

Short. Yes, and I say so.

Detailed. Hold and contact placements record the aligned instant but not a residual, so across 32 job runs only footstep events contribute to the aggregate. The 20 ms figure comes from a separate measurement made on the rendered audio, which does cover all seven events.

In our project. The paper states this limitation explicitly rather than letting "32 job runs" imply more than it does.

Q92. How do you know the model is not simply memorising your test video?

Short. It cannot be, because no model in this project was trained on anything.

Detailed. Every checkpoint is published and pre-trained; my footage was recorded afterwards and has never been in any training set.

In our project. What *is* fair to criticise is that the alignment parameters - band fractions, prominence thresholds, the walking search span - were chosen while looking at this clip. That is why the 20 ms figure is quoted as a demonstration rather than a generalisation.

Q93. Your system leaves intervals silent. Is that not just a failure you renamed?

Short. It is a failure, correctly reported, which is different from a failure hidden.

Detailed. Three reasons silence is the right response: honesty, because the system's claim is that it sounds what it sees; diagnosis, because a reported silence tells you where and why it failed; and perception, because a wrong sound is worse than no sound.

In our project. The interval is marked `no_usable_foley`, the measured values and the reason are recorded, the file is kept on disk for diagnostics, and the user interface shows peak, dynamic range, effective bits, harmonic ratio and required gain.

Q94. If MOSS takes 4.7 minutes per sound, is this system usable?

Short. As a batch tool, yes; as an interactive one, only because of the cache.

Detailed. A fully cached job completes in 39.5 seconds against 556.9 seconds when generation is required. 94 percent of generation time is diffusion, and because the model denoises a fixed-length latent, a short request costs the same as a long one.

In our project. I tried the obvious speed-up and reverted it, for the reasons in Q88.

Q95. What is the weakest part of your system?

Short. Action recognition.

Detailed. Foley can only be as good as the timeline it is given. On one test video the recogniser missed a cup placement entirely, emitted one stirring action under three different labels, and returned a caption rather than an action.

In our project. It is also where the remaining quality is, which is why it is the first item in future work.

Q96. Your paper says the visual microphone never recovered real sound, but you have a recovered file. Which is true?

Short. Both, in order. The paper's limitation section was written before that run.

Detailed. The paper's claim was accurate when written: no real acoustic signal had been recovered. Afterwards I ran the pipeline on a 20,000 frames-per-second clip through `/process-local`, with the capture-rate override set to 20,000 and the mains notch enabled, and recovered a 5-second 20 kHz signal whose dominant energy sits at 305.6, 467.8 and 226.2 Hz - musical-band content rather than drift.

In our project. Three caveats I would give unprompted: the clip is from the original authors' published dataset rather than my own capture, the source file is not retained in the repository, and the paper has not been updated. The repository is the newer evidence and the paper is stale on that one point.

Chapter 80 — Level 14: tricky and hostile questions

Q97. Is this not just calling other people's models?

Short. The models are other people's; the system is not.

Detailed. The engineering is in what the models cannot do: knowing *when* to play a sound, knowing *whether* the sound is usable, running three incompatible stacks on one machine, and reporting honestly when it fails.

In our project. Concretely: visual event localisation with four physics-based strategies, true-attack alignment that removed a 96 ms error, a six-gate validator that rejected 29.6 percent of everything generated, phased inference that cut swap growth from +9.8 GB to +0.01 GB, and a nine-stage pipeline with 64 automated tests.

Q98. Could you not have got the same result with sound effects from a library?

Short. For the sound, sometimes. For the placement, no - and the placement is the hard part.

Detailed. A library gives you a footstep; it does not tell you which frame the foot plants on. Every technique in Chapters 29 and 30 would still be required. The project's own model evaluation even raises this option explicitly, and notes that the rest of Module 3 does not depend on the sound being generated rather than sourced.

In our project. Generation also gives open vocabulary: `impact_football`, `friction_notebook` and `ambient_toast_bread` were all produced for actions nobody wrote a class for.

Q99. Your synchronisation result is one clip. Why should I believe it generalises?

Short. You should not, and I do not claim it does.

Detailed. Seven events on one clip, on the build tuned against it. It demonstrates that sub-frame placement is achievable with this method; it does not demonstrate that it happens in general.

In our project. The aggregate across all recorded runs is median 4.7 ms with 66.7 percent inside half a frame, and I would quote that alongside the 20 ms figure rather than instead of it.

Q100. You did not train anything. What did you actually learn?

Short. How generative models fail, and why measurement matters more than metrics.

Detailed. I learned that a model trained at 10 to 47 seconds returns silence and clicks when asked for 3; that a level-setting algorithm will faithfully amplify an empty file into hiss; that an aggregate quality score can improve while the audio gets worse; that an onset-strength peak is not an attack; and that a safety limit which binds on the common case has silently become the primary control.

In our project. Every one of those is a documented incident with measurements attached.

Q101. Why is your output so quiet and empty?

Short. Because the scene is. Roughly 18 percent of a 10-second timeline carries audio.

Detailed. These are discrete contact events, not a continuous soundtrack. A mug picked up and set down produces two short sounds several seconds apart.

In our project. The silence between events is correct, and a system that filled it would be adding sound no physical event produced.

Q102. What if the video contains an action your system has never seen?

Short. It still gets sounded, by open-vocabulary prompt synthesis.

Detailed. The verb is classified into one of nine archetypes, which determines the strategy, the frame region, the selection rule and the level; a prompt is composed from the archetype's acoustic description and the literal phrase.

In our project. An unrecognised verb defaults to a **continuous ambient texture rather than a sharp transient**, because a misplaced texture is far less jarring than a misplaced impact - and we are guessing by definition.

Q103. What if the action recogniser is simply wrong?

Short. Then the sound is wrong, confidently and precisely placed.

Detailed. There is no downstream check on semantic correctness. The visual localiser finds where the motion is, not whether the label describes it.

In our project. The same is true in Subsystem 1: *"if the model reads the wrong words, they will be lip-synced accurately and wrong."* Both are stated as limitations.

Q104. Your lip reading has no accuracy figure. Is the whole subsystem not unverified?

Short. Recognition accuracy is unverified. Several other things are verified.

Detailed. Verified: that recognition uses no audio, proved by a bit-identical rerun with the audio stream stripped; that input format changes confidence by roughly an order of magnitude; that word timings come from the model's own CTC head at 40 ms resolution; and that two transcriptions are correct.

In our project. Unverified is the word error rate, and the honest reason is that I did not retain ground-truth transcripts.

Q105. What would you do differently if you started again?

Short. Record ground truth from day one, and invest in the action recogniser earlier.

Detailed. A hand-annotated action timeline and verified transcripts for a dozen clips would have cost a day and would have turned qualitative claims into numbers. And I spent most of the effort on the sound half when the recognition half is the binding constraint.

In our project. I would also run a multi-assessor listening test, because the ablation shows that objective scores miss degradations a listener catches.

Chapter 81 — Level 15: "why did you use this?"

Q106. Why Python?

Because every model in this space ships with a Python API, and because FFmpeg, NumPy, SciPy and librosa are the standard tools for exactly this work. There is no serious alternative for the model side.

Q107. Why PyTorch and not TensorFlow?

Because all four checkpoints are distributed as PyTorch weights, and because PyTorch's MPS backend is what makes Apple Silicon usable. Converting would add risk with no benefit.

Q108. Why MPS and not CPU everywhere?

Because Qwen2.5-VL and MOSS are far too slow on CPU. Subsystem 1 *is* on CPU, because its MPS path crashes in `ctc_prefix_score.py` and it is fast enough anyway - 0.45x real time.

Q109. Why not CUDA?

Because there is no NVIDIA hardware here. The whole project is built to a 17.18 GB Apple Silicon constraint, and that constraint is why several published models that work fine elsewhere failed on this material.

Q110. Why 2-second windows and a 1-second stride?

Two seconds is long enough to contain a complete short action and short enough that it usually contains only one. A half-window stride means every instant is seen twice, so an action straddling a boundary is still described. The cost is overlapping spans, which is why midpoint boundary resolution exists.

Q111. Why 8 frames per window?

It is enough for the model to see motion within the window, and few enough to fit the memory budget. It is the value from the validated implementation.

Q112. Why seed 42?

It is the conventional default. What matters is that it is *fixed and recorded*, so results are reproducible, and that it is part of the cache key. Seeds 43 and 44 are used as retries.

Q113. Why 50 denoising steps?

Because 25 sounded worse. The measured quality score moved by 0.6 points, which is noise, so the decision rests on listening - and the source comment says so explicitly: *"the quality gate did NOT catch it - listening did."*

Q114. Why CFG 4.0?

It is the validated setting from the model's own reference configuration. Higher values increase prompt adherence but risk exaggerated output; there is also a recorded observation that a much longer negative prompt at CFG 4.0 collapsed a generation, which is a reason not to push guidance harder without evidence.

Q115. Why a 30-second denoised latent when you only use 10 seconds?

Because shortening it is measurably worse. It is 2.8 times faster and *raises* the quality score, but halves the transient count, multiplies inter-step irregularity by 5.8, and makes the cup-pickup class fail on all three seeds instead of passing on one.

Q116. Why -6 dBFS as the peak cap and not -12?

Because at -12 dBFS the cap bound on most clips, so the cap rather than the per-class RMS targets was setting relative level, flattening the dynamics between events. A guard that fires on the common case has stopped being a guard.

Q117. Why +25 dB as the gain limit?

It is a judgement about how much noise amplification is acceptable. It is enforced twice - once in the quality gate and once independently in the mixer - and a clip needing more is **refused rather than clamped**, because clamping still admits amplified noise.

Q118. Why 24 fps for the motion analysis?

It matches the reference clip's frame rate, so every frame is analysed exactly once with no resampling. 320 x 180 is small enough to hold the whole clip in memory and large enough for band motion to be meaningful.

Q119. Why those specific band fractions?

Empirically, `visual_events.py` records that across three prominence thresholds the 0.62 to 1.00 feet band recovered all four foot plants, whereas a taller 0.55 to 1.00 band recovered only two at the default threshold.

Q120. Why 48 kHz mono and not stereo?

MOSS produces 48 kHz mono natively. Foley for a single subject in a single scene does not need a stereo image, and mono halves the data with no perceptual loss here.

Q121. Why AAC at 192 kbit/s?

Because browsers play AAC in MP4 natively, and 192 kbit/s is transparent for mono material. The master is a 48 kHz mono PCM 16-bit WAV, which is also downloadable, so nothing is lost.

Q122. Why Kokoro and not just Piper?

Because Kokoro was judged markedly more natural on this material. Piper is kept as an automatic in-process fallback so the pipeline still works if Kokoro is unavailable, and macOS `say` is a second fallback.

Q123. Why is the MOSS repository not modified?

So that results can be attributed to the published model. Every compatibility fix lives in a wrapper, and the build asserts that `git status --porcelain` inside `moos/MOSS-TTS` returns empty. It also means an upstream update does not conflict with local changes.

Q124. Why write a job record to disk after every transition?

Because it makes the system inspectable after the fact. There are 122 job records and 32 full reports on disk, and every claim in this handbook about what the pipeline actually did was read out of them. A system that only reports to a browser leaves no evidence.

PART 12

PRESENTATION SCRIPT

Slide by slide, with the words to say. Written to be spoken, not read. Roughly 12 to 15 minutes at a normal pace.

Chapter 82 — The slide-by-slide script

KEY POINT: How to use this

Do not memorise it word for word - that always sounds memorised. Read it aloud four or five times until the *structure* is automatic, then let the wording vary. The numbers are the part to get exactly right.

Slide 1 - Title

On the slide: the project title, your name, your department, your supervisor.

Say:

> "Good morning. My project is called *Reconstructing Audio from Silent Video*. It takes a > video that has no sound and generates the sound for it, using only the pixels. I built it > as three independent subsystems, and everything runs locally on a laptop - nothing goes to > a cloud service."

Slide 2 - The problem

On the slide: three photographs - a surveillance camera, archive film, a phone recording in wind. One line: "*The picture still contains the sound.*"

Say:

> "A great deal of recorded video has no usable audio. Surveillance cameras often record > picture only. Archive footage may have lost its sound track. Phone videos are routinely > ruined by wind or a muted microphone. > > But the visual record still constrains what the audio was. Lips move in a way that limits > what could have been said. A mug meeting a table implies a specific contact sound at a > specific instant. And surfaces in the frame physically vibrate in response to the sound > around them. > > My project asks whether that information can be turned back into audio."

Slide 3 - Existing approaches and why they failed here

On the slide: the two tables from Chapter 1 - closed-vocabulary recognisers, and text-to-audio models with their failure modes.

Say:

> "I did not start by building. I started by measuring what already exists, on my hardware. > > Closed-vocabulary action recognisers failed structurally. VideoMAE returned 'shredding > paper' with confidence 0.55 for a clip of someone handling a cup, with no relevant label > in the top ten. The problem is not that the model is bad - it is that Kinetics labels > describe whole-clip activities, and Foley needs object-contact events. There is no label > for 'a mug is set on a table'. > > Then five sound-generation models. Three of them failed with an almost identical > signature: over 91 percent digital silence with a couple of 30-millisecond clicks. That > consistency was a clue, and it turned out that every one of those runs had asked for two > to four seconds of audio from models trained to generate ten to forty-seven. So the > conclusion was procedural rather than about model identity: generate at the model's native > duration and crop afterwards. That is what my final system does."

Slide 4 - Objectives

On the slide: five bullets.

Say:

> "So my objectives were: reconstruct plausible and *temporally correct* audio on consumer > hardware; prove that recognition happens from pixels alone; make generated sound land on > the correct frame rather than merely inside the correct label; refuse to output audio that > measurement says is unusable; and report every limitation honestly."

Slide 5 - System overview

On the slide: Figure 1.1 - the three subsystems.

Say:

> "Three independent paths. The first reads lips and generates timed speech. The second > recognises physical actions and generates synchronised Foley. The third is the visual > microphone - recovering sound from the vibration of objects. > > They share two contracts. Only the video stream is ever decoded, so any audio already in > the file is never read. And the picture is stream-copied to the output, never re-encoded, > so the output video is bit-identical in its picture stream."

Slide 6 - Subsystem 1: lip reading

On the slide: the preprocessing chain, and the model diagram.

Say:

> "Subsystem 1 uses Auto-AVSR, a 250-million-parameter visual speech recognition model. The > video is standardised to 25 frames per second SDR, MediaPipe finds the face, every frame > is warped onto a mean face and cropped to an 88-by-88 greyscale mouth region, and a 3-D > convolutional frontend feeds a Conformer encoder and a Transformer decoder. > > The 3-D convolution is the important part: it sees space *and time* together, so it > responds to the mouth moving rather than to a still shape."

Slide 7 - Proving there is no audio path

On the slide: the four-row result table from Section 12.7.

Say:

> "A claim that a system reads lips has to be demonstrated, not asserted. So I stripped the > audio stream out of a test recording and re-ran it. The decoded frames were bit-identical > - maximum pixel difference zero across 2.21 billion values - the model input tensor was > bitwise identical, and the output matched exactly: same sentence, same token identifiers, > same decoder score. > > It is also structurally true. The frontend is a `Conv3d` stack that accepts only pixel > tensors. It cannot ingest a waveform."

Slide 8 - Word timing from the CTC head

On the slide: Figure 8.1 - CTC peakiness and forced alignment.

Say:

> "The model returns a sentence with no timestamps, but it has a CTC head that emits > per-frame token probabilities at 25 frames a second. Forced-aligning the decoded tokens > against those probabilities gives a word onset every 40 milliseconds, derived from the > video itself. > > One correction was needed. A CTC peak marks where the model

became *confident*, which is > later than where the mouth began moving. So I measure the mouth motion directly and shift > every anchor back by the difference - about 0.24 seconds on my test clip."

Slide 9 - Subsystem 2: the nine stages

On the slide: Figure 22.1.

Say:

> "Subsystem 2 is the main one. Nine stages, and every one reports real progress from the > backend - there is no fake progress bar. > > A vision-language model, Qwen2.5-VL, watches the video in two-second windows and describes > the physical action in each. Crucially, it is never asked for timestamps - it answers only > 'what is happening here?', and the timing comes from each window's known position on the > timeline. Semantics from the model, timing from the windowing."

Slide 10 - The central problem: where to put the sound

On the slide: a timeline with a "walking 1.5 to 2.5 s" block, and four separate markers under it.

Say:

> "Here is the problem that turned out to be the interesting one. > > The recogniser tells me 'walking, 1.5 to 2.5 seconds'. But you do not hear walking for a > second - you hear four separate footsteps, each at one instant. Stretch a walking sound > across the label and you get audio that is present and obviously wrong. > > It gets worse. On this clip the recogniser labels the first 1.5 seconds as *standing*. But > when I measure lower-body motion, it is 1.708 during 'standing' against 1.711 during the > labelled walk - statistically indistinguishable. The subject is walking from about 0.2 > seconds, and two of the four foot plants fall inside the wrong label."

Slide 11 - Visual event localisation

On the slide: Figure 29.1 - the four detection strategies.

Say:

> "So I run an independent frame-level analysis whose only job is to find those instants. > Frames are decoded to 320 by 180 greyscale, and motion is the mean absolute difference > between consecutive frames inside a horizontal band - the lower third for footsteps, the > upper half for drinking, the middle for the table. > > Then the detection rule depends on the *physics* of the action. A footstep is a motion peak > resolved to the *following* minimum, because the peak is the leg swinging and the plant is > what you hear. A sip is a sustained motion *minimum*, because a sip is the mug held still > at the lips - and I can justify that with a measurement: drinking is by a wide margin the > least visually active interval in the clip. A contact is the last motion peak before rest."

Slide 12 - Alignment on true attack times

On the slide: Figure 11.1 - the envelope with the attack and the strength peak marked.

Say:

> "Once I know the instant, I align the generated clip so that its true envelope attack lands > on it. > > That word 'true' is doing real work. My first implementation aligned onset-*strength* > peaks, which mark where the spectrum changes fastest, not where the sound begins. On my own > assets they lead or lag the real attack by minus 96 to plus 250 milliseconds. One footstep > reported a strength peak at 3.760 seconds whose true attack is at 3.856 - so aligning the > strength peak misplaced the audible sound by exactly 96 milliseconds, and it was audible > before I found it. > > Clips

are shifted, never time-stretched, because stretching changes what a footstep sounds > like. Where the cadence does not match, the residual is absorbed and reported rather than > corrected."

Slide 13 - Not trusting the generated audio

On the slide: the six gates, and the 29.6 percent figure.

Say:

> "The second hard problem is that generative audio models fail in ways that ordinary > processing makes worse. > > Sometimes MOSS returns a near-silent, near-constant file with no usable signal. A > level-setting algorithm will faithfully try to raise that to target, apply forty-something > decibels of gain, and turn quantisation noise into audible hiss. > > So every generated file is measured raw, before any gain, against six gates. Across the 54 > assets I generated during development, the gate rejects 16 - just under 30 percent. Eleven > of those would have needed more than 25 decibels of make-up gain, with a maximum of 42. > Applying 42 decibels to a file whose peak is minus 62 dBFS does not give you a quiet > contact sound; it gives you amplified noise."

Slide 14 - Why one number is not enough

On the slide: the MOSS versus Stable Audio table.

Say:

> "Here is the result I am most pleased with. > > I ran a second sound model, Stable Audio Open, through the identical pipeline. Their median > quality scores are almost identical - 54.5 and 53.2. But their median harmonic ratios differ > by a factor of 22: 0.04 against 0.898. > > Foley is inharmonic. A mug meeting a table is a broadband transient. Stable Audio produced > *musical tones* for object contacts - one output for 'cup placed on a table' was a pure 346 > hertz sine wave. > > The aggregate score does not separate those two models. The harmonic ratio separates them > decisively. That is the case for multi-criteria gating: a single scalar, however carefully > weighted, can be blind to the failure that matters."

Slide 15 - When it fails, it says so

On the slide: a screenshot of the results panel showing a rejected class with its measurements.

Say:

> "The hardest sound class was picking up a ceramic mug. Two attempts failed, and I measured > why. The second was mathematically near-empty: 40 distinct sample values in ten seconds, > 1.06 decibels of dynamic range, 95 percent harmonic content. > > So the system leaves that interval silent and reports it, with the measured numbers, rather > than substituting something. An automated check asserts that no audio is written there. > > But then, when I added multi-candidate generation, the same class failed on seeds 42 and 43 > and passed on seed 44 with a quality score of 85.8. Same prompt, same model, same settings. > It was a sampling failure, not a capability limit - and that is exactly why the retry loop > exists."

Slide 16 - Results

On the slide: the synchronisation table, and the test counts.

Say:

> "The headline result is synchronisation. On my reference clip the worst alignment error is > 20 milliseconds across seven events. One frame at 24 frames a second is 41.7 milliseconds, > so every event is inside half a frame. And that is

measured on the *rendered audio* - I > detect the envelope attacks in the final WAV and compare them to the visual events, rather > than asserting it from the plan. > > I want to be careful about scope, though. That is seven events on one clip, on the build I > tuned against it. Across all 31 recorded runs the median is 4.7 milliseconds with two > thirds of events inside half a frame, and one outlier at 462 milliseconds - which is the > alternative backend's walking asset, whose cadence is 30 percent too fast, and under a > shift-only policy that mismatch accumulates. > > There are 64 automated tests and they all pass."

Slide 17 - Subsystem 3: the visual microphone

On the slide: the Nyquist table and the sub-pixel detection floor.

Say:

> "The third subsystem implements the Visual Microphone of Davis and colleagues from 2014. > Sound makes objects vibrate by a fraction of a pixel, and phase-based analysis can measure > that. > > The honest headline is that it produces one audio sample per video frame, so the highest > recoverable frequency is half the frame rate. At 60 frames a second that is 30 hertz, which > is below speech entirely. That is the sampling theorem, not a bug. > > So I characterised it on synthetic stimuli with known ground truth. It recovers a known > oscillation exactly down to a displacement of 0.02 pixels, and above Nyquist it aliases > exactly as theory predicts - 70 hertz folds to 50, 100 folds to 20, at 120 frames a second. > > I also ran it on a 20,000 frames-per-second clip from the original authors' dataset, and > recovered a five-second signal with clear musical-band content at 305 hertz. Two things > were essential there: overriding the capture frame rate, because high-speed cameras write a > *playback* rate into the file header, and notching out the mains-lighting flicker, which > the algorithm faithfully recovers because it is a real oscillation."

Slide 18 - Limitations

On the slide: five honest bullets.

Say:

> "The limiting component is action recognition. On one test video it missed a cup placement > entirely and emitted one stirring action under three different labels. Everything > downstream is bounded by that. > > I have no word error rate for lip reading, because I did not retain ground-truth > transcripts. My listening judgements were made by me alone. The synchronisation figure is > one clip. And the visual microphone has been demonstrated, not deployed - it needs a > high-speed camera. > > The system is Apple Silicon only, single machine, single job. It is a demonstrator, and I > would rather say that than oversell it."

Slide 19 - Future work

On the slide: the priority list.

Say:

> "In priority order: improve the action timeline, because it bounds everything else - prompt > engineering to elicit actions rather than captions, and label normalisation to merge > synonymous predictions. Then a proper multi-assessor listening test, because my own > ablation shows objective scores missing degradations a listener catches. Then selecting > generated candidates by cadence fit as well as quality score, which would fix the residual > accumulation without time-stretching. And a benchmark comparison against Diff-Foley, > MMAudio and FoleyCrafter to place the method in context."

Slide 20 - Conclusion

On the slide: three sentences.

Say:

> "The central finding is that in generated-Foley systems, *placement* and *validation* matter > as much as generation. > > Anchoring audio to visually measured contact instants, using true envelope attack times, > achieved a worst case of 20 milliseconds - inside half a frame. Measuring every generated > asset before mixing rejected nearly 30 percent of everything I produced, eleven items of > which would otherwise have received more than 25 decibels of automatic gain. > > And two results argue against trusting a single quality number: two backends with nearly > identical median scores differ in median harmonic ratio by a factor of 22, and a setting > change that raised measured dynamic range from 43.7 to 61.1 decibels simultaneously halved > the number of audible transients. > > The system has real limits and I have stated them. Within those limits, it shows that a > laptop-scale pipeline can place generated sound accurately enough to survive frame-level > scrutiny - provided the audio it generates is measured before it is believed. > > Thank you. I am happy to take questions."

82.1 The live demonstration

If you get to demonstrate, do it in this order:

1. **Open the interface** and point at the health indicator. "That is a real check on FFmpeg and both model environments."
1. **Upload the reference clip.** Point at the warning: "This video already contains an audio track. It will be ignored." Say: "That is not a failure - it is the system telling you it will not read the audio."
1. **Press Generate** and let the stage list run. Point at the percentage: "That number comes from the backend. During action recognition it is driven by a file the model process writes after every window."
1. **When the timeline appears mid-run**, stop and explain it. "This has appeared while sound generation is still going. The coloured blocks are what the recogniser said. The white markers are where the sound will actually be anchored - and notice they are not at the block edges. That is the whole idea of the project."
1. **When it finishes**, play it, then open "Show quality measurements" if anything was rejected.
1. **Click "View Processing Report"** and scroll to *validations*. "Every candidate, every seed, every measurement, every reason."

REMEMBER THIS

If the demonstration fails, do not panic and do not apologise repeatedly. Say: *"That is one of the failure paths I built for - let me show you what it reported."* Then open the job record. A system that fails informatively is a better demonstration than one that happens to work.

PART 13

LAST NIGHT BEFORE THE VIVA

Everything you must be able to say, in the smallest space it can be written.

Chapter 83 — The project in ten lines

1. Three subsystems reconstruct audio from silent video, all locally on an Apple M4 with 17.18 GB and no CUDA.
 1. Subsystem 1: Auto-AVSR, 250.4 M parameters, reads lips; CTC forced alignment gives word onsets every 40 ms; Kokoro speaks them on time.
 1. Subsystem 2: Qwen2.5-VL labels actions in 2-second windows; MOSS-SoundEffect v2.0 generates Foley from text; the sound is aligned to measured visual events.
 1. Subsystem 3: the phase-based Visual Microphone recovers sound from sub-pixel vibration.
2. The core contribution is **placement**: a label is a span, a Foley event is an instant, so frame-level motion analysis finds the exact contact frames.
 1. Alignment uses **true envelope attack times**, not onset-strength peaks, which removed a 96 ms audible error.
 1. Clips are **shifted, never time-stretched**; the residual is reported.
 2. Every generated asset is measured raw against **six quality gates**; 29.6 percent of 54 assets were rejected; a class with no usable sound is **left silent and reported**.
1. Worst synchronisation error on the reference clip: **20 ms**, below half a frame at 24 fps, measured on the rendered audio.
 1. Models run as **subprocesses in five isolated environments**, because their dependencies and their 12 GB memory peaks cannot coexist.

Chapter 84 — The numbers you must know

Quantity	Value
Worst synchronisation error, reference clip	20 ms
One frame at 24 fps	41.7 ms
Median synchronisation error, all recorded runs	4.7 ms
Worst across all runs (explained: cadence mismatch)	462.4 ms
Assets generated / rejected	54 / 16 = 29.6 percent
Rejected needing over +25 dB	11 (median +37.0, max +42.1)
Auto-AVSR parameters	250,383,410
Auto-AVSR published WER on LRS3	20.3 percent (their number)
MOSS total parameters	3,508.21 M (DiT 1416.05, text 1720.57, VAE 371.59)
MOSS DiT	30 layers, dim 1536, 12 heads, ffn 8960
Generation time per asset	280.9 s, 94 percent of it diffusion
Job time, cached / uncached	39.5 s / 556.9 s
Peak memory	12.11 GB mean, 12.51 GB worst, of 17.18 GB
Swap growth, with phasing / without	+0.01 GB / +9.80 GB
Automated tests	64 (42 + 22), all passing
Quality-gate checks on the validated build	19, all passing
Curated Foley classes in code	17 (documentation says 16)
Harmonic ratio, MOSS vs Stable Audio (median)	0.040 vs 0.898
Quality score, MOSS vs Stable Audio (median)	54.5 vs 53.2
Foot plants on the reference clip	0.458, 1.083, 1.667, 2.208 s
Sip holds	6.625, 7.792 s
Cup contact	9.833 s
Final mix	peak -6.00 dBFS, RMS -36.87 dBFS, crest 30.87 dB, 0 clipped
Visual microphone detection floor	exact to 0.020 px, fails at 0.015 px
Subsystem 1 real-time factor	0.45x, on CPU

Chapter 85 — Technologies, models and datasets, in three tables

Technologies

Layer	Technology
Frontend	React 18, Vite 6, TypeScript, Tailwind (Subsystem 2); vanilla HTML/CSS/JS (1 and 3)
Backend	FastAPI + Uvicorn (2 and 3); Flask (1)
ML runtime	PyTorch 2.9.1 / 2.5.1 / 2.13, MPS and CPU
Audio	NumPy, SciPy, librosa, soundfile
Vision	MediaPipe, OpenCV, pyrtools
Media	FFmpeg and FFprobe

Models

Model	Role	Size	Licence
Auto-AVSR vsr_trlrs2lrs3vox2avsp_base	lip reading	250.4 M	Apache 2.0 (weights bound by corpus licences)
Qwen2.5-VL-3B-Instruct	action recognition	about 3 B	published checkpoint
MOSS-SoundEffect v2.0	Foley generation	3.5 B across 3 components	Apache-2.0
Stable Audio Open 1.0	alternative Foley backend	-	non-commercial
Kokoro v1.0	text to speech	ONNX	-
Levi and Hassner gender CNN	voice selection	small	-

Datasets (training corpora of the checkpoints; this project trained on none)

Dataset	Contents	Role
LRS2-BBC	BBC television sentences with face tracks	training
LRS3-TED	TED and TEDx talks	training and the 20.3 percent WER benchmark
VoxCeleb2	YouTube interviews, no original transcripts	automatically labelled training
AVSpeech	single visible speaker, clean speech	automatically labelled training

Chapter 86 — Important files

File	Why it matters
backend/services/synchronization.py	Visual events and true-attack alignment - the core contribution
backend/services/foley_validation.py	The six gates and the 0-100 score
backend/services/prompt_map.py	17 curated Foley classes and the resolution tiers
backend/services/prompt_synthesis.py	Open-vocabulary prompts from nine verb archetypes
backend/services/pipeline.py	The nine-stage orchestration
backend/services/audio_processing.py	The mixing chain and the independent gain limit
backend/core/jobs.py	Job store, state machine, background worker
moss/scripts/moss_generate.py	Phased generation with residency assertions
moss/scripts/mps_compat.py	The float64-to-CPU shim, verified bit-exact
03-FoleyCrafter-Test/action-recognition/action_recognition.py	The Qwen windowing and merging
.../resolve_segments.py	Midpoint boundary resolution
02-Auto-AVSR-Test/app/timing.py	CTC forced alignment and the lag correction
02-Auto-AVSR-Test/app/inference.py	The frozen Auto-AVSR wrapper
Acoustic eye/.../visual_microphone.py	The phase-based core
results/qa_polished.json	The 19-check quality gate result
data/jobs/aa4b2f4e6049/report.json	The current end-to-end run

Chapter 87 — Important algorithms

Algorithm	One-line description
Sliding-window action recognition	2 s windows, 1 s stride, 8 frames, free-text output
Action-head merging	Consecutive windows sharing the -ing verb merge into one span
Midpoint boundary resolution	Overlapping spans meet at the midpoint of the overlap, deterministically
Tiered Foley resolution	specific -> generic -> silent -> verb fallback -> synthesis
Flow-matching diffusion with CFG	$nneg + cfg * (npos - nneg)$, 50 Euler steps, two forward passes each
Six-gate validation	effective bits, dynamic range, sustained tone, pure tone, gain, integrity
Multi-candidate selection	seeds 42, 43, 44; stop early at score 45; keep the best
Region-band motion	mean absolute inter-frame difference within a horizontal band
Footstep detection	prominence peak, then walk forward to the following minimum
Hold detection	sustained run below the 40th percentile, take the middle
True attack detection	envelope peak, back-track to the last crossing of 20 percent of the peak
Gait matching	best consecutive run of generated steps by sum of squared interval differences
Active-RMS levelling	RMS over frames above the higher of the 60th percentile and -40 dB
Content-addressed caching	SHA-256 over normalised settings, truncated to 16 hex characters
CTC forced alignment	most probable frame path producing the known token sequence
Phase-based vibration recovery	amplitude-weighted phase difference per pyramid band, aligned and summed

Chapter 88 — Things you must remember

REMEMBER THIS: The ten sentences

1. "A label is a span; a Foley event is an instant."
2. "Semantics from the vision-language model; timing from the windowing."
3. "Onset-strength peaks lead or lag the true attack by minus 96 to plus 250 milliseconds."
4. "Clips are shifted, never time-stretched; the residual is absorbed and reported."
5. "Measured on the rendered audio, not asserted from the plan."
6. "An action with no usable Foley is left silent and reported, never filled with a substitute."
1. "The gate rejected 29.6 percent of everything I generated."
2. "Two backends with nearly identical median scores differ in median harmonic ratio by a factor of 22."
1. "A setting change that raised measured dynamic range from 43.7 to 61.1 dB halved the number of audible transients."
1. "I did not train anything. The engineering is in the pipeline, the placement and the validation."

CAUTION: The five things not to say

1. Do **not** say your system has a word error rate. It does not.
2. Do **not** say 20.3 percent WER is your result. It is the checkpoint authors' result on their benchmark.
1. Do **not** say the visual microphone recovers speech from ordinary video. Nyquist forbids it.
1. Do **not** say your system "understands" what is happening. It recognises and generates.
2. Do **not** claim the 20 ms figure generalises. It is seven events on one clip on the build tuned against it.

Chapter 89 — The fifty questions most likely to be asked

If you are short of time, these are the ones. Full answers are in Part 11 at the numbers shown.

#	Question	See
1	What is your project?	Q1
2	What problem does it solve?	Q2
3	What is the input and output?	Q3
4	Why three subsystems?	Q4
5	What is Foley?	Q5
6	What is your contribution?	Q7
7	Walk me through the pipeline.	Q9
8	What happens when a user uploads a video?	Q10
9	What happens after pressing Generate?	Q11
10	Why <code>resolved_actions</code> and not <code>actions</code> ?	Q13
11	How do the modules communicate?	Q15
12	Did you train any model?	Q17
13	Training versus inference?	Q18
14	Is your system overfitted?	Q19
15	What is <code>bfloat16</code> and why?	Q21
16	What is a CNN?	Q23
17	What is a Transformer? What is attention?	Q25
18	What is a Conformer?	Q26
19	What is softmax? What are logits?	Q28
20	What is a VAE?	Q29
21	What is diffusion?	Q30
22	What is flow matching?	Q31
23	What is classifier-free guidance?	Q32
24	Which models do you use?	Q33
25	Why Qwen and not an action classifier?	Q34
26	Why MOSS and not Stable Audio?	Q35
27	How do you load the model?	Q39
28	What dataset did you use?	Q40
29	What are LRS2 and LRS3?	Q41
30	What biases are in your datasets?	Q43
31	Why FastAPI?	Q45
32	How does the background job work?	Q46
33	Is your progress bar real?	Q48
34	Why subprocesses and not imports?	Q51
35	Why React for one interface only?	Q52

#	Question	See
36	What does TypeScript buy you?	Q56
37	Why 409 and not 404?	Q59
38	What is CORS?	Q60
39	Why 48 kHz?	Q64
40	What is crest factor?	Q65
41	What is harmonic ratio, and why does it matter?	Q67
42	Onset versus attack?	Q68
43	Why never time-stretch?	Q69
44	What is <code>-c:v copy</code> ?	Q72
45	How do you find the exact timestamp?	Q74
46	Why is a footstep the minimum after the peak?	Q75
47	How accurate is your synchronisation?	Q78
48	What is the weakest part of your system?	Q95
49	Is this not just calling other people's models?	Q97
50	What would you do differently?	Q105

PART 14

GLOSSARY AND APPENDICES

Every technical term used in this project, defined simply, defined properly, and located in the code.

Chapter 90 — Glossary A to Z

Format for each entry: **term** - simple meaning; technical meaning; where it appears here.

A

AAC - a compressed audio format. Advanced Audio Coding, a lossy codec operating on frames of 1024 samples. The output MP4's audio track, at 192 kbit/s in Subsystem 2 and 160 kbit/s in Subsystem 1.

Action head - the main verb of an action phrase. The primary `-ing` verb, falling back to the first content word. `action_head()` in `action_recognition.py`; it is what merging compares.

Active RMS - the loudness of the parts that are not silence. RMS computed over frames above a gate set at the higher of the 60th percentile and 40 dB below the loudest frame. `active_rms()` in `audio_processing.py`; it is the level-setting metric.

Alias - a frequency above half the sampling rate masquerading as a lower one. Folding about the Nyquist frequency. Demonstrated exactly in Subsystem 3: at 120 fps, 70 Hz appears as 50 Hz and 100 Hz appears as 20 Hz.

Attack - where a sound starts rising. The leading edge of a transient, found by back-tracking from an envelope maximum to where it last rose through 20 percent of that maximum. `attack_times()` in `synchronization.py` - the alignment anchor.

Attention - a mechanism that lets every position in a sequence look at every other and decide what matters. Scaled dot-product of queries against keys, softmaxed into weights over values. In Auto-AVSR's Conformer and Transformer, in Qwen2.5-VL, and in MOSS's DiT.

Autoencoder - a compressor and decompressor trained together. An encoder mapping input to a compact latent and a decoder mapping back. MOSS's DAC VAE, `hop_length` 960.

B

Batch - several examples processed together. The leading tensor dimension. Always 1 here; this is single-item inference.

Beam search - keeping the best N partial answers alive instead of committing to one. Expanding every hypothesis by every token and keeping the top N by accumulated log-probability. Beam width 40 in Auto-AVSR.

bfloat16 - a 16-bit float that keeps float32's range but loses precision. Halves weight memory. Qwen2.5-VL, and MOSS's parameters after the wrapper's cast.

Blank - CTC's "nothing here" symbol. An extra vocabulary entry removed after collapsing repeats. `model.blank` in `timing.py`.

C

Cache key - a fingerprint of everything that affects the output. SHA-256 over normalised settings, truncated to 16 hex characters. `cache_key()` in `sound_generation.py`.

CFG (classifier-free guidance) - running the model with and without the prompt and pushing towards the difference. `nneg + cfg * (npos - nneg)`. CFG 4.0 for MOSS; it is why each step costs two forward passes.

Channel - one stream of audio, or one feature map in a CNN. Mono is one audio channel; the DiT has 128 latent channels.

Codec - the algorithm that compresses a stream. H.264 for video, AAC for audio.

Conformer - a Transformer block with convolution inside it. Attention for long-range structure plus convolution for local detail. Auto-AVSR's encoder.

Convolution - sliding a small filter over data. A weighted sum computed at every position, producing a feature map. `Conv3d` in Auto-AVSR's frontend, sliding over space and time.

CORS - the browser rule that stops one site calling another. Cross-Origin Resource Sharing headers. `CORSMiddleware` in `main.py`.

Crest factor - how spiky the audio is. Peak divided by RMS, in dB. 30.87 dB in the final mix - high, which is correct for impulsive Foley.

CTC - a way of training a sequence model without knowing which frame goes with which symbol. Connectionist Temporal Classification: a blank symbol, collapse repeats, sum over all paths. Auto-AVSR's second head, weight 0.1 in the beam search, and the source of word timings.

D

DAC - the audio codec whose decoder MOSS uses. Descript Audio Codec (Kumar et al., NeurIPS 2023). `hop_length` 960, so one latent step becomes 960 audio samples.

dBFS - decibels relative to digital full scale. 0 dBFS is the loudest a sample can be; values are always negative in well-behaved audio. Final mix peak -6.00 dBFS.

DC offset - the waveform sitting off centre. A non-zero mean. Removed per clip in `mix()`; offsets of order 1e-4 were observed.

Diffusion - generating by repeatedly removing noise. A fixed forward noising process and a learned reverse process. MOSS, 50 steps.

DiT - a Diffusion Transformer: a Transformer used as the denoiser instead of a U-Net. MOSS's denoiser - 30 layers, dim 1536, 12 heads, ffn 8960.

Dynamic range - the gap between loud and quiet. The 95th minus the 5th percentile of frame levels, measured over signal-bearing frames only and capped at 96 dB. Gate 2 rejects below 6 dB.

E

Effective bits - how much of the 16-bit range is actually used. $16 + \log_2(\text{peak})$. Gate 1 rejects below 9.0, which corresponds to a peak of about -42 dBFS.

Envelope - the outline of a waveform's loudness. The magnitude of the analytic signal from a Hilbert transform, smoothed. `envelope()` in `synchronization.py`.

Epoch - one pass through the training data. Not applicable; nothing is trained here.

F

FFmpeg - the tool that does everything with video and audio. Decoding, filtering, encoding and muxing. Frame extraction, standardisation, and the final mux in all three subsystems.

Flow matching - training a model to point straight from noise to data. Regressing the vector field that transports samples along a probability path. MOSS's training objective; `FlowMatchScheduler` with `sigma_shift` 5.

Foley - everyday sound effects made to match on-screen action. Named after Jack Foley. Generated here rather than performed.

Forced alignment - finding which frame each known symbol occurs on. The most probable frame path producing a given target sequence. `torchaudio.functional.forced_align` in `timing.py`.

Frame rate - pictures per second. 24 fps for the reference clip, 25 fps forced in Subsystem 1, and the *capture* rate is the audio sample rate in Subsystem 3.

G

Generalisation - working on data never seen in training. Every clip this project processes is a generalisation test, because no model here was trained on any of it.

Ground truth - the correct answer. **This project has none for its own recordings**, which is why no word error rate and no recognition accuracy are reported.

H

Harmonic ratio - how tonal a sound is, from 0 to 1. The fraction of energy surviving harmonic-percussive separation. **The single most discriminating metric in the project**: Foley must be inharmonic, and MOSS measures 0.00 to 0.02 on object contacts against Stable Audio's 0.87 to 0.88.

HDR - video with a wider brightness range and a different transfer curve. HLG or PQ transfer, BT.2020 primaries, 10-bit or more. **Rejected** by Subsystem 1, because untone-mapped HDR shifts the model input tensor mean from about 0.06 to about 1.27.

Hilbert transform - a way of getting the envelope of a signal. Produces the analytic signal, whose magnitude is the amplitude envelope. `scipy.signal.hilbert`.

Hop length - how far a window moves each step. In an STFT, samples between frames; in the DAC VAE, audio samples per latent step - **960**.

I

Inference - running a trained model forward. No gradients, no optimiser. **Everything in this project is inference**.

L

Latent - a compact internal representation. The compressed sequence a generative model actually produces. (1, 128, 1500) in MOSS, decoding to 1,440,000 samples.

Limiter - something that reduces only the peaks above a threshold. A soft-knee `tanh` above -6 dBFS with a -3 dBFS ceiling. **Never engaged** in any recorded mix; its gain reduction is always reported.

Logits - raw unnormalised scores. The output of a final linear layer before softmax.

LSTM - a recurrent network with gates so memory survives long sequences. Forget, input and output gates over an additively-updated cell state. **Does not appear in this project**.

M

Mean-face alignment - warping every frame so the face lands in the same place. Removes head translation, rotation and scale so the model sees only articulation. `VideoProcess` in the Auto-AVSR preprocessing.

MPS - Apple's GPU compute backend. Metal Performance Shaders. Used for Qwen2.5-VL and MOSS; **has no float64 support**, which forced the `sinusoidal_embedding_1d` shim.

Mux - putting a video stream and an audio stream into one file. Multiplexing. `ffmpeg -map 0:v:0 -map 1:a:0 -c:v copy -c:a aac`.

N

Negative prompt - a description of what you do not want. The second conditioning that CFG pushes away from. Shared across most classes; deliberately **empty** for drinking.

Nyquist frequency - half the sampling rate; the highest representable frequency. **The entire limitation of Subsystem 3:** at 60 fps it is 30 Hz, which excludes speech.

O

Onset strength - a measure of how fast the spectrum is changing. Spectral flux. **Deliberately not used** as an alignment anchor: it leads or lags the true attack by -96 to +250 ms.

Overfitting - memorising the training data. Not applicable; nothing is trained here.

P

PCM - raw uncompressed audio samples. Pulse Code Modulation. `PCM_16` for every WAV this project writes.

Phoneme / viseme - a distinguishable unit of sound, versus a distinguishable unit of visible mouth shape. Several phonemes share a viseme, which is why lip reading is fundamentally ambiguous - /p/, /b/ and /m/ all look identical.

Prominence - how far a peak rises above the surrounding terrain. Used instead of a plain height threshold, because a threshold admits ripples on the shoulder of a larger peak.

Q

Quantisation noise - the error from rounding to a finite number of levels. Inaudible at full scale; audible when a very quiet file is amplified by 40 dB. **This is precisely what the quality gate exists to prevent.**

R

ResNet - a convolutional network with residual (skip) connections. Lets a deep stack train without the gradient vanishing. Auto-AVSR's `Conv3dResNet`.

RMS - a measure of average energy. Root mean square. -36.87 dBFS in the final mix.

RoPE - rotary position embedding. Rotating queries and keys by an angle proportional to position, so attention depends on relative distance. MOSS's DiT carries three `complex128` RoPE tables, handled specially in `cast_params_only()`.

S

Sample rate - samples per second. 48 kHz throughout Subsystem 2; 24 kHz from Kokoro; equal to the capture frame rate in Subsystem 3.

Seed - a number fixing the random generator. Same seed and settings, byte-identical output. 42 by default; 43 and 44 as retries.

SentencePiece - a sub-word tokenizer. Unigram model with a 5000-token vocabulary in Auto-AVSR; word starts are marked with a special character.

Sigma shift - a parameter warping the noise schedule so steps are spent where they matter. 5.0, read from the checkpoint's own scheduler configuration.

Softmax - turning scores into probabilities that sum to one. Exponentiate and normalise.

Spectral flatness - how noise-like a spectrum is. Geometric mean over arithmetic mean; near 1 for noise, near 0 for a tone. Part of the pure-tone gate.

Spectrogram - a picture of frequency content over time. The magnitude of an STFT. Used in Subsystem 3's output and in the quality metrics.

Steerable pyramid - a bank of oriented, multi-scale filters. Complex-valued here, so local phase can be tracked. The core of Subsystem 3, via `pyrtools`.

STFT - Short-Time Fourier Transform. Sliding a window along a signal and taking a Fourier transform of each. `n_fft=2048` in the quality gate.

Stream copy - passing a compressed stream through untouched. `-c:v copy`. The output picture is bit-identical and there is no generation loss.

Subprocess - a separate operating-system process. How every model in this project is run, so memory, dependencies and failures are isolated.

T

Token - a sub-word unit. What a language model actually reads and writes. 5049 in Auto-AVSR's vocabulary.

Transformer - a sequence model built from attention rather than recurrence. Parallel over positions, unlike an RNN. Auto-AVSR's decoder, Qwen2.5-VL, MOSS's DiT and text encoder.

Transient - a short, sharp sound event. A footstep, a contact. Foley is made of them, which is why dynamic range and crest factor matter.

V

VAE - variational autoencoder. An autoencoder whose latent space is shaped so it can be sampled meaningfully. MOSS's DAC decoder.

Visual microphone - recovering sound from the vibration of objects in video. Phase-based analysis of sub-pixel motion (Davis et al., SIGGRAPH 2014). Subsystem 3.

VLM - vision-language model. A vision encoder feeding a language model, so image patches become tokens. Qwen2.5-VL-3B-Instruct.

VSR - visual speech recognition. Mapping mouth images to words using no audio. Auto-AVSR's `vsr_` checkpoint.

W

WER - word error rate. Insertions plus deletions plus substitutions, divided by the number of reference words. **Not measured in this project**; the 20.3 percent figure on LRS3 is the checkpoint authors' published result.

Z

Zero-crossing snap - moving a cut point to where the waveform crosses zero. Prevents a step discontinuity, which is audible as a click. Nearest crossing within plus or minus 3 ms, with the offset recorded and added back to the placement.

Chapter 91 — Appendix A: how this handbook was verified

Every claim in this handbook was read out of the repository on this machine, or measured while writing it. This appendix records what was inspected, so the handbook can be checked.

91.1 Source files read in full

Module3_Fresh/backend/: main.py, api/routes.py, core/config.py, core/jobs.py, and every module in services/ and runners/, plus tests/test_suite.py.

Module3_Fresh/frontend/src/: every .tsx and .ts file, plus package.json, vite.config.ts, tailwind.config.js, tsconfig.json and index.html.

Module3_Fresh/moss/scripts/: moss_generate.py, moss_phased.py, mps_compat.py.

Module3_Fresh/scripts/: visual_events.py, m3_config.py, run_module3.py.

03-FoleyCrafter-Test/action-recognition/: action_recognition.py, resolve_segments.py.

02-Auto-AVSR-Test/: run_server.py, preprocess_video.py, tts_worker.py, requirements.txt, and app/api.py, inference.py, video_processing.py, timing.py, sync.py, tts.py, gender.py, templates/index.html.

Acoustic eye/acoustic-eye/: README.md, backend/config.py, backend/processing/visual_microphone.py, signal_processing.py, and the pipeline and route structure.

01-Lip-Reading/: av_inference.py, requirements-frozen.txt, and the directory listing.

91.2 Documentation read in full

README.md at the repository root; Module3_Fresh/README.md and HANDOFF.md; Module3_Fresh/docs/architecture.md, api.md, pipeline.md; Module3_Fresh/results/documentation/01_MODULE3_TECHNICAL_DOCUMENTATION.md, 03_PROCESSING_FLOWCHART.md, 04_PROMPTS_REFERENCE.md, 05_RESULTS_AND_QA.md; Module3_Fresh/results/text_to_audio_model_evaluation.md, cup_pickup_moss_v1_report.md, cup_pickup_moss_v2_report.md; 02-Auto-AVSR-Test/README.md; paper/README.md and paper/main.tex in full.

91.3 Data read and recomputed

Source	What was taken
03-FoleyCrafter-Test/action-recognition/results/module2_action_segments.json	The reference timeline, timings, memory
.../results/videomae_test_result.json, temporal_results.json	The rejected recognisers' outputs
Module3_Fresh/data/jobs/aa4b2f4e6049/report.json	The current end-to-end run
Module3_Fresh/data/jobs/b0599ab38c1f/report.json	The coffee-stirring run and the cup-pickup rescue
Module3_Fresh/data/jobs/*.json (122 records)	Status distribution and the nine failure causes
Module3_Fresh/results/qa_polished.json	The 19-check quality gate
Module3_Fresh/results/web_*_generation.json (40 records)	Phase timings, memory, MPS verification

Source	What was taken
Module3_Fresh/moss/checkpoints/.../model_index.json, transformer/config.json	The DiT architecture
paper/experiments/exp1_sync.json	Aggregate synchronisation: 45 events, median 4.7 ms
paper/experiments/exp2_gate.json	Recomputed: 54 assets, 16 rejected, per-backend medians
paper/experiments/exp3_ablation.json	Both ablations
paper/experiments/exp5_vm2.json, exp4_vm.json	The visual microphone characterisation
paper/experiments/exp6_latency.json	All latency and memory means
02-Auto-AVSR-Test/outputs/*.txt (7 transcripts)	Every Subsystem 1 result

91.4 Commands executed while writing

Command	Result
moss/venv-moss/bin/python backend/tests/test_suite.py	42 passed, 0 failed
moss/venv-moss/bin/python backend/tests/test_foley_validation.py	22 passed, 0 failed
moss/venv-moss/bin/python backend/tests/e2e_gate.py	OK - completed in 5 s from cache, all four classes passed validation, no rejected asset reached the mix
A count of FoleySpec entries in prompt_map.py	17 curated classes
A count of @test (decorators	42 and 22
An analysis of Acoustic_eye/recovered/*.wav	20,000 Hz, 5.000 s, dominant 305.6 / 467.8 / 226.2 Hz in the final file
An aggregate over exp2_gate.json	29.6 percent rejected; 11 needing over +25 dB, median +37.0, max +42.1; MOSS 26 percent, Stable Audio 50 percent

91.5 Discrepancies found between the documentation and the code

These are reported rather than smoothed over, because an examiner reading your report and then your code will find them.

Claim in the written documentation	What the code says	Where
"Sixteen Foley classes"	17 - button_press was added later	README, docs/pipeline.md, paper/main.tex
"59 automated tests (36 + 22)"	64 (42 + 22) - tests were added with open-vocabulary synthesis	README, HANDOFF.md
"No supported Foley action was found" listed as a failure mode	No longer possible: pipeline.py now treats it as a partial result, and open vocabulary means resolve() never returns "unsupported"	docs/pipeline.md
"S3 has not recovered real sound"	A 20 kHz, 5-second recovery from a 20,000 fps clip exists in Acoustic_eye/recovered/, produced after the paper text was written	paper/main.tex Section IX
"Eight stages" in one place, nine in another	Nine, as listed in core/jobs.py	docs/pipeline.md heading versus its own content

KEY POINT: How to use this appendix

If an examiner points at a discrepancy, you now already know about it and can say which side is current. That converts a potential embarrassment into evidence that you know your own codebase better than your own report.

Chapter 92 — Appendix B: how to run everything

92.1 Subsystem 2 - action recognition and sound generation

```
cd Module3_Fresh
moss/venv-moss/bin/python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

```
cd Module3_Fresh/frontend && npm run dev
```

Then open <http://localhost:5173>. Interactive API documentation is at <http://127.0.0.1:8000/docs>.

Tests:

```
cd Module3_Fresh
moss/venv-moss/bin/python backend/tests/test_suite.py
moss/venv-moss/bin/python backend/tests/test_foley_validation.py
moss/venv-moss/bin/python backend/tests/e2e_gate.py
```

The validated standalone build:

```
moss/venv-moss/bin/python scripts/run_module3.py
```

92.2 Subsystem 1 - lip reading and speech generation

```
cd 02-Auto-AVSR-Test
python run_server.py --port 5001
```

Then open <http://127.0.0.1:5001>.

92.3 Subsystem 3 - Acoustic Eye

```
cd "Acoustic eye/acoustic-eye"
python run.py
```

For a high-speed clip too large to upload, use `POST /process-local` with the file's path, `capture_fps` set to the true capture rate, and `mains_notch_hz` set to 50 or 60.

92.4 Integrity checks before a demonstration

```
cd Module3_Fresh
shasum -a 256 -c <(grep -E '^[a-f0-9]{64}' results/APPROVED_ASSETS.lock)
(cd moss/MOSS-TTS && git status --porcelain) # must print nothing
```

CAUTION: The one operational rule that has already caused a wasted debugging session

Restart the backend after any code change. A running uvicorn process holds the old modules in memory, so a fix appears not to work. This is written into `HANDOFF.md` because it already happened once.

92.5 Rebuilding this handbook

The Markdown source is `handbook/PROJECT_HANDBOOK.md` and the PDF is `handbook/PROJECT_HANDBOOK.pdf`. The PDF is generated from the Markdown by a build script using ReportLab; the Markdown is the editable source of truth.